Lập Trình NodeJS
Từ Cơ Bản Đến Nâng Cao

Dành cho người mới bắt đầu — từ nền tảng đến cloud serverless

Phần 1: Nền tảng Web · Phần 2: NodeJS Core · Phần 3: Express.js · Phần 4: Database · Phần 5: Nâng cao & Cloud

PHẦN 1 — NỀN TẢNG

Chương 1: Internet và Web Hoạt Động Như Thế Nào?

Trước khi viết một dòng NodeJS nào, bạn cần hiểu sân chơi của mình. Chương này giải thích cách dữ liệu di chuyển từ trình duyệt của bạn đến một server ở góc khuất nào đó trên thế giới — và quay trở lại.

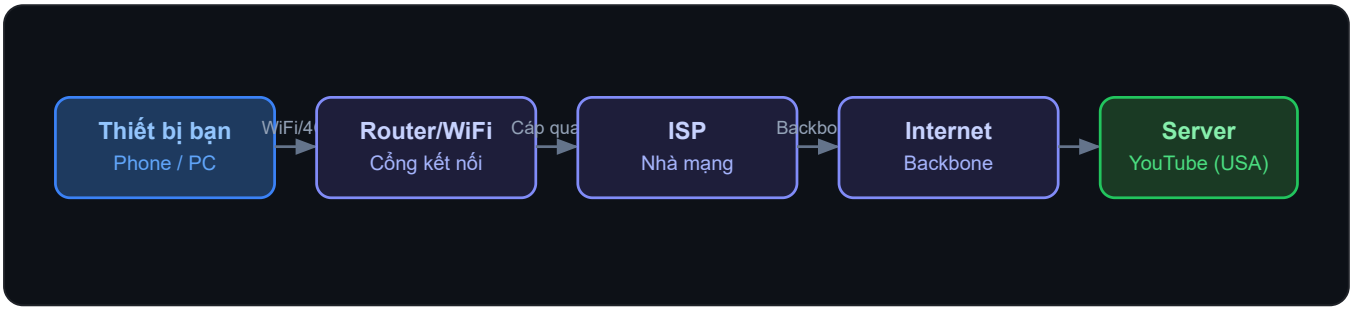
🎯 SAU CHƯƠNG NÀY BẠN SẼ HIỂU ĐƯỢC

- Internet là gì, và Web khác Internet ở chỗ nào
- Client và Server là ai, nói chuyện với nhau như thế nào
- HTTP request và response trông như thế nào bên trong
- URL, IP Address, Domain, DNS hoạt động ra sao
- Frontend và Backend khác nhau ở đâu

1.1 Internet là gì?

Hãy tưởng tượng Internet như một mạng lưới đường cao tốc khổng lồ nối các máy tính lại với nhau. Khi bạn xem YouTube, máy tính của bạn gửi một "gói dữ liệu" đi qua nhiều trạm trung chuyển, qua cáp quang dưới đáy biển, để đến máy chủ của Google ở đâu đó ở Mỹ — và video quay trở lại chỉ trong vài trăm mili-giây.

Internet là hạ tầng vật lý và giao thức (quy ước) để các máy tính giao tiếp với nhau toàn cầu. **Web (World Wide Web)** chỉ là một trong những dịch vụ chạy trên Internet — cùng với Email, FTP, SSH, v.v.



1.2 Client và Server

Trong thế giới web, mọi cuộc trò chuyện đều có hai vai:

- **Client** — người hỏi. Thường là trình duyệt web (Chrome, Firefox), app di động, hoặc bất kỳ chương trình nào *yêu cầu* dữ liệu.
- **Server** — người trả lời. Là máy tính (thường rất mạnh) chạy 24/7, *lắng nghe* và *phản hồi* các yêu cầu.



> 📌 **QUAN TRỌNG**
>
> Server luôn là người **bị động** — nó không tự dưng gửi dữ liệu đến client. Nó chỉ phản hồi khi được hỏi. (Trừ khi dùng WebSocket — nhưng đó là câu chuyện của chương sau.)

1.3 HTTP — Ngôn ngữ của Web

HTTP (HyperText Transfer Protocol) là "ngôn ngữ" mà client và server dùng để nói chuyện. Mỗi cuộc trò chuyện gồm hai phần: **Request** (yêu cầu) và **Response** (phản hồi).

HTTP Request trông như thế này

HTTP Ví dụ HTTP Request Copy

```
GET /api/products?category=phone HTTP/1.1
Host: shop.example.com
Accept: application/json
Authorization: Bearer eyJhbGci ...
User-Agent: Mozilla/5.0 (Windows NT 10.0)
```

Giải thích từng dòng:

- `GET` — phương thức HTTP (hành động muốn làm gì)
- `/api/products?category=phone` — đường dẫn + query string
- `HTTP/1.1` — phiên bản giao thức
- `Host` — tên miền đang gửi đến
- `Authorization` — token xác thực (nếu có)

HTTP Response trông như thế này

HTTP Ví dụ HTTP Response Copy

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 248
Cache-Control: max-age=60

{
  "products": [
    { "id": 1, "name": "iPhone 15", "price": 25000000 },
    { "id": 2, "name": "Samsung S24", "price": 22000000 }
  ]
}
```

Giải thích:

- `200 OK` — status code, nghĩa là thành công
- `Content-Type: application/json` — dữ liệu trả về là JSON
- Dòng trống — phân cách header và body
- Phần JSON bên dưới — body, dữ liệu thực sự

Các HTTP Methods phổ biến

Method	Ý nghĩa	Ví dụ thực tế
`GET`	Lấy dữ liệu	Xem danh sách sản phẩm
`POST`	Tạo mới	Đặt hàng, đăng ký tài khoản
`PUT`	Cập nhật toàn bộ	Sửa thông tin hồ sơ
`PATCH`	Cập nhật một phần	Đổi chỉ avatar
`DELETE`	Xóa	Xóa bài viết

HTTP Status Codes

Nhóm	Ý nghĩa	Ví dụ hay gặp
`2xx`	Thành công ✅	200 OK, 201 Created, 204 No Content
`3xx`	Chuyển hướng 🔀	301 Moved, 302 Found
`4xx`	Lỗi từ client ❌	400 Bad Request, 401 Unauthorized, 404 Not Found
`5xx`	Lỗi từ server 💥	500 Internal Server Error, 503 Service Unavailable

1.4 URL, Domain, IP và DNS

Khi bạn gõ `https://facebook.com/profile/123` vào trình duyệt, URL đó có cấu trúc:



IP Address là gì?

Mỗi máy tính trên Internet có một địa chỉ IP (như số nhà). Ví dụ: `142.250.185.14` là một IP của Google. Nhưng nhớ số IP rất khó, nên chúng ta dùng **domain name** thay thế (như `google.com`).

DNS hoạt động như thế nào?

DNS (Domain Name System) như danh bạ điện thoại của Internet — nó dịch tên miền sang IP.

> 💡 **THỬ NGAY**
>
> Mở Terminal/Command Prompt và gõ: `nslookup google.com` Bạn sẽ thấy IP address thực của Google.

1.5 HTTPS và SSL/TLS

HTTP truyền dữ liệu dưới dạng văn bản thuần — bất kỳ ai nghe giữa đường cũng đọc được. **HTTPS** thêm lớp mã hóa SSL/TLS: dữ liệu được mã hóa trước khi gửi đi, chỉ client và server mới đọc được.

	HTTP	**HTTPS**
Port mặc định	80	443
Mã hóa	Không	SSL/TLS
Dùng cho	Dev local	Mọi website thực tế

1.6 Frontend vs Backend

Khi bạn mở một trang web, có hai thế giới làm việc song song:

	Frontend	**Backend**
Chạy ở đâu	Trình duyệt của người dùng	Server
Ngôn ngữ	HTML, CSS, JavaScript	NodeJS, Python, Java, PHP...
Người dùng thấy	Có (giao diện)	Không (logic ẩn)
Làm gì	Hiển thị, tương tác	Xử lý, lưu dữ liệu, bảo mật
Ví dụ	React, Vue, Angular	Express, Django, Spring

> 📌 **NODEJS LÀ GÌ TRONG BỨC TRANH NÀY?**
>
> NodeJS cho phép bạn chạy JavaScript ở phía **server**. Trước khi có NodeJS, JavaScript chỉ chạy được trong trình duyệt. NodeJS "giải phóng" JavaScript ra khỏi trình duyệt — bạn dùng cùng một ngôn ngữ để viết cả frontend lẫn backend.

1.7 Code mẫu — Xem HTTP Request trong thực tế

Ví dụ 1: Dùng DevTools xem request

Mở Chrome, nhấn `F12` → tab **Network** → tải trang bất kỳ. Bạn sẽ thấy hàng chục request hiện ra.

Ví dụ 2: Gửi HTTP request bằng JavaScript (fetch API)

JAVASCRIPT fetch-example.js Copy

```javascript
// Gửi GET request đến một API công khai
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then(response => {
    console.log('Status:', response.status);  // 200
    console.log('OK?', response.ok);           // true
    return response.json();
  })
  .then(data => {
    console.log('Dữ liệu nhận được:', data);
  })
  .catch(error => {
    console.error('Lỗi:', error);
  });
```

Kết quả khi chạy (trong DevTools Console hoặc NodeJS):

OUTPUT Copy

```
Status: 200
OK? true
Dữ liệu nhận được: {
  id: 1,
  title: 'sunt aut facere repellat ... ',
  body: 'quia et suscipit ... ',
  userId: 1
}
```

Ví dụ 3: Gửi POST request với dữ liệu

JAVASCRIPT post-example.js Copy

```
// Gửi POST request (tạo dữ liệu mới)
fetch('https://jsonplaceholder.typicode.com/posts', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',  // Báo server biết ta gửi JSON
  },
  body: JSON.stringify({                  // Chuyển object JS thành chuỗi
JSON
    title: 'Bài viết mới',
    body: 'Nội dung bài viết...',
    userId: 1,
  }),
})
  .then(response => response.json())
  .then(data => {
    console.log('Tạo thành công:', data);
    // { id: 101, title: 'Bài viết mới', ... }
  });
```

Ví dụ 4: Xem raw HTTP bằng curl (Terminal)

SHELL Chạy trong Terminal Copy

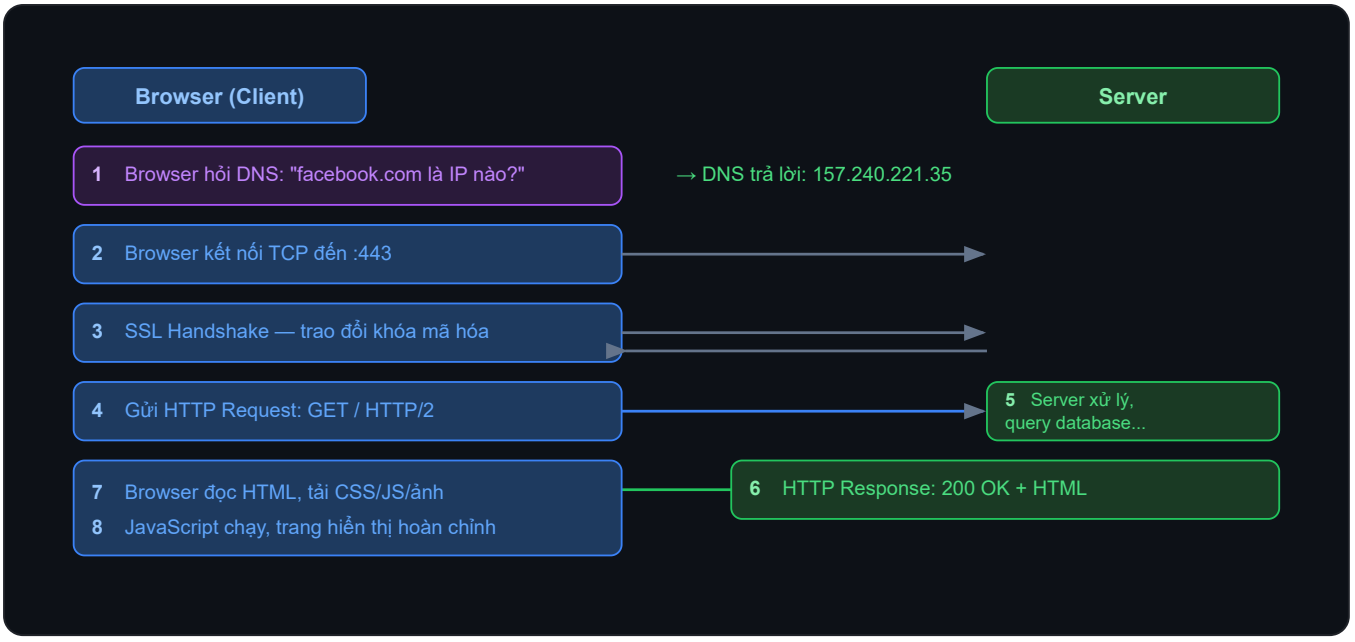
```shell
# Xem toàn bộ HTTP request và response headers
curl -v https://jsonplaceholder.typicode.com/posts/1

# Kết quả sẽ hiện:
# > GET /posts/1 HTTP/2
# > Host: jsonplaceholder.typicode.com
# < HTTP/2 200
# < content-type: application/json; charset=utf-8
# { "userId": 1, "id": 1, ... }
```

> ⚠️ **LƯU Ý**
>
> `curl` có sẵn trên macOS và Linux. Trên Windows 10+, bạn cũng có thể dùng curl trong PowerShell hoặc Command Prompt. Nếu không có, cài curl.se hoặc dùng Postman.

1.8 Tóm tắt cơ chế hoạt động đầy đủ

Khi bạn gõ `https://facebook.com` và nhấn Enter, đây là điều xảy ra:



📚 Tóm tắt chương 1

- **Internet** là hạ tầng mạng toàn cầu; **Web** là một dịch vụ chạy trên Internet
- **Client** gửi request, **Server** trả response — đây là mô hình cơ bản của web
- **HTTP** là giao thức giao tiếp: gồm method (GET/POST...), URL, headers, body và status code
- **DNS** dịch tên miền thành IP address để máy tính có thể kết nối
- **HTTPS** = HTTP + mã hóa SSL/TLS, luôn dùng cho production
- **Frontend** chạy trong trình duyệt; **Backend** chạy trên server — NodeJS thuộc backend

🏋️ Bài tập thực hành

DỄ

1.1 Khám phá DevTools

Mở Chrome, vào **trang bất kỳ** (vd: vnexpress.net), nhấn `F12` → tab Network. Reload trang và quan sát: Có bao nhiêu request được gửi đi? Request nào mất nhiều thời gian nhất? Click vào một request và xem Headers tab — ghi chép lại Status Code và Content-Type.

DỄ

1.2 Dùng fetch trong Console

Mở DevTools Console (F12 → Console). Copy và chạy ví dụ fetch ở mục 1.7. Thử thay đổi URL thành `/posts/5` và xem kết quả thay đổi như thế nào. Thử gọi `/posts/999` — status code là bao nhiêu?

TRUNG BÌNH

1.3 Xây dựng hàm fetchData có error handling

Viết một hàm `fetchData(url)` nhận URL và trả về data. Hàm phải xử lý cả trường hợp lỗi mạng (`catch`) lẫn HTTP error (status code 4xx/5xx). Hint: `response.ok` là `false` khi status code >= 400.

KHÓ

1.4 Mini HTTP Inspector

Viết một trang HTML đơn giản có một ô input để nhập URL và một button "Gửi Request". Khi click, trang gửi GET request đến URL đó và hiển thị: status code, content-type header, và 500 ký tự đầu của response body. Thêm loading indicator khi đang chờ.

PHẦN 1 — NỀN TẢNG

Chương 2: Stateful vs Stateless — Cookie, Session & Browser Storage

HTTP vốn là giao thức "hay quên" — mỗi request là một cuộc trò chuyện mới hoàn toàn. Chương này giải thích tại sao cần lưu "trạng thái" và các cách khác nhau để làm điều đó, từ cookie đến localStorage đến session server.

🎯 SAU CHƯƠNG NÀY BẠN SẼ HIỂU ĐƯỢC

- HTTP là stateless nghĩa là gì và tại sao đây là vấn đề
- Cookie là gì, hoạt động thế nào, và các tùy chọn bảo mật
- Sự khác biệt giữa localStorage, sessionStorage và IndexedDB
- Server Session và JWT Token khác nhau ở đâu
- Khi nào dùng cái nào trong thực tế

2.1 HTTP là Stateless — Vấn đề gì?

Ở chương trước, bạn đã biết HTTP là giao thức giao tiếp giữa client và server. Điều quan trọng cần biết: **HTTP là stateless** — không có trạng thái.

Điều này có nghĩa là: mỗi HTTP request là **độc lập hoàn toàn**. Server không nhớ bạn từ request trước. Hãy xem ví dụ:



Đây là bản chất của HTTP — mỗi request là một tờ giấy trắng. Server không có "ký ức". Vậy làm sao các website biết bạn đã đăng nhập? Đó là lúc chúng ta cần cơ chế lưu **state**.

State là gì?

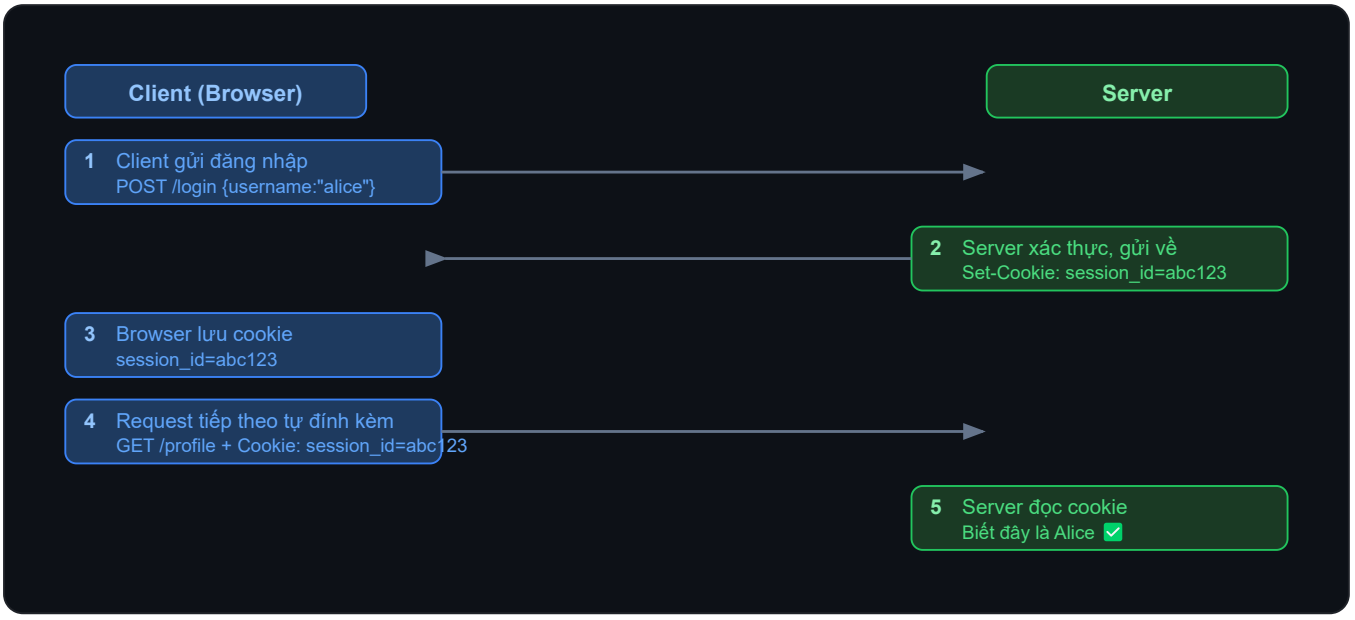
State là thông tin cần được ghi nhớ giữa các request. Ví dụ:

- Bạn đã đăng nhập chưa? (authentication state)
- Giỏ hàng của bạn có gì? (shopping cart state)
- Ngôn ngữ bạn đang dùng là tiếng Việt hay tiếng Anh? (preference state)
- Bạn đang ở trang nào trong wizard nhiều bước? (form state)

2.2 Cookie — Cách Web "Nhớ" Bạn

Cookie là gì?

Cookie là một mẩu dữ liệu nhỏ mà server gửi cho browser, và browser sẽ **tự động gửi lại** trong mọi request tiếp theo đến cùng domain đó.



Tạo cookie với JavaScript (phía client)

JAVASCRIPT cookie-basic.js Copy

```javascript
// Đặt cookie (có hiệu lực 7 ngày)
document.cookie = "username=alice; max-age=604800; path=/";

// Đặt nhiều cookie (mỗi lần chỉ set được 1)
document.cookie = "theme=dark; max-age=2592000; path=/";
document.cookie = "lang=vi; max-age=2592000; path=/";

// Đọc tất cả cookie (trả về chuỗi, không tiện lắm)
console.log(document.cookie);
// "username=alice; theme=dark; lang=vi"

// Hàm tiện ích để đọc 1 cookie theo tên
function getCookie(name) {
  const value = `; ${document.cookie}`;
  const parts = value.split(`; ${name}=`);
  if (parts.length === 2) return parts.pop().split(';').shift();
  return null;
}

console.log(getCookie('username')); // "alice"
console.log(getCookie('theme'));    // "dark"

// Xóa cookie bằng cách đặt max-age = 0
document.cookie = "username=; max-age=0; path=/";
```

Các thuộc tính quan trọng của Cookie

Thuộc tính	Ý nghĩa	Khi nào dùng
`max-age`	Thời gian sống (giây). `max-age=0` xóa cookie	Mọi cookie cần hết hạn
`expires`	Ngày hết hạn cụ thể	Thay thế cho max-age
`path`	Cookie chỉ gửi khi URL bắt đầu bằng path này	`path=/` = toàn site
`domain`	Domain nào nhận cookie	Subdomain sharing
`HttpOnly`	JS không đọc được — chỉ browser gửi tự động	Session token — bảo vệ XSS
`Secure`	Chỉ gửi qua HTTPS	Mọi cookie nhạy cảm
`SameSite`	Kiểm soát khi nào cookie được gửi cross-site	Chống CSRF attack

> ⚠️ **BẢO MẬT COOKIE**
>
> Cookie session token phải luôn có `HttpOnly` và `Secure`. `HttpOnly` ngăn JavaScript đọc cookie (bảo vệ khỏi XSS attack). `Secure` đảm bảo cookie chỉ truyền qua HTTPS (tránh bị nghe lén).

Giới hạn của Cookie

- Tối đa ~4KB mỗi cookie
- Tối đa ~50 cookie mỗi domain
- Tự động gửi trong **mọi** request — tăng bandwidth
- Người dùng có thể tắt cookie

2.3 localStorage và sessionStorage

Không phải lúc nào cũng cần gửi dữ liệu lên server. Đôi khi chỉ cần lưu trong browser. JavaScript có hai cơ chế lưu trữ phía client:

localStorage

Lưu vĩnh viễn trong browser — không mất khi đóng tab, thậm chí đóng browser.

JAVASCRIPT localstorage-demo.js Copy

```javascript
// Lưu dữ liệu đơn giản
localStorage.setItem('theme', 'dark');
localStorage.setItem('lang', 'vi');

// Lưu object (phải JSON.stringify)
const user = { name: 'Alice', age: 25, role: 'admin' };
localStorage.setItem('user', JSON.stringify(user));

// Đọc dữ liệu
const theme = localStorage.getItem('theme');    // "dark"
const lang = localStorage.getItem('lang');      // "vi"
const userStr = localStorage.getItem('user');   // chuỗi JSON
const userObj = JSON.parse(userStr);            // { name: "Alice", ... }

// Xóa
localStorage.removeItem('lang');

// Xóa hết
localStorage.clear();

// Kiểm tra số lượng
console.log(localStorage.length); // số keys

// Duyệt qua tất cả
for (let i = 0; i < localStorage.length; i++) {
  const key = localStorage.key(i);
  const value = localStorage.getItem(key);
  console.log(`${key}: ${value}`);
}
```

sessionStorage

Tương tự localStorage nhưng **xóa ngay khi đóng tab**.

JAVASCRIPT sessionstorage-demo.js Copy

```javascript
// API y hệt localStorage, chỉ khác vòng đời
sessionStorage.setItem('checkout_step', '2');
sessionStorage.setItem('cart_total', '1500000');

// Dùng cho dữ liệu tạm thời trong 1 phiên làm việc
// Ví dụ: lưu bước hiện tại trong form đặt hàng nhiều bước
const currentStep = sessionStorage.getItem('checkout_step');
console.log('Đang ở bước:', currentStep); // "2"

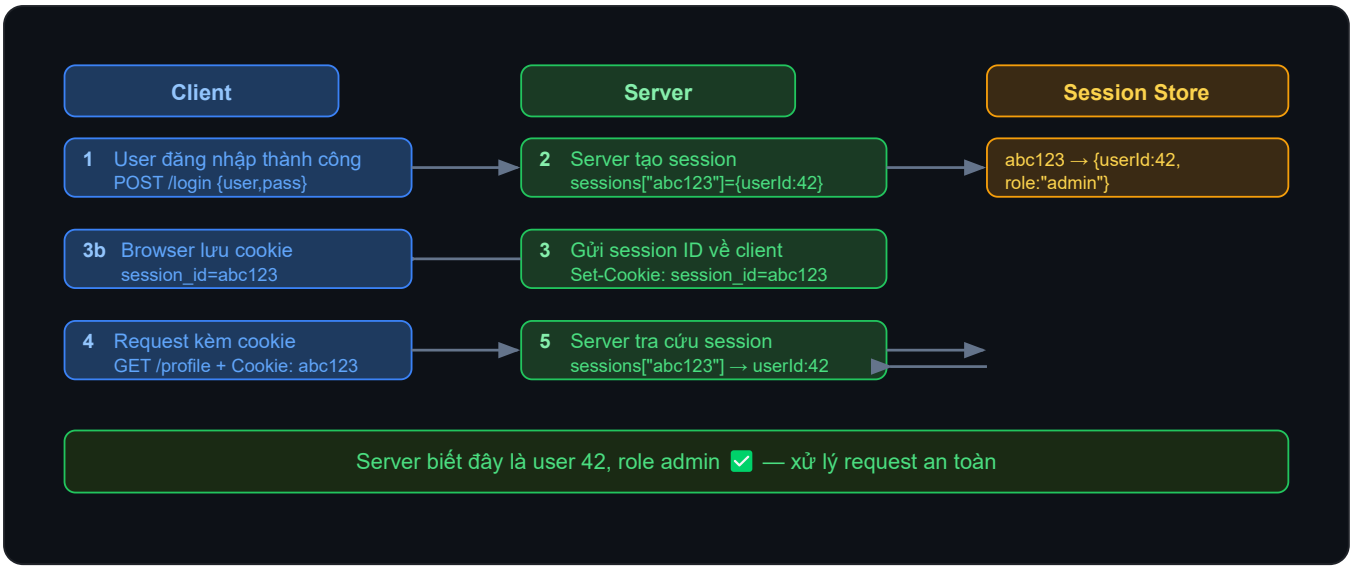
// Khi user đóng tab → sessionStorage tự xóa
// Tab mới mở lại → trống rỗng
```

So sánh Cookie vs localStorage vs sessionStorage

	Cookie	localStorage	sessionStorage
Dung lượng	~4KB	~5-10MB	~5-10MB
Thời gian sống	Theo max-age/expires	Vĩnh viễn	Đến khi đóng tab
Tự gửi lên server	✅ Có	❌ Không	❌ Không
JS có đọc được	Chỉ khi không có HttpOnly	✅ Luôn có	✅ Luôn có
Chia sẻ giữa tab	✅ Có	✅ Có	❌ Mỗi tab riêng
Dùng cho	Auth token, session	Preferences, cache	Form state, wizard

2.4 Server Session — "Nhớ" Phía Server

Cookie chỉ lưu dữ liệu ở client. Nhưng để bảo mật, dữ liệu nhạy cảm (như thông tin user đang đăng nhập) nên lưu ở **server**. Đây là cơ chế Server Session:

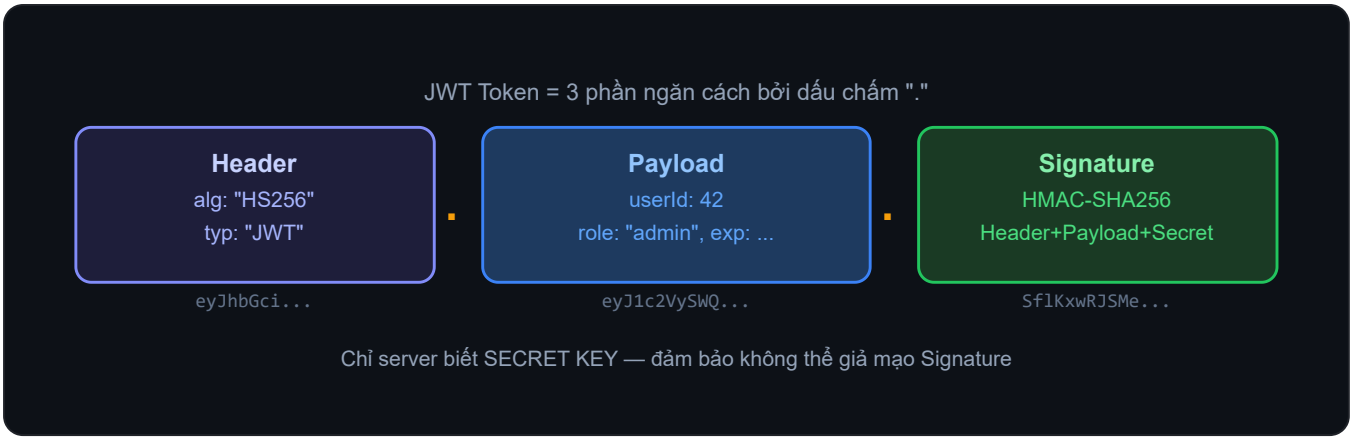


> **📌 TẠI SAO KHÔNG LƯU LUÔN DATA TRONG COOKIE?**
>
> Kỹ thuật này gọi là **session reference** — cookie chỉ chứa một ID ngẫu nhiên, dữ liệu thực sự ở server. Điều này:
>
> - Bảo mật hơn — user không thể giả mạo dữ liệu
> - Có thể lưu lượng lớn dữ liệu (không bị giới hạn 4KB)
> - Có thể vô hiệu hóa session từ server (đăng xuất bắt buộc)

2.5 JWT Token — Session Không Cần Server Lưu

Server Session có một nhược điểm: cần lưu trữ session ở đâu đó (RAM hoặc database). Với hàng triệu user, điều này tốn tài nguyên và phức tạp khi scale.

JWT (JSON Web Token) giải quyết bằng cách mã hóa dữ liệu ngay trong token:


JWT Token = 3 phần ngăn cách bởi dấu chấm "."
Header
alg: "HS256"
typ: "JWT"
.
Payload
userId: 42
role: "admin", exp: ...
.
Signature
HMAC-SHA256
Header+Payload+Secret
eyJhbGci...
eyJ1c2VySWQ...
SflKxwRJSMe...
Chỉ server biết SECRET KEY — đảm bảo không thể giả mạo Signature

JAVASCRIPT jwt-concept.js (chạy trong NodeJS) Copy

```javascript
// Cài đặt: npm install jsonwebtoken
const jwt = require('jsonwebtoken');

const SECRET_KEY = 'khoa-bi-mat-cua-ban-phai-dai-va-kho-doan';

// Tạo token khi đăng nhập thành công
function createToken(userId, role) {
  return jwt.sign(
    { userId, role },           // payload (dữ liệu muốn lưu)
    SECRET_KEY,                  // khóa bí mật để ký
    { expiresIn: '7d' }         // hết hạn sau 7 ngày
  );
}

// Xác thực token từ request
function verifyToken(token) {
  try {
    const decoded = jwt.verify(token, SECRET_KEY);
    return { ok: true, data: decoded };
  } catch (err) {
    return { ok: false, error: err.message };
  }
}

// Sử dụng
const token = createToken(42, 'admin');
console.log('Token:', token);

const result = verifyToken(token);
console.log('Verified:', result);
// { ok: true, data: { userId: 42, role: 'admin', iat: ..., exp: ... } }

// Giả mạo token sẽ fail
const fakeToken = token.slice(0, -5) + 'XXXXX';
```

```
console.log('Fake token:', verifyToken(fakeToken));
// { ok: false, error: 'invalid signature' }
```

So sánh Server Session vs JWT

	Server Session	**JWT Token**
Lưu dữ liệu ở đâu	Server (RAM/DB)	Client (cookie/localStorage)
Vô hiệu hóa tức thì	✅ Dễ (xóa khỏi store)	❌ Khó (phải dùng blacklist)
Scale (nhiều server)	Cần shared store (Redis)	✅ Dễ (verify bằng key)
Lưu được dữ liệu lớn	✅ Có	❌ Token sẽ rất to
Phù hợp cho	Web app truyền thống	API, microservices, mobile app

2.6 Code mẫu — Shopping Cart với localStorage

JAVASCRIPT shopping-cart.js Copy

```
// ========== Shopping Cart với localStorage ==========

const CART_KEY = 'my_cart';

// Đọc giỏ hàng từ localStorage
function getCart() {
  const data = localStorage.getItem(CART_KEY);
  return data ? JSON.parse(data) : [];
}

// Lưu giỏ hàng vào localStorage
function saveCart(cart) {
  localStorage.setItem(CART_KEY, JSON.stringify(cart));
}

// Thêm sản phẩm vào giỏ
function addToCart(product) {
  const cart = getCart();
  const existing = cart.find(item => item.id === product.id);

  if (existing) {
    existing.quantity += 1;   // Tăng số lượng nếu đã có
  } else {
    cart.push({ ...product, quantity: 1 });  // Thêm mới
  }

  saveCart(cart);
  return cart;
}

// Xóa sản phẩm
function removeFromCart(productId) {
```

```
  const cart = getCart().filter(item => item.id !== productId);
  saveCart(cart);
  return cart;
}

// Tính tổng tiền
function getTotal() {
  return getCart().reduce((sum, item) => sum + item.price * item.quantity,
0);
}

// Xóa toàn bộ giỏ hàng
function clearCart() {
  localStorage.removeItem(CART_KEY);
}

// ========== Sử dụng ==========

addToCart({ id: 1, name: 'iPhone 15', price: 25000000 });
addToCart({ id: 2, name: 'AirPods', price: 5000000 });
addToCart({ id: 1, name: 'iPhone 15', price: 25000000 }); // +1 qty

console.log('Giỏ hàng:', getCart());
// [{ id: 1, name: 'iPhone 15', price: 25000000, quantity: 2 },
//  { id: 2, name: 'AirPods', price: 5000000, quantity: 1 }]

console.log('Tổng tiền:', getTotal().toLocaleString('vi-VN'), 'VND');
// 55.000.000 VND

removeFromCart(2);
console.log('Sau khi xóa AirPods:', getCart().length, 'sản phẩm');
```

2.7 Stateful vs Stateless Application

Nhìn lại toàn cảnh, có hai kiểu thiết kế ứng dụng web:

	Stateful	Stateless
Server nhớ gì?	Nhớ thông tin user giữa requests (session)	Không nhớ gì — mỗi request tự đầy đủ
Ví dụ	Web app dùng Server Session	REST API dùng JWT
Scale	Phức tạp hơn (sticky sessions)	Dễ dàng (bất kỳ server nào cũng xử lý được)
Phù hợp	App nội bộ, ít traffic	Public API, microservices, high traffic

> 💡 **XU HƯỚNG HIỆN ĐẠI**
>
> Hầu hết API hiện đại (đặc biệt khi kết hợp với mobile app hoặc SPA) đều chọn **stateless với JWT** vì dễ scale và không cần shared session store. Tuy nhiên Server Session vẫn là lựa chọn tốt cho web app truyền thống.

📚 Tóm tắt chương 2

- **HTTP stateless**: mỗi request độc lập, server không tự nhớ bạn
- **Cookie**: browser tự gửi trong mọi request, giới hạn 4KB, dùng HttpOnly+Secure cho token
- **localStorage**: lưu vĩnh viễn, ~5-10MB, chỉ JS đọc được, không tự gửi lên server
- **sessionStorage**: như localStorage nhưng xóa khi đóng tab
- **Server Session**: cookie chứa ID, server lưu dữ liệu thực — bảo mật, dễ vô hiệu hóa
- **JWT**: dữ liệu mã hóa trong token, server không cần lưu — dễ scale

🏋️ Bài tập thực hành

DỄ

2.1 Khám phá Storage trong DevTools

Mở Chrome DevTools (F12) → tab **Application** → Storage. Vào một website bạn hay dùng (vd: tiki.vn, shopee.vn). Xem nội dung của Cookies, localStorage, sessionStorage. Ghi chép: website lưu gì ở đây? Bạn thấy token hay session ID không?

DỄ

2.2 Lưu và đọc user preferences

Viết code JavaScript (chạy trong Console) để lưu các preferences vào localStorage: theme (dark/light), fontSize (small/medium/large), language (vi/en). Sau đó viết hàm `loadPreferences()` đọc về và log ra. Thử đóng tab và mở lại — data có còn không?

TRUNG BÌNH

2.3 Hoàn thiện Shopping Cart

Dựa vào code mẫu ở mục 2.6, thêm các tính năng: (1) Hàm `updateQuantity(productId, newQuantity)` để thay đổi số lượng (2) Hàm `getItemCount()` trả về tổng số lượng sản phẩm (không phải số loại) (3) Khi quantity = 0, tự động xóa khỏi giỏ. Viết test case để kiểm tra các trường hợp edge case.

KHÓ

2.4 Dark Mode Persistent

Tạo một trang HTML với button toggle Dark/Light mode. Khi click, trang đổi theme và lưu vào localStorage. Khi mở lại trang (reload), trang phải tự apply đúng theme đã lưu **mà không bị flash trắng** trước khi JS chạy. Hint: Script đọc localStorage phải chạy ngay trong `<head>`, trước khi HTML render.

PHẦN 1 — NỀN TẢNG

Chương 3: NodeJS là gì? Cài đặt Môi Trường Lập Trình

Bây giờ bạn đã hiểu web hoạt động như thế nào, đã đến lúc gặp gỡ công cụ chính. Chương này giải thích NodeJS là gì, tại sao nó đặc biệt, và giúp bạn cài đặt đầy đủ môi trường để bắt đầu code.

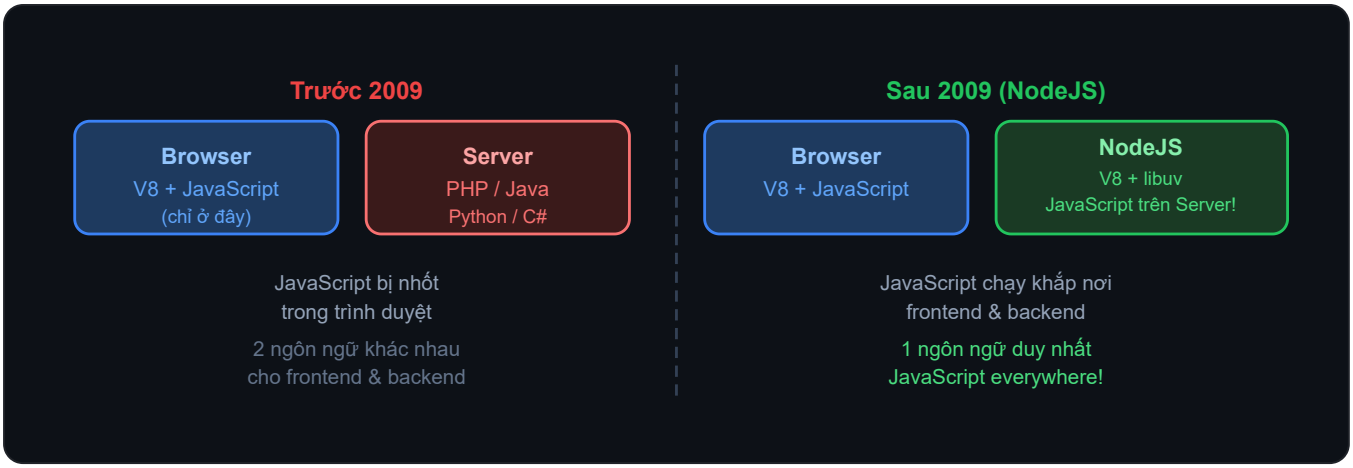
🎯 **SAU CHƯƠNG NÀY BẠN SẼ**

- Hiểu NodeJS là gì và tại sao JavaScript có thể chạy ngoài trình duyệt
- Cài đặt NodeJS, npm và VS Code
- Chạy được chương trình NodeJS đầu tiên
- Hiểu cơ bản về Event Loop — thứ làm NodeJS khác biệt
- Biết cách dùng npm để quản lý packages

3.1 NodeJS là gì?

Trước năm 2009, JavaScript chỉ sống trong trình duyệt. Bạn không thể dùng JS để đọc file, truy cập database, hay xây dựng server.

Năm 2009, **Ryan Dahl** tạo ra NodeJS — một môi trường chạy JavaScript bên ngoài trình duyệt, sử dụng **V8 engine** (engine JS của Chrome) cùng với thư viện **libuv** để xử lý I/O bất đồng bộ.



NodeJS không phải là ngôn ngữ mới hay framework — nó là **runtime environment** (môi trường chạy) cho JavaScript.

Tại sao NodeJS phổ biến?

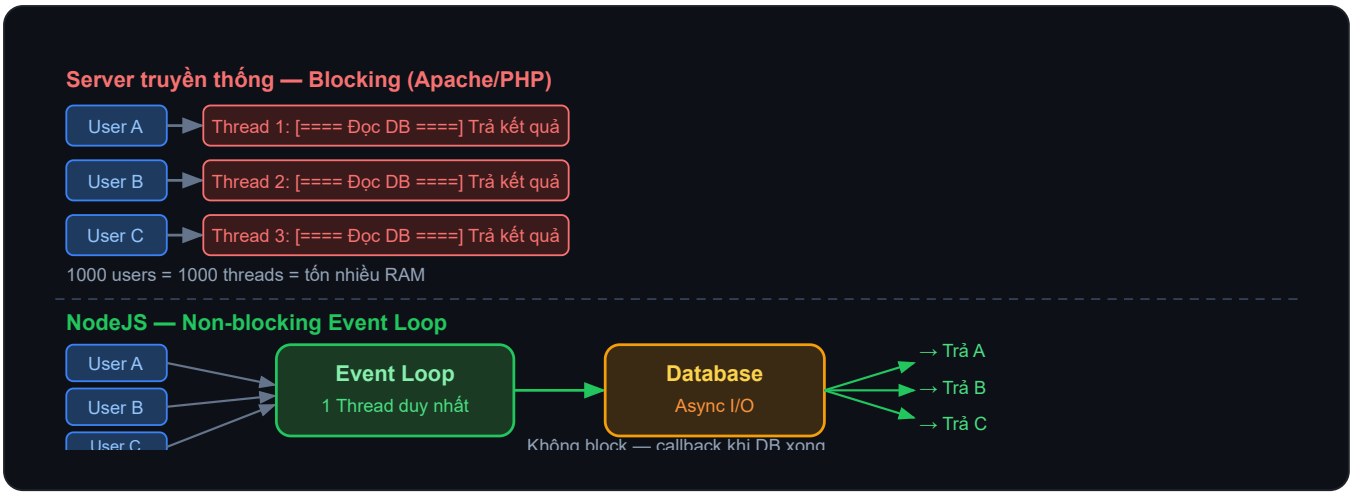
- **Một ngôn ngữ cho cả frontend lẫn backend** — dùng JS ở khắp nơi
- **Non-blocking I/O** — xử lý hàng nghìn kết nối đồng thời hiệu quả
- **Hệ sinh thái npm khổng lồ** — hơn 2 triệu packages
- **Cộng đồng lớn** — dễ tìm tài liệu, giải pháp
- **Nhanh** — V8 engine tối ưu cao, libuv xử lý I/O hiệu quả

NodeJS phù hợp cho gì?

Phù hợp ✅	Không phù hợp ❌
REST API, GraphQL API	CPU-intensive tasks (ML, video encoding)
Real-time apps (chat, gaming)	Tính toán nặng liên tục
Microservices	Heavy computation without worker threads
Streaming (video, audio)	
CLI tools, build tools	

3.2 Event Loop — Trái Tim của NodeJS

Điều làm NodeJS đặc biệt là **non-blocking, event-driven architecture**. Hãy so sánh với cách server truyền thống hoạt động:



NodeJS dùng **Event Loop** để quản lý điều này. Khi có I/O (đọc file, query DB, HTTP request), NodeJS không đứng chờ — nó đăng ký callback và tiếp tục xử lý việc khác. Khi I/O xong, callback được gọi.

JAVASCRIPT event-loop-demo.js Copy

```
const fs = require('fs');

console.log('1 - Bắt đầu đọc file...');

// Đọc file bất đồng bộ - không block
fs.readFile('data.txt', 'utf8', (err, content) => {
  // Callback này chạy SAU khi file đọc xong
  console.log('3 - File đã đọc xong:', content.length, 'ký tự');
});

// Dòng này chạy NGAY LẬP TỨC, không chờ file
console.log('2 - Dòng này chạy trong khi file đang được đọc!');

// Kết quả:
// 1 - Bắt đầu đọc file...
// 2 - Dòng này chạy trong khi file đang được đọc!
// 3 - File đã đọc xong: 1234 ký tự
```

> 📌 **GHI NHỚ**
>
> NodeJS là **single-threaded** nhưng **non-blocking**. Một thread, nhưng không bao giờ "ngồi chờ" I/O — nó luôn làm việc khác trong lúc chờ. Đây là sức mạnh của Event Loop.

3.3 Cài Đặt Môi Trường

Bước 1: Cài NodeJS

Có hai cách cài NodeJS:

Cách 1: Cài trực tiếp (đơn giản)

Vào `nodejs.org` → tải bản LTS (Long Term Support) → cài như phần mềm bình thường.

Cách 2: Dùng nvm — Node Version Manager (khuyên dùng)

nvm cho phép cài nhiều version NodeJS và chuyển đổi dễ dàng — rất hữu ích khi làm nhiều project.

SHELL — Cài nvm trên macOS/Linux — Copy

```
# Cài nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh |
bash

# Restart terminal, sau đó:
nvm install --lts        # Cài version LTS mới nhất
nvm use --lts            # Dùng version LTS
nvm list                 # Xem các version đã cài
```

SHELL — Cài nvm-windows (Windows) — Copy

```
# Tải nvm-windows từ:
# https://github.com/coreybutler/nvm-windows/releases
# Tải file nvm-setup.exe và chạy

# Sau khi cài:
nvm install lts
nvm use lts
```

SHELL Kiểm tra cài đặt thành công Copy

```
node --version    # v20.x.x hoặc cao hơn
npm --version     # 10.x.x hoặc cao hơn
```

Bước 2: Cài VS Code

VS Code là editor tốt nhất cho NodeJS development.

- Tải tại `code.visualstudio.com`
- Extensions nên cài: **ESLint**, **Prettier**, **REST Client**, **GitLens**

Bước 3: Tạo project đầu tiên

SHELL Copy

```
# Tạo thư mục project
mkdir hello-node
cd hello-node

# Khởi tạo project (tạo file package.json)
npm init -y

# Mở bằng VS Code
code .
```

3.4 Chương Trình NodeJS Đầu Tiên

Hello World

JAVASCRIPT hello.js Copy

```javascript
console.log('Hello, NodeJS!');
console.log('Node version:', process.version);
console.log('Platform:', process.platform);
console.log('Current directory:', process.cwd());
```

SHELL Chạy trong Terminal Copy

```shell
node hello.js

# Kết quả:
# Hello, NodeJS!
# Node version: v20.11.0
# Platform: win32
# Current directory: C:\Users\you\hello-node
```

NodeJS REPL — Thử code ngay trong Terminal

SHELL REPL – Read Eval Print Loop Copy

```
# Gõ 'node' không có argument để vào REPL
node

# Trong REPL, gõ code và nhấn Enter để chạy ngay:
> 2 + 2
4
> const name = 'NodeJS'
undefined
> `Hello ${name}!`
'Hello NodeJS!'
> [1,2,3].map(x => x * 2)
[ 2, 4, 6 ]

# Thoát REPL: Ctrl+C hai lần, hoặc gõ .exit
```

HTTP Server trong 5 dòng

JAVASCRIPT server.js Copy

```javascript
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });
  res.end('Xin chào từ NodeJS server!');
});

server.listen(3000, () => {
  console.log('Server đang chạy tại http://localhost:3000');
});
```

SHELL Copy

```shell
node server.js
# Server đang chạy tại http://localhost:3000

# Mở trình duyệt và vào: http://localhost:3000
# Bạn sẽ thấy: "Xin chào từ NodeJS server!"

# Nhấn Ctrl+C để dừng server
```

3.5 Module System trong NodeJS

NodeJS sử dụng hệ thống module để chia code thành các file riêng biệt.

CommonJS (require) — Cách truyền thống

JAVASCRIPT math.js Copy

```
// Định nghĩa và export
function add(a, b) { return a + b; }
function subtract(a, b) { return a - b; }
const PI = 3.14159;

module.exports = { add, subtract, PI };

// Hoặc export từng cái:
// module.exports.add = add;
```

JAVASCRIPT main.js Copy

```
// Import module tự viết
const math = require('./math');           // ./ = cùng thư mục
console.log(math.add(5, 3));              // 8
console.log(math.PI);                     // 3.14159

// Destructuring
const { add, PI } = require('./math');
console.log(add(10, 5));                  // 15

// Import module built-in (không cần ./)
const fs = require('fs');
const path = require('path');

// Import package từ npm (không cần ./)
const express = require('express');
```

ES Modules (import/export) — Cách hiện đại

JAVASCRIPT math.mjs (hoặc thêm "type":"module" vào package.json) Copy

```
// ES Module export
export function add(a, b) { return a + b; }
export const PI = 3.14159;
export default function multiply(a, b) { return a * b; }

// Import
import multiply, { add, PI } from './math.mjs';
console.log(add(2, 3));      // 5
console.log(multiply(4, 5)); // 20
```

💡 **NÊN DÙNG CÁI NÀO?**

Hiện tại hầu hết tutorial và codebase NodeJS dùng **CommonJS (require)**. ES Modules đang dần phổ biến hơn, đặc biệt với Typescript. Trong sách này chúng ta sẽ dùng CommonJS ở phần đầu, chuyển sang ES Modules ở phần sau.

3.6 npm — Node Package Manager

npm là gì?

npm là kho thư viện lớn nhất thế giới với hơn 2 triệu packages. Nó đi kèm với NodeJS và dùng để cài, quản lý các thư viện bên thứ ba.

SHELL Các lệnh npm hay dùng Copy

```shell
# Khởi tạo project mới
npm init -y

# Cài package (thêm vào dependencies)
npm install express
npm install mongoose jsonwebtoken  # Cài nhiều cùng lúc

# Cài package chỉ cho development (không deploy lên production)
npm install --save-dev nodemon jest

# Cài package globally (dùng như CLI tool)
npm install -g nodemon

# Xóa package
npm uninstall express

# Xem danh sách package đã cài
npm list

# Cập nhật packages
npm update

# Kiểm tra lỗi bảo mật
npm audit
npm audit fix
```

package.json — Hồ sơ của project

JSON package.json Copy

```json
{
  "name": "my-node-app",
  "version": "1.0.0",
  "description": "Ứng dụng NodeJS đầu tiên của tôi",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "test": "jest"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^8.0.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.2",
    "jest": "^29.7.0"
  }
}
```

SHELL Chạy scripts Copy

```shell
npm start      # Chạy script "start"
npm run dev    # Chạy script "dev" (dùng nodemon – auto restart khi code
thay đổi)
npm test       # Chạy tests
```

nodemon — Tự động restart server

Khi dev, mỗi lần sửa code bạn phải Ctrl+C rồi `node server.js` lại. Rất mất thời gian. `nodemon` theo dõi file và tự restart server khi có thay đổi.

SHELL Copy

```
npm install --save-dev nodemon
npx nodemon server.js   # Tự restart khi lưu file
```

3.7 Cấu trúc Project NodeJS chuẩn

TEXT Cấu trúc project Express API Copy

```
my-app/
├── src/
│   ├── index.js          # Entry point - khởi động server
│   ├── app.js            # Cấu hình Express app
│   ├── routes/           # Định nghĩa routes
│   │   ├── users.js
│   │   └── products.js
│   ├── controllers/      # Xử lý logic request/response
│   │   ├── userController.js
│   │   └── productController.js
│   ├── models/           # Database models
│   │   ├── User.js
│   │   └── Product.js
│   ├── middleware/       # Custom middleware
│   │   ├── auth.js
│   │   └── errorHandler.js
│   └── utils/            # Helper functions
│       └── logger.js
├── tests/                # Test files
├── .env                  # Environment variables (KHÔNG commit lên git!)
├── .env.example          # Template .env để người khác tham khảo
├── .gitignore
├── package.json
└── README.md
```

TEXT .gitignore Copy

```
node_modules/    # Thư mục này có thể rất lớn, KHÔNG commit
.env             # Chứa secrets, KHÔNG commit
.env.local
dist/
*.log
```

3.8 Script tính toán đầu tiên

Hãy viết một script thực tế hơn Hello World:

JAVASCRIPT calculator.js Copy

```javascript
// Nhận arguments từ command line
// Chạy: node calculator.js 10 + 5
const args = process.argv.slice(2); // Bỏ "node" và "calculator.js"

if (args.length !== 3) {
  console.error('Cách dùng: node calculator.js   ');
  console.error('Ví dụ: node calculator.js 10 + 5');
  process.exit(1); // Thoát với lỗi
}

const [num1Str, operator, num2Str] = args;
const num1 = parseFloat(num1Str);
const num2 = parseFloat(num2Str);

if (isNaN(num1) || isNaN(num2)) {
  console.error('Lỗi: Vui lòng nhập số hợp lệ');
  process.exit(1);
}

let result;
switch (operator) {
  case '+': result = num1 + num2; break;
  case '-': result = num1 - num2; break;
  case '*': result = num1 * num2; break;
  case '/':
    if (num2 === 0) {
      console.error('Lỗi: Không thể chia cho 0');
      process.exit(1);
    }
    result = num1 / num2;
    break;
  default:
    console.error(`Lỗi: Phép tính '${operator}' không được hỗ trợ`);
    process.exit(1);
}
```

```
console.log(`${num1} ${operator} ${num2} = ${result}`);
```

SHELL Copy

```
node calculator.js 10 + 5
# 10 + 5 = 15

node calculator.js 100 / 4
# 100 / 4 = 25

node calculator.js 7.5 "*" 3
# 7.5 * 3 = 22.5

node calculator.js 10 / 0
# Lỗi: Không thể chia cho 0
```

📚 Tóm tắt chương 3

- **NodeJS** là runtime environment cho JavaScript bên ngoài browser, dùng V8 engine + libuv
- **Event Loop** cho phép NodeJS xử lý I/O bất đồng bộ với single thread — hiệu quả cao
- Cài NodeJS qua **nvm** (khuyên dùng) hoặc tải trực tiếp từ nodejs.org
- **CommonJS** (`require/module.exports`) là module system mặc định
- **npm** quản lý packages; **package.json** là hồ sơ project
- **nodemon** giúp tự động restart server khi dev

🏋️ Bài tập thực hành

DỄ

3.1 Cài đặt và Hello World

Cài NodeJS bằng nvm. Chạy `node --version` và `npm --version` để xác nhận. Tạo file `info.js` in ra: Node version, npm version, platform, current user (dùng `process.env.USER` hoặc `process.env.USERNAME` tùy OS).

DỄ

3.2 Module system

Tạo file `utils.js` export 3 hàm: `formatCurrency(amount)` (format số thành tiền VND), `capitalizeFirst(str)` (viết hoa chữ đầu), và `slugify(str)` (chuyển "Xin Chào" thành "xin-chao"). Import và test trong `main.js`.

TRUNG BÌNH

3.3 HTTP Server nâng cao

Nâng cấp server.js để xử lý các route khác nhau: `GET /` → "Trang chủ", `GET /about` → "Giới thiệu", `GET /api/time` → JSON chứa thời gian hiện tại, Route không tồn tại → 404 với message phù hợp.

KHÓ

3.4 CLI Task Manager

Xây dựng CLI tool quản lý todo list đơn giản lưu vào file JSON. Commands: `node todo.js add "Học NodeJS"`, `node todo.js list` (hiện danh sách), `node todo.js done 1` (đánh dấu hoàn thành task số 1), `node todo.js delete 2` (xóa task số 2). Dùng `fs` module để đọc/ghi file JSON.

PHẦN 2 — NODEJS CƠ BẢN

Chương 4: NodeJS Core Modules — fs, path, os, events, crypto

NodeJS đi kèm hàng chục module có sẵn không cần cài thêm. Chương này khám phá những module quan trọng nhất mà bạn sẽ dùng hàng ngày: đọc ghi file, xử lý đường dẫn, lắng nghe sự kiện, mã hóa dữ liệu, và xử lý dữ liệu lớn bằng streams.

🎯 **SAU CHƯƠNG NÀY BẠN SẼ**

- Hiểu module system của NodeJS: CommonJS và ES Modules khác nhau ở đâu
- Đọc, ghi, xóa, đổi tên file với module `fs` — cả sync lẫn async
- Xử lý đường dẫn đa nền tảng an toàn với `path`
- Lấy thông tin hệ thống (CPU, RAM, OS) qua module `os`
- Xây dựng kiến trúc hướng sự kiện với `EventEmitter`
- Tạo hash, UUID và random bytes an toàn với `crypto`
- Xử lý file lớn hiệu quả bằng Streams và `pipe()`

4.1 Module System: require vs import

Khi bạn viết `const fs = require('fs')`, bạn đang dùng hệ thống module của NodeJS để nạp một module vào. NodeJS hỗ trợ hai hệ thống module song song nhau: **CommonJS** (ra đời từ đầu) và **ES Modules** (chuẩn của JavaScript hiện đại).

CommonJS — Hệ thống cũ nhưng vẫn phổ biến

CommonJS ra đời trước khi JavaScript có chuẩn module. Cú pháp rất đơn giản:

JAVASCRIPT math.js Copy

```javascript
// Cách 1: export một object
module.exports = {
  add: (a, b) => a + b,
  subtract: (a, b) => a - b,
  PI: 3.14159
};

// Cách 2: export từng thứ riêng lẻ (exports là alias của module.exports)
exports.multiply = (a, b) => a * b;
exports.divide = (a, b) => {
  if (b === 0) throw new Error('Không thể chia cho 0');
  return a / b;
};

// Cách 3: export một class
class Calculator {
  constructor() {
    this.history = [];
  }
  add(a, b) {
    const result = a + b;
    this.history.push(`${a} + ${b} = ${result}`);
    return result;
  }
}
module.exports = Calculator;
```

JAVASCRIPT app.js Copy

```
// Import module của mình
const math = require('./math');
console.log(math.add(2, 3));     // 5
console.log(math.PI);             // 3.14159

// Destructuring khi import
const { add, subtract } = require('./math');
console.log(subtract(10, 4));    // 6

// Import built-in module của NodeJS
const fs = require('fs');
const path = require('path');
const os = require('os');

// Import từ node_modules (đã cài qua npm)
const express = require('express');
```

ES Modules — Chuẩn hiện đại

ES Modules (ESM) là chuẩn module chính thức của JavaScript, được trình duyệt hỗ trợ native. NodeJS hỗ trợ ESM từ version 12, nhưng cần khai báo:

JSON package.json Copy

```json
{
  "name": "my-project",
  "version": "1.0.0",
  "type": "module"  // Thêm dòng này để dùng ESM toàn project
}
```

JAVASCRIPT math.mjs Copy

```javascript
// Named exports
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;
export const PI = 3.14159;

// Default export
export default class Calculator {
  add(a, b) { return a + b; }
}

// Re-export từ module khác
export { readFile, writeFile } from 'fs/promises';
```

JAVASCRIPT app.mjs Copy

```
// Named imports
import { add, subtract, PI } from './math.mjs';

// Default import
import Calculator from './math.mjs';

// Import tất cả vào namespace
import * as math from './math.mjs';
console.log(math.add(1, 2));    // 3

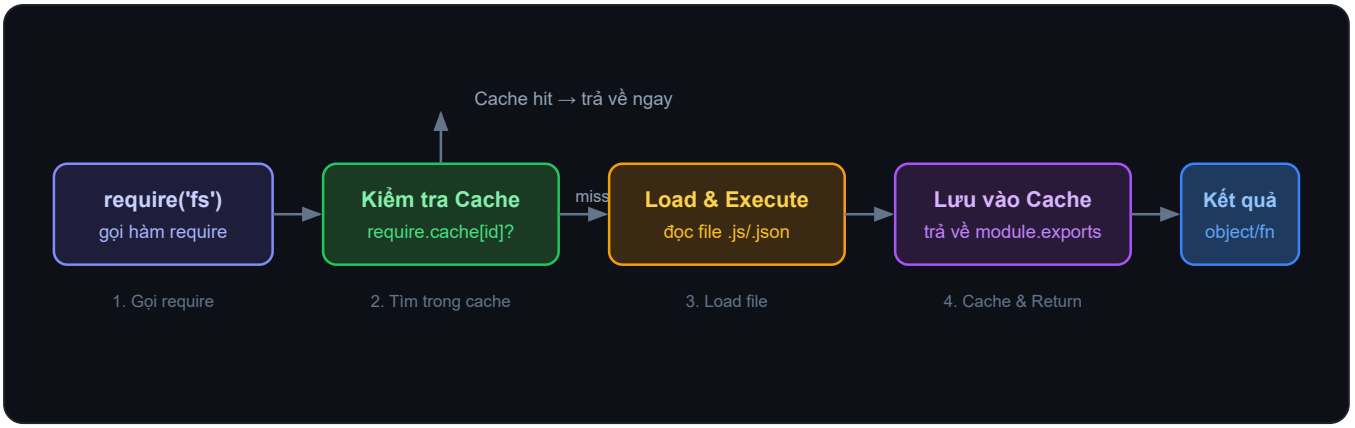
// Import built-in module của NodeJS
import { readFile, writeFile } from 'fs/promises';
import { join, dirname } from 'path';
import { fileURLToPath } from 'url';

// Trong ESM không có __dirname, phải tự tính:
const __filename = fileURLToPath(import.meta.url);
const __dirname = dirname(__filename);
```

> 📌 **COMMONJS HAY ESM?**
>
> Nếu bạn đang bắt đầu dự án mới: dùng **ESM** vì đó là tương lai. Nếu bạn làm việc với codebase cũ hoặc các thư viện chưa hỗ trợ ESM: dùng **CommonJS**. Nhiều dự án thực tế vẫn dùng CommonJS vì hệ sinh thái npm rất lớn và nhiều thư viện chỉ có CommonJS. Trong sách này ta dùng CommonJS cho phần lớn ví dụ vì dễ tiếp cận hơn.

Cơ chế hoạt động của require()

Khi bạn gọi `require('fs')`, NodeJS thực hiện một chuỗi bước:



Nhờ cơ chế cache, nếu bạn `require('./config')` ở 10 chỗ khác nhau trong app, file chỉ được đọc và thực thi **một lần duy nhất**. Đây cũng là cách tạo *singleton* trong NodeJS — rất hữu ích cho kết nối database.

4.2 Module `fs` — Làm việc với File System

Module `fs` là cầu nối giữa NodeJS và hệ thống file của OS. Từ đọc config file, ghi log, tạo thư mục, cho đến đọc hàng triệu dòng dữ liệu — `fs` làm hết.

Đọc file

JAVASCRIPT read-file.js Copy

```javascript
const fs = require('fs');

// ═══ ASYNC (không block) ═══
fs.readFile('./data.txt', 'utf8', (err, data) => {
  if (err) {
    console.error('Lỗi đọc file:', err.message);
    return;
  }
  console.log('Nội dung file:', data);
});

// ═══ SYNC (block toàn bộ process) ═══
// Chỉ dùng khi khởi động app, không dùng trong request handler
try {
  const data = fs.readFileSync('./data.txt', 'utf8');
  console.log('Nội dung:', data);
} catch (err) {
  console.error('Lỗi:', err.message);
}

// ═══ PROMISE (khuyến nghị dùng) ═══
const fsPromises = require('fs/promises');

async function readConfig() {
  try {
    const content = await fsPromises.readFile('./config.json', 'utf8');
    const config = JSON.parse(content);
    return config;
  } catch (err) {
    if (err.code === 'ENOENT') {
      console.log('Config không tồn tại, dùng mặc định');
      return { port: 3000, debug: false };
    }
    throw err;
```

```
  }
}
```

Ghi file

JAVASCRIPT write-file.js Copy

```
const fs = require('fs/promises');

// Ghi file (tạo mới hoặc ghi đè)
async function saveUserData(userId, data) {
  const filePath = `./data/users/${userId}.json`;
  const json = JSON.stringify(data, null, 2);  // indent 2 spaces
  await fs.writeFile(filePath, json, 'utf8');
  console.log(`Đã lưu dữ liệu user ${userId}`);
}

// Thêm vào cuối file (không xóa nội dung cũ)
async function appendLog(message) {
  const timestamp = new Date().toISOString();
  const logLine = `[${timestamp}] ${message}\n`;
  await fs.appendFile('./app.log', logLine, 'utf8');
}

// Ví dụ sử dụng
(async () => {
  await saveUserData('user123', { name: 'Nguyễn Văn A', age: 25 });
  await appendLog('User user123 đã đăng nhập');
  await appendLog('User user123 đã xem trang chủ');
})();
```

Thao tác với thư mục và file

JAVASCRIPT file-operations.js Copy

```javascript
const fs = require('fs/promises');
const path = require('path');

// Liệt kê file trong thư mục
async function listFiles(dirPath) {
  const entries = await fs.readdir(dirPath, { withFileTypes: true });
  for (const entry of entries) {
    const fullPath = path.join(dirPath, entry.name);
    if (entry.isDirectory()) {
      console.log(`📁 ${entry.name}/`);
    } else {
      const stat = await fs.stat(fullPath);
      const sizeKB = (stat.size / 1024).toFixed(1);
      console.log(`📄 ${entry.name} (${sizeKB} KB)`);
    }
  }
}

// Tạo thư mục (kể cả nested)
async function ensureDir(dirPath) {
  await fs.mkdir(dirPath, { recursive: true });
  // recursive: true = không lỗi nếu đã tồn tại
}

// Xóa file
async function deleteFile(filePath) {
  try {
    await fs.unlink(filePath);
    console.log(`Đã xóa: ${filePath}`);
  } catch (err) {
    if (err.code === 'ENOENT') {
      console.log('File không tồn tại, bỏ qua');
    } else {
      throw err;
    }
```

```
  }
}

// Di chuyển/đổi tên file
async function moveFile(src, dest) {
  // Đảm bảo thư mục đích tồn tại
  await ensureDir(path.dirname(dest));
  await fs.rename(src, dest);
}

// Lấy thông tin file
async function getFileInfo(filePath) {
  const stat = await fs.stat(filePath);
  return {
    size: stat.size,
    sizeFormatted: formatBytes(stat.size),
    created: stat.birthtime,
    modified: stat.mtime,
    isFile: stat.isFile(),
    isDirectory: stat.isDirectory()
  };
}

function formatBytes(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`;
  return `${(bytes / 1024 / 1024).toFixed(1)} MB`;
}

// Sử dụng
(async () => {
  await ensureDir('./output/reports');
  await listFiles('./output');
  const info = await getFileInfo('./package.json');
  console.log('package.json:', info);
})();
```

💡 **CODE MẪU: HÀM COPYFILE BẤT ĐỒNG BỘ**

JAVASCRIPT copy-file.js Copy

```
const fs = require('fs/promises');
const path = require('path');

/**
 * Copy file từ src đến dest
 * Tạo thư mục đích tự động nếu chưa có
 */
async function copyFile(src, dest) {
  // Đảm bảo file nguồn tồn tại
  await fs.access(src);

  // Tạo thư mục đích nếu cần
  await fs.mkdir(path.dirname(dest), { recursive: true });

  // Copy file (NodeJS có sẵn hàm này)
  await fs.copyFile(src, dest);

  // Verify kết quả
  const [srcStat, destStat] = await Promise.all([
    fs.stat(src),
    fs.stat(dest)
  ]);

  if (srcStat.size !== destStat.size) {
    throw new Error(`Copy thất bại: kích thước file không khớp`);
  }

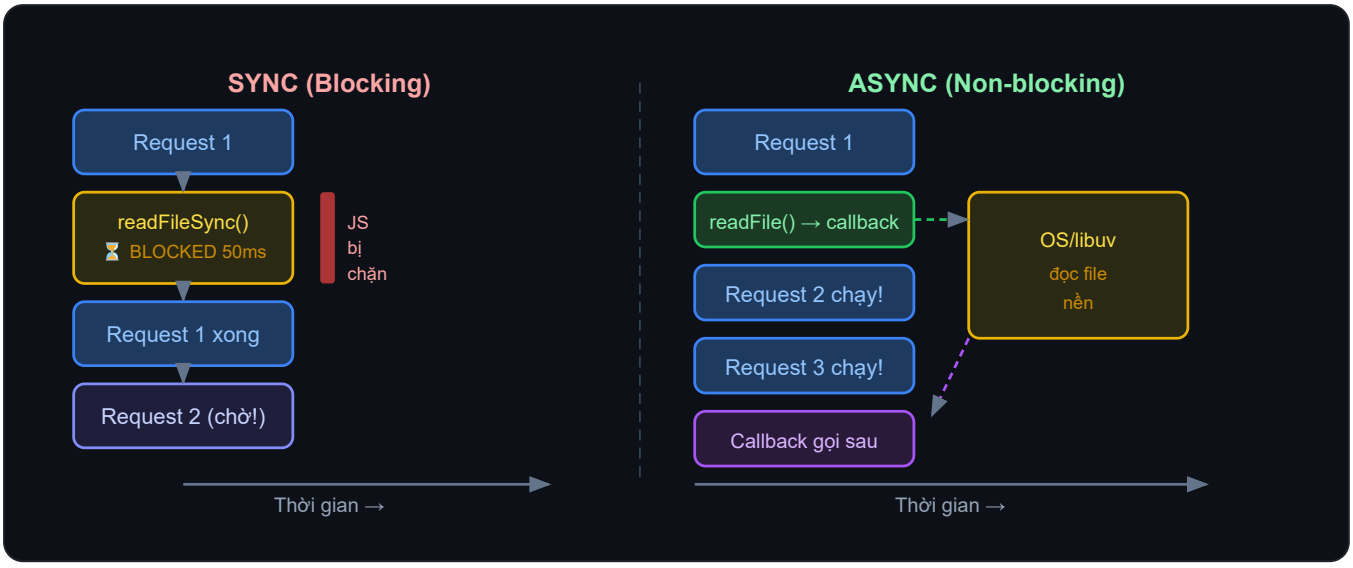
  console.log(`✅ Đã copy ${src} → ${dest}
(${formatBytes(srcStat.size)})`);
}

function formatBytes(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`;
  return `${(bytes / 1024 / 1024).toFixed(1)} MB`;
```

```
}

// Sử dụng
copyFile('./config.json', './backup/config_backup.json')
  .catch(err => console.error('Lỗi:', err.message));
```

Sync vs Async I/O — Tại sao quan trọng?



Với blocking I/O, trong khi NodeJS đọc file, *toàn bộ chương trình đứng yên* — không xử lý được request nào khác. Với non-blocking, NodeJS giao việc đọc file cho OS và tiếp tục xử lý việc khác. Khi OS đọc xong, callback được gọi. Đây là lý do NodeJS có thể xử lý hàng nghìn kết nối đồng thời chỉ với một thread.

4.3 Module `path` — Xử lý đường dẫn

Một trong những lỗi phổ biến nhất khi lập trình NodeJS là viết cứng đường dẫn kiểu `'data/users/file.txt'`. Trên Windows đường dẫn dùng `\`, trên Linux/Mac dùng `/`. Module `path` giải quyết vấn đề này.

JAVASCRIPT path-examples.js Copy

```javascript
const path = require('path');

// path.join() – nối các phần đường dẫn, xử lý / và \ tự động
const filePath = path.join('data', 'users', 'john', 'profile.json');
// Linux/Mac: 'data/users/john/profile.json'
// Windows:  'data\users\john\profile.json'

// path.join xử lý ../ thông minh
const resolved = path.join('/home/user', 'docs', '../images', 'photo.jpg');
// '/home/user/images/photo.jpg'

// path.resolve() – tạo absolute path từ relative path
const absPath = path.resolve('data', 'config.json');
// '/current/working/directory/data/config.json'

// path.resolve với tham số đầu là absolute → dùng ngay
const dbPath = path.resolve('/etc', 'myapp', 'database.conf');
// '/etc/myapp/database.conf'

// Phân tích đường dẫn
const fullPath = '/home/phamt/project/src/server.js';

console.log(path.dirname(fullPath));   // '/home/phamt/project/src'
console.log(path.basename(fullPath));  // 'server.js'
console.log(path.extname(fullPath));   // '.js'
console.log(path.basename(fullPath, '.js')); // 'server' (không có
extension)

// path.parse() → object đầy đủ
const parsed = path.parse(fullPath);
// {
//   root: '/',
//   dir: '/home/phamt/project/src',
//   base: 'server.js',
//   ext: '.js',
```

```
//   name: 'server'
// }

// path.format() → ngược lại với parse
const newPath = path.format({
  dir: '/home/phamt/project/src',
  name: 'server.backup',
  ext: '.js'
});
// '/home/phamt/project/src/server.backup.js'

// __dirname và __filename trong CommonJS
console.log(__dirname);   // thư mục chứa file đang chạy
console.log(__filename);  // full path của file đang chạy

// Dùng __dirname để tạo absolute path an toàn
const configPath = path.join(__dirname, '..', 'config', 'app.json');
// Luôn đúng dù bạn chạy script từ đâu

// path.sep — ký tự phân cách của OS hiện tại
console.log(path.sep);  // '/' trên Unix, '\\' trên Windows

// path.delimiter — ký tự phân cách PATH env
console.log(path.delimiter); // ':' trên Unix, ';' trên Windows
```

💡 **CODE MẪU: TÌM FILE THEO EXTENSION**

JAVASCRIPT find-files.js Copy

```
const fs = require('fs/promises');
const path = require('path');

/**
 * Tìm tất cả file có extension nhất định trong thư mục (đệ quy)
 * @param {string} dir - Thư mục cần tìm
 * @param {string} ext - Extension cần tìm (vd: '.js', '.json')
 * @returns {Promise<string[]>} - Mảng absolute path của các file
 */
async function findFilesByExtension(dir, ext) {
  const results = [];

  async function walk(currentDir) {
    const entries = await fs.readdir(currentDir, { withFileTypes: true
});

    for (const entry of entries) {
      const fullPath = path.join(currentDir, entry.name);

      if (entry.isDirectory()) {
        // Bỏ qua node_modules và .git
        if (entry.name !== 'node_modules' && entry.name !== '.git') {
          await walk(fullPath);
        }
      } else if (path.extname(entry.name) === ext) {
        results.push(fullPath);
      }
    }
  }

  await walk(dir);
  return results;
}

// Sử dụng
```

```
(async () => {
  const jsFiles = await findFilesByExtension('./src', '.js');
  console.log(`Tìm thấy ${jsFiles.length} file JavaScript:`);
  jsFiles.forEach(f => console.log(' -', f));
})();
```

4.4 Module `os` — Thông tin hệ thống

Module `os` cung cấp thông tin về hệ điều hành đang chạy. Hữu ích khi bạn cần build scripts đa nền tảng hoặc monitor tài nguyên hệ thống.

JAVASCRIPT system-info.js Copy

```javascript
const os = require('os');

// Thông tin cơ bản
console.log('Platform:', os.platform());    // 'linux', 'darwin', 'win32'
console.log('Architecture:', os.arch());    // 'x64', 'arm64', 'ia32'
console.log('Hostname:', os.hostname());    // tên máy tính
console.log('OS type:', os.type());         // 'Linux', 'Darwin',
'Windows_NT'
console.log('OS release:', os.release());  // version của OS

// Thư mục
console.log('Home dir:', os.homedir());    // '/home/phamt' hoặc
'C:\Users\phamt'
console.log('Temp dir:', os.tmpdir());     // '/tmp' hoặc
'C:\Users\phamt\AppData\Local\Temp'

// CPU
const cpus = os.cpus();
console.log(`CPU: ${cpus[0].model}`);      // 'Intel(R) Core(TM) i7-...'
console.log(`Số nhân: ${cpus.length}`);   // 8

// RAM
const totalGB = (os.totalmem() / 1024 / 1024 / 1024).toFixed(1);
const freeGB = (os.freemem() / 1024 / 1024 / 1024).toFixed(1);
const usedPercent = ((1 - os.freemem() / os.totalmem()) * 100).toFixed(1);
console.log(`RAM: ${freeGB}GB free / ${totalGB}GB total (${usedPercent}%
used)`);

// Uptime hệ thống
const uptimeSecs = os.uptime();
const uptimeHours = (uptimeSecs / 3600).toFixed(1);
console.log(`Uptime: ${uptimeHours} giờ`);

// Network interfaces
const nets = os.networkInterfaces();
```

```
for (const [name, interfaces] of Object.entries(nets)) {
  for (const iface of interfaces) {
    if (!iface.internal && iface.family === 'IPv4') {
      console.log(`${name}: ${iface.address}`);
    }
  }
}

// Load average (Unix only - CPU load 1/5/15 phút)
if (os.platform() !== 'win32') {
  const [load1, load5, load15] = os.loadavg();
  console.log(`Load avg: ${load1.toFixed(2)} ${load5.toFixed(2)}
${load15.toFixed(2)}`);
}

// EOL (End Of Line) - \n trên Unix, \r\n trên Windows
const content = `Line 1${os.EOL}Line 2${os.EOL}Line 3`;
console.log('EOL char:', JSON.stringify(os.EOL)); // '"\n"' hoặc '"\r\n"'
```

💡 **SYSTEM INFO REPORTER**

JAVASCRIPT system-reporter.js Copy

```
const os = require('os');

function printSystemReport() {
  const separator = '—'.repeat(50);

  console.log(separator);
  console.log('  SYSTEM INFORMATION REPORT');
  console.log(separator);

  // OS Info
  console.log('\n📋 Operating System:');
  console.log(`   Platform  : ${os.platform()} (${os.type()})`);
  console.log(`   Release   : ${os.release()}`);
  console.log(`   Arch      : ${os.arch()}`);
  console.log(`   Hostname  : ${os.hostname()}`);

  // CPU
  const cpus = os.cpus();
  const cpuModel = cpus[0]?.model || 'Unknown';
  console.log('\n🖥️  CPU:');
  console.log(`   Model     : ${cpuModel.trim()}`);
  console.log(`   Cores     : ${cpus.length}`);

  // Memory
  const totalMB = Math.round(os.totalmem() / 1024 / 1024);
  const freeMB = Math.round(os.freemem() / 1024 / 1024);
  const usedMB = totalMB - freeMB;
  const usedPct = Math.round((usedMB / totalMB) * 100);
  const bar = '█'.repeat(Math.round(usedPct / 5)) + '░'.repeat(20 -
Math.round(usedPct / 5));

  console.log('\n💾 Memory:');
  console.log(`   Total     : ${totalMB} MB`);
  console.log(`   Used      : ${usedMB} MB (${usedPct}%)`);
  console.log(`   Free      : ${freeMB} MB`);
```

```
  console.log(`   [${bar}] ${usedPct}%`);

  // Network
  const nets = os.networkInterfaces();
  const ips = [];
  for (const [name, ifaces] of Object.entries(nets)) {
    for (const iface of ifaces) {
      if (!iface.internal && iface.family === 'IPv4') {
        ips.push(`${name}: ${iface.address}`);
      }
    }
  }

  console.log('\n🌐 Network:');
  ips.forEach(ip => console.log(`   ${ip}`));

  // Dirs
  console.log('\n📁 Directories:');
  console.log(`   Home      : ${os.homedir()}`);
  console.log(`   Temp      : ${os.tmpdir()}`);

  console.log('\n' + separator);
}

printSystemReport();
```

4.5 Module `events` — EventEmitter Pattern

Pattern hướng sự kiện (Event-driven) là nền tảng của NodeJS. Thay vì gọi trực tiếp một hàm, bạn *phát ra* (emit) sự kiện, và các *listener* đã đăng ký sẽ nhận và xử lý. Module `events` cung cấp class `EventEmitter` để làm điều này.

Cách dùng cơ bản

JAVASCRIPT event-emitter.js Copy

```
const EventEmitter = require('events');

// Tạo emitter
const emitter = new EventEmitter();

// Đăng ký listener với .on() – gọi MỖI LẦN sự kiện xảy ra
emitter.on('data', (payload) => {
  console.log('Nhận được data:', payload);
});

// Đăng ký listener với .once() – chỉ gọi MỘT LẦN DUY NHẤT
emitter.once('connect', () => {
  console.log('Đã kết nối! (chỉ hiện 1 lần)');
});

// Phát sự kiện với .emit()
emitter.emit('connect');        // In ra 'Đã kết nối!'
emitter.emit('connect');        // Không in gì (đã dùng once)
emitter.emit('data', { id: 1, value: 'hello' }); // In ra data
emitter.emit('data', { id: 2, value: 'world' }); // In ra data lần 2

// Xóa listener
function myHandler(data) {
  console.log('Handler:', data);
}
emitter.on('message', myHandler);
emitter.emit('message', 'test');  // Gọi handler
emitter.off('message', myHandler); // Xóa listener (alias của
removeListener)
emitter.emit('message', 'test2'); // Không gọi handler nữa

// Xóa tất cả listener của một event
emitter.removeAllListeners('data');

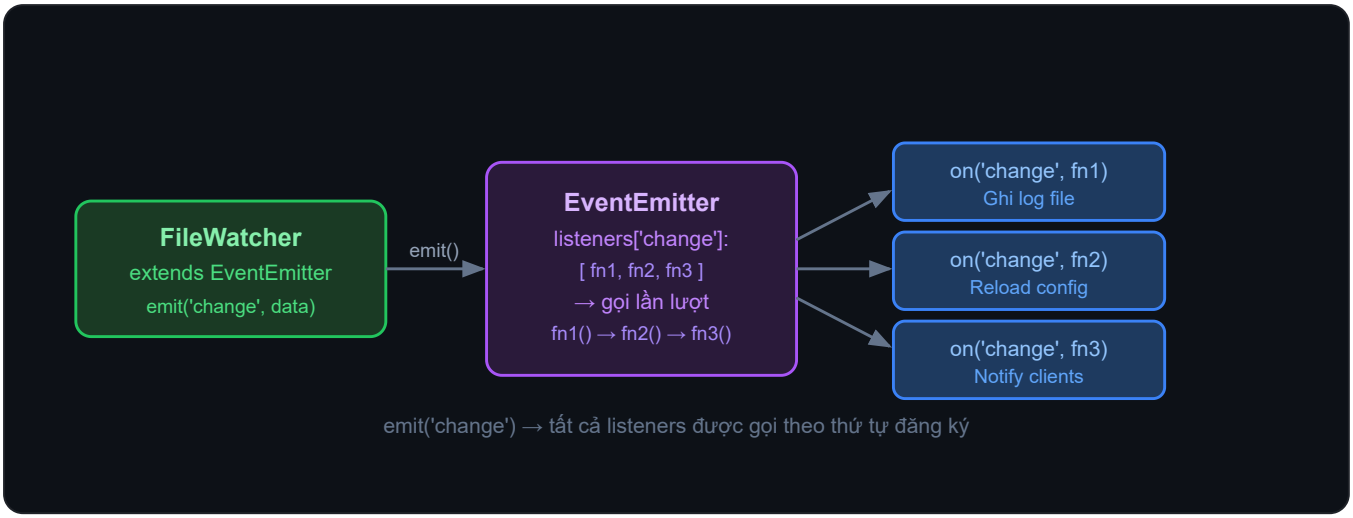
// Đếm số listener
```

```
console.log('Số listener của data:', emitter.listenerCount('data')); // 0

// Xem danh sách tên events đang có listener
console.log('Events:', emitter.eventNames());

// Tránh memory leak: mặc định cảnh báo nếu > 10 listener/event
emitter.setMaxListeners(20); // Tăng giới hạn nếu cần
```

Kế thừa EventEmitter — Pattern quan trọng



JAVASCRIPT file-watcher.js Copy

```
const EventEmitter = require('events');
const fs = require('fs');
const path = require('path');

/**
 * FileWatcher: theo dõi thay đổi của file, phát sự kiện
 * Kế thừa EventEmitter để có đầy đủ on/emit/off
 */
class FileWatcher extends EventEmitter {
  constructor(watchDir, intervalMs = 2000) {
    super();
    this.watchDir = path.resolve(watchDir);
    this.intervalMs = intervalMs;
    this.fileStats = new Map(); // filename → { mtime, size }
    this.timer = null;
    this.isRunning = false;
  }

  // Bắt đầu theo dõi
  start() {
    if (this.isRunning) return;
    this.isRunning = true;

    // Lấy trạng thái ban đầu
    this._scanDirectory();

    // Quét định kỳ
    this.timer = setInterval(() => {
      this._scanDirectory();
    }, this.intervalMs);

    this.emit('watching', { dir: this.watchDir });
    console.log(`Đang theo dõi: ${this.watchDir}`);
  }
```

```
  // Dừng theo dõi
  stop() {
    if (!this.isRunning) return;
    clearInterval(this.timer);
    this.isRunning = false;
    this.emit('stopped');
    console.log('Đã dừng theo dõi');
  }

  _scanDirectory() {
    try {
      const files = fs.readdirSync(this.watchDir);

      // Kiểm tra file mới và file thay đổi
      for (const filename of files) {
        const filePath = path.join(this.watchDir, filename);
        try {
          const stat = fs.statSync(filePath);
          const prev = this.fileStats.get(filename);

          if (!prev) {
            // File mới
            this.fileStats.set(filename, {
              mtime: stat.mtime.getTime(),
              size: stat.size
            });
            if (Object.keys(this.fileStats).length > 1) {
              // Không emit cho lần scan đầu tiên
              this.emit('add', { filename, path: filePath, size: stat.size
});
            }
          } else if (stat.mtime.getTime() !== prev.mtime || stat.size !==
prev.size) {
            // File thay đổi
            this.fileStats.set(filename, {
              mtime: stat.mtime.getTime(),
              size: stat.size
            });
            this.emit('change', {
              filename,
              path: filePath,
```

```
              oldSize: prev.size,
              newSize: stat.size,
              mtime: stat.mtime
            });
          }
        } catch {
          // File không thể đọc, bỏ qua
        }
      }

      // Kiểm tra file bị xóa
      for (const [filename] of this.fileStats) {
        if (!files.includes(filename)) {
          this.fileStats.delete(filename);
          this.emit('remove', {
            filename,
            path: path.join(this.watchDir, filename)
          });
        }
      }
    } catch (err) {
      this.emit('error', err);
    }
  }
}

// ═══ Sử dụng ═══
const watcher = new FileWatcher('./watched-folder', 1000);

watcher.on('watching', ({ dir }) => {
  console.log(`✅ Bắt đầu theo dõi: ${dir}`);
});

watcher.on('add', ({ filename, size }) => {
  console.log(`📄 File mới: ${filename} (${size} bytes)`);
});

watcher.on('change', ({ filename, oldSize, newSize }) => {
  const diff = newSize - oldSize;
  const sign = diff >= 0 ? '+' : '';
  console.log(`📝 File thay đổi: ${filename} (${sign}${diff} bytes)`);
```

```
});

watcher.on('remove', ({ filename }) => {
  console.log(`🗑️  File xóa: ${filename}`);
});

watcher.on('error', (err) => {
  console.error(`❌ Lỗi:`, err.message);
});

// Bắt đầu theo dõi
watcher.start();

// Dừng sau 30 giây
setTimeout(() => watcher.stop(), 30000);
```

4.6 Module `crypto` — Mã hóa cơ bản

Module `crypto` cung cấp các hàm mã hóa mật mã học (cryptographic). Dùng để tạo hash, tạo token ngẫu nhiên an toàn, UUID, và nhiều thứ khác.

> ⚠️ **LƯU Ý VỀ HASH PASSWORD**
>
> **Không bao giờ** dùng SHA-256 hay MD5 để hash password người dùng. Những hàm này quá nhanh, dễ bị tấn công brute-force. Dùng **bcrypt** hoặc **argon2** cho password — chúng được thiết kế đặc biệt chậm để chống brute-force. Crypto trong phần này chủ yếu dùng cho token, checksum, và ID.

JAVASCRIPT crypto-examples.js Copy

```javascript
const crypto = require('crypto');

// ═══ HASH DATA ═══
// Tạo SHA-256 hash của một chuỗi
function hashData(data) {
  return crypto.createHash('sha256')
    .update(data)
    .digest('hex');  // 'hex' | 'base64' | 'binary'
}

console.log(hashData('hello'));
// '2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824'

// MD5 (dùng cho checksum file, KHÔNG dùng cho password)
function md5(data) {
  return crypto.createHash('md5').update(data).digest('hex');
}

// Hash file để verify integrity
const fs = require('fs');
function hashFile(filePath) {
  return new Promise((resolve, reject) => {
    const hash = crypto.createHash('sha256');
    const stream = fs.createReadStream(filePath);
    stream.on('data', chunk => hash.update(chunk));
    stream.on('end', () => resolve(hash.digest('hex')));
    stream.on('error', reject);
  });
}

// ═══ RANDOM BYTES ═══
// Tạo bytes ngẫu nhiên an toàn về mật mã học (cryptographically secure)
const randomBuf = crypto.randomBytes(32); // 32 bytes = 256 bits
console.log(randomBuf.toString('hex'));     // 64 ký tự hex
console.log(randomBuf.toString('base64')); // base64 encoding
```

```
// ═══ GENERATE SECURE TOKEN ═══
function generateSecureToken(byteLength = 32) {
  return crypto.randomBytes(byteLength).toString('hex');
}
// Dùng cho: session token, email verification token, API key, reset
password token

const sessionToken = generateSecureToken();
console.log('Session token:', sessionToken);
// 'a3f8b2c1...' (64 ký tự, cực kỳ khó đoán)

const apiKey = generateSecureToken(24);
console.log('API key:', apiKey);

// ═══ UUID v4 ═══
// Từ NodeJS v14.17+
const uuid = crypto.randomUUID();
console.log('UUID:', uuid);
// '550e8400-e29b-41d4-a716-446655440000'

// ═══ HMAC — Hash-based Message Authentication Code ═══
// Verify rằng message không bị giả mạo
function createHmac(data, secret) {
  return crypto.createHmac('sha256', secret)
    .update(data)
    .digest('hex');
}

function verifyHmac(data, signature, secret) {
  const expected = createHmac(data, secret);
  // timingSafeEqual để tránh timing attack
  const expectedBuf = Buffer.from(expected, 'hex');
  const signatureBuf = Buffer.from(signature, 'hex');
  if (expectedBuf.length ≠ signatureBuf.length) return false;
  return crypto.timingSafeEqual(expectedBuf, signatureBuf);
}

const SECRET = process.env.HMAC_SECRET || 'my-secret-key';
const message = JSON.stringify({ userId: 123, action: 'login' });
const sig = createHmac(message, SECRET);
```

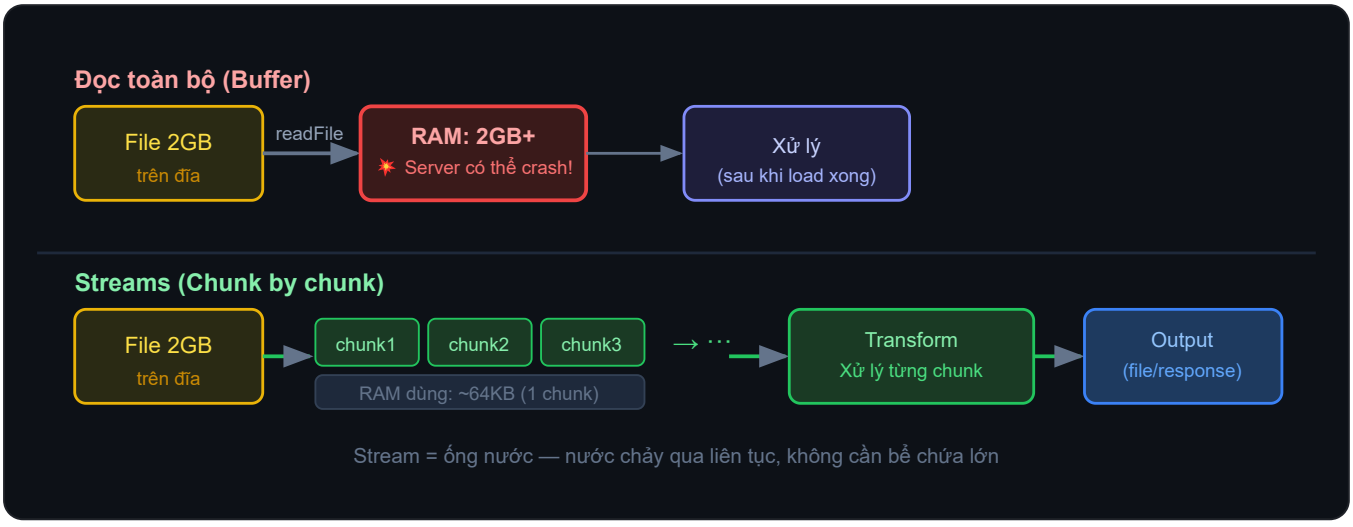
```
console.log('Signature:', sig);
console.log('Valid:', verifyHmac(message, sig, SECRET)); // true

// ═══ COMPLETE TOKEN EXAMPLE ═══
function generateApiCredentials() {
  return {
    apiKey: `ak_${generateSecureToken(16)}`,
    apiSecret: generateSecureToken(32),
    createdAt: new Date().toISOString(),
    id: crypto.randomUUID()
  };
}

console.log('API Credentials:', generateApiCredentials());
```

4.7 Streams — Xử lý dữ liệu lớn

Hãy tưởng tượng bạn cần đọc file log 2GB. Nếu dùng `fs.readFile()`, NodeJS sẽ cố nạp 2GB vào RAM — server sẽ crash hoặc cực kỳ chậm. Streams giải quyết vấn đề này bằng cách xử lý dữ liệu theo từng *chunk* nhỏ.



JAVASCRIPT streams.js Copy

```javascript
const fs = require('fs');
const zlib = require('zlib');
const path = require('path');

// ═══ COPY FILE LỚN BẰNG STREAM ═══
function copyLargeFile(src, dest) {
  return new Promise((resolve, reject) => {
    const readStream = fs.createReadStream(src);
    const writeStream = fs.createWriteStream(dest);

    // pipe() kết nối readable → writable, xử lý backpressure tự động
    readStream.pipe(writeStream);

    writeStream.on('finish', () => {
      console.log(`✅ Copy xong: ${src} → ${dest}`);
      resolve();
    });

    readStream.on('error', reject);
    writeStream.on('error', reject);
  });
}

// ═══ NÉN FILE BẰNG GZIP ═══
async function compressFile(inputPath) {
  const outputPath = inputPath + '.gz';

  return new Promise((resolve, reject) => {
    const readStream = fs.createReadStream(inputPath);
    const gzipStream = zlib.createGzip();
    const writeStream = fs.createWriteStream(outputPath);

    // Pipe chain: read → gzip → write
    readStream
      .pipe(gzipStream)
```

```
      .pipe(writeStream);

    writeStream.on('finish', () => {
      // So sánh kích thước
      const origSize = fs.statSync(inputPath).size;
      const gzipSize = fs.statSync(outputPath).size;
      const ratio = ((1 - gzipSize / origSize) * 100).toFixed(1);
      console.log(`✅ Nén xong: ${outputPath}`);
      console.log(`   ${formatBytes(origSize)} → ${formatBytes(gzipSize)}
(giảm ${ratio}%)`);
      resolve(outputPath);
    });

    readStream.on('error', reject);
    writeStream.on('error', reject);
  });
}

// ═══ ĐỌC FILE TỪNG DÒNG (Line by line) ═══
const readline = require('readline');

async function processLargeLog(logPath) {
  const fileStream = fs.createReadStream(logPath);
  const rl = readline.createInterface({
    input: fileStream,
    crlfDelay: Infinity  // xử lý cả \r\n (Windows)
  });

  let lineCount = 0;
  let errorCount = 0;

  for await (const line of rl) {
    lineCount++;
    if (line.includes('ERROR')) errorCount++;
  }

  console.log(`Tổng dòng: ${lineCount}`);
  console.log(`Số lỗi: ${errorCount}`);
}

// ═══ TRANSFORM STREAM ═══
```

```
const { Transform } = require('stream');

// Tạo Transform stream tùy chỉnh (uppercase)
const upperCaseTransform = new Transform({
  transform(chunk, encoding, callback) {
    this.push(chunk.toString().toUpperCase());
    callback();
  }
});

// Đọc → uppercase → ghi
fs.createReadStream('input.txt')
  .pipe(upperCaseTransform)
  .pipe(fs.createWriteStream('output.txt'));

function formatBytes(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`;
  return `${(bytes / 1024 / 1024).toFixed(1)} MB`;
}

// Sử dụng
(async () => {
  await copyLargeFile('large-data.csv', 'backup/large-data.csv');
  await compressFile('large-data.csv');
})();
```

📌 KHI NÀO DÙNG STREAMS?

- File lớn hơn vài chục MB
- Xử lý video/audio streaming
- Đọc dữ liệu từ database theo batch
- HTTP responses lớn (cần stream về client)
- Real-time data processing (log aggregation, ETL)

📚 Tóm tắt chương 4

- **Module System:** CommonJS dùng `require/module.exports`, ES Modules dùng `import/export`. NodeJS cache module sau lần load đầu tiên.
- **fs module:** Luôn ưu tiên dùng `fs/promises` (async) thay vì sync. Dùng `readdir({withFileTypes:true})` để phân biệt file và thư mục. Error code `ENOENT` = file không tồn tại.
- **path module:** Dùng `path.join()` thay vì nối string thủ công. Dùng `__dirname` kết hợp `path.join()` để tạo absolute path ổn định.
- **os module:** Truy cập thông tin platform, CPU, RAM, network interfaces. Dùng `os.platform()` để viết code đa nền tảng.
- **EventEmitter:** Kế thừa EventEmitter để thêm khả năng event-driven vào class. Dùng `once()` cho sự kiện chỉ xảy ra một lần. Nhớ gọi `removeAllListeners()` khi cleanup.
- **crypto module:** `randomBytes()` cho token an toàn, `randomUUID()` cho ID duy nhất, `createHash()` cho checksum. **Không dùng** crypto để hash password — dùng bcrypt.
- **Streams:** Xử lý file lớn mà không chiếm nhiều RAM. `pipe()` kết nối streams. `readline` để đọc file text từng dòng.

🏋️ Bài tập thực hành

DỄ

4.1 Đọc file CSV và đếm dòng

Viết script `count-lines.js` nhận đường dẫn file CSV từ command line (`process.argv[2]`), đọc file đó và in ra: tổng số dòng, số dòng có nội dung (không tính dòng trắng), và tên các cột (dòng header đầu tiên). Ví dụ: `node count-lines.js data.csv`

Gợi ý: Dùng `readline` để đọc từng dòng, split bằng `,` để lấy header.

TRUNG BÌNH

4.2 Xây dựng EventBus Singleton

Tạo file `event-bus.js` export một EventBus singleton có thể dùng ở mọi nơi trong app để pub/sub events. EventBus cần có:

- `subscribe(event, handler)` — đăng ký lắng nghe
- `unsubscribe(event, handler)` — hủy đăng ký

- `publish(event, data)` — phát event
- `publishOnce(event, data)` — phát event, tự xóa sau khi xử lý
- Logging: in ra console khi có event được publish (có thể tắt bằng `setDebug(false)`)

Viết thêm file `test-bus.js` để test EventBus từ nhiều module khác nhau.

KHÓ

4.3 CLI File Organizer

Xây dựng CLI tool `organizer.js` tự động sắp xếp file trong một thư mục theo extension. Khi chạy `node organizer.js ./downloads`, tool sẽ:

- Scan thư mục được chỉ định (không đệ quy)
- Nhóm file theo extension: `.jpg/.png/.gif` → `images/`, `.pdf/.doc/.docx` → `documents/`, `.mp3/.wav/.flac` → `audio/`, `.mp4/.mkv/.avi` → `video/`, `.zip/.rar/.tar.gz` → `archives/`, còn lại → `others/`
- In ra preview những gì sẽ move (dry-run mode với flag `--dry-run`)
- Hỏi xác nhận trước khi thực hiện
- Move file vào subfolder tương ứng
- In báo cáo: đã move X file, Y file bỏ qua, tổng dung lượng

Thách thức thêm: Xử lý trường hợp file trùng tên tại đích đến (thêm số đuôi: `photo.jpg` → `photo_1.jpg`).

PHẦN 2 — NODEJS CƠ BẢN

Chương 5: Async Programming — Callback, Promise, async/await

Lập trình bất đồng bộ là linh hồn của NodeJS. Chương này giải thích tại sao cần async, các pattern từ cổ điển (callback) đến hiện đại (async/await), cách xử lý lỗi đúng cách, và cách Event Loop thực sự hoạt động bên trong.

> 🎯 **SAU CHƯƠNG NÀY BẠN SẼ**
>
> - Hiểu tại sao lập trình bất đồng bộ là cốt lõi của NodeJS
> - Viết và đọc hiểu code dùng callback, Promise, và async/await
> - Xử lý lỗi đúng cách trong cả ba pattern
> - Chạy nhiều Promise song song với `Promise.all`, `Promise.allSettled`
> - Tránh những lỗi phổ biến: callback hell, unhandled rejection, await trong vòng lặp
> - Hiểu Event Loop: call stack, microtask queue, task queue và thứ tự thực thi

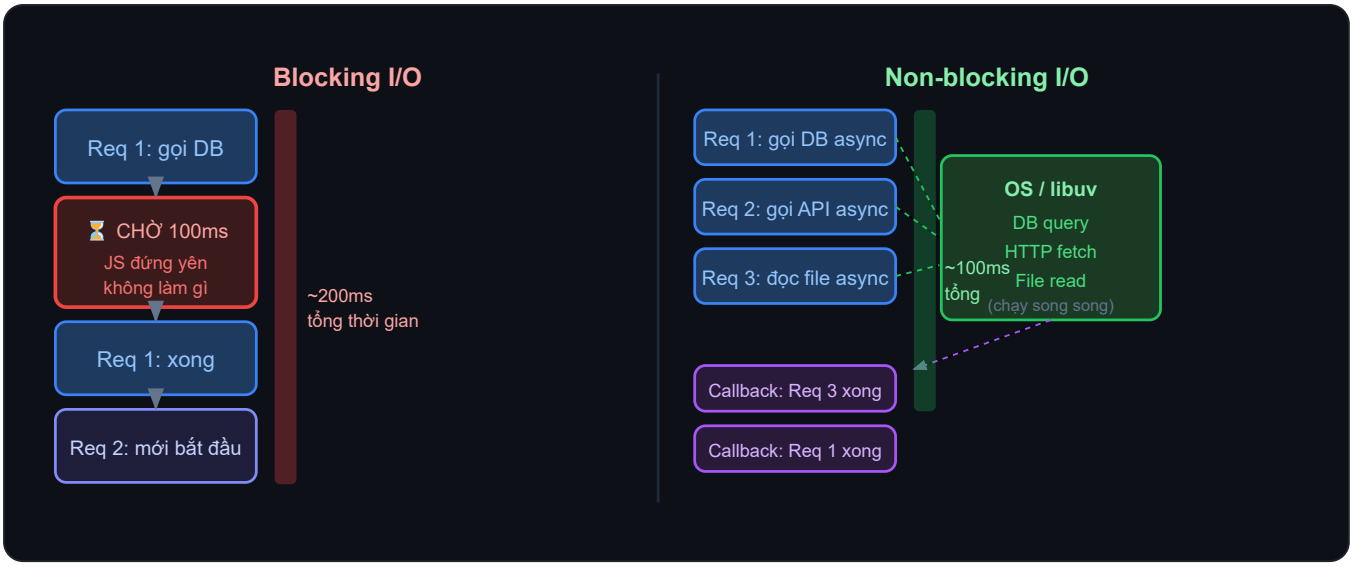
5.1 Vấn đề: Tại sao cần Async?

JavaScript là **single-threaded** — chỉ có một luồng xử lý, làm một việc tại một thời điểm. Nếu một thao tác mất 200ms (đọc file, gọi API, truy vấn database), toàn bộ chương trình phải đứng chờ — không xử lý được gì khác.

Phép so sánh: Nhà hàng vs Cửa hàng tạp hóa

Hãy tưởng tượng bạn là nhân viên phục vụ tại một nhà hàng có 10 bàn:

- **Blocking (cửa hàng tạp hóa):** Bạn nhận order của bàn 1, đứng ở bếp chờ 20 phút cho đến khi món ăn xong, mang ra rồi mới quay lại nhận order của bàn 2. Khách ở bàn 2–10 phải chờ cả tiếng!
- **Non-blocking (nhà hàng thực sự):** Bạn nhận order của tất cả 10 bàn, giao mỗi order cho bếp (OS/libuv). Trong khi bếp nấu, bạn tiếp tục phục vụ khách khác. Khi bếp gọi "Bàn 3 xong!", bạn mang ra ngay. Tất cả được phục vụ hiệu quả.



Trong khi chờ database trả lời (thường 10–100ms), CPU của bạn hoàn toàn rảnh rỗi. Non-blocking I/O tận dụng thời gian chờ đó để xử lý các request khác. Đây là lý do NodeJS có thể xử lý **hàng nghìn request đồng thời** chỉ với một thread.

5.2 Callback Pattern

Callback là cách đầu tiên JavaScript xử lý async. Ý tưởng: *"Khi xong việc này, gọi cho tôi biết bằng cách gọi hàm callback này."*

Error-first Callback Convention

Quy ước trong NodeJS: tham số đầu tiên của callback **luôn là** error (nếu không có lỗi thì là `null`). Tham số thứ hai trở đi là kết quả.

JAVASCRIPT callbacks.js Copy

```
const fs = require('fs');

// Error-first callback: (error, result) => {}
fs.readFile('./config.json', 'utf8', (err, data) => {
  // Luôn kiểm tra lỗi TRƯỚC
  if (err) {
    console.error('Không đọc được file:', err.message);
    return; // QUAN TRỌNG: return để không chạy tiếp
  }

  // Chỉ dùng data khi không có lỗi
  const config = JSON.parse(data);
  console.log('Port:', config.port);
});

// Tự viết hàm dùng error-first callback
function fetchUserFromDB(userId, callback) {
  // Giả lập query database (thực tế là async)
  setTimeout(() => {
    if (userId <= 0) {
      callback(new Error('userId phải là số dương'));
      return;
    }
    // Thành công
    callback(null, {
      id: userId,
      name: 'Nguyễn Văn A',
      email: 'a@example.com'
    });
  }, 100);  // giả lập 100ms query time
}

// Sử dụng
fetchUserFromDB(1, (err, user) => {
  if (err) {
```

```
    console.error('Lỗi:', err.message);
    return;
  }
  console.log('User:', user);
});
```

Callback Hell — Cái bẫy của người mới

Vấn đề xảy ra khi bạn cần làm nhiều thao tác async nối tiếp nhau. Code lồng nhau ngày càng sâu, trông như một cái kim tự tháp:

JAVASCRIPT callback-hell.js Copy

```javascript
// ❌ Callback Hell – Pyramid of Doom
// Kịch bản: đọc config → kết nối DB → lấy user → lấy orders → tính tổng

fs.readFile('./config.json', 'utf8', (err, configData) => {
  if (err) { handleError(err); return; }

  const config = JSON.parse(configData);

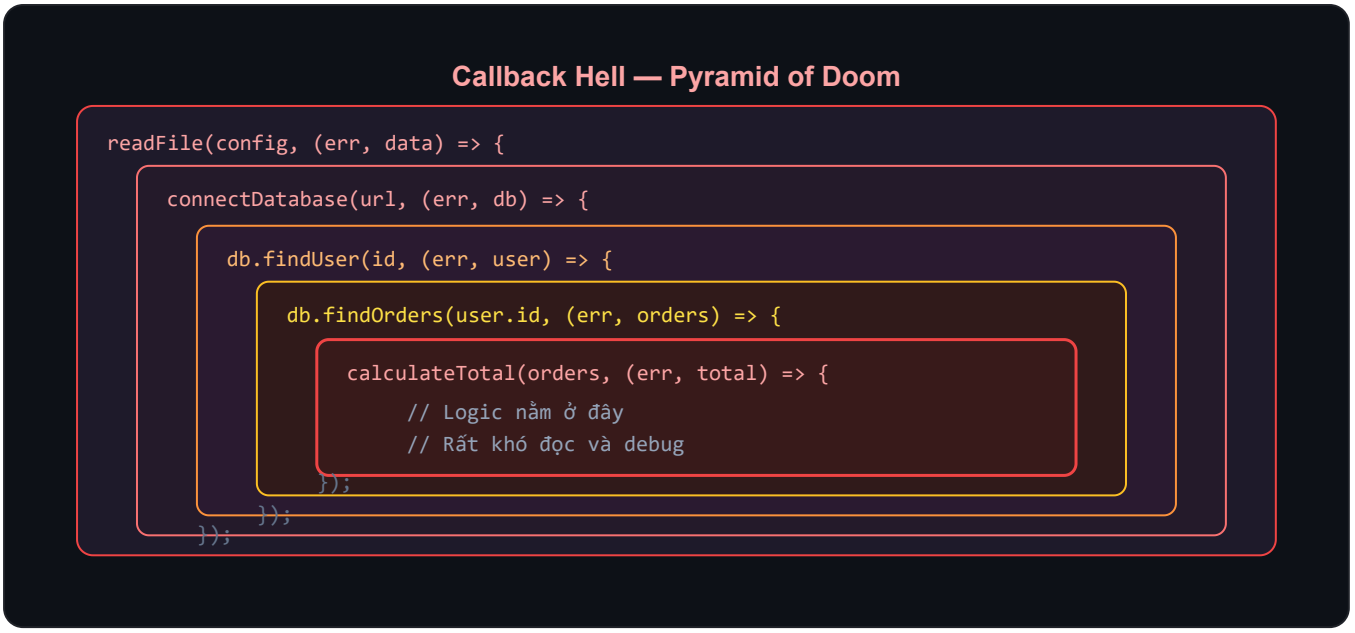
  connectDatabase(config.dbUrl, (err, db) => {
    if (err) { handleError(err); return; }

    db.findUser(userId, (err, user) => {
      if (err) { handleError(err); return; }

      db.findOrders(user.id, (err, orders) => {
        if (err) { handleError(err); return; }

        calculateTotal(orders, (err, total) => {
          if (err) { handleError(err); return; }

          // Cuối cùng mới đến kết quả
          // Nhưng đến đây code đã thụt vào quá sâu!
          console.log(`User ${user.name} có tổng đơn hàng: ${total}`);
        });
      });
    });
  });
});
```



Vấn đề của callback hell không chỉ là code xấu. Khó xử lý lỗi đúng chỗ, khó tái sử dụng logic, không thể dùng `try/catch`, và rất khó test. Promise ra đời để giải quyết.

5.3 Promise — Giải pháp cho Callback Hell

Promise là object đại diện cho một giá trị *có thể có trong tương lai* (hoặc thất bại). Thay vì truyền callback vào, bạn nhận về một Promise object và xử lý sau.

3 trạng thái của Promise



JAVASCRIPT promises.js Copy

```
// ═══ TẠO PROMISE TỪ ĐẦU ═══
function fetchUserFromDB(userId) {
  return new Promise((resolve, reject) => {
    // Thực hiện công việc async
    setTimeout(() => {
      if (userId <= 0) {
        reject(new Error('userId phải là số dương'));
        return;
      }
      resolve({
        id: userId,
        name: 'Nguyễn Văn A',
        email: 'a@example.com'
      });
    }, 100);
  });
}

// ═══ DÙNG .then() .catch() .finally() ═══
fetchUserFromDB(1)
  .then(user => {
    console.log('User:', user.name);
    return user.id; // Giá trị trả về từ .then() → .then() tiếp theo nhận được
  })
  .then(userId => {
    console.log('User ID:', userId);
  })
  .catch(err => {
    // Bắt lỗi từ BẤT KỲ .then() nào phía trên
    console.error('Lỗi:', err.message);
  })
  .finally(() => {
    // Luôn chạy, dù thành công hay thất bại
    console.log('Done!');
```

```
  });

// ═══ PROMISE CHAINING ═══
// Rewrite callback hell thành promise chain — phẳng và dễ đọc hơn nhiều
function processUserOrder(userId) {
  return readConfig('./config.json')
    .then(config => connectDatabase(config.dbUrl))
    .then(db => db.findUser(userId))
    .then(user => db.findOrders(user.id)) // db không accessible! xem phiên bản fix dưới
    .then(orders => calculateTotal(orders))
    .then(total => {
      console.log(`Tổng đơn hàng: ${total}`);
      return total;
    })
    .catch(err => {
      console.error('Lỗi trong pipeline:', err.message);
      throw err; // re-throw để caller biết có lỗi
    });
}

// Fix: dùng closure để giữ reference
function processUserOrderFixed(userId) {
  let _db; // giữ db reference

  return readConfig('./config.json')
    .then(config => connectDatabase(config.dbUrl))
    .then(db => {
      _db = db;
      return _db.findUser(userId);
    })
    .then(user => _db.findOrders(user.id))
    .then(orders => calculateTotal(orders))
    .catch(err => {
      console.error('Lỗi:', err.message);
      throw err;
    });
}

// ═══ PROMISIFY CALLBACK FUNCTIONS ═══
const util = require('util');
```

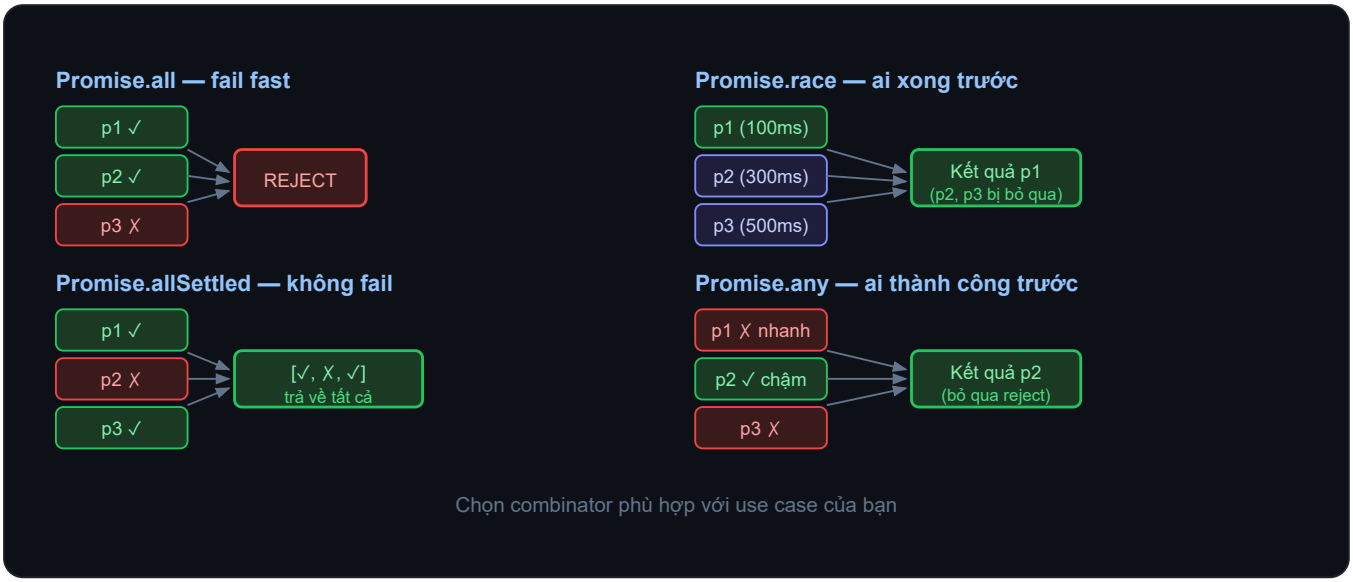
```
const fs = require('fs');

// Chuyển callback-style thành Promise
const readFile = util.promisify(fs.readFile);
const writeFile = util.promisify(fs.writeFile);

// Giờ dùng được .then()
readFile('./data.json', 'utf8')
  .then(data ⇒ JSON.parse(data))
  .then(obj ⇒ console.log('Data:', obj))
  .catch(err ⇒ console.error('Lỗi:', err));
```

5.4 Promise Combinators

Khi cần xử lý nhiều Promise cùng lúc, JavaScript cung cấp các hàm "combinator". Mỗi hàm có hành vi khác nhau khi gặp Promise bị reject.



Chọn combinator phù hợp với use case của bạn

JAVASCRIPT promise-combinators.js Copy

```
// Giả lập fetch API
function fetchUser(id) {
  return new Promise(resolve =>
    setTimeout(() => resolve({ id, name: `User ${id}` }), Math.random() *
200)
  );
}

function fetchPosts(userId) {
  return new Promise(resolve =>
    setTimeout(() => resolve([`Post của user ${userId}`]), Math.random() *
300)
  );
}

// ═══ Promise.all – chờ tất cả, fail nhanh nếu 1 cái fail ═══
async function loadDashboard(userId) {
  console.time('dashboard');

  // Song song: fetch user VÀ posts cùng lúc (không await lần lượt!)
  const [user, posts, settings] = await Promise.all([
    fetchUser(userId),
    fetchPosts(userId),
    fetchSettings(userId)
  ]);

  console.timeEnd('dashboard'); // Nhanh hơn await lần lượt rất nhiều

  return { user, posts, settings };
}

// ═══ Promise.allSettled – chờ tất cả, không quan tâm có fail không ═══
async function fetchMultipleSources(urls) {
  const results = await Promise.allSettled(
    urls.map(url => fetch(url).then(r => r.json()))
```

```
  );

  const successful = [];
  const failed = [];

  for (const result of results) {
    if (result.status === 'fulfilled') {
      successful.push(result.value);
    } else {
      failed.push(result.reason.message);
    }
  }

  console.log(`Thành công: ${successful.length}, Thất bại:
${failed.length}`);
  return successful;
}

// === Promise.race - timeout pattern ===
function withTimeout(promise, ms) {
  const timeout = new Promise((_, reject) =>
    setTimeout(() => reject(new Error(`Timeout sau ${ms}ms`)), ms)
  );
  return Promise.race([promise, timeout]);
}

// Fetch với timeout 5 giây
withTimeout(fetchUser(1), 5000)
  .then(user => console.log('User:', user))
  .catch(err => console.error('Lỗi:', err.message));

// === Promise.any - dùng server nhanh nhất trong nhiều mirrors ===
async function fetchFromFastestMirror(path) {
  const mirrors = [
    `https://mirror1.example.com${path}`,
    `https://mirror2.example.com${path}`,
    `https://mirror3.example.com${path}`
  ];

  try {
    const data = await Promise.any(
```

```
      mirrors.map(url => fetch(url).then(r => {
        if (!r.ok) throw new Error(`HTTP ${r.status}`);
        return r.json();
      }))
    );
    return data;
  } catch (err) {
    // AggregateError: tất cả đều fail
    throw new Error('Tất cả mirrors đều không trả lời');
  }
}
```

5.5 async/await — Syntax đẹp nhất

`async/await` là "syntactic sugar" trên Promise — bên dưới vẫn là Promise, nhưng code trông như synchronous, dễ đọc và viết hơn nhiều.

JAVASCRIPT async-await.js Copy

```javascript
const fs = require('fs/promises');

// async function luôn trả về Promise
async function readAndParseJSON(filePath) {
  // await tạm dừng function, chờ Promise resolve
  const content = await fs.readFile(filePath, 'utf8');
  const data = JSON.parse(content);
  return data; // Được wrap trong Promise.resolve(data) tự động
}

// Sử dụng: vẫn là Promise từ bên ngoài
readAndParseJSON('./config.json')
  .then(config => console.log(config))
  .catch(err => console.error(err));

// Hoặc await trong async context khác
async function main() {
  const config = await readAndParseJSON('./config.json');
  console.log('Port:', config.port);
}

// ═══ SO SÁNH Promise chain vs async/await ═══

// Promise chain
function getUserDataPromise(userId) {
  return connectDB()
    .then(db => db.findUser(userId))
    .then(user => {
      return db.findOrders(user.id) // db không accessible!
        .then(orders => ({ user, orders }));
    })
    .then(({ user, orders }) => ({
      ...user,
      orderCount: orders.length
    }));
```

```
}

// async/await - sạch hơn nhiều, dễ đọc như sync code
async function getUserData(userId) {
  const db = await connectDB();
  const user = await db.findUser(userId);
  const orders = await db.findOrders(user.id);  // db accessible!

  return {
    ...user,
    orderCount: orders.length
  };
}

// ═══ SEQUENTIAL vs PARALLEL ═══

// ❌ SAI: await trong vòng lặp = sequential (chậm)
async function fetchUsersSequential(ids) {
  const users = [];
  for (const id of ids) {
    const user = await fetchUser(id); // Chờ từng cái một!
    users.push(user);
  }
  return users;
  // Nếu mỗi fetch mất 100ms và có 10 id → tổng 1000ms
}

// ✅ ĐÚNG: song song bằng Promise.all
async function fetchUsersParallel(ids) {
  const users = await Promise.all(ids.map(id => fetchUser(id)));
  return users;
  // Tất cả fetch chạy song song → chỉ tốn ~100ms
}

// ✅ ĐÚNG khi cần sequential nhưng muốn code đẹp
async function processSequentially(items) {
  const results = [];
  for (const item of items) {
    // reduce cũng dùng được nhưng phức tạp hơn
    const result = await processItem(item);
    results.push(result);
```

```
  }
  return results;
}

// ═══ TOP-LEVEL AWAIT (ES2022) ═══
// Trong file .mjs hoặc package.json có "type": "module"
// Không cần wrap trong async function
// const config = await readAndParseJSON('./config.json');
// console.log(config);  // Chạy ở top level!
```

5.6 Error Handling Đúng Cách

Async code có nhiều pitfall về error handling. Bỏ sót một `catch` có thể làm server crash hoặc tệ hơn — chạy tiếp với dữ liệu sai.

JAVASCRIPT error-handling.js Copy

```javascript
// ═══ try/catch với async/await ═══
async function processOrder(orderId) {
  try {
    const order = await db.findOrder(orderId);

    if (!order) {
      throw new Error(`Order ${orderId} không tồn tại`);
    }

    const payment = await paymentService.charge(order.total);
    await db.updateOrder(orderId, { status: 'paid', paymentId: payment.id
});

    return { success: true, order };

  } catch (err) {
    // Xử lý từng loại lỗi khác nhau
    if (err.code === 'ORDER_NOT_FOUND') {
      return { success: false, error: 'Order không tồn tại' };
    }
    if (err.code === 'PAYMENT_DECLINED') {
      await db.updateOrder(orderId, { status: 'payment_failed' });
      return { success: false, error: 'Thanh toán bị từ chối' };
    }
    // Lỗi không biết: log và re-throw
    console.error(`Lỗi không xử lý được trong processOrder(${orderId}):`,
err);
    throw err;

  } finally {
    // Luôn chạy: cleanup resources
    await db.releaseConnection();
  }
}
```

```
// ═══ Custom Error Classes ═══
class AppError extends Error {
  constructor(message, code, statusCode = 500) {
    super(message);
    this.name = 'AppError';
    this.code = code;
    this.statusCode = statusCode;
    Error.captureStackTrace(this, this.constructor);
  }
}

class NotFoundError extends AppError {
  constructor(resource, id) {
    super(`${resource} với id '${id}' không tồn tại`, 'NOT_FOUND', 404);
    this.name = 'NotFoundError';
  }
}

class ValidationError extends AppError {
  constructor(field, message) {
    super(`Validation thất bại: ${field} — ${message}`, 'VALIDATION_ERROR',
400);
    this.name = 'ValidationError';
    this.field = field;
  }
}

// Sử dụng
async function getUser(id) {
  if (!id || typeof id !== 'number') {
    throw new ValidationError('id', 'phải là số nguyên dương');
  }

  const user = await db.findById(id);
  if (!user) {
    throw new NotFoundError('User', id);
  }

  return user;
}
```

```
// ═══ UNHANDLED PROMISE REJECTION ═══
// Cực kỳ nguy hiểm — không có catch thì NodeJS bỏ qua lỗi

// ❌ BAD: không có catch
fetchData().then(data ⇒ processData(data));
// Nếu processData() throw → lỗi bị nuốt mất

// ✅ GOOD: luôn có catch
fetchData()
  .then(data ⇒ processData(data))
  .catch(err ⇒ console.error('Lỗi:', err));

// Bắt unhandled rejection toàn app (safety net)
process.on('unhandledRejection', (reason, promise) ⇒ {
  console.error('Unhandled Promise Rejection:', reason);
  // Trong production: log và graceful shutdown
  process.exit(1);
});

// ═══ RETRY VỚI EXPONENTIAL BACKOFF ═══
async function retryWithBackoff(fn, maxRetries = 3, baseDelayMs = 1000) {
  let lastError;

  for (let attempt = 1; attempt ≤ maxRetries; attempt++) {
    try {
      return await fn();
    } catch (err) {
      lastError = err;

      // Không retry nếu là lỗi client (4xx)
      if (err.statusCode ≥ 400 && err.statusCode < 500) {
        throw err;
      }

      if (attempt ═══ maxRetries) break;

      // Exponential backoff: 1s, 2s, 4s, 8s ...
      const delay = baseDelayMs * Math.pow(2, attempt - 1);
      const jitter = Math.random() * 500; // Random jitter để tránh
thundering herd
      console.log(`Attempt ${attempt} thất bại, thử lại sau ${delay +
```

```
jitter}ms ... `);
      await new Promise(resolve => setTimeout(resolve, delay + jitter));
    }
  }

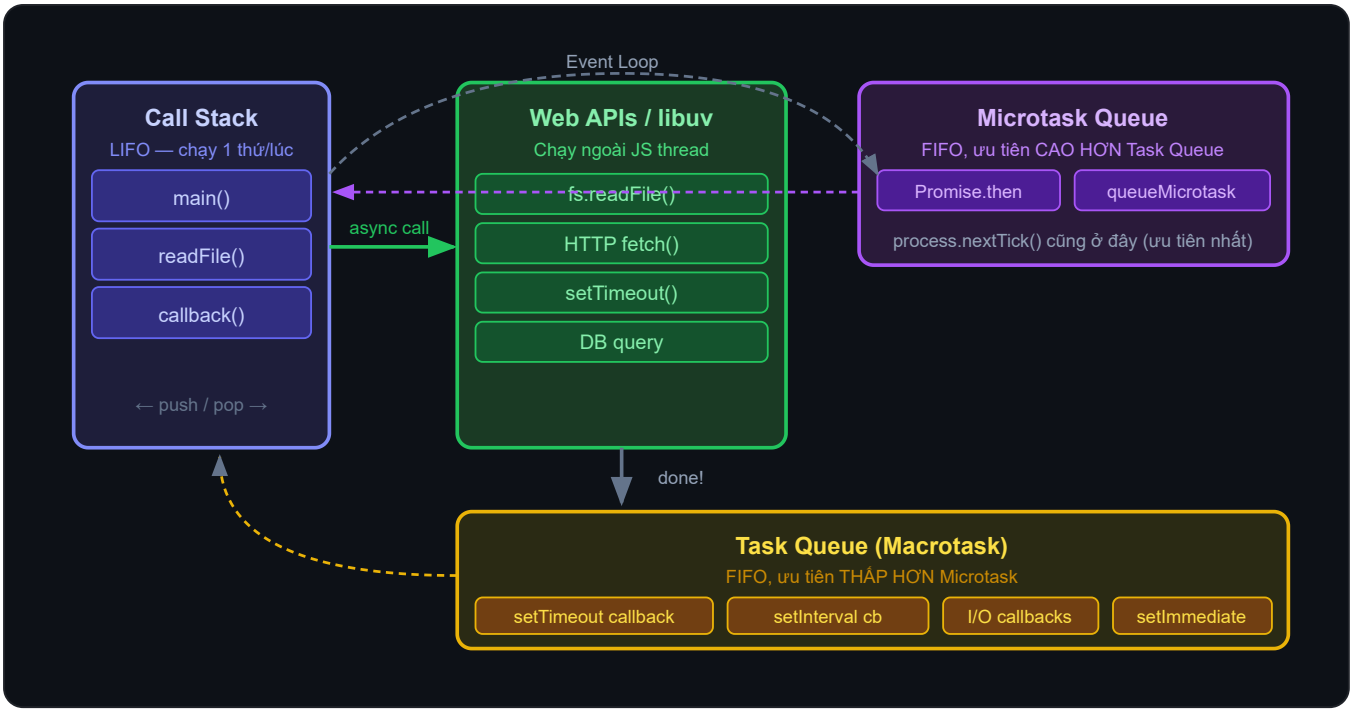
  throw new Error(`Thất bại sau ${maxRetries} lần: ${lastError.message}`);
}

// Sử dụng
const data = await retryWithBackoff(
  () => fetch('https://api.example.com/data').then(r => r.json()),
  3,    // max 3 lần
  1000  // bắt đầu với 1 giây
);
```

5.7 Event Loop — Hiểu Sâu Bên Trong

Event Loop là *trái tim* của NodeJS. Nó quyết định code nào chạy khi nào. Hiểu Event Loop giúp bạn debug được những bug khó hiểu nhất và viết code hiệu quả hơn.

Các thành phần của Event Loop



Thứ tự thực thi: process.nextTick vs Promise.then vs setTimeout

JAVASCRIPT event-loop-demo.js Copy

```
console.log('1. Synchronous - chạy ngay');

setTimeout(() => {
  console.log('5. setTimeout(0) - Macrotask (chạy sau cùng)');
}, 0);

Promise.resolve()
  .then(() => {
    console.log('3. Promise.then - Microtask');
  })
  .then(() => {
    console.log('4. Promise.then lần 2 - vẫn Microtask');
  });

process.nextTick(() => {
  console.log('2. process.nextTick - Microtask ưu tiên cao nhất');
});

console.log('1b. Synchronous - vẫn đang chạy sync');

// Output:
// 1. Synchronous - chạy ngay
// 1b. Synchronous - vẫn đang chạy sync
// 2. process.nextTick - Microtask ưu tiên cao nhất
// 3. Promise.then - Microtask
// 4. Promise.then lần 2 - vẫn Microtask
// 5. setTimeout(0) - Macrotask (chạy sau cùng)

// ═══ VÍ DỤ PHỨC TẠP HƠN ═══
console.log('--- Ví dụ 2 ---');

setTimeout(() => console.log('timeout 1'), 0);
setTimeout(() => console.log('timeout 2'), 0);

Promise.resolve()
```

```
  .then(() => {
    console.log('promise 1');
    process.nextTick(() => console.log('nextTick trong promise'));
  })
  .then(() => console.log('promise 2'));

process.nextTick(() => {
  console.log('nextTick 1');
  process.nextTick(() => console.log('nextTick lồng nhau'));
});

// Output:
// nextTick 1
// nextTick lồng nhau     ← nextTick được gọi trong nextTick cũng ưu tiên
cao
// promise 1
// nextTick trong promise ← nextTick trong promise.then cũng được ưu tiên
// promise 2
// timeout 1
// timeout 2

// ═══ setImmediate vs setTimeout(fn, 0) ═══
// Trong I/O callback: setImmediate chạy trước setTimeout
// Ngoài I/O: thứ tự không đảm bảo

const fs = require('fs');
fs.readFile('./file.txt', () => {
  setTimeout(() => console.log('timeout'), 0);
  setImmediate(() => console.log('immediate'));
  // Output đảm bảo:
  // immediate
  // timeout
});
```

⚠️ **ĐỪNG BLOCK EVENT LOOP**

Mọi code synchronous dài sẽ block Event Loop. Ví dụ: tính toán nặng, vòng lặp triệu lần, parse JSON file 100MB. Trong thời gian đó NodeJS không thể xử lý request nào khác. Giải pháp: dùng `worker_threads` cho CPU-intensive tasks, hoặc chia nhỏ công việc bằng `setImmediate`.

Thực hành tốt: Không block Event Loop

JAVASCRIPT non-blocking-heavy.js Copy

```
// ❌ Block Event Loop - tính fibonacci ngay trên main thread
function fibSlow(n) {
  if (n ≤ 1) return n;
  return fibSlow(n - 1) + fibSlow(n - 2);
}
// fibSlow(45) → block 5+ giây!

// ✅ Dùng Worker Threads cho CPU-intensive tasks
const { Worker, isMainThread, parentPort, workerData } =
require('worker_threads');

if (isMainThread) {
  // Code chạy trên main thread
  function fibInWorker(n) {
    return new Promise((resolve, reject) ⇒ {
      const worker = new Worker(__filename, { workerData: n });
      worker.on('message', resolve);
      worker.on('error', reject);
    });
  }

  // Main thread không bị block
  fibInWorker(45).then(result ⇒ {
    console.log('Kết quả fib(45):', result);
  });

  // Server vẫn chạy bình thường trong khi worker tính toán
  console.log('Server vẫn xử lý requests bình thường...');

} else {
  // Code chạy trong worker thread
  function fib(n) {
    if (n ≤ 1) return n;
    return fib(n - 1) + fib(n - 2);
  }
```

```
  parentPort.postMessage(fib(workerData));
}

// ✅ Chia nhỏ công việc bằng setImmediate (cho array lớn)
async function processLargeArray(items, processFn) {
  const BATCH_SIZE = 100;
  const results = [];

  for (let i = 0; i < items.length; i += BATCH_SIZE) {
    const batch = items.slice(i, i + BATCH_SIZE);
    const batchResults = batch.map(processFn);
    results.push(...batchResults);

    // Nhường Event Loop sau mỗi batch
    await new Promise(resolve => setImmediate(resolve));
  }

  return results;
}
```

📚 Tóm tắt chương 5

- **Tại sao Async:** NodeJS single-threaded nên I/O blocking sẽ làm nghẽn toàn bộ server. Non-blocking I/O cho phép xử lý nhiều request đồng thời chỉ với 1 thread.
- **Callback:** Pattern cổ điển, dùng error-first convention `(err, result) => {}`. Tránh callback hell bằng cách extract functions ra ngoài hoặc dùng Promise.
- **Promise:** Object đại diện cho giá trị tương lai. 3 trạng thái: pending, fulfilled, rejected. Dùng `.then()`, `.catch()`, `.finally()`. Có thể chain.
- **Promise Combinators:** `Promise.all` — song song, fail fast; `Promise.allSettled` — song song, không fail; `Promise.race` — ai xong trước; `Promise.any` — ai thành công trước.
- **async/await:** Syntactic sugar trên Promise. Dùng `try/catch` thay `.catch()`. Cẩn thận `await` trong vòng lặp — dùng `Promise.all` nếu muốn song song.
- **Error Handling:** Luôn có `catch`. Dùng custom error classes để phân biệt loại lỗi. Bắt `unhandledRejection` toàn app làm safety net.

- **Event Loop:** Microtask (Promise.then, process.nextTick) ưu tiên hơn Macrotask (setTimeout, I/O callbacks). process.nextTick ưu tiên nhất trong microtask. Không bao giờ block Event Loop bằng code CPU-intensive.

🏋️ Bài tập thực hành

DỄ

5.1 Promisify với util.promisify

Chuyển đổi ba hàm callback-style sau sang Promise dùng `util.promisify`:

- `fs.readFile` — đọc file
- `fs.writeFile` — ghi file
- `dns.lookup` — phân giải DNS của một domain

Viết một script nhận domain từ `process.argv[2]`, phân giải IP bằng `dns.lookup` promisified, sau đó ghi kết quả vào file `dns-results.json`. Xử lý đầy đủ error khi domain không tồn tại.

TRUNG BÌNH

5.2 fetchWithTimeout

Viết hàm `fetchWithTimeout(url, timeoutMs, options = {})` thỏa mãn:

- Fetch URL với `node-fetch` hoặc built-in `fetch` (NodeJS 18+)
- Nếu request mất hơn `timeoutMs` milliseconds thì reject với lỗi rõ ràng
- Nếu response status không OK (4xx, 5xx) thì reject với status code
- Trả về parsed JSON nếu Content-Type là application/json, text nếu không
- Hỗ trợ abort signal để cancel request khi timeout

Test với các URL thực tế: JSONPlaceholder API, một URL không tồn tại, và mô phỏng timeout.

KHÓ

5.3 asyncPool — Concurrency Limiter

Viết hàm `asyncPool(concurrencyLimit, tasks)` trong đó:

- `concurrencyLimit`: số task chạy đồng thời tối đa
- `tasks`: mảng các hàm `() ⇒ Promise`
- Hàm chạy tối đa `concurrencyLimit` tasks cùng lúc

- Khi một task hoàn thành, task tiếp theo được bắt đầu ngay
- Trả về mảng kết quả theo thứ tự ban đầu của tasks (không phải theo thứ tự hoàn thành)
- Nếu có task nào fail, collect tất cả errors và throw AggregateError ở cuối

Test case: Tạo 20 tasks mỗi task delay ngẫu nhiên 100-500ms, chạy với limit = 5. Verify rằng không bao giờ có hơn 5 tasks chạy đồng thời (dùng counter). So sánh thời gian chạy với limit = 1, 5, 10, 20.

PHẦN 2 — NODEJS CƠ BẢN

Chương 6: Tự Xây Dựng HTTP Server với NodeJS Thuần

Trước khi dùng Express, hãy hiểu những gì xảy ra bên dưới. Chương này xây dựng HTTP server từ module `http` thuần — không framework, không package ngoài — giúp bạn thực sự hiểu cách một web framework hoạt động như thế nào.

> 🎯 **SAU CHƯƠNG NÀY BẠN SẼ**
>
> - Tạo HTTP server bằng module `http` thuần của NodeJS
> - Đọc method, URL, headers và body từ request
> - Gửi response với status code và headers đúng chuẩn HTTP
> - Implement routing thủ công không cần framework
> - Parse request body dạng JSON và form-urlencoded
> - Phục vụ static files từ filesystem
> - Hiểu tại sao cần framework như Express thay vì dùng thuần

6.1 Module http — Tạo Server Đầu Tiên

Module `http` là một trong những core modules quan trọng nhất của NodeJS. Nó cho phép bạn tạo HTTP server mà không cần cài bất kỳ package nào từ npm.

Hàm chính là `http.createServer(requestListener)` — nhận một callback được gọi mỗi khi có request đến. Callback này nhận 2 tham số:

- `req` (IncomingMessage) — đối tượng đại diện cho request đến, là một **Readable Stream**
- `res` (ServerResponse) — đối tượng để gửi response về, là một **Writable Stream**

JAVASCRIPT server-basic.js Copy

```javascript
// Không cần npm install gì cả – http là built-in module
const http = require('http');

// Tạo server – callback gọi mỗi khi có HTTP request đến
const server = http.createServer((req, res) => {
  // Gửi response với status 200 và content type text/plain
  res.writeHead(200, {
    'Content-Type': 'text/plain; charset=utf-8'
  });
  res.end('Xin chào từ NodeJS HTTP Server!');
});

// Lắng nghe cổng 3000
server.listen(3000, '127.0.0.1', () => {
  console.log('Server đang chạy tại http://localhost:3000');
  console.log('Nhấn Ctrl+C để dừng.');
});

// Xử lý lỗi server
server.on('error', (err) => {
  if (err.code === 'EADDRINUSE') {
    console.error('Cổng 3000 đang bị dùng bởi ứng dụng khác!');
  } else {
    console.error('Lỗi server:', err.message);
  }
  process.exit(1);
});
```

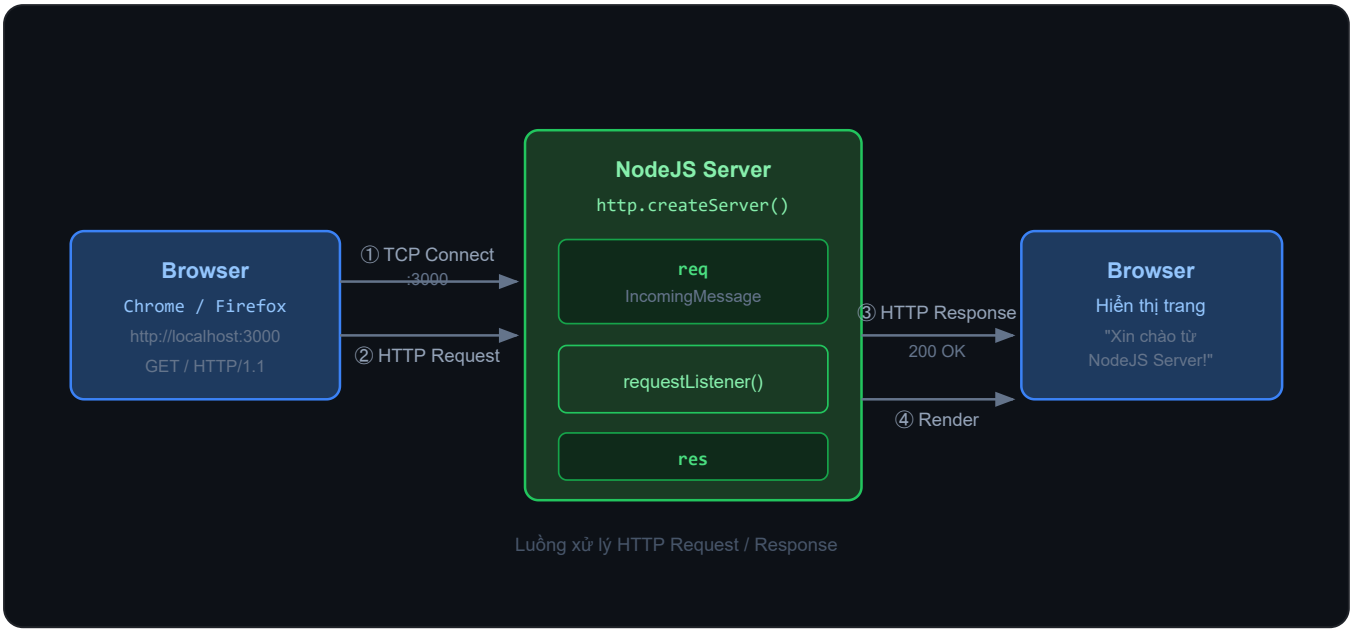
SHELL Chạy server Copy

```
node server-basic.js
# Server đang chạy tại http://localhost:3000

# Mở tab terminal khác và test với curl:
curl http://localhost:3000
# Xin chào từ NodeJS HTTP Server!

# Hoặc mở trình duyệt → http://localhost:3000
```

Hãy xem toàn bộ luồng từ browser đến server và ngược lại:



Luồng xử lý HTTP Request / Response

> 📌 **LƯU Ý VỀ SERVER.LISTEN()**
>
> `server.listen(port)` là bất đồng bộ — callback chạy khi server đã sẵn sàng nhận kết nối. Nếu không truyền `host`, server sẽ lắng nghe trên tất cả interfaces (`0.0.0.0`). Truyền `'127.0.0.1'` nếu chỉ muốn truy cập từ máy local.

6.2 Request Object — Đọc Dữ Liệu Đến

Đối tượng `req` (kiểu `http.IncomingMessage`) chứa tất cả thông tin về request đến. Đây là một **Readable Stream** nên bạn cần đọc dữ liệu theo cách stream.

Các thuộc tính cơ bản của req

JAVASCRIPT read-request.js Copy

```
const http = require('http');

const server = http.createServer((req, res) => {
  // ── Method: GET, POST, PUT, DELETE, PATCH ...
  console.log('Method:', req.method);        // 'GET'

  // ── URL đầy đủ bao gồm query string
  console.log('URL:', req.url);              // '/users?page=2&limit=10'

  // ── HTTP version
  console.log('HTTP version:', req.httpVersion); // '1.1'

  // ── Tất cả request headers (lowercase keys)
  console.log('Headers:', req.headers);
  console.log('Content-Type:', req.headers['content-type']);
  console.log('User-Agent:', req.headers['user-agent']);
  console.log('Authorization:', req.headers['authorization']);

  // ── Remote IP
  console.log('Client IP:', req.socket.remoteAddress);

  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Đã nhận request');
});

server.listen(3000);
```

Parse URL và Query String

`req.url` trả về chuỗi đầy đủ như `/users?page=2&name=nguyen`. Để làm việc thuận tiện, dùng class `URL` built-in:

JAVASCRIPT parse-url.js Copy

```
const http = require('http');

const server = http.createServer((req, res) => {
  // Cách hiện đại: dùng class URL (Node 10+)
  // Cần base URL để parse tương đối
  const url = new URL(req.url, `http://${req.headers.host}`);

  console.log('Pathname:', url.pathname);     // '/users'
  console.log('Search:', url.search);         // '?page=2&name=nguyen'

  // Đọc query params dễ dàng
  const page = url.searchParams.get('page');         // '2'
  const name = url.searchParams.get('name');         // 'nguyen'
  const limit = url.searchParams.get('limit') ?? '10'; // '10' (default)

  // searchParams tự động decode URL encoding
  // ?q=xin+ch%C3%A0o → 'xin chào'

  // Cách cũ (vẫn hoạt động):
  // const qs = req.url.split('?')[1] || '';
  // const params = new URLSearchParams(qs);

  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ pathname: url.pathname, page, name, limit }));
});

server.listen(3000);
```

Đọc Request Body

Vì `req` là Readable Stream, body không có sẵn ngay lập tức. Bạn phải "tích lũy" dữ liệu từ các `data` events và đợi `end` event:

JAVASCRIPT read-body.js Copy

```javascript
const http = require('http');

/**
 * Đọc toàn bộ body của request, trả về Promise
 * @param {http.IncomingMessage} req
 * @param {number} [maxBytes=1048576] - giới hạn 1MB
 */
function readBody(req, maxBytes = 1_048_576) {
  return new Promise((resolve, reject) => {
    const chunks = [];
    let totalSize = 0;

    req.on('data', (chunk) => {
      totalSize += chunk.length;

      // Bảo vệ: từ chối body quá lớn
      if (totalSize > maxBytes) {
        req.destroy(); // Hủy kết nối
        reject(new Error('Request body quá lớn (tối đa 1MB)'));
        return;
      }
      chunks.push(chunk);
    });

    req.on('end', () => {
      // Ghép các Buffer thành một và chuyển sang string
      resolve(Buffer.concat(chunks).toString('utf8'));
    });

    req.on('error', (err) => {
      reject(err);
    });
  });
}

/**
 * Đọc và parse JSON body, trả về Promise
 */
```

```
async function readJSONBody(req) {
  const raw = await readBody(req);
  if (!raw.trim()) return {};
  try {
    return JSON.parse(raw);
  } catch {
    throw new Error('Request body không phải JSON hợp lệ');
  }
}

/**
 * Đọc và parse form body (application/x-www-form-urlencoded)
 * Ví dụ: "name=nguyen&age=25" → { name: 'nguyen', age: '25' }
 */
async function readFormBody(req) {
  const raw = await readBody(req);
  return Object.fromEntries(new URLSearchParams(raw));
}

// ─── Sử dụng ──────────────────────────────────────────────
const server = http.createServer(async (req, res) => {
  if (req.method === 'POST' && req.url === '/json') {
    try {
      const body = await readJSONBody(req);
      console.log('Nhận được JSON:', body);
      res.writeHead(200, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ received: body }));
    } catch (err) {
      res.writeHead(400, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ error: err.message }));
    }
    return;
  }

  res.writeHead(404);
  res.end('Not found');
});

server.listen(3000, () => console.log('Server chạy tại :3000'));
```

SHELL Test với curl Copy

```
# Test POST JSON
curl -X POST http://localhost:3000/json \
  -H "Content-Type: application/json" \
  -d '{"name":"Nguyen","age":25}'

# Kết quả: {"received":{"name":"Nguyen","age":25}}
```

6.3 Response Object – Gửi Dữ Liệu Về Client

Đối tượng

res

(kiểu

http.ServerResponse

) là Writable Stream

để gửi response về client. Có 3 bước cần làm: đặt status, đặt headers, gửi body.

Các phương thức của res

JAVASCRIPT response-methods.js Copy

```
// ── Cách 1: set riêng lẻ
res.statusCode = 201;
res.setHeader('Content-Type', 'application/json');
res.setHeader('X-Request-Id', 'abc-123');
res.end(JSON.stringify({ id: 1, name: 'Nguyen' }));

// ── Cách 2: writeHead – set status + nhiều headers cùng lúc (nhanh hơn)
res.writeHead(201, {
  'Content-Type': 'application/json',
  'X-Request-Id': 'abc-123',
  'Cache-Control': 'no-cache'
});
res.end(JSON.stringify({ id: 1 }));

// ── Gửi body nhiều phần (streaming)
res.writeHead(200, { 'Content-Type': 'text/plain' });
res.write('Phần 1 ... ');   // ghi một phần
res.write('Phần 2 ... ');   // ghi thêm
res.end('Xong!');           // kết thúc

// ── QUAN TRỌNG: res.end() PHẢI được gọi – nếu không client sẽ chờ mãi!
```

Helper Functions – Tái sử dụng trong mọi handler





JAVASCRIPT response-helpers.js Copy

```
'use strict';

/**
 * Gửi JSON response
 * @param {http.ServerResponse} res
 * @param {number} status - HTTP status code
 * @param {any} data - Dữ liệu sẽ được JSON.stringify
 */
function sendJSON(res, status, data) {
  const body = JSON.stringify(data, null, 0); // compact JSON
  res.writeHead(status, {
    'Content-Type': 'application/json; charset=utf-8',
    'Content-Length': Buffer.byteLength(body),  // byte length, không
phải char length
  });
  res.end(body);
}

/**
 * Gửi error response chuẩn
 */
function sendError(res, status, message, details = null) {
  sendJSON(res, status, {
    error: {
      status,
      message,
      ...(details && { details }),
      timestamp: new Date().toISOString(),
    }
  });
}

/**
 * Gửi HTML response
 */
function sendHTML(res, status, html) {
  res.writeHead(status, {
    'Content-Type': 'text/html; charset=utf-8',
```

```
      'Content-Length': Buffer.byteLength(html),
    });
    res.end(html);
  }

  /**
   * Redirect về URL khác
   */
  function redirect(res, url, permanent = false) {
    res.writeHead(permanent ? 301 : 302, { Location: url });
    res.end();
  }

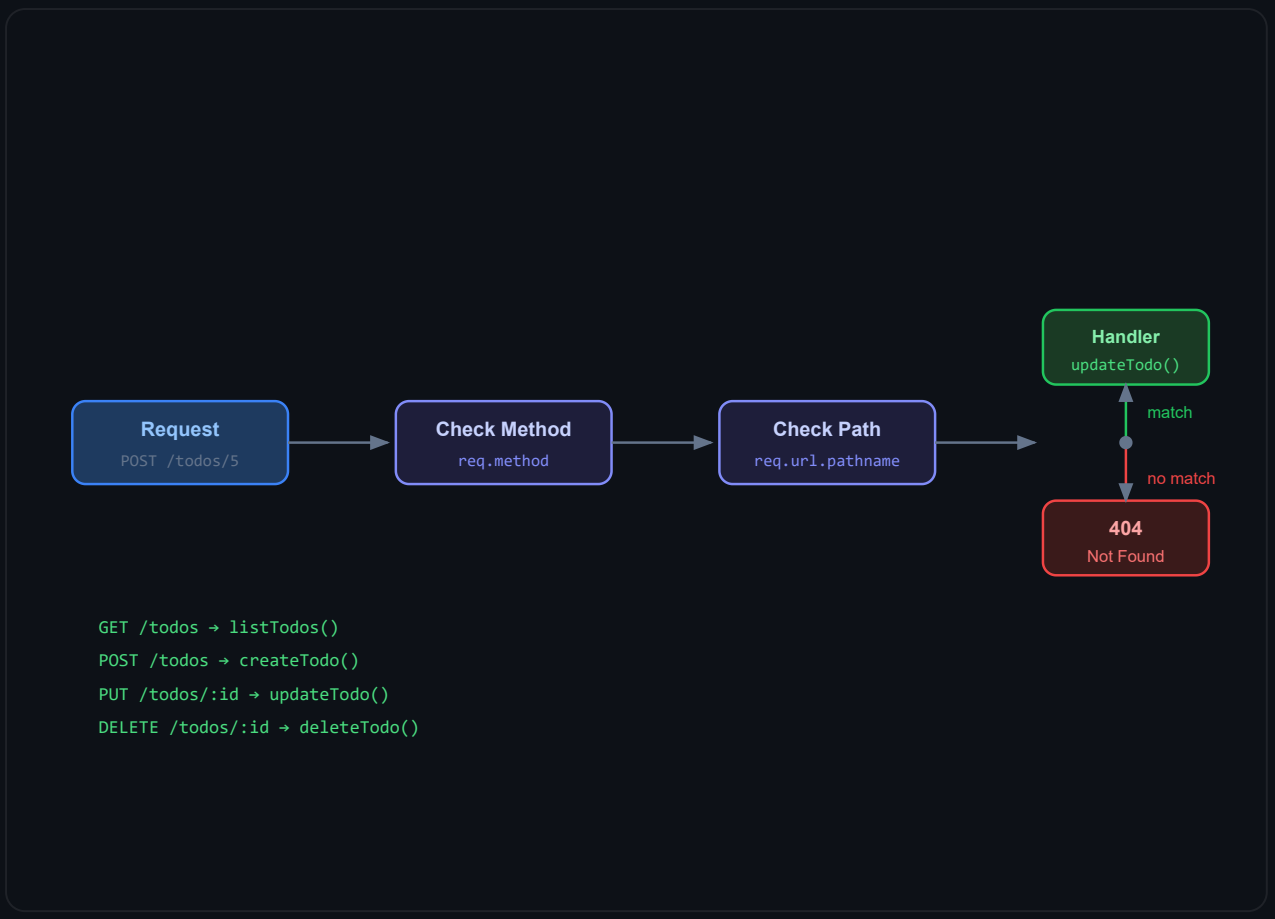
  module.exports = { sendJSON, sendError, sendHTML, redirect };

  // ─── Ví dụ dùng ──────────────────────────────────────────
  // const { sendJSON, sendError } = require('./response-helpers');
  //
  // server.createServer((req, res) => {
  //   sendJSON(res, 200, { users: [], total: 0 });
  //   sendError(res, 404, 'User không tồn tại');
  // });
```

6.4 Routing Thủ Công

Routing là quá trình quyết định handler nào xử lý request dựa vào method và URL.

Không có framework, bạn phải tự so sánh:



Todo REST API – Ví dụ đầy đủ (không dùng package ngoài)

Hãy xây dựng một Todo API hoàn chỉnh với GET / POST / PUT / DELETE, lưu dữ liệu trong

Map

(in-memory, mất khi restart server):





JAVASCRIPT todo-server.js Copy

```
'use strict';
const http = require('http');
const { URL } = require('url');

// ── In-memory store ──────────────────────────────────────
const todos = new Map();
let nextId = 1;

// ── Utilities ────────────────────────────────────────────
function sendJSON(res, status, data) {
  const body = JSON.stringify(data);
  res.writeHead(status, {
    'Content-Type': 'application/json; charset=utf-8',
    'Content-Length': Buffer.byteLength(body),
    'Access-Control-Allow-Origin': '*',  // Cho phép CORS
  });
  res.end(body);
}

function sendError(res, status, message) {
  sendJSON(res, status, { error: message });
}

function readJSONBody(req) {
  return new Promise((resolve, reject) => {
    let raw = '';
    req.on('data', chunk => { raw += chunk; });
    req.on('end', () => {
      try { resolve(raw ? JSON.parse(raw) : {}); }
      catch { reject(new Error('Invalid JSON')); }
    });
    req.on('error', reject);
  });
}

// ── Parse route params từ URL ────────────────────────────
// matchRoute('/todos/:id', '/todos/5') → { id: '5' } hoặc null
function matchRoute(pattern, pathname) {
```

```
  const patternParts = pattern.split('/');
  const pathParts = pathname.split('/');
  if (patternParts.length !== pathParts.length) return null;

  const params = {};
  for (let i = 0; i < patternParts.length; i++) {
    if (patternParts[i].startsWith(':')) {
      // Segment là param (:id, :slug ...)
      params[patternParts[i].slice(1)] =
decodeURIComponent(pathParts[i]);
    } else if (patternParts[i] !== pathParts[i]) {
      return null; // Không khớp
    }
  }
  return params;
}

// ── Route Handlers ──────────────────────────────────────────

// GET /todos - Trả về danh sách tất cả todos
function listTodos(req, res) {
  const url = new URL(req.url, `http://localhost`);
  const done = url.searchParams.get('done'); // filter: ?done=true

  let list = Array.from(todos.values());
  if (done === 'true') list = list.filter(t => t.done);
  if (done === 'false') list = list.filter(t => !t.done);

  sendJSON(res, 200, { data: list, total: list.length });
}

// GET /todos/:id - Trả về 1 todo theo id
function getTodo(req, res, params) {
  const id = parseInt(params.id, 10);
  if (!todos.has(id)) {
    return sendError(res, 404, `Todo #${id} không tồn tại`);
  }
  sendJSON(res, 200, todos.get(id));
}

// POST /todos - Tạo todo mới
```

```
async function createTodo(req, res) {
  let body;
  try {
    body = await readJSONBody(req);
  } catch {
    return sendError(res, 400, 'Request body phải là JSON hợp lệ');
  }

  const text = (body.text || '').trim();
  if (!text) {
    return sendError(res, 422, 'Trường "text" là bắt buộc và không được
rỗng');
  }
  if (text.length > 200) {
    return sendError(res, 422, 'Trường "text" không được quá 200 ký
tự');
  }

  const todo = {
    id: nextId++,
    text,
    done: false,
    createdAt: new Date().toISOString(),
  };
  todos.set(todo.id, todo);
  sendJSON(res, 201, todo);
}

// PUT /todos/:id — Cập nhật todo
async function updateTodo(req, res, params) {
  const id = parseInt(params.id, 10);
  if (!todos.has(id)) {
    return sendError(res, 404, `Todo #${id} không tồn tại`);
  }

  let body;
  try {
    body = await readJSONBody(req);
  } catch {
    return sendError(res, 400, 'Request body phải là JSON hợp lệ');
  }
```

```
  const existing = todos.get(id);
  const updated = {
     ...existing,
    text: (body.text !== undefined) ? String(body.text).trim() :
existing.text,
    done: (body.done !== undefined) ? Boolean(body.done) :
existing.done,
    updatedAt: new Date().toISOString(),
  };

  if (!updated.text) {
    return sendError(res, 422, 'Trường "text" không được rỗng');
  }

  todos.set(id, updated);
  sendJSON(res, 200, updated);
}

// DELETE /todos/:id — Xóa todo
function deleteTodo(req, res, params) {
  const id = parseInt(params.id, 10);
  if (!todos.has(id)) {
    return sendError(res, 404, `Todo #${id} không tồn tại`);
  }
  todos.delete(id);
  res.writeHead(204); // No Content
  res.end();
}

// ── Router ──────────────────────────────────────────────────
const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, `http://localhost`);
  const pathname = url.pathname;
  const method = req.method;

  // Hỗ trợ CORS preflight
  if (method === 'OPTIONS') {
    res.writeHead(204, {
      'Access-Control-Allow-Origin': '*',
      'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE,
```

```
  OPTIONS',
        'Access-Control-Allow-Headers': 'Content-Type',
      });
      return res.end();
    }

    try {
      // GET /todos
      if (method === 'GET' && pathname === '/todos') {
        return listTodos(req, res);
      }

      // POST /todos
      if (method === 'POST' && pathname === '/todos') {
        return createTodo(req, res);
      }

      // GET /todos/:id
      let params = matchRoute('/todos/:id', pathname);
      if (params) {
        if (method === 'GET')    return getTodo(req, res, params);
        if (method === 'PUT')    return updateTodo(req, res, params);
        if (method === 'DELETE') return deleteTodo(req, res, params);
      }

      // GET /health - health check endpoint
      if (method === 'GET' && pathname === '/health') {
        return sendJSON(res, 200, {
          status: 'ok',
          uptime: process.uptime(),
          timestamp: new Date().toISOString(),
          todoCount: todos.size,
        });
      }

      // Không khớp route nào → 404
      sendError(res, 404, `Route ${method} ${pathname} không tồn tại`);

    } catch (err) {
      console.error('Lỗi server không mong muốn:', err);
      sendError(res, 500, 'Lỗi server nội bộ');
```

```
  }
});

server.listen(3000, () => {
  console.log('Todo API đang chạy tại http://localhost:3000');
  console.log('Endpoints:');
  console.log('  GET    /todos       - Danh sách todos');
  console.log('  POST   /todos       - Tạo todo mới');
  console.log('  GET    /todos/:id   - Lấy todo theo id');
  console.log('  PUT    /todos/:id   - Cập nhật todo');
  console.log('  DELETE /todos/:id   - Xóa todo');
  console.log('  GET    /health      - Health check');
});
```

SHELL

Test API với curl

Copy

```
# Tạo todo mới
curl -X POST http://localhost:3000/todos \
  -H "Content-Type: application/json" \
  -d '{"text":"Học NodeJS HTTP module"}'
# {"id":1,"text":"Học NodeJS HTTP
module","done":false,"createdAt":"..."}

# Tạo thêm vài todo
curl -X POST http://localhost:3000/todos \
  -H "Content-Type: application/json" \
  -d '{"text":"Đọc tài liệu EventEmitter"}'

# Lấy danh sách
curl http://localhost:3000/todos
# {"data":[...],"total":2}

# Lấy todo #1
curl http://localhost:3000/todos/1

# Cập nhật: đánh dấu done=true
curl -X PUT http://localhost:3000/todos/1 \
  -H "Content-Type: application/json" \
  -d '{"done":true}'

# Filter: chỉ lấy todos chưa xong
curl "http://localhost:3000/todos?done=false"

# Xóa todo #1
curl -X DELETE http://localhost:3000/todos/1
# (204 No Content)

# Health check
curl http://localhost:3000/health
```

6.5 Phục Vụ Static Files

Một use case quan trọng của HTTP server là phục vụ file tĩnh (HTML, CSS, JS, images).

Cách làm: map URL path sang filesystem path, đọc file, trả về với Content-Type đúng.



JAVASCRIPT static-server.js Copy

```
'use strict';
const http = require('http');
const fs = require('fs');
const path = require('path');
const { URL } = require('url');

// Thư mục chứa static files
const PUBLIC_DIR = path.join(__dirname, 'public');

// Map extension → MIME type
const MIME_TYPES = {
  '.html': 'text/html; charset=utf-8',
  '.css': 'text/css; charset=utf-8',
  '.js': 'application/javascript; charset=utf-8',
  '.json': 'application/json; charset=utf-8',
  '.png': 'image/png',
  '.jpg': 'image/jpeg',
  '.jpeg': 'image/jpeg',
  '.gif': 'image/gif',
  '.svg': 'image/svg+xml',
  '.ico': 'image/x-icon',
  '.woff2': 'font/woff2',
  '.pdf': 'application/pdf',
};

function getMimeType(filePath) {
  const ext = path.extname(filePath).toLowerCase();
  return MIME_TYPES[ext] || 'application/octet-stream';
}

const server = http.createServer((req, res) => {
  const url = new URL(req.url, 'http://localhost');
  let pathname = url.pathname;

  // Bảo mật: chặn path traversal attack (../../etc/passwd)
  // path.normalize loại bỏ '../', path.resolve tính absolute path
  const safePath = path.normalize(pathname).replace(/^(\.\.[\/\\])+/,
'');
```

```
  let filePath = path.join(PUBLIC_DIR, safePath);

  // Nếu path là thư mục → tìm index.html
  if (fs.existsSync(filePath) && fs.statSync(filePath).isDirectory()) {
    filePath = path.join(filePath, 'index.html');
  }

  // Kiểm tra file có tồn tại và nằm trong PUBLIC_DIR không
  if (!filePath.startsWith(PUBLIC_DIR)) {
    res.writeHead(403, { 'Content-Type': 'text/plain' });
    return res.end('403 Forbidden');
  }

  fs.stat(filePath, (statErr, stats) => {
    if (statErr || !stats.isFile()) {
      res.writeHead(404, { 'Content-Type': 'text/html; charset=utf-8'
});
      return res.end(`

<h1>404 – Không tìm thấy</h1>


File

  ${pathname}

 không tồn tại.


        ← Về trang chủ
      `);
    }

    const mimeType = getMimeType(filePath);
    const fileSize = stats.size;

    res.writeHead(200, {
      'Content-Type': mimeType,
      'Content-Length': fileSize,
      'Last-Modified': stats.mtime.toUTCString(),
      // Cache 1 giờ cho assets, không cache HTML
```

```
      'Cache-Control': mimeType.startsWith('text/html') ? 'no-cache' :
'max-age=3600',
    });

    // Đọc và gửi file bằng stream (hiệu quả cho file lớn)
    fs.createReadStream(filePath).pipe(res);
  });
});

server.listen(3000, () => {
  console.log(`Static server: http://localhost:3000`);
  console.log(`Serving files from: ${PUBLIC_DIR}`);
});
```

⚠️ **BẢO MẬT PATH TRAVERSAL**

Nếu không kiểm tra, kẻ tấn công có thể gửi request với path như

`GET /../../../etc/passwd`

để đọc file hệ thống.

Luôn dùng

`path.normalize()`

và kiểm tra file path có bắt đầu bằng PUBLIC_DIR không.

6.6 HTTP Server với Streams

Đối với file lớn (video, PDF ...), việc đọc toàn bộ vào RAM rồi gửi là không hiệu quả.

Stream cho phép gửi dữ liệu **từng phần** — client nhận được dữ liệu ngay

trong khi file vẫn đang được đọc từ disk.



JAVASCRIPT stream-server.js Copy

```
'use strict';
const http = require('http');
const fs = require('fs');
const path = require('path');

const server = http.createServer((req, res) => {
  const filePath = path.join(__dirname, 'assets', 'video.mp4');

  fs.stat(filePath, (err, stats) => {
    if (err) {
      res.writeHead(404);
      return res.end('File không tồn tại');
    }

    const fileSize = stats.size;

    // —— Hỗ trợ Range Request (cho video streaming, resume download)
——
    const rangeHeader = req.headers['range'];

    if (rangeHeader) {
      // Ví dụ: "bytes=0-999999"
      const [, rangeStr] = rangeHeader.match(/bytes=(\d*)-(\d*)/) ||
[];
      const parts = rangeHeader.replace('bytes=', '').split('-');
      const start = parseInt(parts[0], 10);
      const end = parts[1] ? parseInt(parts[1], 10) : fileSize - 1;
      const chunkSize = end - start + 1;

      res.writeHead(206, { // 206 Partial Content
        'Content-Range': `bytes ${start}-${end}/${fileSize}`,
        'Accept-Ranges': 'bytes',
        'Content-Length': chunkSize,
        'Content-Type': 'video/mp4',
      });

      // Stream chỉ phần được yêu cầu
      fs.createReadStream(filePath, { start, end }).pipe(res);
```

```
    } else {
      // Stream toàn bộ file
      res.writeHead(200, {
        'Content-Length': fileSize,
        'Content-Type': 'video/mp4',
        'Accept-Ranges': 'bytes',
      });
      fs.createReadStream(filePath).pipe(res);
    }
  });
});

// ── So sánh: WITHOUT stream (xấu cho file lớn) ──────────────
// fs.readFile(filePath, (err, data) => {
//   res.writeHead(200, { 'Content-Length': data.length });
//   res.end(data);  // ← Đọc hết vào RAM trước khi gửi!
// });

// ── WITH stream (tốt) ────────────────────────────────────────
// fs.createReadStream(filePath).pipe(res);
// → Đọc từng chunk 64KB → gửi ngay → đọc chunk tiếp theo
// → RAM sử dụng: ~64KB thay vì toàn bộ file

server.listen(3000, () => console.log('Stream server:
http://localhost:3000'));
```

💡 **.PIPE() HOẠT ĐỘNG NHƯ THẾ NÀO?**

```
 tự động xử lý backpressure – nếu client nhận chậm,
          readStream sẽ tạm dừng đọc. Điều này ngăn RAM bị tràn khi
server phục vụ file lớn
          cho nhiều client đồng thời. Khi stream đọc kết thúc,

  pipe()

 tự gọi

  res.end()

.
```

6.7 Vì Sao Cần Framework (Express)?

Qua các ví dụ trên, bạn đã thấy server thuần hoàn toàn có thể dùng được.

Nhưng khi project lớn hơn, các vấn đề này xuất hiện:

Tính năng	HTTP thuần	Express.js
Routing	Tự viết if/else, matchRoute()	app.get('/todos/:id', handler)
Body parsing	Tự viết readJSONBody(), 20+ dòng	express.json() – 1 dòng
Static files	Tự viết static server, xử lý MIME, security	express.static('public')
Middleware	Không có – phải gọi thủ công trong mỗi handler	app.use(middleware)
Error handling	Try/catch trong mỗi handler	Error middleware tập trung
Route grouping	Không có	express.Router()
Số dòng code cho Todo API	~130 dòng	~40 dòng
Học được gì?	Hiểu sâu HTTP, streams, protocols	Productive nhanh, code sạch hơn

📌 VẪN CÓ LÚC DÙNG HTTP THUẦN

Có những trường hợp dùng

http

module trực tiếp vẫn tốt hơn:

microservice siêu nhỏ cần binary nhỏ, proxy server, WebSocket server,

hoặc khi muốn toàn quyền kiểm soát. Frameworks như Express, Fastify đều

build trên top của

http

module – nên hiểu nền tảng sẽ giúp bạn debug dễ hơn nhiều.

📚 Tóm tắt chương 6

- **http.createServer(callback)** tạo server, callback nhận (req, res)

- **req** (IncomingMessage) là Readable Stream – đọc req.method, req.url, req.headers, body qua events data + end

- **res** (ServerResponse) là Writable Stream – dùng writeHead(), write(), end()

- Routing thủ công = so sánh

  `method + pathname`

  , có thể extract params từ URL

- Static file server cần: map URL → filesystem path, xử lý MIME type, bảo vệ path traversal

- `fs.createReadStream().pipe(res)`

  hiệu quả hơn

  `fs.readFile()`

  cho file lớn

- Framework như Express giải quyết boilerplate code, nhưng biết nền tảng giúp debug sâu hơn

🏋️ Bài tập thực hành

DỄ

6.1 Health Check Endpoint

Thêm route `GET /health` vào Todo server trả về JSON: `{ status: "ok", uptime: ..., timestamp: ..., memory: process.memoryUsage() }`.

Test bằng curl và kiểm tra response headers có `Content-Type: application/json` không.

TRUNG BÌNH

6.2 Mini Router Class

Viết class

`Router`

có các method

`.get(path, handler)`

,

`.post(path, handler)`

,

`.put(path, handler)`

,

`.delete(path, handler)`

và

`.handle(req, res)`

để dispatch request. Method

`handle`

nên

support route params (

`/users/:id`

) và trả

`req.params`

trong handler.

Refactor Todo server để dùng Router này.

KHÓ

6.3 Static Server với ETag và Conditional Request

Nâng cấp static file server để hỗ trợ: (1) Tính **ETag** dựa trên `mtime + size` của file. (2) Đọc header `If-None-Match` trong request — nếu ETag khớp → trả **304 Not Modified** và không gửi body. (3) Thêm header `Cache-Control: max-age=0, must-revalidate` cho HTML files. Test với curl dùng flag `-v` để thấy headers, sau đó test lần 2 với `-H "If-None-Match: <etag>"`.



PHẦN 2 — NODEJS CƠ BẢN

Chương 7: npm Ecosystem — Quản Lý Packages và Dependencies

npm là kho thư viện lớn nhất thế giới với hơn 2 triệu packages và hàng tỷ lượt tải mỗi tuần. Hiểu rõ npm giúp bạn quản lý project chuyên nghiệp, tránh các pitfalls phổ biến về phiên bản, bảo mật và hiệu suất.

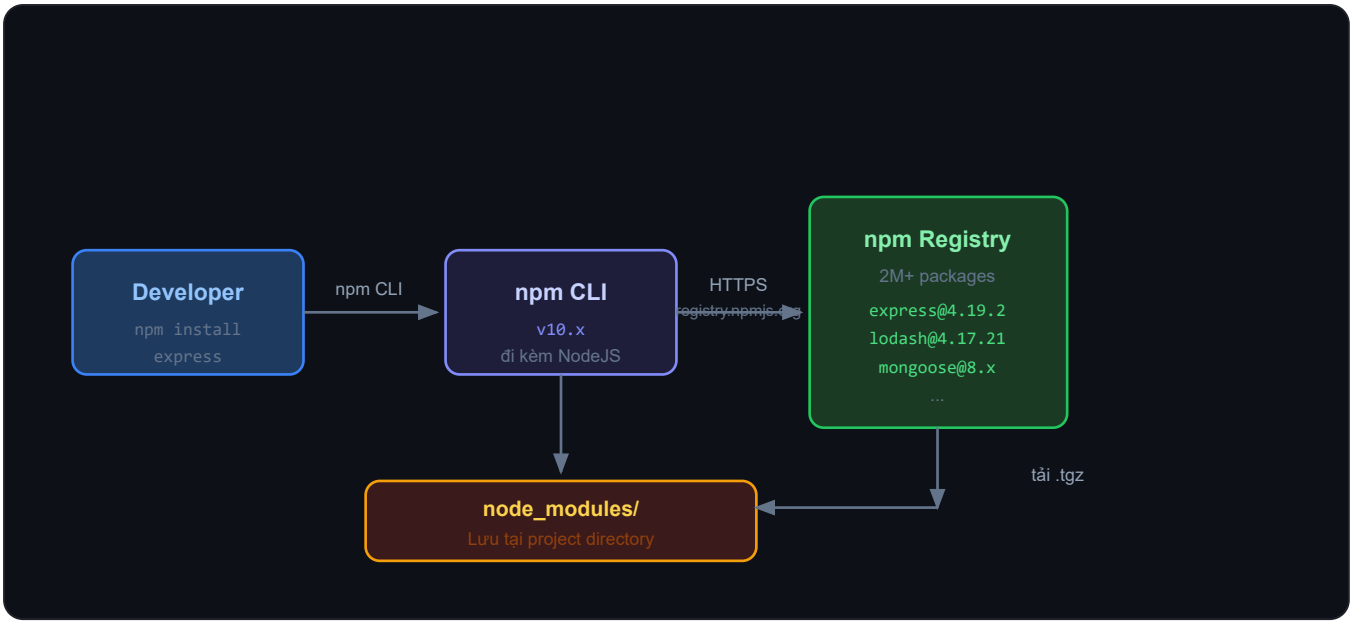
🎯 **SAU CHƯƠNG NÀY BẠN SẼ**

- Hiểu npm là gì và sự khác biệt giữa npm, yarn, pnpm
- Đọc và viết `package.json` đúng chuẩn với từng field
- Phân biệt `dependencies`, `devDependencies`, `peerDependencies`
- Hiểu Semantic Versioning và các range operators (^, ~, *)
- Viết npm scripts hiệu quả với pre/post hooks
- Tạo và publish npm package của riêng mình
- Dùng `npm audit` để kiểm tra bảo mật
- Hiểu sự khác biệt giữa `npm install` và `npm ci`

7.1 npm là gì?

npm (Node Package Manager) thực ra là **3 thứ riêng biệt**:

- **CLI tool** — lệnh `npm` bạn chạy trong terminal
- **Registry** — cơ sở dữ liệu packages tại `registry.npmjs.org`
- **Website** — `npmjs.com` để tìm kiếm và xem docs của packages



npm vs yarn vs pnpm

Thuộc tính	npm	yarn	pnpm
Tạo bởi	npm Inc. (GitHub)	Meta (Facebook)	Zoltan Kochan
Cài đặt	Đi kèm NodeJS	`npm i -g yarn`	`npm i -g pnpm`
Tốc độ	Trung bình	Nhanh (parallel)	Rất nhanh (content-addressable)
Disk space	Nhiều (copy files)	Nhiều (copy files)	Ít nhất (symlinks + store)
Lock file	package-lock.json	yarn.lock	pnpm-lock.yaml
Workspaces	Có (npm v7+)	Có	Có (tốt nhất)
Nên dùng khi	Mặc định, đơn giản	Monorepo lớn	Disk quan trọng, CI nhanh

> 💡 **NÊN DÙNG CÁI NÀO?**
>
> Nếu bạn mới bắt đầu, dùng **npm** vì nó đã có sẵn, không cần cài thêm. Khi làm monorepo hoặc team lớn, **pnpm** tiết kiệm disk và CI nhanh hơn đáng kể. Đừng mix — trong một project chỉ dùng một package manager.

7.2 package.json — Hồ Sơ Project

`package.json` là "hồ sơ" của project NodeJS — chứa metadata, dependencies, scripts và cấu hình. Tạo bằng `npm init` hoặc `npm init -y` (bỏ qua hỏi).

JSON package.json — Ví dụ đầy đủ có giải thích Copy

```
{
  // ─── Định danh ──────────────────────────────────────────
  "name": "my-node-api",          // Tên package (lowercase, no spaces, no
uppercase)
  "version": "1.2.3",             // Semantic version: MAJOR.MINOR.PATCH
  "description": "REST API cho ứng dụng quản lý tasks",
  "keywords": ["nodejs", "rest-api", "tasks"],
  "homepage": "https://github.com/user/my-node-api#readme",
  "license": "MIT",

  // ─── Tác giả & Đóng góp viên ────────────────────────────
  "author": {
    "name": "Nguyen Van A",
    "email": "nguyen@example.com",
    "url": "https://nguyenvana.dev"
  },
  "contributors": [
    "Tran Thi B "
  ],

  // ─── Source code ────────────────────────────────────────
  "repository": {
    "type": "git",
    "url": "https://github.com/user/my-node-api.git"
  },
  "bugs": { "url": "https://github.com/user/my-node-api/issues" },

  // ─── Entry points ───────────────────────────────────────
  "main": "src/index.js",         // Entry point cho CommonJS (require())
  "module": "dist/index.mjs",     // Entry point cho ES Modules
  "exports": {                    // Hiện đại hơn "main" — kiểm soát exports
chi tiết
    ".": {
      "import": "./dist/index.mjs",
      "require": "./src/index.js"
    },
    "./utils": "./src/utils.js"   // Cho phép: require('my-package/utils')
  },
```

```
"types": "dist/index.d.ts",     // TypeScript type definitions

// ─── Files được include khi publish ──────────────────────
"files": [
  "dist/",
  "src/",
  "README.md"
],

// ─── Runtime requirements ────────────────────────────────
"engines": {
  "node": "≥18.0.0",           // Yêu cầu Node ≥ 18
  "npm": "≥9.0.0"
},

// ─── Ngăn publish nhầm lên npm registry ──────────────────
"private": true,               // true = không thể npm publish

// ─── Scripts ─────────────────────────────────────────────
"scripts": {
  "start": "node src/index.js",
  "dev": "nodemon src/index.js",
  "build": "tsc",
  "test": "jest --coverage",
  "lint": "eslint src/",
  "format": "prettier --write src/"
},

// ─── Dependencies ────────────────────────────────────────
"dependencies": {
  "express": "^4.19.2",        // Cần cho PRODUCTION
  "mongoose": "^8.3.1",
  "dotenv": "^16.4.5"
},

"devDependencies": {           // Chỉ cần khi DEVELOPMENT / TESTING
  "nodemon": "^3.1.0",
  "jest": "^29.7.0",
  "eslint": "^9.3.0",
  "prettier": "^3.2.5",
  "@types/express": "^4.17.21"
```

```
  },

  "peerDependencies": {           // HOST PROJECT phải cài – dùng khi viết
plugin/library
    "react": "≥18.0.0"
  },

  "optionalDependencies": {       // Có thể thiếu – package vẫn hoạt động
nếu cài thất bại
    "fsevents": "^2.3.3"          // Chỉ hoạt động trên macOS
  }
}
```

Tại sao phân biệt dependencies và devDependencies?

Khi deploy lên production, bạn thường chạy `npm install --omit=dev` — lệnh này bỏ qua `devDependencies`. Điều này giúp:

- Giảm kích thước thư mục `node_modules` đáng kể (đôi khi 50-70%)
- Deploy nhanh hơn, Docker image nhỏ hơn
- Giảm attack surface bảo mật (ít packages = ít vulnerability)

SHELL Quy tắc phân loại Copy

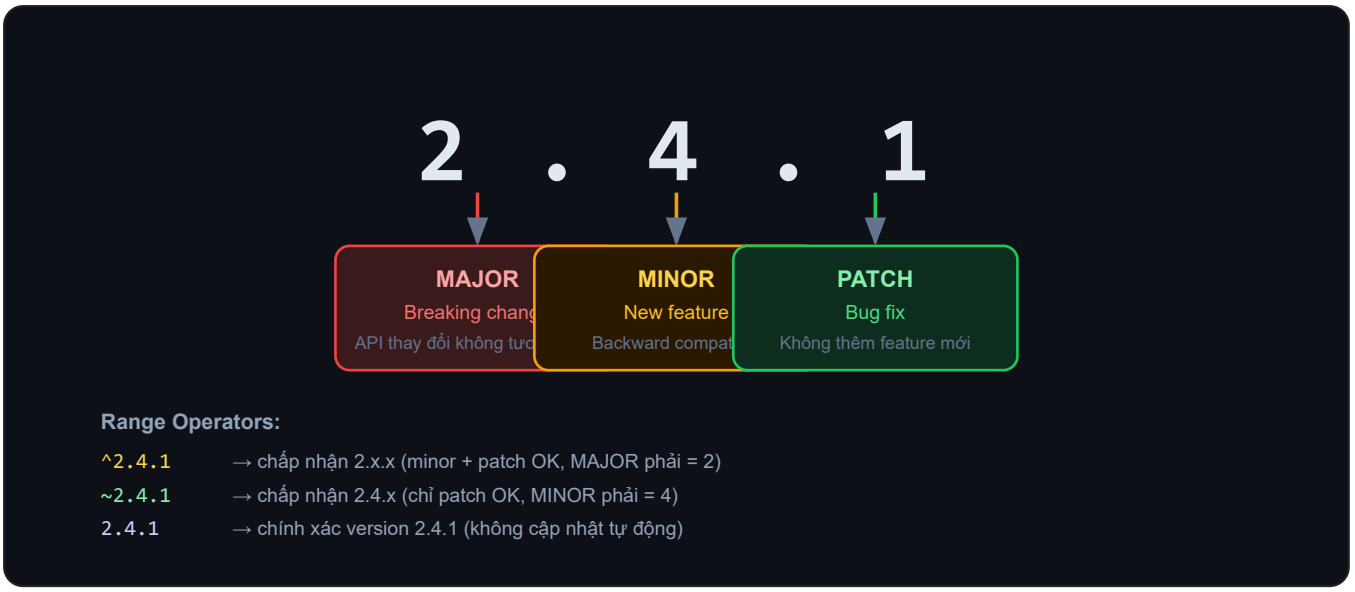
```
# ─── dependencies: cần cho production ──────────────────────────
npm install express mongoose jsonwebtoken bcrypt dotenv
# Đây là những package mà server ĐANG CHẠY cần dùng

# ─── devDependencies: chỉ cần khi development/build ────────────
npm install --save-dev nodemon jest eslint prettier typescript
# Những package này chỉ cần khi bạn đang VIẾT CODE

# ─── Cài production chỉ (khi deploy) ──────────────────────────
npm install --omit=dev
# Hoặc (cách cũ): npm install --production
```

7.3 Semantic Versioning — Đọc Số Version

npm dùng chuẩn **Semantic Versioning (semver)** cho tất cả packages. Hiểu semver giúp bạn biết khi nào update an toàn và khi nào có thể gây breaking change.



Bảng tổng hợp Range Operators

Operator	Ví dụ	Chấp nhận	Không chấp nhận
`^` (caret)	`^2.4.1`	2.4.1, 2.5.0, 2.9.9	3.0.0, 1.9.9
`~` (tilde)	`~2.4.1`	2.4.1, 2.4.8, 2.4.99	2.5.0, 3.0.0
Chính xác	`2.4.1`	2.4.1 (chỉ version này)	2.4.2, 2.5.0
`*` (wildcard)	`*`	Mọi version (kể cả major)	(không có giới hạn)
`≥`	`≥2.4.0`	2.4.0, 3.0.0, 99.0.0	2.3.9
Range	`2.0.0 - 3.0.0`	2.0.0 đến 3.0.0	1.x, 3.0.1+

⚠ TẠI SAO KHÔNG DÙNG * (WILDCARD) CHO MỌI THỨ?

Dùng `*` nghĩa là "chấp nhận bất kỳ version nào kể cả major". Nếu package X ra version 3.0.0 với breaking changes, app của bạn sẽ tự động "nâng cấp" và có thể bị lỗi ngay lập tức. Đây là lý do `package-lock.json` tồn tại — để lock chính xác version đã test.

Kiểm tra và cập nhật packages

SHELL — Quản lý versions — Copy

```
# Xem packages nào có update
npm outdated
# Package  Current  Wanted  Latest
# express  4.17.1   4.19.2  4.19.2    ← Wanted = trong range ^4.17.1
# mongoose 7.0.0    7.6.1   8.3.1     ← Latest 8.x nhưng Wanted vẫn 7.x
(major!)

# Cập nhật packages trong range đã khai báo (an toàn)
npm update

# Cập nhật một package cụ thể lên latest (CÓ THỂ BREAKING!)
npm install express@latest

# Kiểm tra semver: version nào khớp range?
node -e "console.log(require('semver').satisfies('2.5.0', '^2.4.1'))"
# true
```

7.4 package-lock.json và node_modules

`package-lock.json` là file quan trọng mà nhiều người bỏ qua hoặc thậm chí thêm vào `.gitignore` (sai!). Hãy hiểu tại sao nó quan trọng:

Vấn đề không có lock file

⚠️ **"WORKS ON MY MACHINE" VÌ KHÔNG CÓ LOCK FILE**

Developer A cài dependencies hôm nay → package X 2.4.1 được cài.
Developer B pull code tuần sau → `npm install` → package X 2.5.0 được cài (có bug mới).
CI server → X 2.5.1 được cài.
Production → X 2.5.2 được cài.
Tất cả đang chạy các version KHÁC NHAU, dù cùng một source code!

package-lock.json giải quyết vấn đề này

JSON package-lock.json (fragment) Copy

```
{
  "name": "my-app",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "my-app",
      "version": "1.0.0",
      "dependencies": { "express": "^4.19.2" }
    },
    "node_modules/express": {
      "version": "4.19.2",          // ← EXACT version, không phải ^4.19.2
      "resolved": "https://registry.npmjs.org/express/-/express-4.19.2.tgz",
      "integrity": "sha512-...",    // ← SHA hash để verify file không bị
tamper
      "dependencies": {
        "accepts": "~1.3.8",        // ← Dependencies của express cũng bị
lock!
        "body-parser": "1.20.2",
        "debug": "2.6.9",
        ...
      }
    },
    "node_modules/accepts": {
      "version": "1.3.8",           // ← Transitive dep cũng lock
      ...
    }
  }
}
```

`package-lock.json` lock cả **transitive dependencies** (dependencies của dependencies). Nhờ vậy, mọi người trong team, CI và production đều cài *y hệt* nhau.

Quy tắc node_modules và git

TEXT .gitignore Copy

```
# ─── KHÔNG commit ──────────────────────────────────────────
node_modules/          # Rất lớn, có thể tái tạo từ package-lock.json
.env                   # Chứa secrets (API keys, passwords)
.env.local
.env.production
dist/                  # Build output — tái tạo được từ source
*.log
npm-debug.log*
.DS_Store              # macOS metadata

# ─── NÊN commit ────────────────────────────────────────────
# package.json         ← Danh sách dependencies
# package-lock.json    ← Lock file để reproducible installs
# .env.example         ← Template không chứa secrets thật
```

TEXT .env.example Copy

```
# Copy file này thành .env và điền giá trị thật
# ĐỪNG bao giờ commit .env!

NODE_ENV=development
PORT=3000
MONGODB_URI=mongodb://localhost:27017/mydb
JWT_SECRET=your-secret-key-here
REDIS_URL=redis://localhost:6379
```

npm install vs npm ci

Lệnh	npm install	npm ci
Mục đích	Development, thêm/xóa packages	CI/CD, production deploy
package-lock.json	Cập nhật nếu cần	KHÔNG bao giờ thay đổi
Hành vi khi lock lệch	Tự update lock file	Báo lỗi và dừng
node_modules	Update incremental	Xóa sạch rồi cài lại
Tốc độ	Chậm hơn	Nhanh hơn (không resolve)
Dùng khi	Hàng ngày khi dev	GitHub Actions, Docker build

YAML .github/workflows/ci.yml Copy

```yaml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'         # Cache node_modules giữa các runs
      - run: npm ci            # ← Dùng ci, không dùng install!
      - run: npm test
```

7.5 npm Scripts — Automation

Thay vì ghi nhớ các lệnh dài, đặt chúng vào `scripts` trong `package.json`. npm scripts chạy trong một shell environment đặc biệt nơi `node_modules/.bin` được thêm vào PATH — nên bạn có thể dùng tên binary trực tiếp.

Scripts cơ bản

JSON package.json – scripts section Copy

```json
{
  "scripts": {
    // ── Built-in scripts (không cần "run") ──────────────────────
    "start": "node src/index.js",      // npm start
    "test": "jest",                    // npm test
    "stop": "...",                     // npm stop
    "restart": "...",                  // npm restart

    // ── Custom scripts (cần "npm run ") ─────────────────────────
    "dev": "nodemon src/index.js",     // npm run dev
    "dev:debug": "node --inspect src/index.js", // npm run dev:debug

    // ── Build ────────────────────────────────────────────────────
    "build": "tsc --project tsconfig.json",
    "build:watch": "tsc --watch",

    // ── Lint & format ────────────────────────────────────────────
    "lint": "eslint src/ --ext .js,.ts",
    "lint:fix": "eslint src/ --fix",
    "format": "prettier --write \"src/**/*.{js,ts,json}\"",

    // ── Database ─────────────────────────────────────────────────
    "db:seed": "node scripts/seed.js",
    "db:migrate": "node scripts/migrate.js",

    // ── Kết hợp nhiều lệnh (cross-platform dùng npm-run-all) ─────
    "check": "npm run lint && npm run test",
    "clean": "rimraf dist/ node_modules/.cache/"
  }
}
```

pre và post hooks

npm tự động chạy script có tên `pre<scriptname>` trước và `post<scriptname>` sau:

JSON — package.json — pre/post hooks — Copy

```json
{
  "scripts": {
    "pretest": "npm run lint",       // Chạy TRƯỚC npm test
    "test": "jest",
    "posttest": "npm run coverage",  // Chạy SAU npm test thành công

    "prebuild": "npm run clean",     // Dọn dẹp trước khi build
    "build": "tsc",
    "postbuild": "node scripts/copy-assets.js", // Copy assets sau build

    "preinstall": "node check-node-version.js",
    "postinstall": "node scripts/setup.js"
    // postinstall chạy sau npm install — dùng để setup, generate files ...
  }
}
```

SHELL — Chạy npm test — thứ tự thực thi — Copy

```shell
npm test
# 1. Chạy pretest (npm run lint)
# 2. Chạy test (jest)
# 3. Nếu test thành công → chạy posttest
# Nếu bước nào fail → dừng ngay, không chạy bước sau
```

Environment variables trong scripts

JSON package.json – env vars Copy

```json
{
  "scripts": {
    // ── Set trực tiếp (KHÔNG cross-platform – chỉ Unix/Mac) ──────────
    "test:unix": "NODE_ENV=test jest",

    // ── cross-env: chạy được trên mọi OS (Windows, Mac, Linux) ──────
    "test": "cross-env NODE_ENV=test jest",
    "dev": "cross-env NODE_ENV=development PORT=3001 nodemon src/index.js",
    "build:prod": "cross-env NODE_ENV=production npm run build"
  }
}

// Cài cross-env:
// npm install --save-dev cross-env
```

npx — Chạy package mà không cần cài global

SHELL npx usage Copy

```shell
# Chạy package mà không cài global
npx create-react-app my-app      # Tải tạm thời, chạy, xóa
npx prisma migrate dev           # Chạy prisma từ devDependencies
npx jest --watchAll              # Chạy jest local (không global)

# ── Tại sao dùng npx thay vì cài global? ──────────────────────
# Global:
npm install -g create-react-app  # Cài vào hệ thống
create-react-app my-app          # Có thể outdated!

# npx:
npx create-react-app@latest my-app  # Luôn dùng version mới nhất
# Không làm ô nhiễm global npm, mỗi lần chạy là version đúng
```

Ví dụ thực tế: Scripts cho Express API project

JSON package.json Copy

```
{
  "name": "my-express-api",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "start":        "node dist/index.js",
    "dev":          "cross-env NODE_ENV=development nodemon --watch src --
ext ts src/index.ts",
    "build":        "tsc --project tsconfig.json",
    "prebuild":     "rimraf dist/",
    "test":         "cross-env NODE_ENV=test jest --forceExit",
    "test:watch":   "jest --watchAll",
    "test:cov":     "jest --coverage --coverageDirectory=coverage",
    "lint":         "eslint src/ --ext .ts",
    "lint:fix":     "eslint src/ --ext .ts --fix",
    "format":       "prettier --write \"src/**/*.ts\"",
    "typecheck":    "tsc --noEmit",
    "validate":     "npm run typecheck && npm run lint && npm test",
    "db:seed":      "cross-env NODE_ENV=development ts-node
scripts/seed.ts",
    "docker:up":    "docker-compose up -d",
    "docker:down":  "docker-compose down"
  },
  "dependencies": {
    "express": "^4.19.2",
    "mongoose": "^8.3.1",
    "dotenv": "^16.4.5",
    "zod": "^3.23.4"
  },
  "devDependencies": {
    "typescript": "^5.4.5",
    "ts-node": "^10.9.2",
    "nodemon": "^3.1.0",
    "jest": "^29.7.0",
    "ts-jest": "^29.1.4",
    "@types/express": "^4.17.21",
    "@types/node": "^20.12.11",
    "eslint": "^9.3.0",
```

```
    "prettier": "^3.2.5",
    "rimraf": "^5.0.7",
    "cross-env": "^7.0.3"
  }
}
```

7.6 Tạo và Publish npm Package

npm không chỉ là nơi tải packages — bạn cũng có thể đóng góp cho cộng đồng bằng cách publish package của riêng mình. Hãy tạo một package thực tế: **format-vnd** — format số thành tiền VND.

SHELL — Tạo cấu trúc package — Copy

```
# Tạo thư mục package
mkdir format-vnd
cd format-vnd
npm init  # Điền đầy đủ thông tin

# Cài dependencies (nếu cần)
# Package này không cần dependencies nào!

# Cấu trúc thư mục:
# format-vnd/
# ├── src/
# │   └── index.js      ← Source chính
# ├── test/
# │   └── index.test.js ← Tests
# ├── package.json
# ├── .npmignore        ← Files không publish
# └── README.md
```

JAVASCRIPT src/index.js Copy

```javascript
'use strict';

/**
 * Format số thành chuỗi tiền VND
 * @param {number} amount - Số tiền
 * @param {object} [options]
 * @param {boolean} [options.symbol=true] - Thêm ký hiệu ₫ hoặc VND
 * @param {string} [options.style='symbol'] - 'symbol' (₫) hoặc 'code' (VND)
 * @returns {string}
 *
 * @example
 * formatVND(1500000)              // '1.500.000 ₫'
 * formatVND(1500000, { style: 'code' }) // '1.500.000 VND'
 * formatVND(1500000, { symbol: false }) // '1.500.000'
 */
function formatVND(amount, options = {}) {
  const { symbol = true, style = 'symbol' } = options;

  if (typeof amount !== 'number' || isNaN(amount)) {
    throw new TypeError(`formatVND: expected a number, got ${typeof
amount}`);
  }

  // Dùng Intl.NumberFormat cho format chuẩn
  const formatter = new Intl.NumberFormat('vi-VN', {
    style: 'decimal',
    minimumFractionDigits: 0,
    maximumFractionDigits: 0,
  });

  const formatted = formatter.format(Math.abs(amount));
  const prefix = amount < 0 ? '-' : '';

  if (!symbol) return `${prefix}${formatted}`;
  if (style === 'code') return `${prefix}${formatted} VND`;
  return `${prefix}${formatted} ₫`;
}
```

```
/**
 * Parse chuỗi tiền VND về number
 * @param {string} str - Ví dụ: '1.500.000 ₫' hoặc '1500000'
 * @returns {number}
 */
function parseVND(str) {
  if (typeof str !== 'string') {
    throw new TypeError(`parseVND: expected a string, got ${typeof str}`);
  }
  // Xóa ký hiệu tiền, dấu chấm ngăn cách
  const cleaned = str
    .replace(/[₫VND\s]/g, '')
    .replace(/\./g, '');
  const num = parseInt(cleaned, 10);
  if (isNaN(num)) throw new Error(`parseVND: không thể parse "${str}"`);
  return num;
}

/**
 * So sánh hai giá trị tiền (trả về âm/0/dương như Array.sort comparator)
 */
function compareVND(a, b) {
  return a - b;
}

module.exports = { formatVND, parseVND, compareVND };
```

JAVASCRIPT test/index.test.js Copy

```
const { formatVND, parseVND } = require('../src/index');

describe('formatVND', () => {
  test('format số dương cơ bản', () => {
    expect(formatVND(1500000)).toBe('1.500.000 ₫');
    expect(formatVND(0)).toBe('0 ₫');
    expect(formatVND(999)).toBe('999 ₫');
  });

  test('format số âm', () => {
    expect(formatVND(-50000)).toBe('-50.000 ₫');
  });

  test('options: symbol=false', () => {
    expect(formatVND(1500000, { symbol: false })).toBe('1.500.000');
  });

  test('options: style=code', () => {
    expect(formatVND(1500000, { style: 'code' })).toBe('1.500.000 VND');
  });

  test('throw khi không phải number', () => {
    expect(() => formatVND('1000')).toThrow(TypeError);
    expect(() => formatVND(NaN)).toThrow(TypeError);
  });
});

describe('parseVND', () => {
  test('parse string có ký hiệu', () => {
    expect(parseVND('1.500.000 ₫')).toBe(1500000);
    expect(parseVND('1.500.000 VND')).toBe(1500000);
  });

  test('parse string số thuần', () => {
    expect(parseVND('1500000')).toBe(1500000);
  });
});
```

JSON package.json cho format-vnd Copy

```json
{
  "name": "format-vnd",
  "version": "1.0.0",
  "description": "Format và parse số tiền Việt Nam Đồng (VND)",
  "main": "src/index.js",
  "exports": {
    ".": {
      "require": "./src/index.js"
    }
  },
  "files": [
    "src/"
  ],
  "scripts": {
    "test": "jest",
    "prepublishOnly": "npm test"  // Tự động test trước khi publish
  },
  "keywords": ["vnd", "vietnam", "currency", "format", "money"],
  "author": "Nguyen Van A ",
  "license": "MIT",
  "devDependencies": {
    "jest": "^29.7.0"
  },
  "engines": {
    "node": "≥14.0.0"
  }
}
```

TEXT .npmignore Copy

```
# Files KHÔNG publish lên npm registry
test/
*.test.js
*.spec.js
.github/
.vscode/
coverage/
.env*
*.log
```

Publish lên npm registry

SHELL Publish flow Copy

```
# 1. Đăng ký tài khoản tại npmjs.com

# 2. Đăng nhập
npm login
# Username: your-username
# Password: ****
# Email: your@email.com

# 3. Kiểm tra những gì sẽ được publish
npm pack --dry-run
# Danh sách files sẽ được đóng gói

# 4. Publish
npm publish
# + format-vnd@1.0.0

# 5. Khi có thay đổi — bump version trước khi publish lại
npm version patch   # 1.0.0 → 1.0.1 (bug fix)
npm version minor   # 1.0.0 → 1.1.0 (new feature)
npm version major   # 1.0.0 → 2.0.0 (breaking change)
# Lệnh trên tự tạo git commit + tag

npm publish         # Publish version mới

# 6. Publish package scoped (namespace cá nhân)
# Đổi name thành "@username/format-vnd" trong package.json
npm publish --access public  # Scoped packages mặc định là private
```

7.7 Bảo Mật và Best Practices

npm audit — Kiểm tra vulnerability

SHELL Security audit Copy

```shell
# Kiểm tra tất cả packages có vulnerability không
npm audit
# ── Kết quả mẫu ──────────────────────────────────────────────
# found 3 vulnerabilities (1 low, 2 moderate)
#
# moderate  Prototype Pollution in lodash
# Package   lodash
# Severity  moderate
# Via       some-old-package > lodash@3.10.1
# Fix:      npm audit fix
# More info https:// ...

# Tự động fix các vulnerability có thể fix được
npm audit fix

# Fix kể cả các breaking change (dùng cẩn thận)
npm audit fix --force

# Chỉ audit, không fail (dùng trong CI script)
npm audit --audit-level=high || true
# (|| true) ngăn CI fail nếu chỉ có low/moderate issues
```

Kiểm tra package trước khi cài

Không phải package nào trên npm cũng tốt. Trước khi cài một package mới, hãy kiểm tra:

Tiêu chí	Cách kiểm tra	Red flags
Downloads/tuần	npmjs.com → package stats	< 1.000 downloads/tuần
Last publish	npmjs.com → versions tab	Không update > 2 năm
Open issues	GitHub repository	Nhiều issues bị bỏ lơ
Bundle size	bundlephobia.com	Quá lớn so với tính năng
Dependencies	`npm install --dry-run`	Kéo theo 100+ packages
License	package.json / GitHub	GPL (copyleft) cho app thương mại

Best Practices tổng hợp

SHELL Best practices cheat sheet Copy

```shell
# ✅ Luôn commit package-lock.json
git add package-lock.json

# ✅ Dùng npm ci trong CI/CD (không dùng npm install)
npm ci

# ✅ Không commit node_modules
echo "node_modules/" >> .gitignore

# ✅ Không commit .env
echo ".env" >> .gitignore

# ✅ Kiểm tra security định kỳ
npm audit

# ✅ Chỉ cài những gì bạn thực sự cần
# (Tránh "package obesity" - cài package chỉ dùng 1 function)

# ✅ Lock phiên bản Node trong package.json
# "engines": { "node": "≥20.0.0" }

# ✅ Dùng .nvmrc để team dùng cùng Node version
echo "20" > .nvmrc
# Khi pull code: nvm use (đọc .nvmrc tự động)

# ✅ Xem size thực của node_modules
du -sh node_modules/    # Linux/Mac
# dir node_modules /s   # Windows

# ✅ Tìm package lớn nhất để tối ưu
npx cost-of-modules
```

npm workspaces — Monorepo đơn giản

Từ npm v7, có thể quản lý nhiều packages trong một repo mà không cần công cụ riêng. Đây là cách setup monorepo đơn giản nhất:

TEXT — Cấu trúc monorepo — Copy

```
my-monorepo/
├── package.json          ← Root package.json (khai báo workspaces)
├── package-lock.json     ← Một lock file duy nhất cho toàn repo
├── node_modules/         ← Shared node_modules (deduped)
└── packages/
    ├── shared-utils/     ← Package dùng chung
    │   ├── package.json  (name: "@myrepo/shared-utils")
    │   └── src/index.js
    └── api/              ← API server
        ├── package.json  (name: "@myrepo/api")
        └── src/index.js
```

JSON — package.json (root) — Copy

```
{
  "name": "my-monorepo",
  "private": true,
  "workspaces": [
    "packages/*"     // Tất cả thư mục trong packages/ đều là workspace
  ],
  "scripts": {
    // Chạy script trong TẤT CẢ workspaces
    "build": "npm run build --workspaces",
    "test":  "npm test --workspaces --if-present",

    // Chạy trong workspace CỤ THỂ
    "dev:api": "npm run dev --workspace=packages/api"
  }
}
```

JSON packages/api/package.json Copy

```json
{
  "name": "@myrepo/api",
  "version": "0.1.0",
  "dependencies": {
    // Tham chiếu workspace khác – npm tự resolve local
    "@myrepo/shared-utils": "*",
    "express": "^4.19.2"
  },
  "scripts": {
    "dev": "nodemon src/index.js",
    "start": "node src/index.js"
  }
}
```

SHELL Dùng workspaces Copy

```shell
# Cài tất cả dependencies cho toàn monorepo (root directory)
npm install

# Cài package cho workspace cụ thể
npm install axios --workspace=packages/api

# Chạy script trong workspace cụ thể
npm run dev --workspace=packages/api

# npm tự động symlink @myrepo/shared-utils → packages/shared-utils
# nên require('@myrepo/shared-utils') sẽ dùng code local!
```

📚 Tóm tắt chương 7

- **npm** gồm 3 thứ: CLI tool, registry (npmjs.com), website — `yarn` và `pnpm` là alternatives
- **package.json** chứa metadata, dependencies và scripts; `private: true` ngăn publish nhầm

- **dependencies** cần cho production; **devDependencies** chỉ cho development/build; **peerDependencies** host phải tự cài
- **Semver**: `^` cho phép minor+patch, `~` chỉ cho phép patch, exact version lock chặt nhất
- **package-lock.json** lock exact version của mọi transitive dep — luôn commit file này
- Dùng **npm ci** trong CI/CD; dùng **npm install** khi development
- **npm scripts** hỗ trợ pre/post hooks, chạy binaries từ node_modules/.bin tự động
- **npm audit** kiểm tra vulnerability — chạy định kỳ và trong CI
- **npm workspaces** quản lý monorepo mà không cần công cụ riêng

🏋️ Bài tập thực hành

DỄ

7.1 Khám phá npm

Tạo project mới, cài 3 packages: `chalk` (in màu terminal), `dayjs` (format ngày), `nanoid` (generate ID ngắn). Viết script `npm run info` in ra: Node version, npm version, danh sách top-level packages đã cài (`npm list --depth=0`), và ngày giờ hiện tại dùng dayjs. Giải thích sự khác biệt giữa `npm list` và `npm list --depth=0` .

TRUNG BÌNH

7.2 Package tiện ích vn-utils

Tạo package `vn-utils` export 3 functions: `formatCurrency(amount, currency?)` — format VND hoặc USD, `formatDate(date, format?)` — format ngày theo chuẩn DD/MM/YYYY, `slugify(str)` — chuyển "Xin Chào NodeJS" thành "xin-chao-nodejs" (bao gồm bỏ dấu tiếng Việt). Viết unit tests dùng Jest với ít nhất 3 test case mỗi function. Thêm `prepublishOnly` hook để test tự chạy trước khi publish.

KHÓ

7.3 Monorepo với npm workspaces

Setup monorepo gồm 2 packages: **packages/shared-utils** — export `formatDate()` và `validateEmail()`. **packages/api** — Express server dùng shared-utils, có route `POST /validate` nhận `{ email }` và trả về kết quả validate. Yêu cầu: (1) Root `package.json` khai báo workspaces. (2) api dùng `"@myapp/shared-utils": "*"` trong dependencies — npm symlink local. (3) Viết script ở root: `npm run dev` khởi động api, `npm test --workspaces` chạy tests của tất cả packages.

PHẦN 3 — WEB APPLICATION VỚI EXPRESS

Chương 8: Express.js — Framework Web Phổ Biến Nhất

Express biến một HTTP server thuần cồng kềnh thành vài dòng code gọn gàng. Chương này học cách xây dựng REST API hoàn chỉnh với Express — routing, params, body, và tổ chức code chuẩn.

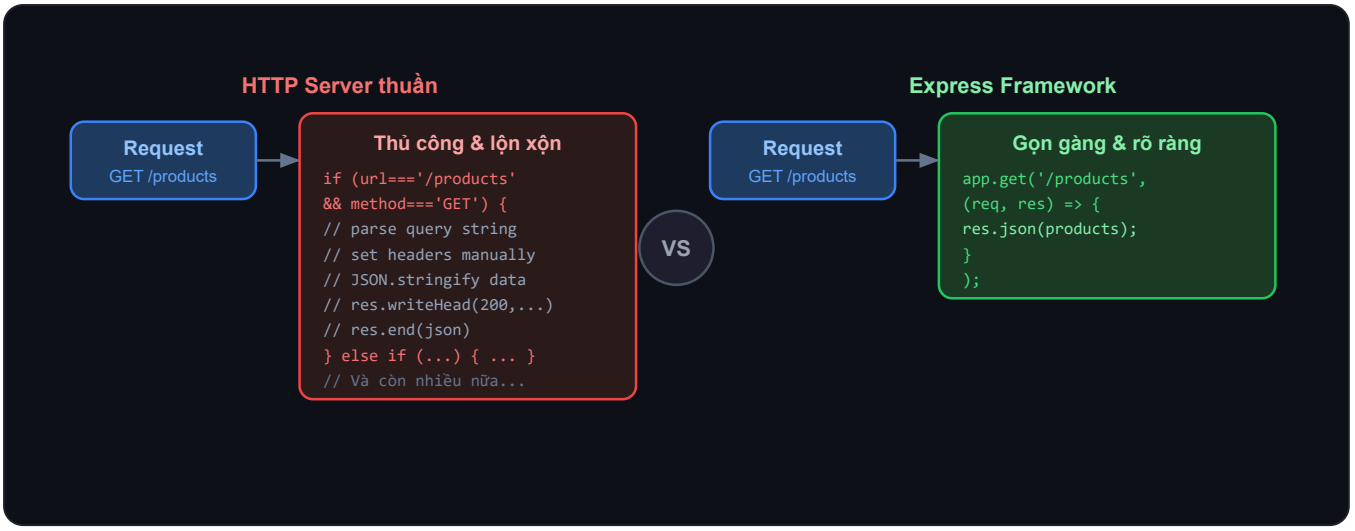
🎯 **SAU CHƯƠNG NÀY BẠN SẼ BIẾT**

- Cài đặt và cấu hình Express app từ đầu
- Định nghĩa routes với đầy đủ HTTP methods (GET, POST, PUT, PATCH, DELETE)
- Đọc route params, query strings, và request body
- Tổ chức routes gọn gàng với Express Router
- Xây dựng REST API hoàn chỉnh với CRUD operations

8.1 Tại sao cần Express?

Ở chương 6, bạn đã viết HTTP server thuần bằng module `http` của Node. Nó hoạt động, nhưng khi ứng dụng lớn hơn — nhiều routes, nhiều methods, parse body, validate data — code sẽ nhanh chóng trở thành một mớ `if/else` lộn xộn.

Hãy xem cùng một tính năng "Lấy danh sách sản phẩm" được viết theo hai cách:



Express không phải là một framework "nặng" như Django hay Rails. Nó là một **thin layer** (lớp mỏng) nằm trên module `http` của Node, cung cấp:

- **Routing** — khớp URL với handler function một cách thanh lịch
- **Middleware** — pipeline xử lý request/response
- **Request/Response helpers** — `req.params`, `req.body`, `res.json()`...
- **Extensibility** — hàng nghìn middleware từ npm

> 📌 **EXPRESS LÀ SỐ 1**
>
> Express là framework Node.js được tải xuống nhiều nhất trên npm — trên 30 triệu lần/tuần. Hầu hết các framework khác (NestJS, Fastify, Koa) đều lấy cảm hứng hoặc build on top của nó. Hiểu Express = hiểu nền tảng của hầu hết Node.js web development.

Đây là cùng một tính năng viết bằng hai cách — quan sát sự khác biệt:

JAVASCRIPT server-raw.js — HTTP thuần Copy

```
const http = require('http');
const url = require('url');

const products = [
  { id: 1, name: 'Laptop', price: 25000000 },
  { id: 2, name: 'Mouse', price: 350000 },
];

const server = http.createServer((req, res) => {
  const parsed = url.parse(req.url, true);
  const pathname = parsed.pathname;
  const method = req.method;

  // Phải tự xử lý từng route
  if (pathname === '/products' && method === 'GET') {
    res.writeHead(200, {
      'Content-Type': 'application/json',
      'X-Powered-By': 'Node.js'
    });
    res.end(JSON.stringify({ success: true, data: products }));

  } else if (pathname.match(/^\/products\/\d+$/) && method === 'GET') {
    const id = parseInt(pathname.split('/')[2]);
    const product = products.find(p => p.id === id);
    if (product) {
      res.writeHead(200, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ success: true, data: product }));
    } else {
      res.writeHead(404, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ success: false, error: 'Not found' }));
    }

  } else {
    res.writeHead(404, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'Route not found' }));
  }
});
```

```
server.listen(3000);
```

JAVASCRIPT server-express.js – Với Express Copy

```
const express = require('express');
const app = express();

const products = [
  { id: 1, name: 'Laptop', price: 25000000 },
  { id: 2, name: 'Mouse', price: 350000 },
];

// Express tự handle routing, parsing, và response helpers
app.get('/products', (req, res) => {
  res.json({ success: true, data: products });
});

app.get('/products/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const product = products.find(p => p.id === id);

  if (!product) {
    return res.status(404).json({ success: false, error: 'Not found' });
  }

  res.json({ success: true, data: product });
});

app.listen(3000, () => console.log('Server chạy tại
http://localhost:3000'));
```

8.2 Cài đặt và Hello World

Tạo project mới và cài Express:

BASH Terminal Copy

```bash
mkdir my-express-app
cd my-express-app
npm init -y
npm install express
```

Tạo file `app.js` với server Express đầu tiên:

JAVASCRIPT app.js Copy

```javascript
const express = require('express');

// Tạo Express application
const app = express();

// Cổng server lắng nghe
const PORT = process.env.PORT || 3000;

// Route đơn giản nhất
app.get('/', (req, res) => {
  res.send('Xin chào từ Express!');
});

app.get('/hello', (req, res) => {
  res.json({
    message: 'Hello, World!',
    timestamp: new Date().toISOString(),
    framework: 'Express.js'
  });
});

// Khởi động server
app.listen(PORT, () => {
  console.log(`✅ Server đang chạy tại http://localhost:${PORT}`);
  console.log(`📝 Môi trường: ${process.env.NODE_ENV || 'development'}`);
});
```

Chạy server:

```bash
node app.js
# ✅ Server đang chạy tại http://localhost:3000

# Thử với curl:
curl http://localhost:3000/hello
# {"message":"Hello, World!","timestamp":"...","framework":"Express.js"}
```

> 💡 **NODEMON CHO DEVELOPMENT**
>
> Cài `npm install -D nodemon` và thêm script `"dev": "nodemon app.js"` vào `package.json`. Sau đó chạy `npm run dev` — server tự restart mỗi khi bạn sửa code, tiết kiệm rất nhiều thời gian.

8.3 Routing — Xử Lý Các HTTP Methods

Express cung cấp một method tương ứng với mỗi HTTP method. Quy tắc CRUD rất rõ ràng:

- `app.get(path, handler)` — Đọc dữ liệu (Read)
- `app.post(path, handler)` — Tạo mới (Create)
- `app.put(path, handler)` — Cập nhật toàn bộ (Replace)
- `app.patch(path, handler)` — Cập nhật một phần (Partial update)
- `app.delete(path, handler)` — Xóa (Delete)
- `app.all(path, handler)` — Khớp với mọi HTTP method

Ví dụ đầy đủ CRUD cho resource "products" dùng in-memory array (chưa dùng database):

JAVASCRIPT app.js — CRUD đầy đủ Copy

```
const express = require('express');
const app = express();

// Middleware quan trọng: parse JSON request body
app.use(express.json());

// "Database" tạm thời trong bộ nhớ
let products = [
  { id: 1, name: 'Laptop Dell XPS 13', price: 28000000, category:
'electronics', stock: 15 },
  { id: 2, name: 'Chuột Logitech MX Master', price: 1800000, category:
'electronics', stock: 50 },
  { id: 3, name: 'Bàn phím Keychron K2', price: 2200000, category:
'electronics', stock: 30 },
];
let nextId = 4; // Auto-increment ID

// ── GET /products ── Lấy danh sách (có thể filter)
app.get('/products', (req, res) => {
  let result = [...products];

  // Filter theo category: GET /products?category=electronics
  if (req.query.category) {
    result = result.filter(p => p.category === req.query.category);
  }

  // Filter theo giá tối đa: GET /products?maxPrice=5000000
  if (req.query.maxPrice) {
    result = result.filter(p => p.price <= parseInt(req.query.maxPrice));
  }

  res.json({
    success: true,
    total: result.length,
    data: result
  });
});
```

```
// ── GET /products/:id ── Lấy 1 sản phẩm
app.get('/products/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const product = products.find(p => p.id === id);

  if (!product) {
    return res.status(404).json({
      success: false,
      error: `Không tìm thấy sản phẩm với id=${id}`
    });
  }

  res.json({ success: true, data: product });
});

// ── POST /products ── Tạo sản phẩm mới
app.post('/products', (req, res) => {
  const { name, price, category, stock } = req.body;

  // Validate required fields
  if (!name || price === undefined || !category) {
    return res.status(400).json({
      success: false,
      error: 'Thiếu thông tin bắt buộc: name, price, category'
    });
  }

  const newProduct = {
    id: nextId++,
    name,
    price: Number(price),
    category,
    stock: stock ?? 0,
    createdAt: new Date().toISOString()
  };

  products.push(newProduct);

  // 201 Created
  res.status(201).json({
    success: true,
```

```
    message: 'Tạo sản phẩm thành công',
    data: newProduct
  });
});

// ── PUT /products/:id ── Cập nhật toàn bộ (replace)
app.put('/products/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = products.findIndex(p => p.id === id);

  if (index === -1) {
    return res.status(404).json({ success: false, error: 'Không tìm thấy sản
phẩm' });
  }

  const { name, price, category, stock } = req.body;
  if (!name || price === undefined || !category) {
    return res.status(400).json({ success: false, error: 'PUT yêu cầu đầy đủ
tất cả fields' });
  }

  products[index] = { id, name, price: Number(price), category, stock: stock
?? 0 };
  res.json({ success: true, data: products[index] });
});

// ── PATCH /products/:id ── Cập nhật một phần
app.patch('/products/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = products.findIndex(p => p.id === id);

  if (index === -1) {
    return res.status(404).json({ success: false, error: 'Không tìm thấy sản
phẩm' });
  }

  // Merge dữ liệu cũ với dữ liệu mới
  products[index] = { ...products[index], ...req.body };
  res.json({ success: true, data: products[index] });
});
```

```
// ── DELETE /products/:id ── Xóa sản phẩm
app.delete('/products/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = products.findIndex(p => p.id === id);

  if (index === -1) {
    return res.status(404).json({ success: false, error: 'Không tìm thấy sản
phẩm' });
  }

  const deleted = products.splice(index, 1)[0];
  res.json({
    success: true,
    message: `Đã xóa sản phẩm "${deleted.name}"`,
    data: deleted
  });
});

app.listen(3000, () => console.log('Server tại http://localhost:3000'));
```

> 💡 **MULTIPLE CALLBACKS TRÊN 1 ROUTE**
>
> Express cho phép bạn đặt nhiều handler function cho 1 route. Mỗi function phải gọi `next()` để chuyển sang function tiếp theo — đây chính là cơ chế middleware (Chương 9):
>
> `app.get('/products', checkAuth, logRequest, (req, res) => { ... })`

8.4 Request Object — Đọc Dữ Liệu Từ Client

Object `req` (request) chứa tất cả thông tin mà client gửi lên. Đây là những thuộc tính quan trọng nhất:

JAVASCRIPT request-info.js — Debug tool Copy

```javascript
// Route debug: hiển thị mọi thứ trong request
// Truy cập: GET /debug/info?page=2&limit=10
app.get('/debug/info', (req, res) => {
  res.json({
    // ── Route Parameters ──────────────────────
    // /users/:userId/posts/:postId → req.params
    params: req.params,

    // ── Query String ──────────────────────────
    // /debug/info?page=2&limit=10&tag=nodejs
    query: req.query,
    // req.query.page → '2' (luôn là string!)
    // Cần convert: parseInt(req.query.page)

    // ── Request Body ──────────────────────────
    // Cần middleware: app.use(express.json())
    body: req.body,

    // ── HTTP Headers ──────────────────────────
    headers: {
      contentType: req.get('Content-Type'),     // 'application/json'
      authorization: req.get('Authorization'),  // 'Bearer abc123'
      userAgent: req.get('User-Agent'),
      accept: req.get('Accept'),
    },

    // ── Request Meta ──────────────────────────
    method: req.method,       // 'GET'
    url: req.url,             // '/debug/info?page=2&limit=10'
    path: req.path,           // '/debug/info' (không có query string)
    hostname: req.hostname,   // 'localhost'
    ip: req.ip,               // '::1' hoặc '127.0.0.1'
    protocol: req.protocol,   // 'http' hoặc 'https'
    secure: req.secure,       // true nếu HTTPS

    // ── Cookies ───────────────────────────────
    // Cần middleware 'cookie-parser'
    cookies: req.cookies,
```

```
  });
});

// ── Ví dụ đọc route params ──
// Route: /users/:userId/posts/:postId
app.get('/users/:userId/posts/:postId', (req, res) => {
  const { userId, postId } = req.params;
  // req.params = { userId: '42', postId: '7' }
  // Lưu ý: params luôn là STRING, cần parseInt nếu muốn số

  res.json({
    message: `Bài viết ${postId} của user ${userId}`,
    userId: parseInt(userId),  // convert sang number
    postId: parseInt(postId),
  });
});

// ── Ví dụ đọc query string ──
// GET /search?q=nodejs&page=2&limit=10&sort=newest
app.get('/search', (req, res) => {
  const {
    q = '',           // từ khóa tìm kiếm
    page = '1',       // trang hiện tại
    limit = '10',     // số kết quả mỗi trang
    sort = 'newest'   // cách sắp xếp
  } = req.query;

  res.json({
    keyword: q,
    page: parseInt(page),
    limit: Math.min(parseInt(limit), 100), // Giới hạn tối đa 100
    sort,
    results: [] // Giả sử data ở đây
  });
});
```

⚠ PARAMS VÀ QUERY LUÔN LÀ STRING

`req.params.id` và `req.query.page` luôn trả về **string**, ngay cả khi client gửi số. Bạn phải tự convert: `parseInt(req.params.id)` hoặc `Number(req.query.limit)`. Nếu không, so sánh như `product.id ===`

> `req.params.id` sẽ luôn `false` vì `1 !== '1'`.

8.5 Response Object — Gửi Dữ Liệu Về Client

Object `res` cung cấp nhiều phương thức để gửi response. Quan trọng nhất:

JAVASCRIPT response-methods.js Copy

```
// ── res.json() ── Gửi JSON (phổ biến nhất)
// Tự set Content-Type: application/json
// Tự JSON.stringify
app.get('/json-example', (req, res) => {
  res.json({ name: 'Alice', age: 30 });
});

// ── res.status().json() ── Chaining với status code
app.get('/not-found-example', (req, res) => {
  res.status(404).json({ error: 'Không tìm thấy' });
  // Hoặc:
  // res.status(201).json({ message: 'Tạo thành công' })
  // res.status(400).json({ error: 'Dữ liệu không hợp lệ' })
  // res.status(500).json({ error: 'Lỗi server' })
});

// ── res.send() ── Gửi nhiều loại dữ liệu
app.get('/send-example', (req, res) => {
  res.send('Hello String');          // text/html
  // res.send(Buffer.from('data'))   // application/octet-stream
  // res.send({ key: 'value' })      // application/json (nhưng dùng
res.json() rõ hơn)
});

// ── res.sendFile() ── Gửi file
const path = require('path');
app.get('/download', (req, res) => {
  res.sendFile(path.join(__dirname, 'files', 'report.pdf'));
});

// ── res.redirect() ── Chuyển hướng
app.get('/old-path', (req, res) => {
  res.redirect('/new-path');              // 302 (mặc định)
  // res.redirect(301, '/permanent');    // 301 Permanent Redirect
});

// ── res.set() ── Set custom header
app.get('/custom-header', (req, res) => {
```

```
  res.set('X-Custom-Header', 'my-value');
  res.set({
    'Cache-Control': 'no-cache',
    'X-Rate-Limit': '100'
  });
  res.json({ ok: true });
});

// ── Cookie ──
app.get('/set-cookie', (req, res) => {
  res.cookie('token', 'abc123', {
    httpOnly: true,    // Không thể đọc bởi JavaScript
    secure: true,      // Chỉ gửi qua HTTPS
    maxAge: 7 * 24 * 60 * 60 * 1000  // 7 ngày (milliseconds)
  });
  res.json({ message: 'Cookie đã được set' });
});

app.get('/clear-cookie', (req, res) => {
  res.clearCookie('token');
  res.json({ message: 'Cookie đã được xóa' });
});
```

Trong thực tế, bạn nên tạo một helper function để trả response nhất quán trên toàn bộ API:

JAVASCRIPT helpers/apiResponse.js Copy

```
/**
 * Helper tạo response API nhất quán
 *
 * Ví dụ dùng:
 *   apiResponse(res, 200, { users }, 'Lấy danh sách thành công')
 *   apiResponse(res, 404, null, 'Không tìm thấy', false)
 */
function apiResponse(res, statusCode, data = null, message = '', success =
true) {
  const response = {
    success,
    message,
    ...(data !== null && { data })
  };

  return res.status(statusCode).json(response);
}

// Shortcuts hay dùng
const ok = (res, data, message = 'Thành công') =>
  apiResponse(res, 200, data, message);

const created = (res, data, message = 'Tạo thành công') =>
  apiResponse(res, 201, data, message);

const notFound = (res, message = 'Không tìm thấy') =>
  apiResponse(res, 404, null, message, false);

const badRequest = (res, message = 'Dữ liệu không hợp lệ') =>
  apiResponse(res, 400, null, message, false);

const serverError = (res, message = 'Lỗi server nội bộ') =>
  apiResponse(res, 500, null, message, false);

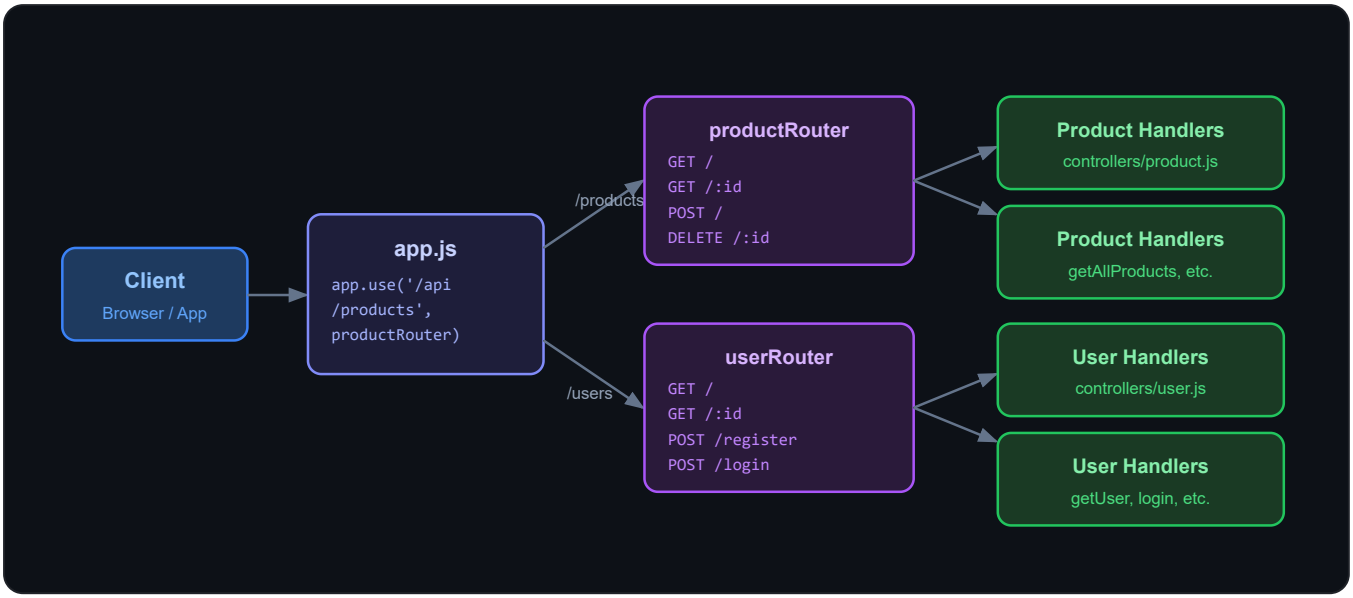
module.exports = { apiResponse, ok, created, notFound, badRequest,
serverError };

// ── Cách dùng trong route ──
```

```
// const { ok, notFound } = require('./helpers/apiResponse');
// app.get('/users/:id', (req, res) => {
//   const user = findUser(req.params.id);
//   if (!user) return notFound(res, 'Không tìm thấy người dùng');
//   return ok(res, user, 'Lấy thông tin thành công');
// });
```

8.6 Express Router — Tổ Chức Code Theo Module

Khi ứng dụng lớn dần, không nên nhét tất cả routes vào một file `app.js`. Express cung cấp `express.Router()` để tách routes thành các module riêng biệt.



Cấu trúc thư mục được đề xuất:

TEXT — Cấu trúc project — Copy

```
my-express-app/
├── app.js              ← Entry point, mount routers
├── routes/
│   ├── products.js     ← Router cho /api/products
│   └── users.js        ← Router cho /api/users
├── package.json
└── node_modules/
```

JAVASCRIPT routes/products.js Copy

```
const express = require('express');
const router = express.Router();

// "Database" in-memory
let products = [
  { id: 1, name: 'Laptop', price: 28000000 },
  { id: 2, name: 'Mouse', price: 350000 },
];
let nextId = 3;

// Lưu ý: path ở đây là RELATIVE so với mount point
// Khi mount tại '/api/products', thì:
//   router.get('/')       → GET /api/products
//   router.get('/:id')    → GET /api/products/:id

router.get('/', (req, res) => {
  res.json({ success: true, data: products });
});

router.get('/:id', (req, res) => {
  const product = products.find(p => p.id === parseInt(req.params.id));
  if (!product) return res.status(404).json({ success: false, error: 'Không
tìm thấy' });
  res.json({ success: true, data: product });
});

router.post('/', (req, res) => {
  const { name, price } = req.body;
  if (!name || price === undefined) {
    return res.status(400).json({ success: false, error: 'Thiếu name hoặc
price' });
  }
  const product = { id: nextId++, name, price: Number(price) };
  products.push(product);
  res.status(201).json({ success: true, data: product });
});

router.put('/:id', (req, res) => {
```

```
  const idx = products.findIndex(p => p.id === parseInt(req.params.id));
  if (idx === -1) return res.status(404).json({ success: false, error:
'Không tìm thấy' });
  products[idx] = { ...products[idx], ...req.body };
  res.json({ success: true, data: products[idx] });
});

router.delete('/:id', (req, res) => {
  const idx = products.findIndex(p => p.id === parseInt(req.params.id));
  if (idx === -1) return res.status(404).json({ success: false, error:
'Không tìm thấy' });
  const [deleted] = products.splice(idx, 1);
  res.json({ success: true, data: deleted });
});

module.exports = router;
```

JAVASCRIPT routes/users.js Copy

```
const express = require('express');
const router = express.Router();

let users = [
  { id: 1, name: 'Nguyễn Văn An', email: 'an@example.com', role: 'admin' },
  { id: 2, name: 'Trần Thị Bình', email: 'binh@example.com', role: 'user' },
];

router.get('/', (req, res) => {
  // Lọc theo role: GET /api/users?role=admin
  const { role } = req.query;
  const result = role ? users.filter(u => u.role === role) : users;
  res.json({ success: true, data: result });
});

router.get('/:id', (req, res) => {
  const user = users.find(u => u.id === parseInt(req.params.id));
  if (!user) return res.status(404).json({ success: false, error: 'Không tìm
thấy user' });
  res.json({ success: true, data: user });
});

router.post('/register', (req, res) => {
  const { name, email } = req.body;
  if (!name || !email) {
    return res.status(400).json({ success: false, error: 'Cần name và email'
});
  }
  const exists = users.find(u => u.email === email);
  if (exists) {
    return res.status(409).json({ success: false, error: 'Email đã tồn tại'
});
  }
  const user = { id: users.length + 1, name, email, role: 'user' };
  users.push(user);
  res.status(201).json({ success: true, data: user });
});
```

```
module.exports = router;
```

JAVASCRIPT app.js – Entry point Copy

```
const express = require('express');
const app = express();

// ── Global Middleware ──
app.use(express.json());                        // Parse JSON body
app.use(express.urlencoded({ extended: true })); // Parse form data
app.use(express.static('public'));              // Serve static files từ thư
mục public/

// ── Import Routes ──
const productRoutes = require('./routes/products');
const userRoutes = require('./routes/users');

// ── Mount Routes ──
// productRoutes xử lý tất cả routes bắt đầu bằng /api/products
app.use('/api/products', productRoutes);
// userRoutes xử lý tất cả routes bắt đầu bằng /api/users
app.use('/api/users', userRoutes);

// ── Health Check ──
app.get('/health', (req, res) => {
  res.json({ status: 'OK', uptime: process.uptime() });
});

// ── 404 Handler (phải đặt SAU tất cả routes) ──
app.use((req, res) => {
  res.status(404).json({
    success: false,
    error: `Route ${req.method} ${req.path} không tồn tại`
  });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server tại http://localhost:${PORT}`);
  console.log('Routes:');
  console.log('  GET    /api/products');
  console.log('  GET    /api/products/:id');
```

```
  console.log('  POST   /api/products');
  console.log('  PUT    /api/products/:id');
  console.log('  DELETE /api/products/:id');
  console.log('  GET    /api/users');
  console.log('  POST   /api/users/register');
});
```

8.7 Built-in Middleware

Express đi kèm một số middleware tích hợp sẵn. Thứ tự đặt middleware RẤT QUAN TRỌNG — chúng phải được đặt TRƯỚC các routes cần dùng đến chúng:

JAVASCRIPT app.js — Built-in middleware Copy

```
const express = require('express');
const app = express();

// ── 1. express.json() ──────────────────────────────────────
// Parse request body khi Content-Type: application/json
// Không có middleware này, req.body sẽ là undefined với JSON request
app.use(express.json({
  limit: '10mb',     // Giới hạn kích thước body (mặc định 100kb)
  strict: true       // Chỉ chấp nhận object và array (không chấp nhận
primitive)
}));

// ── 2. express.urlencoded() ────────────────────────────────
// Parse request body khi Content-Type: application/x-www-form-urlencoded
// Dùng khi submit HTML form truyền thống
app.use(express.urlencoded({
  extended: true,  // true = dùng qs library (hỗ trợ nested objects)
                   // false = dùng querystring library (đơn giản hơn)
  limit: '10mb'
}));

// ── 3. express.static() ────────────────────────────────────
// Serve static files (HTML, CSS, JS, images) từ thư mục chỉ định
// Truy cập: GET /logo.png → đọc file public/logo.png
app.use(express.static('public', {
  maxAge: '1d',      // Cache 1 ngày trên client
  etag: true,        // Bật ETag cho caching thông minh
  index: 'index.html' // File mặc định khi truy cập thư mục
}));

// Serve từ nhiều thư mục:
// app.use('/uploads', express.static('uploads'));
// app.use('/assets', express.static('assets'));

// Routes đặt SAU middleware
app.get('/api/test', (req, res) => {
  // req.body đã được parse, sẵn sàng dùng
  res.json({ received: req.body });
```

```
});

app.listen(3000);
```

> ⚠️ **THỨ TỰ MIDDLEWARE QUAN TRỌNG**
>
> Middleware được thực thi theo thứ tự bạn khai báo. Đặt `express.json()` TRƯỚC route handlers. Nếu đặt sau, `req.body` sẽ là `undefined` khi route handler chạy, vì JSON chưa được parse.

8.8 Dự án thực hành: API Blog đơn giản

Áp dụng tất cả kiến thức trong chương để xây dựng một Blog API hoàn chỉnh. API này dùng in-memory storage (chưa có database) nhưng có đầy đủ các tính năng cơ bản.

JAVASCRIPT routes/posts.js — Blog API đầy đủ Copy

```
const express = require('express');
const router = express.Router();

// ── Data Store (in-memory) ──
let posts = [
  {
    id: 1,
    title: 'Express.js là gì?',
    content: 'Express là web framework cho Node.js ... ',
    author: 'Alice',
    tags: ['nodejs', 'express', 'backend'],
    createdAt: '2024-01-15T08:00:00.000Z',
    updatedAt: '2024-01-15T08:00:00.000Z',
  },
  {
    id: 2,
    title: 'REST API Best Practices',
    content: 'Thiết kế REST API tốt cần chú ý đến ... ',
    author: 'Bob',
    tags: ['api', 'rest', 'backend'],
    createdAt: '2024-01-20T10:30:00.000Z',
    updatedAt: '2024-01-20T10:30:00.000Z',
  },
  {
    id: 3,
    title: 'JavaScript Async/Await',
    content: 'Async/await giúp code bất đồng bộ trở nên ... ',
    author: 'Alice',
    tags: ['javascript', 'async'],
    createdAt: '2024-02-01T14:00:00.000Z',
    updatedAt: '2024-02-01T14:00:00.000Z',
  },
];

let comments = [
  { id: 1, postId: 1, author: 'Charlie', content: 'Bài viết rất hay!',
createdAt: '2024-01-16T09:00:00.000Z' },
  { id: 2, postId: 1, author: 'Diana', content: 'Cảm ơn, rất hữu ích.',
```

```
createdAt: '2024-01-17T11:00:00.000Z' },
  { id: 3, postId: 2, author: 'Eve', content: 'Tôi có câu hỏi về ... ',
createdAt: '2024-01-21T15:00:00.000Z' },
];

let nextPostId = 4;
let nextCommentId = 4;

// ── GET /api/posts ── Lấy danh sách, hỗ trợ filter
router.get('/', (req, res) => {
  const { author, tag, page = '1', limit = '10' } = req.query;

  let result = [...posts];

  // Filter theo author
  if (author) {
    result = result.filter(p =>
      p.author.toLowerCase() === author.toLowerCase()
    );
  }

  // Filter theo tag
  if (tag) {
    result = result.filter(p => p.tags.includes(tag));
  }

  // Sắp xếp mới nhất trước
  result.sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt));

  // Pagination
  const pageNum = Math.max(1, parseInt(page));
  const limitNum = Math.min(50, Math.max(1, parseInt(limit)));
  const startIdx = (pageNum - 1) * limitNum;
  const paginatedResult = result.slice(startIdx, startIdx + limitNum);

  res.json({
    success: true,
    data: paginatedResult,
    pagination: {
      total: result.length,
      page: pageNum,
```

```
      limit: limitNum,
      totalPages: Math.ceil(result.length / limitNum),
      hasNext: startIdx + limitNum < result.length,
      hasPrev: pageNum > 1,
    }
  });
});

// ── GET /api/posts/search ── Tìm kiếm theo title hoặc content
// Phải đặt TRƯỚC route /:id vì 'search' sẽ bị hiểu là :id
router.get('/search', (req, res) => {
  const { q } = req.query;
  if (!q || q.trim() === '') {
    return res.status(400).json({ success: false, error: 'Cần cung cấp từ
khóa tìm kiếm ?q=' });
  }

  const keyword = q.toLowerCase().trim();
  const results = posts.filter(p =>
    p.title.toLowerCase().includes(keyword) ||
    p.content.toLowerCase().includes(keyword) ||
    p.tags.some(t => t.includes(keyword))
  );

  res.json({ success: true, total: results.length, keyword: q, data: results
});
});

// ── GET /api/posts/stats ── Thống kê
router.get('/stats', (req, res) => {
  const postWithMostComments = posts.reduce((best, post) => {
    const count = comments.filter(c => c.postId === post.id).length;
    const bestCount = comments.filter(c => c.postId === best.id).length;
    return count > bestCount ? post : best;
  }, posts[0]);

  const authorStats = posts.reduce((acc, post) => {
    acc[post.author] = (acc[post.author] || 0) + 1;
    return acc;
  }, {});
```

```
  res.json({
    success: true,
    data: {
      totalPosts: posts.length,
      totalComments: comments.length,
      postWithMostComments: postWithMostComments
        ? { ...postWithMostComments, commentCount: comments.filter(c =>
c.postId === postWithMostComments.id).length }
        : null,
      postsByAuthor: authorStats,
      allTags: [...new Set(posts.flatMap(p => p.tags))].sort(),
    }
  });
});

// ── GET /api/posts/:id ── Chi tiết bài viết
router.get('/:id', (req, res) => {
  const id = parseInt(req.params.id);
  if (isNaN(id)) {
    return res.status(400).json({ success: false, error: 'ID phải là số' });
  }

  const post = posts.find(p => p.id === id);
  if (!post) {
    return res.status(404).json({ success: false, error: `Không tìm thấy bài
viết id=${id}` });
  }

  const postComments = comments.filter(c => c.postId === id);

  res.json({
    success: true,
    data: {
      ...post,
      commentCount: postComments.length
    }
  });
});

// ── POST /api/posts ── Tạo bài viết mới
router.post('/', (req, res) => {
```

```
  const { title, content, author, tags = [] } = req.body;

  // Validate
  const errors = [];
  if (!title || title.trim() === '') errors.push('title là bắt buộc');
  if (!content || content.trim() === '') errors.push('content là bắt buộc');
  if (!author || author.trim() === '') errors.push('author là bắt buộc');
  if (!Array.isArray(tags)) errors.push('tags phải là array');

  if (errors.length > 0) {
    return res.status(400).json({ success: false, errors });
  }

  const now = new Date().toISOString();
  const newPost = {
    id: nextPostId++,
    title: title.trim(),
    content: content.trim(),
    author: author.trim(),
    tags: tags.map(t => String(t).toLowerCase().trim()),
    createdAt: now,
    updatedAt: now,
  };

  posts.push(newPost);
  res.status(201).json({ success: true, data: newPost });
});

// ── PUT /api/posts/:id — Cập nhật toàn bộ bài viết
router.put('/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const idx = posts.findIndex(p => p.id === id);
  if (idx === -1) return res.status(404).json({ success: false, error:
'Không tìm thấy' });

  const { title, content, author, tags } = req.body;
  if (!title || !content || !author) {
    return res.status(400).json({ success: false, error: 'Cần đầy đủ title,
content, author' });
  }
```

```
  posts[idx] = {
    ...posts[idx],
    title: title.trim(),
    content: content.trim(),
    author: author.trim(),
    tags: Array.isArray(tags) ? tags : posts[idx].tags,
    updatedAt: new Date().toISOString(),
  };

  res.json({ success: true, data: posts[idx] });
});

// ── DELETE /api/posts/:id — Xóa bài viết
router.delete('/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const idx = posts.findIndex(p => p.id === id);
  if (idx === -1) return res.status(404).json({ success: false, error:
'Không tìm thấy' });

  const [deleted] = posts.splice(idx, 1);
  // Xóa cả comments liên quan
  const deletedComments = comments.filter(c => c.postId === id).length;
  comments = comments.filter(c => c.postId !== id);

  res.json({
    success: true,
    message: `Đã xóa bài viết và ${deletedComments} comments liên quan`,
    data: deleted
  });
});

// ── GET /api/posts/:id/comments — Lấy comments của bài viết
router.get('/:id/comments', (req, res) => {
  const id = parseInt(req.params.id);
  const post = posts.find(p => p.id === id);
  if (!post) return res.status(404).json({ success: false, error: 'Bài viết
không tồn tại' });

  const postComments = comments.filter(c => c.postId === id);
  res.json({ success: true, total: postComments.length, data: postComments
});
```

```
});

// ── POST /api/posts/:id/comments ── Thêm comment
router.post('/:id/comments', (req, res) => {
  const id = parseInt(req.params.id);
  const post = posts.find(p => p.id === id);
  if (!post) return res.status(404).json({ success: false, error: 'Bài viết
không tồn tại' });

  const { author, content } = req.body;
  if (!author || !content) {
    return res.status(400).json({ success: false, error: 'Cần author và
content' });
  }

  const comment = {
    id: nextCommentId++,
    postId: id,
    author: author.trim(),
    content: content.trim(),
    createdAt: new Date().toISOString()
  };
  comments.push(comment);
  res.status(201).json({ success: true, data: comment });
});

module.exports = router;
```

JAVASCRIPT app.js — Blog API entry point Copy

```
const express = require('express');
const app = express();

app.use(express.json());
app.use(express.urlencoded({ extended: true }));

const postRoutes = require('./routes/posts');
app.use('/api/posts', postRoutes);

// Nếu đã làm routes/users.js:
// const userRoutes = require('./routes/users');
// app.use('/api/users', userRoutes);

app.get('/', (req, res) => {
  res.json({
    name: 'Blog API',
    version: '1.0.0',
    endpoints: {
      posts: {
        'GET /api/posts': 'Danh sách bài viết (có filter & pagination)',
        'GET /api/posts/search?q=keyword': 'Tìm kiếm bài viết',
        'GET /api/posts/stats': 'Thống kê blog',
        'GET /api/posts/:id': 'Chi tiết bài viết',
        'POST /api/posts': 'Tạo bài viết mới',
        'PUT /api/posts/:id': 'Cập nhật bài viết',
        'DELETE /api/posts/:id': 'Xóa bài viết',
        'GET /api/posts/:id/comments': 'Comments của bài viết',
        'POST /api/posts/:id/comments': 'Thêm comment',
      }
    }
  });
});

app.use((req, res) => {
  res.status(404).json({ success: false, error: 'Route không tồn tại' });
});
```

```
app.listen(3000, () => console.log('Blog API tại http://localhost:3000'));
```

Kiểm tra API bằng curl:

BASH — Testing với curl — Copy

```
# Lấy tất cả bài viết
curl http://localhost:3000/api/posts

# Tạo bài viết mới
curl -X POST http://localhost:3000/api/posts \
  -H "Content-Type: application/json" \
  -d '{"title":"Bài mới","content":"Nội dung bài
viết","author":"Alice","tags":["nodejs"]}'

# Tìm kiếm
curl "http://localhost:3000/api/posts/search?q=express"

# Xem comments của bài viết 1
curl http://localhost:3000/api/posts/1/comments

# Thêm comment
curl -X POST http://localhost:3000/api/posts/1/comments \
  -H "Content-Type: application/json" \
  -d '{"author":"Bob","content":"Rất hay!"}'

# Thống kê
curl http://localhost:3000/api/posts/stats
```

💡 **TEST API NHANH HƠN VỚI REST CLIENT**

Thay vì dùng curl, bạn có thể cài extension **REST Client** cho VS Code và viết file `.http` để test API. Hoặc dùng **Postman** / **Insomnia** — các công cụ GUI giúp test API trực quan hơn nhiều.

📋 Tóm tắt chương 8

- Express là thin layer trên Node `http` module — đơn giản hóa routing, request/response
- Route handlers nhận `(req, res)` hoặc `(req, res, next)`
- `req.params`, `req.query`, `req.body` — ba cách đọc dữ liệu từ client
- `res.json()`, `res.status()`, `res.send()` — gửi response
- `express.Router()` giúp tách routes thành module riêng, dễ quản lý
- Built-in middleware: `express.json()`, `express.urlencoded()`, `express.static()`
- Đặt middleware TRƯỚC route handlers cần dùng đến chúng

🏋️ Bài tập thực hành

DỄ

8.1 Thêm route tìm kiếm

Thêm route `GET /api/posts/search?q=keyword` vào Blog API trên để tìm kiếm bài viết theo title. So sánh không phân biệt chữ hoa/thường. Nếu không có kết quả, trả về array rỗng (không phải 404). Nếu thiếu param `q`, trả về 400 Bad Request.

TRUNG BÌNH

8.2 Route thống kê blog

Implement `GET /api/stats` trả về object chứa: `totalPosts`, `totalComments`, `postWithMostComments` (object đầy đủ + commentCount), `topAuthors` (array 3 author có nhiều bài nhất), `popularTags` (5 tags xuất hiện nhiều nhất). Gợi ý: dùng `Array.reduce()` để đếm.

KHÓ

8.3 Pagination thực sự

Implement pagination hoàn chỉnh cho `GET /api/posts`. Query params: `?page=1&limit=10`. Response phải có: `{ data, total, page, limit, totalPages, hasNext, hasPrev }`. Thêm validation: page phải >= 1, limit phải trong [1, 100], nếu page vượt quá totalPages thì trả về data rỗng nhưng vẫn có thông tin pagination đúng. Viết ít nhất 3 test case bằng curl để kiểm tra edge cases.

PHẦN 3 — WEB APPLICATION VỚI EXPRESS

Chương 9: Middleware và Error Handling

Middleware là trái tim của Express — mọi thứ từ authentication đến logging đến CORS đều là middleware. Hiểu middleware giúp bạn xây dựng ứng dụng có kiến trúc sạch và mở rộng được.

> 🎯 **SAU CHƯƠNG NÀY BẠN SẼ BIẾT**
>
> - Middleware là gì và cơ chế `next()` hoạt động ra sao
> - Viết custom middleware: logger, request ID, authentication check
> - Dùng các middleware phổ biến: cors, helmet, morgan, express-rate-limit
> - Implement error handling middleware đúng chuẩn Express
> - Validate request body với Zod schema
> - Xử lý async errors an toàn với asyncHandler pattern

9.1 Middleware là gì?

Middleware là những function nằm giữa khi request đến và khi response được gửi về. Chúng tạo thành một **pipeline** (ống dẫn) — request đi qua từng middleware theo thứ tự, mỗi cái có thể đọc, sửa đổi request/response, hoặc dừng pipeline lại.

Mọi middleware trong Express đều có cùng chữ ký:

JAVASCRIPT — Cấu trúc middleware — Copy

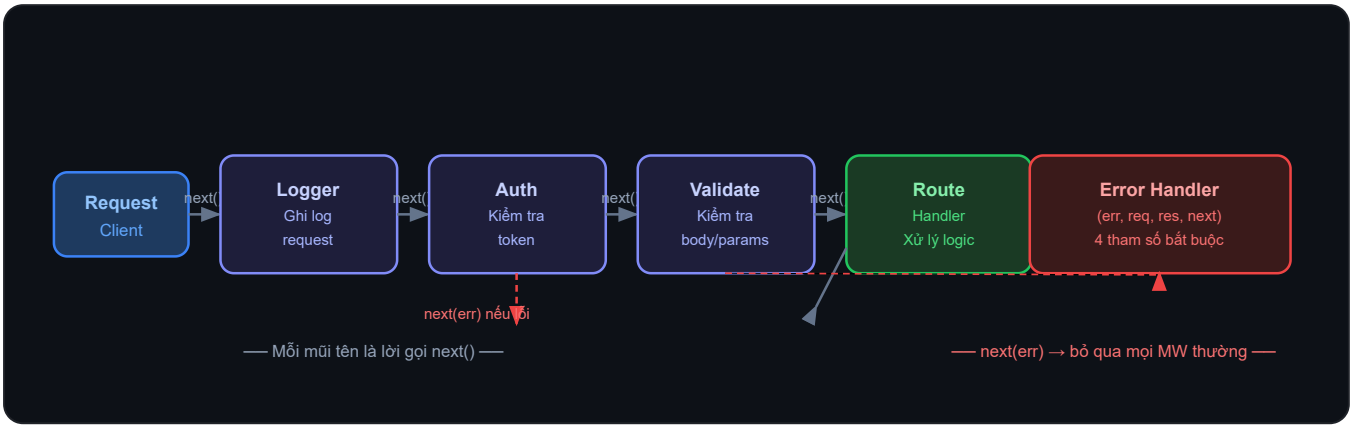
```
// Middleware function signature
function myMiddleware(req, res, next) {
  // req  - request object (có thể đọc và sửa đổi)
  // res  - response object (có thể gửi response và dừng chain)
  // next - function để chuyển sang middleware tiếp theo

  // 3 lựa chọn:

  // 1. Gọi next() - chuyển sang middleware/handler tiếp theo
  next();

  // 2. Gửi response - kết thúc request-response cycle
  // res.json({ ... });

  // 3. Gọi next(error) - chuyển sang error handler
  // next(new Error('Something went wrong'));
}
```



📌 QUY TẮC VÀNG CỦA MIDDLEWARE

Mỗi middleware **phải làm 1 trong 3 việc**: gọi `next()`, gọi `next(error)`, hoặc gửi response (`res.json()`, `res.send()`...). Nếu không làm gì trong 3 cái này, request sẽ bị treo mãi mãi và client timeout.

9.2 Phân loại Middleware trong Express

JAVASCRIPT types-of-middleware.js Copy

```
const express = require('express');
const app = express();
const router = express.Router();

// ── 1. Application-level Middleware ──────────────────────
// Áp dụng cho MỌI request đến app
// app.use() không có path → áp dụng tất cả
app.use((req, res, next) => {
  console.log(`[APP] ${req.method} ${req.url}`);
  next();
});

// app.use() có path → chỉ áp dụng cho routes bắt đầu bằng path đó
app.use('/api', (req, res, next) => {
  console.log('[API] Request đến API');
  next();
});

// ── 2. Router-level Middleware ────────────────────────────
// Chỉ áp dụng cho routes trong router này
router.use((req, res, next) => {
  console.log('[ROUTER] Middleware của router');
  next();
});

router.get('/items', (req, res) => {
  res.json({ items: [] });
});

app.use('/shop', router);

// ── 3. Error-handling Middleware ──────────────────────────
// Đặc biệt: PHẢI có đúng 4 tham số (err, req, res, next)
// Đặt SAU TẤT CẢ routes và middleware khác
app.use((err, req, res, next) => {
```

```
  console.error('Error:', err.message);
  res.status(err.statusCode || 500).json({
    success: false,
    error: err.message
  });
});

// ── 4. Built-in Middleware ──────────────────────────────
app.use(express.json());          // Parse JSON body
app.use(express.urlencoded({ extended: true })); // Parse form data
app.use(express.static('public')); // Serve static files

// ── 5. Third-party Middleware (cài từ npm) ──────────────
// const cors = require('cors');
// const helmet = require('helmet');
// const morgan = require('morgan');
// app.use(cors());
// app.use(helmet());
// app.use(morgan('dev'));
```

> ⚠️ **THỨ TỰ KHAI BÁO MIDDLEWARE**
>
> Middleware được thực thi theo đúng thứ tự bạn khai báo bằng `app.use()`. Quy tắc thực tế cho một app production:
>
> 1. Security middleware (helmet, cors, rate-limit)
> 2. Logging (morgan)
> 3. Body parsing (express.json)
> 4. Route handlers
> 5. 404 handler
> 6. Error handler (luôn cuối cùng)

9.3 Viết Custom Middleware

Custom middleware là nơi bạn đặt logic tái sử dụng. Dưới đây là 3 middleware thực tế và hữu ích:

JAVASCRIPT middleware/requestLogger.js Copy

```javascript
/**
 * Middleware 1: Request Logger
 * Ghi log thời gian, method, path, và status code của mỗi request
 */
function requestLogger(req, res, next) {
  const start = Date.now();
  const timestamp = new Date().toISOString();

  // Ghi log khi request đến
  console.log(`→ [${timestamp}] ${req.method} ${req.url}`);

  // Hook vào sự kiện 'finish' của response để ghi log khi xong
  // res.on('finish') được gọi sau khi response đã được gửi
  res.on('finish', () => {
    const duration = Date.now() - start;
    const statusEmoji = res.statusCode < 400 ? '✅' : '❌';

    console.log(
      `← ${statusEmoji} [${res.statusCode}] ${req.method} ${req.url} ` +
      `(${duration}ms)`
    );
  });

  next(); // Chuyển sang middleware tiếp theo
}

module.exports = requestLogger;

// Cách dùng:
// const requestLogger = require('./middleware/requestLogger');
// app.use(requestLogger);
```

JAVASCRIPT middleware/addRequestId.js Copy

```javascript
const { randomUUID } = require('crypto');

/**
 * Middleware 2: Add Request ID
 * Tạo unique ID cho mỗi request — giúp tracing log khi debug
 */
function addRequestId(req, res, next) {
  // Dùng request-id từ client nếu có (useful cho distributed tracing)
  // Nếu không thì tự tạo mới
  req.id = req.get('X-Request-Id') || randomUUID();

  // Gắn ID vào response header để client biết
  res.set('X-Request-Id', req.id);

  next();
}

module.exports = addRequestId;

// Bây giờ trong mọi route, bạn có thể dùng req.id
// app.get('/test', (req, res) => {
//   console.log(`Request ${req.id}: processing...`);
//   res.json({ requestId: req.id });
// });
```

JAVASCRIPT middleware/checkApiKey.js Copy

```javascript
/**
 * Middleware 3: API Key Authentication
 * Kiểm tra API key trong header trước khi cho phép truy cập
 */

// Giả sử danh sách API keys hợp lệ (trong thực tế lưu trong DB hoặc env)
const VALID_API_KEYS = new Set([
  process.env.API_KEY_1 || 'dev-key-abc123',
  process.env.API_KEY_2 || 'dev-key-xyz789',
]);

function checkApiKey(req, res, next) {
  // Đọc API key từ header X-API-Key
  const apiKey = req.get('X-API-Key');

  if (!apiKey) {
    return res.status(401).json({
      success: false,
      error: 'Thiếu API key. Cung cấp qua header X-API-Key.'
    });
  }

  if (!VALID_API_KEYS.has(apiKey)) {
    return res.status(403).json({
      success: false,
      error: 'API key không hợp lệ hoặc đã hết hạn.'
    });
  }

  // API key hợp lệ — cho phép tiếp tục
  next();
}

module.exports = checkApiKey;

// Áp dụng cho toàn bộ app:
// app.use(checkApiKey);
```

```
// Hoặc chỉ áp dụng cho một route cụ thể:
// app.get('/api/sensitive-data', checkApiKey, (req, res) => { ... });

// Hoặc áp dụng cho một nhóm routes:
// app.use('/api/admin', checkApiKey);
```

JAVASCRIPT middleware/measureTime.js Copy

```
/**
 * Middleware 4: Measure Processing Time
 * Gắn thời gian xử lý vào response header
 */
function measureTime(req, res, next) {
  const startHrTime = process.hrtime.bigint(); // Độ chính xác nanosecond

  res.on('finish', () => {
    const endHrTime = process.hrtime.bigint();
    const durationMs = Number(endHrTime - startHrTime) / 1_000_000; //
Convert to ms

    // Ghi vào console
    console.log(`⏱️  ${req.method} ${req.path}:
${durationMs.toFixed(3)}ms`);

    // Gắn vào header để frontend có thể thấy (chỉ dùng khi dev)
    // res.set('X-Response-Time', `${durationMs.toFixed(2)}ms`);
    // Lưu ý: set header sau khi response đã gửi sẽ bị lỗi
    // Nên set header TRƯỚC khi gửi response
  });

  // Set header TRƯỚC khi response gửi
  const origJson = res.json.bind(res);
  res.json = function(body) {
    const endHrTime = process.hrtime.bigint();
    const durationMs = Number(endHrTime - startHrTime) / 1_000_000;
    res.set('X-Response-Time', `${durationMs.toFixed(2)}ms`);
    return origJson(body);
  };

  next();
}
```

9.4 Middleware phổ biến từ npm

Hầu hết các tính năng phổ biến đã có middleware sẵn trên npm, không cần tự viết lại từ đầu:

BASH Terminal Copy

```
npm install cors helmet morgan express-rate-limit
```

cors — Cross-Origin Resource Sharing

Mặc định, trình duyệt chặn request từ domain A đến API ở domain B (bảo mật same-origin policy). CORS là cách server "cho phép" các domain khác gọi API của mình.

JAVASCRIPT cors-config.js Copy

```
const cors = require('cors');

// ── Option 1: Cho phép TẤT CẢ origins (không an toàn cho production)
app.use(cors());

// ── Option 2: Chỉ cho phép domain cụ thể
app.use(cors({
  origin: 'https://myapp.com'
}));

// ── Option 3: Config đầy đủ cho production
const corsOptions = {
  // Danh sách origins được phép
  origin: function(origin, callback) {
    const allowedOrigins = [
      'https://myapp.com',
      'https://www.myapp.com',
      'http://localhost:3000',  // Development
      'http://localhost:5173',  // Vite dev server
    ];

    // Cho phép request không có origin (vd: Postman, curl, mobile app)
    if (!origin) return callback(null, true);

    if (allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error(`CORS: Origin ${origin} không được phép`));
    }
  },

  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization', 'X-API-Key'],
  exposedHeaders: ['X-Request-Id', 'X-Rate-Limit-Remaining'],
  credentials: true,   // Cho phép gửi cookies và Authorization header
  maxAge: 86400,       // Cache preflight request 24 giờ (giảm OPTIONS
requests)
};
```

```
app.use(cors(corsOptions));

// ── Áp dụng CORS cho route cụ thể ──
app.get('/public-api', cors(), (req, res) => {
  // Chỉ route này cho phép tất cả origins
  res.json({ public: true });
});
```

helmet — Security Headers

Helmet tự động thêm hơn 15 HTTP security headers vào mọi response, bảo vệ app khỏi các lỗ hổng phổ biến như XSS, clickjacking, MIME sniffing...

JAVASCRIPT helmet-config.js Copy

```javascript
const helmet = require('helmet');

// Cách đơn giản nhất – dùng mặc định (đủ tốt cho hầu hết app)
app.use(helmet());

// Headers helmet thêm vào:
// Content-Security-Policy        – Kiểm soát tài nguyên được load
// X-Frame-Options: SAMEORIGIN    – Chống clickjacking (iframe embedding)
// X-Content-Type-Options: nosniff – Chống MIME sniffing
// Strict-Transport-Security      – Bắt buộc dùng HTTPS (HSTS)
// X-XSS-Protection               – Bật XSS filter của browser
// Referrer-Policy                – Kiểm soát thông tin referrer
//  ...  và nhiều header khác

// Config nâng cao nếu cần
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'", "'unsafe-inline'", 'cdn.example.com'],
      styleSrc: ["'self'", "'unsafe-inline'", 'fonts.googleapis.com'],
      imgSrc: ["'self'", 'data:', 'https:'],
      fontSrc: ["'self'", 'fonts.gstatic.com'],
    }
  },
  // Tắt một header cụ thể nếu gây conflict:
  // xFrameOptions: false,
  crossOriginEmbedderPolicy: false, // Tắt nếu dùng CDN resources
}));
```

morgan — HTTP Request Logging

JAVASCRIPT morgan-config.js Copy

```javascript
const morgan = require('morgan');
const fs = require('fs');
const path = require('path');

// ── Format sẵn có ──
// 'combined' – Apache combined log format (dùng cho production)
// 'common'   – Apache common log format
// 'dev'      – Màu sắc, ngắn gọn (dùng khi development)
// 'tiny'     – Rất ngắn
// 'short'    – Ngắn gọn, có response time

// Development: log ra console với màu sắc
if (process.env.NODE_ENV !== 'production') {
  app.use(morgan('dev'));
  // Output: GET /api/products 200 15.234 ms - 234
}

// Production: log ra file
if (process.env.NODE_ENV === 'production') {
  // Tạo thư mục logs nếu chưa có
  const logsDir = path.join(__dirname, 'logs');
  if (!fs.existsSync(logsDir)) fs.mkdirSync(logsDir, { recursive: true });

  // Stream ghi ra file
  const accessLogStream = fs.createWriteStream(
    path.join(logsDir, 'access.log'),
    { flags: 'a' } // append mode
  );

  app.use(morgan('combined', { stream: accessLogStream }));
}

// ── Custom Token ── Thêm thông tin custom vào log
morgan.token('request-id', (req) => req.id || '-');
morgan.token('body-size', (req) => {
  const body = JSON.stringify(req.body);
  return body ? `${body.length}B` : '-';
});
```

```
// Format custom
app.use(morgan('[:date[clf]] ":method :url" :status :response-time ms -
:request-id'));

// ── Bỏ qua log health check (giảm noise) ──
app.use(morgan('dev', {
  skip: (req) => req.path === '/health' || req.path === '/favicon.ico'
}));
```

express-rate-limit — Giới hạn số request

JAVASCRIPT rate-limit-config.js Copy

```
const rateLimit = require('express-rate-limit');

// ── Rate limit tổng quát cho toàn app ──
const globalLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 phút
  max: 200,                 // Tối đa 200 requests / 15 phút / IP
  standardHeaders: true,    // Thêm RateLimit-* headers vào response
  legacyHeaders: false,     // Tắt X-RateLimit-* headers cũ
  message: {
    success: false,
    error: 'Quá nhiều request từ IP này. Thử lại sau 15 phút.',
  }
});

app.use(globalLimiter); // Áp dụng cho tất cả

// ── Rate limit chặt hơn cho endpoint nhạy cảm ──
const loginLimiter = rateLimit({
  windowMs: 60 * 60 * 1000, // 1 giờ
  max: 5,                   // Chỉ 5 lần login thất bại / giờ / IP
  skipSuccessfulRequests: true, // Không đếm request thành công (chỉ đếm
lỗi)
  message: {
    success: false,
    error: 'Quá nhiều lần đăng nhập thất bại. Tài khoản bị tạm khóa 1 giờ.',
  }
});

app.post('/api/auth/login', loginLimiter, authController.login);

// ── Rate limit cho API (dùng API key) ──
const apiLimiter = rateLimit({
  windowMs: 60 * 1000,  // 1 phút
  max: 60,              // 60 requests/phút (1 request/giây)
  keyGenerator: (req) => {
    // Rate limit theo API key thay vì IP
    return req.get('X-API-Key') || req.ip;
```

```
  },
});
```

Setup đầy đủ cho production app

JAVASCRIPT app.js — Production setup Copy

```
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
const rateLimit = require('express-rate-limit');

const requestLogger = require('./middleware/requestLogger');
const addRequestId = require('./middleware/addRequestId');

const app = express();

// ── BƯỚC 1: Security (đặt đầu tiên) ──────────────────────
app.use(helmet());

app.use(cors({
  origin: process.env.ALLOWED_ORIGINS?.split(',') ||
['http://localhost:3000'],
  credentials: true,
}));

app.use(rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 300,
  message: { success: false, error: 'Quá nhiều request, thử lại sau.' }
}));

// ── BƯỚC 2: Request tracing ──────────────────────────────
app.use(addRequestId);  // Tạo X-Request-Id

// ── BƯỚC 3: Logging ──────────────────────────────────────
app.use(morgan(process.env.NODE_ENV === 'production' ? 'combined' : 'dev'));
// app.use(requestLogger); // Hoặc dùng custom logger của mình

// ── BƯỚC 4: Body parsing ─────────────────────────────────
app.use(express.json({ limit: '5mb' }));
app.use(express.urlencoded({ extended: true, limit: '5mb' }));

// ── BƯỚC 5: Static files ─────────────────────────────────
```

```
app.use(express.static('public'));

// ── BƯỚC 6: Routes ──────────────────────────────────────
const postRoutes = require('./routes/posts');
const userRoutes = require('./routes/users');

app.use('/api/posts', postRoutes);
app.use('/api/users', userRoutes);

app.get('/health', (req, res) => {
  res.json({ status: 'OK', uptime: Math.floor(process.uptime()), requestId:
req.id });
});

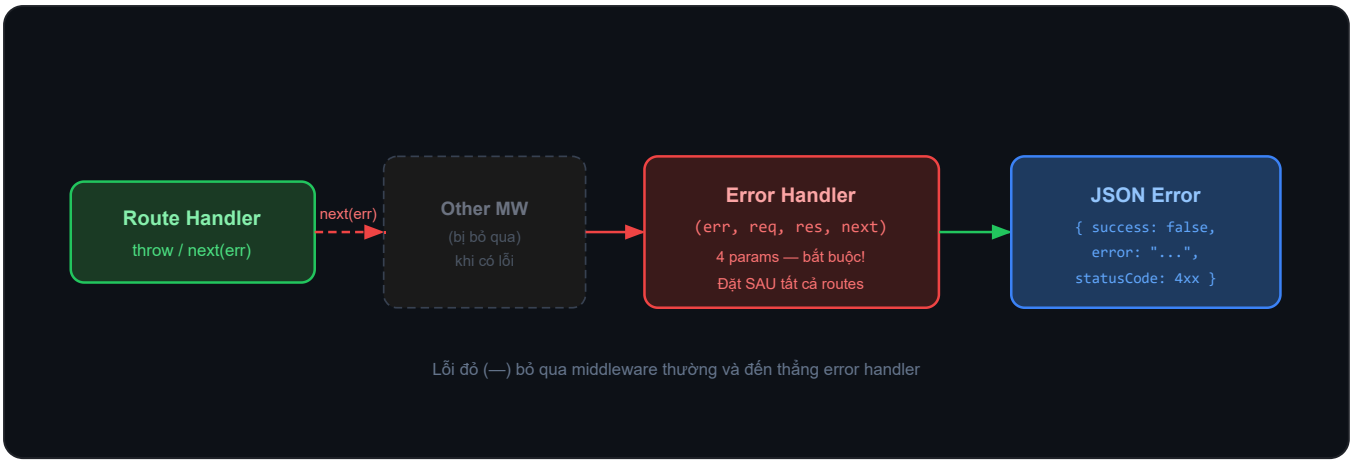
// ── BƯỚC 7: 404 Handler ─────────────────────────────────
app.use((req, res) => {
  res.status(404).json({
    success: false,
    error: `Không tìm thấy: ${req.method} ${req.path}`
  });
});

// ── BƯỚC 8: Error Handler (luôn cuối cùng) ──────────────
app.use((err, req, res, next) => {
  console.error(`[${req.id}] Error:`, err.stack);
  res.status(err.statusCode || 500).json({
    success: false,
    error: err.message || 'Lỗi server nội bộ',
    requestId: req.id,
    ...(process.env.NODE_ENV === 'development' && { stack: err.stack })
  });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Server: http://localhost:${PORT}`));
```

9.5 Error Handling Middleware

Error handling là một trong những phần quan trọng nhất — ứng dụng production phải xử lý lỗi một cách có kiểm soát, không để lộ thông tin nhạy cảm, và luôn trả về response có cấu trúc nhất quán.



JAVASCRIPT errors/AppError.js Copy

```javascript
/**
 * Custom Error class cho ứng dụng
 * Mở rộng Error mặc định với statusCode và các thông tin thêm
 */
class AppError extends Error {
  constructor(message, statusCode = 500, details = null) {
    super(message);          // Gọi Error constructor với message
    this.name = 'AppError';  // Tên lỗi
    this.statusCode = statusCode;
    this.details = details;  // Thông tin thêm (vd: validation errors)
    this.isOperational = true; // Lỗi có chủ ý (ko phải bug bất ngờ)

    // Ghi lại stack trace
    Error.captureStackTrace(this, this.constructor);
  }

  // Factory methods tiện lợi
  static badRequest(message, details) {
    return new AppError(message, 400, details);
  }

  static unauthorized(message = 'Chưa đăng nhập hoặc token hết hạn') {
    return new AppError(message, 401);
  }

  static forbidden(message = 'Không có quyền truy cập') {
    return new AppError(message, 403);
  }

  static notFound(resource = 'Tài nguyên') {
    return new AppError(`${resource} không tìm thấy`, 404);
  }

  static conflict(message) {
    return new AppError(message, 409);
  }

  static tooManyRequests(message = 'Quá nhiều request') {
```

```
    return new AppError(message, 429);
  }
}

module.exports = AppError;
```

JAVASCRIPT middleware/errorHandler.js Copy

```
const AppError = require('../errors/AppError');

/**
 * Global Error Handler Middleware
 * PHẢI có đúng 4 tham số (err, req, res, next)
 * PHẢI đặt SAU TẤT CẢ route handlers và middleware khác
 */
function errorHandler(err, req, res, next) {
  // Log lỗi ra console (trong production nên dùng logging service)
  if (process.env.NODE_ENV === 'development') {
    console.error('🔴 Error:', err);
  } else {
    // Production: chỉ log lỗi không mong đợi
    if (!err.isOperational) {
      console.error('🔴 Unexpected error:', err);
    }
  }

  // Xử lý các loại lỗi phổ biến từ thư viện bên ngoài
  let error = err;

  // Mongoose: CastError (ID không đúng format)
  if (err.name === 'CastError') {
    error = AppError.badRequest(`Giá trị "${err.value}" không hợp lệ cho
field ${err.path}`);
  }

  // Mongoose: Duplicate key (E11000)
  if (err.code === 11000) {
    const field = Object.keys(err.keyValue)[0];
    error = AppError.conflict(`${field} "${err.keyValue[field]}" đã tồn
tại`);
  }

  // Zod: Validation error
  if (err.name === 'ZodError') {
    const errors = err.errors.map(e => ({
      field: e.path.join('.'),
```

```
      message: e.message
    }));
    error = AppError.badRequest('Dữ liệu không hợp lệ', errors);
  }

  // JWT: Token lỗi
  if (err.name === 'JsonWebTokenError') {
    error = AppError.unauthorized('Token không hợp lệ');
  }
  if (err.name === 'TokenExpiredError') {
    error = AppError.unauthorized('Token đã hết hạn, vui lòng đăng nhập
lại');
  }

  // Express: Body too large
  if (err.type === 'entity.too.large') {
    error = AppError.badRequest('Request body quá lớn');
  }

  // Chuẩn bị response
  const statusCode = error.statusCode || 500;
  const response = {
    success: false,
    error: {
      message: error.message || 'Lỗi server nội bộ',
      ...(error.details && { details: error.details }),
    },
    requestId: req.id, // Từ addRequestId middleware
  };

  // Trong development: thêm stack trace
  if (process.env.NODE_ENV === 'development') {
    response.error.stack = err.stack;
  }

  res.status(statusCode).json(response);
}

module.exports = errorHandler;

// ── Cách dùng AppError trong routes ──
```

```
// const AppError = require('../errors/AppError');
//
// app.get('/users/:id', async (req, res, next) => {
//   try {
//     const user = await User.findById(req.params.id);
//     if (!user) throw AppError.notFound('User');
//     res.json({ success: true, data: user });
//   } catch (err) {
//     next(err); // Chuyển lỗi đến errorHandler
//   }
// });
```

9.6 Request Validation với Zod

Validate dữ liệu từ client là bắt buộc — không bao giờ tin tưởng dữ liệu từ bên ngoài. Zod là thư viện validation hiện đại, type-safe, và có thông báo lỗi rõ ràng:

BASH Terminal Copy

```
npm install zod
```

JAVASCRIPT schemas/userSchema.js Copy

```javascript
const { z } = require('zod');

// Schema đăng ký người dùng
const registerSchema = z.object({
  name: z.string({
    required_error: 'Tên là bắt buộc',
    invalid_type_error: 'Tên phải là chuỗi ký tự',
  })
    .min(2, { message: 'Tên phải có ít nhất 2 ký tự' })
    .max(100, { message: 'Tên không được quá 100 ký tự' })
    .trim(),

  email: z.string({
    required_error: 'Email là bắt buộc',
  })
    .email({ message: 'Email không đúng định dạng' })
    .toLowerCase(),

  password: z.string({
    required_error: 'Mật khẩu là bắt buộc',
  })
    .min(8, { message: 'Mật khẩu phải có ít nhất 8 ký tự' })
    .regex(/[A-Z]/, { message: 'Mật khẩu phải có ít nhất 1 chữ hoa' })
    .regex(/[0-9]/, { message: 'Mật khẩu phải có ít nhất 1 chữ số' }),

  confirmPassword: z.string({
    required_error: 'Xác nhận mật khẩu là bắt buộc',
  }),

  role: z.enum(['user', 'admin'], {
    errorMap: () => ({ message: 'Role phải là user hoặc admin' }),
  }).default('user'),

  age: z.number()
    .int({ message: 'Tuổi phải là số nguyên' })
    .min(13, { message: 'Phải ít nhất 13 tuổi' })
    .max(120, { message: 'Tuổi không hợp lệ' })
    .optional(),
```

```
}).refine(
  // Cross-field validation: password phải khớp confirmPassword
  (data) => data.password === data.confirmPassword,
  {
    message: 'Mật khẩu xác nhận không khớp',
    path: ['confirmPassword'], // Lỗi gắn vào field nào
  }
);

// Schema tạo bài viết
const createPostSchema = z.object({
  title: z.string({ required_error: 'Tiêu đề là bắt buộc' })
    .min(5, { message: 'Tiêu đề phải có ít nhất 5 ký tự' })
    .max(200, { message: 'Tiêu đề không được quá 200 ký tự' })
    .trim(),

  content: z.string({ required_error: 'Nội dung là bắt buộc' })
    .min(20, { message: 'Nội dung phải có ít nhất 20 ký tự' })
    .trim(),

  tags: z.array(z.string().toLowerCase().trim())
    .min(1, { message: 'Cần ít nhất 1 tag' })
    .max(10, { message: 'Tối đa 10 tags' })
    .default([]),

  status: z.enum(['draft', 'published']).default('draft'),

  // URL thumbnail (optional)
  thumbnail: z.string().url({ message: 'thumbnail phải là URL hợp lệ'
}).optional(),
});

// Schema query params (pagination)
const paginationSchema = z.object({
  page: z.string().transform(Number).pipe(
    z.number().int().min(1, { message: 'page phải ≥ 1' })
  ).default('1'),

  limit: z.string().transform(Number).pipe(
    z.number().int().min(1).max(100, { message: 'limit tối đa 100' })
```

```
  ).default('10'),

  sort: z.enum(['newest', 'oldest', 'popular']).default('newest'),
});

module.exports = { registerSchema, createPostSchema, paginationSchema };
```

JAVASCRIPT middleware/validate.js Copy

```
/**
 * Middleware factory: validate request với Zod schema
 *
 * @param {Object} schemas - { body?, query?, params? }
 * @returns Express middleware
 *
 * Cách dùng:
 *   router.post('/', validate({ body: createPostSchema }), handler)
 *   router.get('/', validate({ query: paginationSchema }), handler)
 */
function validate(schemas) {
  return (req, res, next) => {
    const errors = [];

    try {
      // Validate từng phần của request
      if (schemas.body) {
        const result = schemas.body.safeParse(req.body);
        if (!result.success) {
          result.error.errors.forEach(e => {
            errors.push({ location: 'body', field: e.path.join('.'),
message: e.message });
          });
        } else {
          req.body = result.data; // Replace với data đã được transform
        }
      }

      if (schemas.query) {
        const result = schemas.query.safeParse(req.query);
        if (!result.success) {
          result.error.errors.forEach(e => {
            errors.push({ location: 'query', field: e.path.join('.'),
message: e.message });
          });
        } else {
          req.query = result.data;
        }
```

```
      }

      if (schemas.params) {
        const result = schemas.params.safeParse(req.params);
        if (!result.success) {
          result.error.errors.forEach(e => {
            errors.push({ location: 'params', field: e.path.join('.'),
message: e.message });
          });
        } else {
          req.params = result.data;
        }
      }

      if (errors.length > 0) {
        return res.status(400).json({
          success: false,
          error: 'Dữ liệu không hợp lệ',
          details: errors,
        });
      }

      next();
    } catch (err) {
      next(err);
    }
  };
}

module.exports = validate;

// ── Ví dụ sử dụng ──
// const validate = require('../middleware/validate');
// const { createPostSchema, paginationSchema } =
require('../schemas/postSchema');
//
// router.get('/', validate({ query: paginationSchema }), async (req, res)
=> {
//   // req.query đã được validate và transform
//   const { page, limit, sort } = req.query; // Đã là number, không phải
string!
```

```
//   ...
// });
//
// router.post('/', validate({ body: createPostSchema }), async (req, res)
⇒ {
//   // req.body đã được sanitize (trim, lowercase, etc.)
//   const { title, content, tags } = req.body;
//   ...
// });
```

9.7 Async Middleware và Route Handlers

Đây là một lỗi rất phổ biến: khi dùng `async/await` trong route handler, nếu có lỗi bị throw, nó KHÔNG tự động đi đến error handler. Bạn phải tự bắt lỗi:

JAVASCRIPT async-error-problem.js Copy

```javascript
// ❌ SAI: Lỗi không đến error handler, app bị crash!
app.get('/users/:id', async (req, res) => {
  const user = await User.findById(req.params.id); // Lỗi DB → crash!
  res.json(user);
});

// ✅ CÁCH 1: try/catch thủ công (an toàn nhưng verbose)
app.get('/users/:id', async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ error: 'Not found' });
    res.json(user);
  } catch (err) {
    next(err); // Chuyển lỗi đến error handler
  }
});

// ✅ CÁCH 2: asyncHandler wrapper (sạch hơn, recommend)
const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
  // Nếu fn throw hoặc promise reject → next(err) được gọi tự động
};

// Dùng:
app.get('/users/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id); // Lỗi → next(err) tự
động!
  if (!user) throw AppError.notFound('User');
  res.json({ success: true, data: user });
}));

// ✅ CÁCH 3: Dùng package 'express-async-errors' (zero config)
// npm install express-async-errors
// require('express-async-errors'); // Đặt ở đầu app.js, TRƯỚC khi import
express
// Sau đó tất cả async handlers tự động catch lỗi
```

JAVASCRIPT utils/asyncHandler.js — Implementation đầy đủ Copy

```
/**
 * asyncHandler — Wrap async function để tự động catch lỗi
 * và chuyển đến Express error handler
 *
 * Không cần try/catch trong mọi route handler nữa!
 */
const asyncHandler = (fn) => {
  return function asyncHandlerWrapper(req, res, next) {
    return Promise.resolve(fn(req, res, next)).catch(next);
  };
};

module.exports = asyncHandler;

// ── Ví dụ sử dụng đầy đủ ──
const asyncHandler = require('../utils/asyncHandler');
const AppError = require('../errors/AppError');

// Giả sử có UserService
const UserService = {
  findById: async (id) => {
    if (id === '0') throw new Error('Database connection failed');
    if (id === '999') return null;
    return { id, name: 'Test User', email: 'test@example.com' };
  },
  create: async (data) => ({ id: Math.random(), ...data }),
  update: async (id, data) => ({ id, ...data }),
  delete: async (id) => true,
};

// GET /users/:id — không cần try/catch!
router.get('/:id', asyncHandler(async (req, res) => {
  const user = await UserService.findById(req.params.id);
  if (!user) throw AppError.notFound('User');
  res.json({ success: true, data: user });
}));

// POST /users — validation + async
```

```
router.post('/', validate({ body: createUserSchema }), asyncHandler(async
(req, res) => {
  const user = await UserService.create(req.body);
  res.status(201).json({ success: true, data: user });
}));

// PUT /users/:id
router.put('/:id', asyncHandler(async (req, res) => {
  const { id } = req.params;
  const existing = await UserService.findById(id);
  if (!existing) throw AppError.notFound('User');

  const updated = await UserService.update(id, req.body);
  res.json({ success: true, data: updated });
}));

// DELETE /users/:id
router.delete('/:id', asyncHandler(async (req, res) => {
  const user = await UserService.findById(req.params.id);
  if (!user) throw AppError.notFound('User');
  await UserService.delete(req.params.id);
  res.json({ success: true, message: 'Xóa thành công' });
}));
```

💡 **TÓM TẮT QUY TRÌNH LỖI CHUẨN**

1. Dùng `asyncHandler` bọc tất cả async route handlers
2. Dùng `throw AppError.notFound()` thay vì `res.status(404) ...`
3. Error tự động đến global `errorHandler` middleware
4. Error handler chuẩn hóa response và log lỗi
5. Client luôn nhận JSON với cấu trúc nhất quán

9.8 Kết hợp tất cả: Middleware stack hoàn chỉnh

Đây là ví dụ hoàn chỉnh kết hợp tất cả những gì đã học trong chương này:

JAVASCRIPT app.js — Middleware stack đầy đủ Copy

```
require('dotenv').config(); // Load .env file

const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
const rateLimit = require('express-rate-limit');

// Custom middleware
const addRequestId = require('./middleware/addRequestId');
const errorHandler = require('./middleware/errorHandler');

// Routes
const postRoutes = require('./routes/posts');
const userRoutes = require('./routes/users');

const app = express();

// ═══════════════════════════════════════════════
// MIDDLEWARE STACK (thứ tự quan trọng!)
// ═══════════════════════════════════════════════

// 1. Security — đặt đầu tiên
app.use(helmet());
app.use(cors({
  origin: process.env.ALLOWED_ORIGINS?.split(',') ||
['http://localhost:3000'],
  credentials: true,
}));

// 2. Rate limiting
const limiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 200 });
app.use(limiter);

// 3. Request ID (tracing)
app.use(addRequestId);

// 4. Logging
```

```
app.use(morgan(process.env.NODE_ENV === 'production' ? 'combined' : 'dev'));

// 5. Body parsing
app.use(express.json({ limit: '5mb' }));
app.use(express.urlencoded({ extended: true }));

// 6. Static files
app.use(express.static('public'));

// ═══════════════════════════════════════════════
// ROUTES
// ═══════════════════════════════════════════════

app.get('/health', (req, res) => {
  res.json({
    status: 'OK',
    uptime: Math.floor(process.uptime()),
    requestId: req.id,
    timestamp: new Date().toISOString(),
  });
});

app.use('/api/posts', postRoutes);
app.use('/api/users', userRoutes);

// ═══════════════════════════════════════════════
// ERROR HANDLING (luôn cuối cùng)
// ═══════════════════════════════════════════════

// 404: Route không tồn tại
app.use((req, res, next) => {
  const err = new Error(`Route không tồn tại: ${req.method} ${req.path}`);
  err.statusCode = 404;
  next(err);
});

// Global error handler
app.use(errorHandler);

// ═══════════════════════════════════════════════
// START SERVER
```

```
// ═══════════════════════════════════════════════════

const PORT = process.env.PORT || 3000;
const server = app.listen(PORT, () => {
  console.log(`🚀 Server: http://localhost:${PORT}`);
  console.log(`📍 Môi trường: ${process.env.NODE_ENV || 'development'}`);
});

// Graceful shutdown - tắt server sạch khi nhận SIGTERM/SIGINT
process.on('SIGTERM', () => {
  console.log('SIGTERM nhận được. Đang đóng server...');
  server.close(() => {
    console.log('Server đã đóng.');
    process.exit(0);
  });
});

process.on('SIGINT', () => {
  console.log('\nSIGINT nhận được. Đang đóng server...');
  server.close(() => process.exit(0));
});

// Catch unhandled promise rejections (safety net)
process.on('unhandledRejection', (reason, promise) => {
  console.error('Unhandled Rejection tại:', promise, 'Lý do:', reason);
  // Trong production: log và gracefully shutdown
});

module.exports = app; // Export để dùng trong tests
```

📋 Tóm tắt chương 9

- Middleware là function `(req, res, next)` — phải gọi `next()`, gửi response, hoặc `next(err)`
- Thứ tự đặt middleware quan trọng: security → logging → parsing → routes → 404 → error
- Error handling middleware có 4 tham số `(err, req, res, next)` — đặt cuối cùng
- `next(err)` từ bất kỳ đâu đều nhảy đến error handler, bỏ qua mọi middleware thường

- cors, helmet, morgan, express-rate-limit — bộ tứ middleware cơ bản cho production
- Zod giúp validate request body với schema rõ ràng và thông báo lỗi tiếng Việt
- `asyncHandler` wrapper giải quyết vấn đề async errors không tự đến error handler
- `AppError` class chuẩn hóa việc throw errors với status code và message phù hợp

🏋️ Bài tập thực hành

DỄ

9.1 Middleware đo thời gian xử lý

Viết middleware `measureTime` ghi ra console thời gian xử lý của mỗi request theo format: `[METHOD] /path → Xms (STATUS)`. Middleware phải dùng `process.hrtime.bigint()` để đo chính xác đến millisecond. Gắn thêm header `X-Response-Time` vào response. Test với ít nhất 3 routes khác nhau.

TRUNG BÌNH

9.2 Cache middleware với TTL

Viết middleware factory `cacheResponse(seconds)` — lưu response vào `Map` với key là URL. Nếu request đến và cache còn hạn, trả về cached version kèm header `X-Cache: HIT`. Nếu hết hạn hoặc chưa có, xử lý bình thường và cache kết quả với header `X-Cache: MISS`. Lưu ý: chỉ cache GET requests và status 200. Test: `app.get('/api/products', cacheResponse(30), handler)`.

KHÓ

9.3 Request Sanitizer Middleware

Xây dựng middleware `requestSanitizer(options)` tự động:

- **Trim strings**: xóa whitespace đầu/cuối tất cả string values trong `req.body`
- **Chặn XSS**: xóa HTML tags (thay `<script>`, `<img>`... bằng chuỗi rỗng) dùng regex
- **Validate content-length**: reject nếu body lớn hơn `options.maxSize` bytes
- **Field whitelist**: reject request nếu body chứa fields không nằm trong `options.allowedFields`

Middleware phải hoạt động đệ quy với nested objects và arrays. Viết test cases cho từng tính năng.

PHẦN 3 — WEB APPLICATION VỚI EXPRESS

Chương 10: Cookie, Session và Authentication

Mọi ứng dụng đều cần biết "ai đang đăng nhập". Chương này xây dựng hệ thống authentication hoàn chỉnh từ đăng ký, đăng nhập, bảo vệ routes, đến refresh token — hiểu rõ tại sao mỗi kỹ thuật tồn tại trước khi dùng nó.

> 🎯 **SAU CHƯƠNG NÀY BẠN SẼ HIỂU ĐƯỢC**
>
> - Set và đọc cookie với Express, các options bảo mật quan trọng
> - Implement server session với express-session
> - Hash password an toàn với bcrypt — tại sao không dùng MD5/SHA
> - Tạo và verify JWT token
> - Viết auth middleware bảo vệ routes
> - Hiểu và implement refresh token flow

10.1 Cookie với Express

Cookie là các cặp key-value nhỏ mà server *yêu cầu* browser lưu trữ và *tự động gửi kèm* trong mọi request tiếp theo. Đây là nền tảng của trạng thái trong HTTP — một giao thức vốn dĩ stateless.

Để làm việc với cookie trong Express, cài thêm `cookie-parser` — một middleware tự động parse `Cookie` header thành object `req.cookies`.

SHELL Terminal Copy

```
npm install express cookie-parser
```

JAVASCRIPT app.js – Thiết lập cookie-parser Copy

```
const express = require('express');
const cookieParser = require('cookie-parser');

const app = express();

// Đăng ký middleware cookie-parser
// 'mySecretKey' dùng để ký signed cookies (tùy chọn)
app.use(cookieParser('mySecretKey'));

// SET COOKIE: lưu tùy chọn giao diện người dùng
app.post('/preferences', (req, res) => {
  const { theme, language } = req.body;

  res.cookie('theme', theme, {
    maxAge: 30 * 24 * 60 * 60 * 1000, // 30 ngày (milliseconds)
    httpOnly: false,                   // JS frontend có thể đọc
    secure: process.env.NODE_ENV === 'production', // chỉ HTTPS khi
production
    sameSite: 'lax',                   // bảo vệ CSRF một phần
  });

  res.cookie('lang', language, {
    maxAge: 30 * 24 * 60 * 60 * 1000,
    httpOnly: false,
    sameSite: 'lax',
  });

  res.json({ message: 'Đã lưu tùy chọn' });
});

// READ COOKIE: đọc tùy chọn đã lưu
app.get('/preferences', (req, res) => {
  const theme = req.cookies.theme || 'light';
  const lang  = req.cookies.lang  || 'vi';
```

```
  res.json({ theme, lang });
});

// CLEAR COOKIE: xóa một cookie cụ thể
app.delete('/preferences', (req, res) => {
  res.clearCookie('theme');
  res.clearCookie('lang');
  res.json({ message: 'Đã xóa tùy chọn' });
});
```

Các options cookie quan trọng

Option	Giá trị mẫu	Mục đích
`httpOnly`	`true`	JavaScript phía client không thể đọc cookie — ngăn XSS đánh cắp token
`secure`	`true`	Chỉ gửi cookie qua HTTPS — không gửi qua HTTP thường
`sameSite`	`'strict'` / `'lax'` / `'none'`	Kiểm soát khi nào cookie được gửi trong cross-site requests — bảo vệ CSRF
`maxAge`	`86400000`	Thời gian sống tính bằng milliseconds (86400000 = 1 ngày)
`expires`	`new Date( ... )`	Ngày hết hạn tuyệt đối — dùng maxAge hoặc expires, không dùng cả hai
`domain`	`'.example.com'`	Cookie chia sẻ được giữa subdomains (app.example.com, api.example.com)
`path`	`'/'`	Cookie chỉ gửi khi URL bắt đầu bằng path này
`signed`	`true`	Ký cookie bằng secret — phát hiện nếu client tự ý sửa giá trị



BẢO MẬT COOKIE

Luôn bật `httpOnly: true` cho cookie chứa session ID hoặc token xác thực. Cookie không có `httpOnly` có thể bị đọc bởi script XSS, dẫn đến chiếm tài khoản. Ngoài ra, `secure: true` phải bật trên production để chặn đánh cắp cookie qua mạng không mã hóa.

10.2 Server Session với express-session

Cookie lưu dữ liệu *trực tiếp* trong trình duyệt. Session thì khác: server chỉ gửi cho browser một chuỗi ID ngẫu nhiên (session ID), còn dữ liệu thực sự được lưu *trên server*. Browser chỉ biết cái ID, không biết nội dung.

SHELL Terminal Copy

```
npm install express-session
```

JAVASCRIPT app.js – Cấu hình express-session Copy

```
const session = require('express-session');

app.use(session({
  secret: process.env.SESSION_SECRET || 'dev-secret-change-in-production',
  resave: false,           // không lưu lại session nếu không thay đổi
  saveUninitialized: false, // không tạo session cho request chưa đăng nhập
  cookie: {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    maxAge: 24 * 60 * 60 * 1000, // 1 ngày
    sameSite: 'lax',
  },
}));

// Dữ liệu người dùng giả (không cần DB)
const users = [
  { id: 1, email: 'alice@example.com', password: 'hashed_pw_1', name:
'Alice' },
  { id: 2, email: 'bob@example.com',   password: 'hashed_pw_2', name: 'Bob'
},
];

// POST /login – đăng nhập bằng session
app.post('/login', express.json(), (req, res) => {
  const { email, password } = req.body;

  // Tìm user (trong thực tế sẽ dùng bcrypt.compare)
  const user = users.find(u => u.email === email);
  if (!user || password !== 'secret123') {
    return res.status(401).json({ error: 'Email hoặc mật khẩu không đúng'
});
  }

  // Lưu thông tin vào session – object này tồn tại trên SERVER
  req.session.userId   = user.id;
```

```
  req.session.userName = user.name;
  req.session.loginAt  = new Date().toISOString();

  res.json({ message: `Chào mừng ${user.name}!` });
});

// GET /profile – lấy thông tin người dùng từ session
app.get('/profile', (req, res) => {
  if (!req.session.userId) {
    return res.status(401).json({ error: 'Chưa đăng nhập' });
  }

  const user = users.find(u => u.id === req.session.userId);
  res.json({
    name:    user.name,
    email:   user.email,
    loginAt: req.session.loginAt,
  });
});

// POST /logout – hủy session
app.post('/logout', (req, res) => {
  req.session.destroy(err => {
    if (err) return res.status(500).json({ error: 'Không thể đăng xuất' });

    res.clearCookie('connect.sid'); // tên mặc định của session cookie
    res.json({ message: 'Đã đăng xuất thành công' });
  });
});
```

MEMORYSTORE — CHỈ DÙNG CHO DEVELOPMENT

Theo mặc định, `express-session` lưu session trong RAM (`MemoryStore`). Điều này có nghĩa là: mỗi khi restart server, tất cả session bị xóa; và khi scale lên nhiều server, mỗi server có session riêng — user đăng nhập ở server 1 sẽ bị logout khi request tới server 2. Trong production, dùng `connect-redis` hoặc `connect-mongo` để lưu session vào database dùng chung.

10.3 Hash Password với bcrypt

Không bao giờ lưu mật khẩu dạng plain text. Nếu database bị tấn công, toàn bộ mật khẩu của người dùng bị lộ. Nhưng tại sao không dùng MD5 hay SHA-256 thông thường?

Vấn đề với MD5 và SHA-256 cho password

- **Quá nhanh:** GPU hiện đại có thể hash hàng tỷ SHA-256 mỗi giây. Attacker có thể brute-force toàn bộ dictionary trong vài giờ.
- **Rainbow table:** Bảng hash đã tính sẵn của hàng triệu mật khẩu phổ biến. `md5('123456')` luôn cho cùng một kết quả.
- **Không có salt:** Hai người dùng cùng mật khẩu sẽ có cùng hash — lộ một lộ tất cả.

bcrypt giải quyết tất cả: nó cố tình chậm (có thể điều chỉnh), tự sinh salt ngẫu nhiên, và salt được nhúng vào trong chuỗi hash kết quả.

SHELL Terminal Copy

```
# bcryptjs = pure JavaScript, không cần build native addon
# bcrypt   = native C++ binding, nhanh hơn nhưng cần build tools
npm install bcryptjs
```

JAVASCRIPT auth/password.js — Hash và verify password Copy

```javascript
const bcrypt = require('bcryptjs');

// ========== HASH PASSWORD ==========
// saltRounds = "cost factor" — số vòng lặp = 2^saltRounds
// saltRounds 10 → ~100ms/hash; 12 → ~400ms/hash; 14 → ~1.5s/hash
// Production khuyến nghị: 10–12

const saltRounds = 12;

async function hashPassword(plainPassword) {
  // bcrypt.hash tự sinh salt, hash, và gộp thành một chuỗi:
  // $2b$12$[22 ký tự salt][31 ký tự hash]
  const hash = await bcrypt.hash(plainPassword, saltRounds);
  return hash;
}

// ========== VERIFY PASSWORD ==========
async function verifyPassword(plainPassword, storedHash) {
  // bcrypt.compare đọc salt từ storedHash, hash lại plainPassword,
  // rồi so sánh — trả về true/false
  const isMatch = await bcrypt.compare(plainPassword, storedHash);
  return isMatch;
}

// ========== DEMO ==========
async function demo() {
  const password = 'MySecurePassword123!';

  console.log('Đang hash ...');
  const hash = await hashPassword(password);
  console.log('Hash:', hash);
  // Ví dụ: $2b$12$LQv3c1yqBWVHxkd0LHAkCOYz6TtxMQJqhN8/NekdBnj6q20UvE6Vy

  console.log('\nKiểm tra mật khẩu đúng:');
  console.log(await verifyPassword(password, hash));     // true
```

```
  console.log('\nKiểm tra mật khẩu sai:');
  console.log(await verifyPassword('sai_password', hash)); // false

  // Mỗi lần hash cho kết quả KHÁC NHAU do salt ngẫu nhiên
  const hash2 = await hashPassword(password);
  console.log('\nHash lần 2 (salt khác):', hash !== hash2); // true
}

demo();
```

JAVASCRIPT routes/auth.js — Đăng ký và đăng nhập với bcrypt Copy

```
const express = require('express');
const bcrypt  = require('bcryptjs');
const router  = express.Router();

// In-memory store (thay bằng database trong thực tế)
const users = new Map(); // key: email, value: { id, email, name,
passwordHash }
let nextId = 1;

// POST /auth/register
router.post('/register', async (req, res) => {
  try {
    const { name, email, password } = req.body;

    // Validation cơ bản
    if (!name || !email || !password)
      return res.status(400).json({ error: 'Vui lòng điền đầy đủ thông tin'
});
    if (password.length < 8)
      return res.status(400).json({ error: 'Mật khẩu tối thiểu 8 ký tự' });
    if (users.has(email))
      return res.status(409).json({ error: 'Email đã được đăng ký' });

    const passwordHash = await bcrypt.hash(password, 12);

    const user = { id: nextId++, name, email, passwordHash };
    users.set(email, user);

    // Không trả về passwordHash
    res.status(201).json({ message: 'Đăng ký thành công', userId: user.id
});
  } catch (err) {
    res.status(500).json({ error: 'Lỗi server: ' + err.message });
  }
});
```

```
// POST /auth/login
router.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;

    const user = users.get(email);
    // Luôn gọi bcrypt.compare kể cả khi user không tồn tại
    // để tránh timing attack (so sánh thời gian phản hồi)
    const fakeHash =
'$2b$12$invalidhashfortimingprotection000000000000000000000000';
    const isMatch = user
      ? await bcrypt.compare(password, user.passwordHash)
      : await bcrypt.compare(password, fakeHash);

    if (!user || !isMatch) {
      return res.status(401).json({ error: 'Email hoặc mật khẩu không đúng'
});
    }

    req.session.userId = user.id;
    res.json({ message: `Chào mừng, ${user.name}!` });
  } catch (err) {
    res.status(500).json({ error: 'Lỗi server: ' + err.message });
  }
});

module.exports = router;
```

10.4 JWT Authentication

JSON Web Token (JWT) là một cách khác để xác thực người dùng — không cần lưu session trên server. Thay vào đó, server tạo một token có chứa thông tin user và chữ ký mật mã. Client lưu token này và gửi kèm mỗi request.

Cấu trúc JWT

JWT gồm 3 phần phân cách bằng dấu chấm: `Header.Payload.Signature`

TEXT — Ví dụ JWT token — Copy

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9          ← Header (base64url)
.eyJ1c2VySWQiOjEsImVtYWlsIjoiYWxpY2VAZXhhbXBsZS5jb20iLCJpYXQiOjE2OTAwMDAwMDA
sImV4cCI6MTY5MDYwNDgwMH0=  ← Payload (base64url)
.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c        ← Signature (HMAC-
SHA256)

// Payload sau khi decode:
{
  "userId": 1,
  "email": "alice@example.com",
  "iat": 1690000000,    // issued at
  "exp": 1690604800     // expiration
}
```

PAYLOAD KHÔNG BÍ MẬT

Phần Payload của JWT chỉ được encode bằng base64url — **không phải mã hóa**. Bất kỳ ai cũng có thể đọc nội dung. Chỉ có Signature là được ký bằng secret key. Đừng bao giờ đặt thông tin nhạy cảm (password, thẻ tín dụng) vào payload.

SHELL — Terminal — Copy

```
npm install jsonwebtoken
```

JAVASCRIPT utils/jwt.js — Tạo và xác minh JWT Copy

```
const jwt = require('jsonwebtoken');

const ACCESS_TOKEN_SECRET  = process.env.JWT_ACCESS_SECRET  || 'access-
secret-dev';
const REFRESH_TOKEN_SECRET = process.env.JWT_REFRESH_SECRET || 'refresh-
secret-dev';

// Tạo access token (ngắn hạn)
function signAccessToken(payload) {
  return jwt.sign(payload, ACCESS_TOKEN_SECRET, {
    expiresIn: '15m', // 15 phút
    algorithm: 'HS256',
  });
}

// Tạo refresh token (dài hạn)
function signRefreshToken(payload) {
  return jwt.sign(payload, REFRESH_TOKEN_SECRET, {
    expiresIn: '30d', // 30 ngày
    algorithm: 'HS256',
  });
}

// Xác minh access token
function verifyAccessToken(token) {
  try {
    return jwt.verify(token, ACCESS_TOKEN_SECRET);
    // Trả về payload nếu hợp lệ
    // Throw JsonWebTokenError nếu chữ ký sai
    // Throw TokenExpiredError nếu hết hạn
  } catch (err) {
    return null; // Token không hợp lệ
  }
}
```

```
// Xác minh refresh token
function verifyRefreshToken(token) {
  try {
    return jwt.verify(token, REFRESH_TOKEN_SECRET);
  } catch (err) {
    return null;
  }
}

module.exports = {
  signAccessToken,
  signRefreshToken,
  verifyAccessToken,
  verifyRefreshToken,
};
```

Lưu JWT ở đâu — localStorage vs httpOnly cookie

Cách lưu	Ưu điểm	Rủi ro
`localStorage`	Dễ gửi kèm request, không bị giới hạn subdomain	Dễ bị đánh cắp bởi XSS — bất kỳ script nào cũng đọc được
`sessionStorage`	Tự xóa khi đóng tab	Vẫn dễ bị XSS; mất khi tab đóng — không tiện lợi
httpOnly cookie	JS không đọc được → XSS không lấy được token	Cần xử lý CSRF; không hoạt động với mobile app native

Khuyến nghị: lưu access token trong memory (biến JS) và refresh token trong `httpOnly` cookie. Khi trang tải lại, dùng refresh token để lấy access token mới.

10.5 Auth Middleware

Middleware xác thực đọc token từ request, xác minh tính hợp lệ, và nếu hợp lệ thì đính kèm thông tin user vào `req.user` để các handler sau sử dụng.

JAVASCRIPT middlewares/auth.js — Auth middleware đầy đủ Copy

```
const jwt = require('jsonwebtoken');

const ACCESS_TOKEN_SECRET = process.env.JWT_ACCESS_SECRET || 'access-secret-dev';

// ========== HELPER: asyncHandler ==========
// Bọc async function để tự động truyền lỗi sang next()
const asyncHandler = fn => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

// ========== CLASS LỖI TÙY CHỈNH ==========
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}

// ========== AUTH MIDDLEWARE (bắt buộc) ==========
const authMiddleware = asyncHandler(async (req, res, next) => {
  // Đọc token từ Authorization header hoặc cookie
  let token;
  const authHeader = req.headers.authorization;

  if (authHeader && authHeader.startsWith('Bearer ')) {
    token = authHeader.split(' ')[1];
  } else if (req.cookies?.access_token) {
    token = req.cookies.access_token;
  }

  if (!token) {
    throw new AppError('Chưa đăng nhập — vui lòng cung cấp token', 401);
  }
```

```
  let decoded;
  try {
    decoded = jwt.verify(token, ACCESS_TOKEN_SECRET);
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      throw new AppError('Token đã hết hạn — vui lòng refresh', 401);
    }
    throw new AppError('Token không hợp lệ', 401);
  }

  // Đính kèm user info vào request để dùng ở handler
  req.user = decoded; // { userId, email, role, iat, exp }
  next();
});

// ========== OPTIONAL AUTH MIDDLEWARE ==========
// Không fail nếu không có token; chỉ đính kèm user nếu token hợp lệ
const optionalAuth = (req, res, next) => {
  const authHeader = req.headers.authorization;
  const token = authHeader?.startsWith('Bearer ')
    ? authHeader.split(' ')[1]
    : req.cookies?.access_token;

  if (!token) return next(); // Tiếp tục mà không có user

  try {
    req.user = jwt.verify(token, ACCESS_TOKEN_SECRET);
  } catch {
    // Token không hợp lệ → bỏ qua, không fail
  }
  next();
};

// ========== ROLE-BASED MIDDLEWARE FACTORY ==========
// Dùng: router.get('/admin', authMiddleware, requireRole('admin'), handler)
// Hoặc: router.get('/mod', authMiddleware, requireRole(['admin',
'moderator']), handler)
const requireRole = (roles) => {
  const allowed = Array.isArray(roles) ? roles : [roles];
  return (req, res, next) => {
    if (!req.user) {
```

```
      return res.status(401).json({ error: 'Chưa xác thực' });
    }
    if (!allowed.includes(req.user.role)) {
      return res.status(403).json({
        error: `Không có quyền — yêu cầu role: ${allowed.join(', ')}`,
      });
    }
    next();
  };
};

module.exports = { authMiddleware, optionalAuth, requireRole, AppError,
asyncHandler };
```

JAVASCRIPT routes/protected.js – Sử dụng auth middleware Copy

```javascript
const express = require('express');
const { authMiddleware, requireRole } = require('../middlewares/auth');
const router = express.Router();

// Áp dụng authMiddleware cho tất cả routes trong file này
router.use(authMiddleware);

// Route bình thường – chỉ cần đăng nhập
router.get('/dashboard', (req, res) => {
  res.json({
    message: `Xin chào ${req.user.email}`,
    userId:  req.user.userId,
  });
});

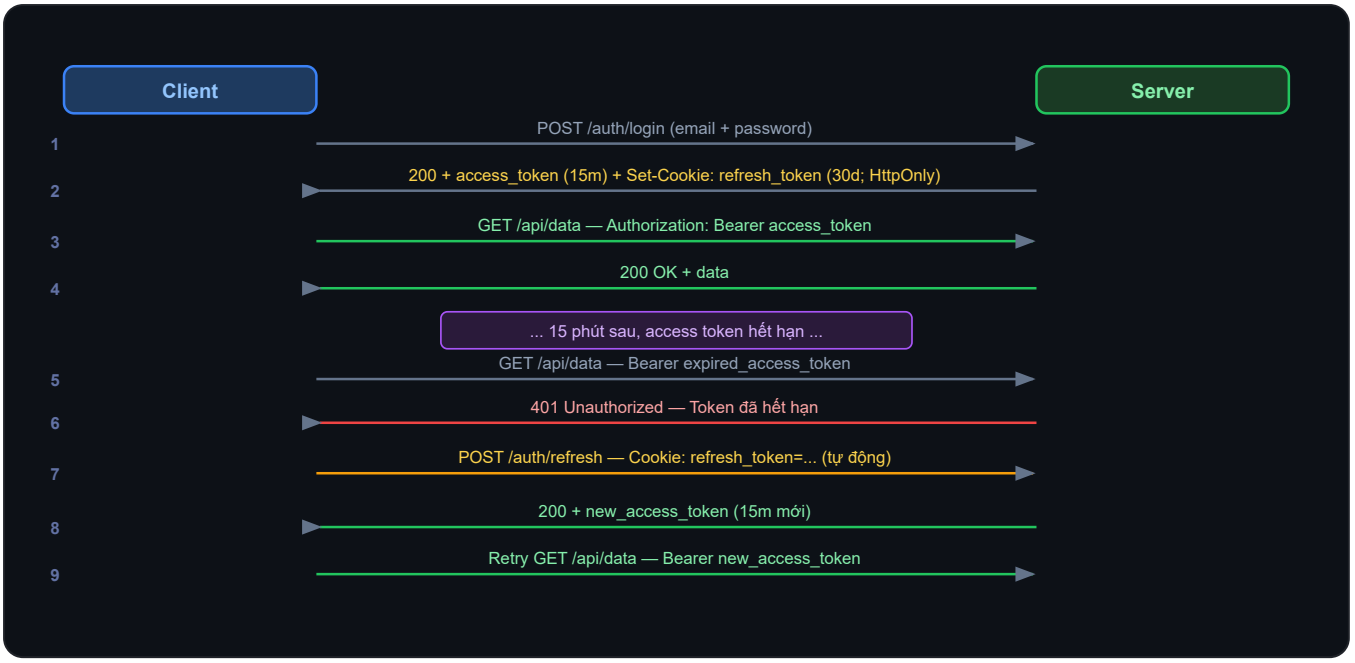
// Route cần role admin
router.get('/admin/users', requireRole('admin'), (req, res) => {
  res.json({ users: [/* danh sách users */] });
});

// Route cho nhiều roles
router.delete('/posts/:id', requireRole(['admin', 'moderator']), (req, res)
=> {
  res.json({ message: `Đã xóa bài ${req.params.id}` });
});

module.exports = router;
```

10.6 Refresh Token Pattern

Access token có vòng đời ngắn (15 phút) để giảm rủi ro nếu bị đánh cắp. Refresh token có vòng đời dài (30 ngày) và được lưu trong `httpOnly` cookie — client dùng nó để lấy access token mới mà không cần đăng nhập lại.



JAVASCRIPT routes/auth.js — Refresh token endpoint Copy

```
const { signAccessToken, verifyRefreshToken } = require('../utils/jwt');

// POST /auth/refresh
router.post('/refresh', (req, res) => {
  const refreshToken = req.cookies.refresh_token;

  if (!refreshToken) {
    return res.status(401).json({ error: 'Không có refresh token' });
  }

  const decoded = verifyRefreshToken(refreshToken);
  if (!decoded) {
    // Xóa cookie không hợp lệ
    res.clearCookie('refresh_token');
    return res.status(401).json({ error: 'Refresh token không hợp lệ hoặc đã
hết hạn' });
  }

  // Tạo access token mới
  const newAccessToken = signAccessToken({
    userId: decoded.userId,
    email:  decoded.email,
    role:   decoded.role,
  });

  res.json({ accessToken: newAccessToken });
});

// POST /auth/logout — Xóa refresh token cookie
router.post('/logout', (req, res) => {
  res.clearCookie('refresh_token', {
    httpOnly: true,
    secure:   process.env.NODE_ENV === 'production',
    sameSite: 'lax',
  });
```

```
  res.json({ message: 'Đăng xuất thành công' });
});
```

10.7 Dự án: Hệ thống Authentication Hoàn chỉnh

Tổng hợp tất cả kiến thức trên để xây dựng một auth system đầy đủ: đăng ký, đăng nhập, refresh token, bảo vệ routes, và thông tin user hiện tại. Dùng in-memory store, không cần database.

TEXT — Cấu trúc thư mục dự án — Copy

```
auth-project/
├── app.js              ← Entry point
├── utils/
│   └── jwt.js          ← sign/verify helpers
├── middlewares/
│   └── auth.js         ← authMiddleware, optionalAuth, requireRole
├── routes/
│   └── auth.js         ← /auth/* endpoints
├── store/
│   └── users.js        ← In-memory user store
└── package.json
```

JAVASCRIPT store/users.js — In-memory user store Copy

```
// In-memory store — trong thực tế thay bằng MongoDB/PostgreSQL
const users = new Map();
let nextId = 1;

module.exports = {
  findByEmail: (email) => users.get(email),
  findById:    (id)    => [...users.values()].find(u => u.id === id),
  create: (userData) => {
    const user = { id: nextId++, ...userData, createdAt: new Date() };
    users.set(userData.email, user);
    return user;
  },
  count: () => users.size,
};
```

JAVASCRIPT routes/auth.js — Tất cả auth routes Copy

```
const express  = require('express');
const bcrypt   = require('bcryptjs');
const router   = express.Router();
const userStore = require('../store/users');
const { signAccessToken, signRefreshToken, verifyRefreshToken } =
require('../utils/jwt');
const { authMiddleware } = require('../middlewares/auth');

const COOKIE_OPTIONS = {
  httpOnly: true,
  secure:   process.env.NODE_ENV === 'production',
  sameSite: 'lax',
  maxAge:   30 * 24 * 60 * 60 * 1000, // 30 ngày
};

// ─────────────────────────────────────────────
// POST /auth/register
// ─────────────────────────────────────────────
router.post('/register', async (req, res) => {
  try {
    const { name, email, password } = req.body;

    if (!name?.trim() || !email?.trim() || !password)
      return res.status(400).json({ error: 'Vui lòng điền đầy đủ: tên,
email, mật khẩu' });

    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email))
      return res.status(400).json({ error: 'Định dạng email không hợp lệ'
});

    if (password.length < 8)
      return res.status(400).json({ error: 'Mật khẩu tối thiểu 8 ký tự' });

    if (userStore.findByEmail(email))
      return res.status(409).json({ error: 'Email đã được đăng ký' });
```

```
    const passwordHash = await bcrypt.hash(password, 12);
    const user = userStore.create({ name: name.trim(), email:
email.toLowerCase(), passwordHash, role: 'user' });

    const accessToken  = signAccessToken({ userId: user.id, email:
user.email, role: user.role });
    const refreshToken = signRefreshToken({ userId: user.id, email:
user.email, role: user.role });

    res.cookie('refresh_token', refreshToken, COOKIE_OPTIONS);

    res.status(201).json({
      message:     'Đăng ký thành công',
      accessToken,
      user: { id: user.id, name: user.name, email: user.email },
    });
  } catch (err) {
    res.status(500).json({ error: 'Lỗi server: ' + err.message });
  }
});

// ─────────────────────────────────────────────
// POST /auth/login
// ─────────────────────────────────────────────
router.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password)
      return res.status(400).json({ error: 'Vui lòng nhập email và mật khẩu'
});

    const user = userStore.findByEmail(email.toLowerCase());
    const fakeHash =
'$2b$12$XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX';
    const isMatch = user
      ? await bcrypt.compare(password, user.passwordHash)
      : (await bcrypt.compare(password, fakeHash), false);

    if (!user || !isMatch)
```

```
      return res.status(401).json({ error: 'Email hoặc mật khẩu không đúng' });

    const accessToken  = signAccessToken({ userId: user.id, email: user.email, role: user.role });
    const refreshToken = signRefreshToken({ userId: user.id, email: user.email, role: user.role });

    res.cookie('refresh_token', refreshToken, COOKIE_OPTIONS);

    res.json({
      message:     'Đăng nhập thành công',
      accessToken,
      user: { id: user.id, name: user.name, email: user.email, role: user.role },
    });
  } catch (err) {
    res.status(500).json({ error: 'Lỗi server: ' + err.message });
  }
});

// ─────────────────────────────────────────────
// POST /auth/refresh
// ─────────────────────────────────────────────
router.post('/refresh', (req, res) => {
  const token = req.cookies.refresh_token;
  if (!token) return res.status(401).json({ error: 'Không có refresh token' });

  const decoded = verifyRefreshToken(token);
  if (!decoded) {
    res.clearCookie('refresh_token');
    return res.status(401).json({ error: 'Refresh token không hợp lệ hoặc đã hết hạn' });
  }

  // Kiểm tra user vẫn còn tồn tại
  const user = userStore.findById(decoded.userId);
  if (!user) {
    res.clearCookie('refresh_token');
    return res.status(401).json({ error: 'Tài khoản không tồn tại' });
```

```
  }

  const accessToken = signAccessToken({ userId: user.id, email: user.email,
role: user.role });
  res.json({ accessToken });
});

// ─────────────────────────────────────────────
// POST /auth/logout
// ─────────────────────────────────────────────
router.post('/logout', (req, res) => {
  res.clearCookie('refresh_token', { httpOnly: true, secure:
process.env.NODE_ENV === 'production' });
  res.json({ message: 'Đã đăng xuất thành công' });
});

// ─────────────────────────────────────────────
// GET /auth/me - Thông tin user hiện tại (protected)
// ─────────────────────────────────────────────
router.get('/me', authMiddleware, (req, res) => {
  const user = userStore.findById(req.user.userId);
  if (!user) return res.status(404).json({ error: 'Không tìm thấy user' });

  res.json({
    id:        user.id,
    name:      user.name,
    email:     user.email,
    role:      user.role,
    createdAt: user.createdAt,
  });
});

module.exports = router;
```

JAVASCRIPT app.js – Entry point hoàn chỉnh Copy

```
const express      = require('express');
const cookieParser = require('cookie-parser');
const authRoutes   = require('./routes/auth');
const { authMiddleware } = require('./middlewares/auth');

const app = express();

app.use(express.json());
app.use(cookieParser());
app.use('/auth', authRoutes);

// Route được bảo vệ – ví dụ
app.get('/api/secret', authMiddleware, (req, res) => {
  res.json({ secret: 'Dữ liệu bí mật chỉ user đăng nhập mới thấy', user:
req.user });
});

// Global error handler
app.use((err, req, res, next) => {
  const status = err.statusCode || 500;
  res.status(status).json({ error: err.message || 'Lỗi server' });
});

app.listen(3000, () => console.log('Auth server chạy tại
http://localhost:3000'));

// Test nhanh với curl:
// curl -X POST http://localhost:3000/auth/register \
//   -H "Content-Type: application/json" \
//   -d '{"name":"Alice","email":"alice@test.com","password":"secret123"}'
```

Tóm tắt chương 10

- **Cookie** lưu dữ liệu nhỏ ở browser; dùng `httpOnly + secure + sameSite` để bảo mật
- **Session** chỉ lưu ID ở browser, data ở server; `MemoryStore` chỉ dùng cho dev
- **bcrypt** hash password với salt ngẫu nhiên và cost factor — không thể reverse; không dùng MD5/SHA
- **JWT** là stateless token, payload có thể đọc được, signature bảo vệ tính toàn vẹn
- **Auth middleware** đọc token từ header hoặc cookie, verify, đính kèm `req.user`
- **Refresh token** dài hạn trong httpOnly cookie, access token ngắn hạn trong memory

Bài tập thực hành

DỄ

10.1 optionalAuth middleware

Thêm middleware `optionalAuth` vào dự án — không trả về lỗi 401 nếu chưa login, nhưng nếu có token hợp lệ thì đính kèm `req.user`. Dùng nó cho route `GET /posts`: nếu đã login thì hiện thêm trường `isOwner: true` cho những bài viết của user đó.

TRUNG BÌNH

10.2 requireRole middleware factory

Implement `requireRole(roles)` nhận cả string lẫn array string. Test với 3 roles: `'user'`, `'moderator'`, `'admin'`. Route `DELETE /posts/:id` chỉ cho phép moderator và admin. Route `GET /admin/stats` chỉ cho admin. Viết test case thủ công bằng curl.

KHÓ

10.3 JWT Blacklist khi đăng xuất

Implement token blacklist: khi user logout, thêm access token hiện tại vào một `Set` in-memory (hoặc Map kèm expiry). Trong `authMiddleware`, check token có trong blacklist không trước khi cho qua. Xử lý thêm: tự động dọn token đã hết hạn ra khỏi blacklist để tránh memory leak (dùng cron hoặc lazy cleanup).

PHẦN 3 — WEB APPLICATION VỚI EXPRESS

Chương 11: Templating Engine và Server-Side Rendering

Không phải mọi app đều cần React hay Vue. Server-side rendering đơn giản, SEO-friendly, và phù hợp cho nhiều loại web. Chương này xây dựng website blog thực sự với EJS — từ template đầu tiên đến hệ thống layout, form handling, và flash messages.

🎯 SAU CHƯƠNG NÀY BẠN SẼ HIỂU ĐƯỢC

- SSR vs CSR vs SSG — khi nào dùng cái nào
- Cài đặt và dùng EJS templating engine với Express
- Tạo layout chung và partial templates (header, footer)
- Handle form submission và POST-Redirect-GET pattern
- Flash messages để thông báo kết quả cho user sau redirect
- Xây dựng blog website đầy đủ với CRUD operations

11.1 SSR vs CSR vs SSG

Có ba cách chính để đưa nội dung từ server đến trình duyệt của người dùng. Hiểu sự khác biệt giúp bạn chọn đúng công cụ cho từng loại dự án.



Khi nào dùng cái nào?

Loại dự án	Phương pháp phù hợp	Lý do
Blog cá nhân, trang marketing	**SSG** (Next.js, Hugo, Astro)	Nội dung ít thay đổi, cần SEO, tốc độ tối đa
Website tin tức, thương mại điện tử	**SSR** (Next.js, Express + EJS)	Nội dung thay đổi theo giờ, cần SEO, dữ liệu động
Dashboard, ứng dụng quản lý	**CSR** (React, Vue, Angular)	UX phức tạp, dữ liệu real-time, không cần SEO
Trang docs, portfolio	**SSG**	Không cần server, deploy lên GitHub Pages miễn phí
Web app hybrid	**SSR + CSR** (Next.js full)	Trang marketing → SSR/SSG; trang app → CSR

11.2 EJS — Embedded JavaScript Templates

EJS (Embedded JavaScript) là templating engine phổ biến nhất cho Express. Nó cho phép nhúng code JavaScript trực tiếp vào HTML — học trong 30 phút, mạnh đủ dùng cho hầu hết dự án SSR thực tế.

SHELL Terminal Copy

```
npm install ejs express
```

JAVASCRIPT app.js – Thiết lập EJS với Express Copy

```
const express = require('express');
const path    = require('path');
const app     = express();

// Nói với Express dùng EJS làm view engine
app.set('view engine', 'ejs');

// Thư mục chứa các file template (mặc định là ./views)
app.set('views', path.join(__dirname, 'views'));

// Serve static files (CSS, JS, images)
app.use(express.static(path.join(__dirname, 'public')));

// Route đầu tiên
app.get('/', (req, res) => {
  // res.render('template-name', { biến truyền vào template })
  res.render('index', {
    title:   'Trang chủ',
    message: 'Chào mừng đến với EJS!',
    users: [
      { name: 'Alice', age: 28 },
      { name: 'Bob',   age: 32 },
      { name: 'Carol', age: 25 },
    ],
  });
});

app.listen(3000, () => console.log('Server tại http://localhost:3000'));
```

Các EJS tags cần biết

HTML views/index.ejs – Ví dụ đầy đủ các tags Copy

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <!-- <%= %> output có escape HTML (an toàn, dùng với user input) -->
  <title><%= title %></title>
</head>
<body>

  <h1><%= message %></h1>

  <!-- <% %> thực thi code JS - if/else -->
  <% if (users.length === 0) { %>
    <p>Không có người dùng nào.</p>
  <% } else { %>
    <p>Có <%= users.length %> người dùng:</p>

    <!-- <% %> thực thi code JS - for loop -->
    <ul>
      <% users.forEach((user, index) => { %>
        <li>
          <!-- #{index+1} để đánh số 1-based -->
          <%= index + 1 %>. <strong><%= user.name %></strong> - <%= user.age
%> tuổi
        </li>
      <% }) %>
    </ul>
  <% } %>

  <!-- <%- %> output KHÔNG escape - dùng khi bạn tin tưởng nội dung -->
  <!-- NGUY HIỂM nếu dùng với user input chưa sanitize -->
  <%- '<em>HTML thô này được render bình thường</em>' %>

  <!-- <%# %> comment - không xuất ra HTML -->
  <%# Dòng này sẽ không xuất hiện trong source HTML %>
```

```
</body>
</html>
```

Tag	Tên gọi	Dùng khi nào
`<%= value %>`	Output (escaped)	Xuất giá trị biến — tự động escape HTML (<, >, &) — an toàn với user input
`<%- html %>`	Output (raw)	Xuất HTML thô — chỉ dùng khi nội dung do bạn kiểm soát (không phải user input)
`<% code %>`	Scriptlet	Chạy code JS: if, for, khai báo biến, gọi function
`<%# comment %>`	Comment	Ghi chú trong template — không xuất ra HTML
`<%- include(path) %>`	Include	Nhúng file template khác vào — dùng cho partials

XSS VÀ EJS ESCAPING

Luôn dùng `<%= %>` (escaped) khi hiển thị dữ liệu từ user. Nếu dùng `<%- %>` với input từ người dùng, attacker có thể nhúng script độc hại vào trang — đây là lỗ hổng XSS (Cross-Site Scripting). Chỉ dùng `<%- %>` khi bạn tự tạo HTML (ví dụ: include partial template).

11.3 Layout và Partials

Vấn đề lớn nhất khi có nhiều trang: lặp code header và footer. Giải pháp là dùng partials — các template nhỏ có thể include vào nhiều trang.

TEXT Cấu trúc thư mục views/ Copy

```
views/
├── partials/
│   ├── header.ejs    ← DOCTYPE, head, mở thẻ body, nav
│   ├── footer.ejs    ← đóng body, scripts
│   └── navbar.ejs    ← navigation bar (optional)
├── index.ejs         ← trang chủ
├── posts/
│   ├── index.ejs     ← list bài viết
│   ├── show.ejs      ← chi tiết bài viết
│   ├── new.ejs       ← form tạo bài
│   └── edit.ejs      ← form sửa bài
└── 404.ejs           ← trang lỗi
```

HTML views/partials/header.ejs Copy

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title><%= typeof title !== 'undefined' ? title : 'Blog NodeJS' %></title>
  <link rel="stylesheet" href="/css/style.css">
</head>
<body>

<nav style="background:#1e293b;padding:12px 24px;display:flex;align-
items:center;gap:24px;">
  <a href="/" style="color:#fff;font-weight:700;text-decoration:none;font-
size:1.1rem;">
    📝 Blog NodeJS
  </a>
  <a href="/posts" style="color:#94a3b8;text-decoration:none;">Tất cả
bài</a>
  <a href="/posts/new" style="color:#94a3b8;text-decoration:none;">Viết bài
mới</a>
  <span style="flex:1"></span>
  <% if (typeof currentUser !== 'undefined' && currentUser) { %>
    <span style="color:#94a3b8;font-size:0.85rem;">Xin chào, <%=
currentUser.name %></span>
    <a href="/auth/logout" style="color:#ef4444;text-decoration:none;font-
size:0.85rem;">Đăng xuất</a>
  <% } else { %>
    <a href="/login" style="color:#0ea5e9;text-decoration:none;font-
size:0.85rem;">Đăng nhập</a>
  <% } %>
</nav>

<main style="max-width:800px;margin:32px auto;padding:0 16px;">
```

HTML views/partials/footer.ejs Copy

```
</main>

<footer style="text-align:center;padding:32px;color:#64748b;font-
size:0.85rem;border-top:1px solid #e2e8f0;margin-top:48px;">
  <p>&copy; <%= new Date().getFullYear() %> Blog NodeJS – Học EJS với
Express</p>
</footer>

<script src="/js/main.js"></script>
</body>
</html>
```

HTML views/index.ejs – Trang chủ dùng partials Copy

```
<!-- Include header – truyền biến title -->
<%- include('./partials/header', { title: 'Trang chủ – Blog NodeJS' }) %>

<h1 style="font-size:2rem;font-weight:800;margin-bottom:8px;">Bài viết mới
nhất</h1>
<p style="color:#64748b;margin-bottom:32px;">Có <%= posts.length %> bài
viết</p>

<% if (posts.length === 0) { %>
  <div style="text-align:center;padding:48px;background:#f8fafc;border-
radius:12px;">
    <p style="color:#64748b;">Chưa có bài viết nào.</p>
    <a href="/posts/new" style="color:#0ea5e9;">Viết bài đầu tiên →</a>
  </div>
<% } %>

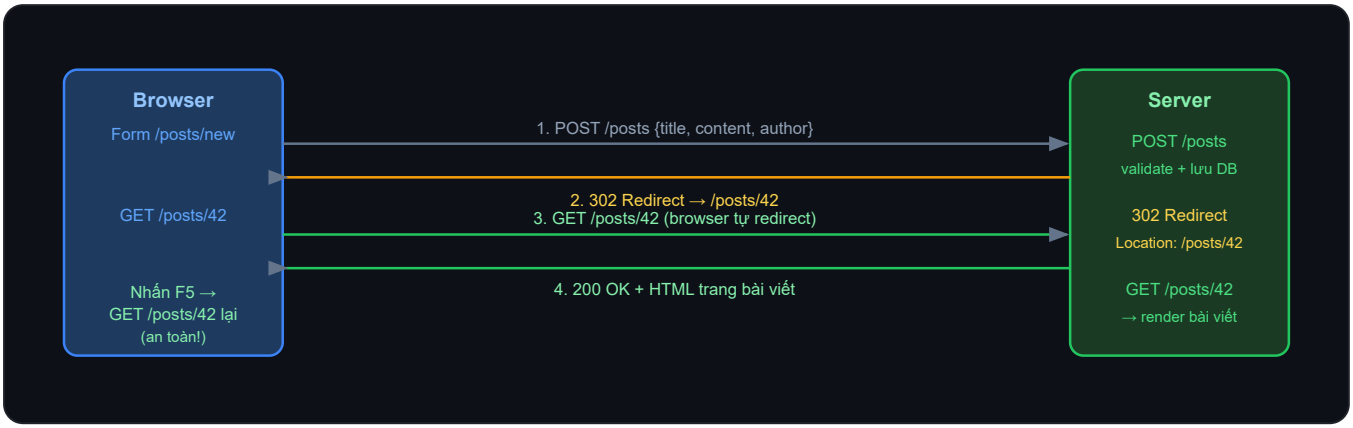
<div style="display:grid;gap:20px;">
  <% posts.forEach(post => { %>
    <article style="border:1px solid #e2e8f0;border-
radius:10px;padding:20px;transition:box-shadow 0.15s;">
      <h2 style="font-size:1.2rem;margin-bottom:8px;">
        <a href="/posts/<%= post.id %>" style="color:#1e293b;text-
decoration:none;">
          <%= post.title %>
        </a>
      </h2>
      <p style="color:#64748b;font-size:0.9rem;margin-bottom:12px;">
        <%= post.excerpt %>
      </p>
      <div style="display:flex;gap:16px;align-items:center;font-
size:0.8rem;color:#94a3b8;">
        <span>✍️ <%= post.author %></span>
        <span>🗓️ <%= new Date(post.createdAt).toLocaleDateString('vi-VN') %>
</span>
        <a href="/posts/<%= post.id %>" style="color:#0ea5e9;margin-
```

```
left:auto;">Đọc tiếp →</a>
      </div>
    </article>
  <% }) %>
</div>

<!— Include footer —>
<%- include('./partials/footer') %>
```

11.4 Form Handling và POST-Redirect-GET

HTML forms gửi dữ liệu đến server qua HTTP POST. Vấn đề kinh điển: nếu user nhấn F5 sau khi submit form, trình duyệt sẽ gửi lại request POST — tạo duplicate entry trong database. Pattern **POST-Redirect-GET** giải quyết điều này.



JAVASCRIPT routes/posts.js – Form handling với PRG pattern Copy

```javascript
const express = require('express');
const router  = express.Router();

// Parse form data (application/x-www-form-urlencoded)
router.use(express.urlencoded({ extended: false }));

// In-memory post store
const posts = [];
let nextId = 1;

// ─── GET /posts – Danh sách bài viết ───
router.get('/', (req, res) => {
  // Sắp xếp mới nhất trước
  const sorted = [...posts].sort((a, b) => b.createdAt - a.createdAt);
  res.render('posts/index', { posts: sorted, title: 'Tất cả bài viết' });
});

// ─── GET /posts/new – Form tạo bài mới ───
// PHẢI đặt trước /posts/:id để tránh "new" bị parse thành id
router.get('/new', (req, res) => {
  res.render('posts/new', { title: 'Viết bài mới', errors: [], old: {} });
});

// ─── POST /posts – Xử lý submit form ───
router.post('/', (req, res) => {
  const { title, content, author } = req.body;
  const errors = [];

  // Server-side validation
  if (!title?.trim())   errors.push('Tiêu đề không được để trống');
  if (!content?.trim()) errors.push('Nội dung không được để trống');
  if (!author?.trim())  errors.push('Tên tác giả không được để trống');
  if (title?.length > 200) errors.push('Tiêu đề tối đa 200 ký tự');

  if (errors.length > 0) {
```

```
    // Render lại form với lỗi và dữ liệu cũ (không redirect)
    return res.status(422).render('posts/new', {
      title: 'Viết bài mới',
      errors,
      old: { title, content, author }, // giữ lại input của user
    });
  }

  const post = {
    id:        nextId++,
    title:     title.trim(),
    content:   content.trim(),
    author:    author.trim(),
    excerpt:   content.trim().slice(0, 150) + (content.length > 150 ? '...'
: ''),
    createdAt: Date.now(),
    updatedAt: Date.now(),
  };
  posts.push(post);

  // POST-Redirect-GET: redirect sau khi lưu thành công
  res.redirect(`/posts/${post.id}`);
});

// ─── GET /posts/:id - Chi tiết bài viết ──
router.get('/:id', (req, res) => {
  const post = posts.find(p => p.id === parseInt(req.params.id));
  if (!post) return res.status(404).render('404', { title: 'Không tìm thấy'
});

  res.render('posts/show', { post, title: post.title });
});

// ─── GET /posts/:id/edit - Form sửa bài ──
router.get('/:id/edit', (req, res) => {
  const post = posts.find(p => p.id === parseInt(req.params.id));
  if (!post) return res.status(404).render('404', { title: 'Không tìm thấy'
});

  res.render('posts/edit', { post, title: `Sửa: ${post.title}`, errors: []
});
```

```
});

// ── POST /posts/:id/update – Cập nhật bài (dùng POST thay PUT vì HTML
form) ──
router.post('/:id/update', (req, res) => {
  const post = posts.find(p => p.id === parseInt(req.params.id));
  if (!post) return res.status(404).render('404', { title: 'Không tìm thấy'
});

  const { title, content, author } = req.body;
  const errors = [];

  if (!title?.trim())   errors.push('Tiêu đề không được để trống');
  if (!content?.trim()) errors.push('Nội dung không được để trống');

  if (errors.length > 0) {
    return res.status(422).render('posts/edit', { post: { ...post,
...req.body }, title: `Sửa: ${post.title}`, errors });
  }

  post.title     = title.trim();
  post.content   = content.trim();
  post.author    = author?.trim() || post.author;
  post.excerpt   = content.trim().slice(0, 150) + '...';
  post.updatedAt = Date.now();

  res.redirect(`/posts/${post.id}`);
});

// ── POST /posts/:id/delete – Xóa bài ──
router.post('/:id/delete', (req, res) => {
  const index = posts.findIndex(p => p.id === parseInt(req.params.id));
  if (index === -1) return res.status(404).render('404', { title: 'Không tìm
thấy' });

  posts.splice(index, 1);
  res.redirect('/posts');
});

module.exports = { router, posts };
```

11.5 Flash Messages với connect-flash

Flash message là thông báo chỉ hiển thị *một lần* — sau khi đọc, nó biến mất. Hoàn hảo cho phản hồi sau form submission: "Đã lưu thành công!" hay "Đã xóa bài viết". Package `connect-flash` lưu flash message vào session, cho phép hiện sau redirect.

SHELL Terminal Copy

```
npm install connect-flash express-session
```

JAVASCRIPT app.js – Cấu hình connect-flash Copy

```
const session = require('express-session');
const flash   = require('connect-flash');

// Flash cần session để hoạt động
app.use(session({
  secret: 'blog-secret',
  resave: false,
  saveUninitialized: false,
}));

app.use(flash());

// Middleware để truyền flash messages vào TẤT CẢ templates
// (không cần truyền thủ công từng route)
app.use((req, res, next) => {
  res.locals.success = req.flash('success');
  res.locals.error   = req.flash('error');
  res.locals.warning = req.flash('warning');
  next();
});

// Sử dụng trong route:
router.post('/', (req, res) => {
  // ... tạo bài viết ...
  req.flash('success', 'Bài viết đã được đăng thành công!');
  res.redirect(`/posts/${post.id}`);
});

router.post('/:id/delete', (req, res) => {
  // ... xóa bài viết ...
  req.flash('success', 'Đã xóa bài viết');
  req.flash('warning', 'Hành động này không thể hoàn tác');
  res.redirect('/posts');
});
```

HTML views/partials/flash.ejs – Component flash messages Copy

```
<!— Hiển thị flash messages – include trong header.ejs —>

<% if (typeof success !== 'undefined' && success.length > 0) { %>
  <% success.forEach(msg => { %>
    <div style="background:#d1fae5;border:1px solid #6ee7b7;border-left:4px
solid #10b981;
                border-radius:8px;padding:12px 16px;margin:12px
0;display:flex;
                align-items:center;gap:10px;" role="alert">
      <span style="font-size:1.1rem;">✅</span>
      <span style="color:#065f46;font-weight:500;"><%= msg %></span>
      <button onclick="this.parentElement.remove()"
              style="margin-
left:auto;background:none;border:none;cursor:pointer;
                     color:#6ee7b7;font-size:1.2rem;">×</button>
    </div>
  <% }) %>
<% } %>

<% if (typeof error !== 'undefined' && error.length > 0) { %>
  <% error.forEach(msg => { %>
    <div style="background:#fee2e2;border:1px solid #fca5a5;border-left:4px
solid #ef4444;
                border-radius:8px;padding:12px 16px;margin:12px
0;display:flex;
                align-items:center;gap:10px;" role="alert">
      <span style="font-size:1.1rem;">❌</span>
      <span style="color:#991b1b;font-weight:500;"><%= msg %></span>
      <button onclick="this.parentElement.remove()"
              style="margin-
left:auto;background:none;border:none;cursor:pointer;
                     color:#fca5a5;font-size:1.2rem;">×</button>
    </div>
  <% }) %>
<% } %>
```

```
<% if (typeof warning !== 'undefined' && warning.length > 0) { %>
  <% warning.forEach(msg => { %>
    <div style="background:#fef3c7;border:1px solid #fcd34d;border-left:4px
solid #f59e0b;
                border-radius:8px;padding:12px 16px;margin:12px
0;display:flex;
                align-items:center;gap:10px;" role="alert">
      <span style="font-size:1.1rem;">⚠️</span>
      <span style="color:#92400e;font-weight:500;"><%= msg %></span>
    </div>
  <% }) %>
<% } %>
```

11.6 Dự án: Blog Website với EJS

Tổng hợp tất cả: xây dựng một blog website hoàn chỉnh với CRUD, flash messages, layout chung, và validation. Dùng in-memory store — có thể dễ dàng thay bằng MongoDB ở chương 12.

TEXT

Cấu trúc dự án blog hoàn chỉnh

Copy

```
blog-ejs/
├── app.js
├── package.json
├── routes/
│   └── posts.js
├── views/
│   ├── partials/
│   │   ├── header.ejs
│   │   ├── footer.ejs
│   │   └── flash.ejs
│   ├── index.ejs          ← trang chủ
│   ├── 404.ejs
│   └── posts/
│       ├── index.ejs      ← danh sách bài
│       ├── show.ejs       ← chi tiết bài
│       ├── new.ejs        ← form tạo bài
│       └── edit.ejs       ← form sửa bài
└── public/
    └── css/
        └── style.css
```

JAVASCRIPT app.js – Entry point blog hoàn chỉnh Copy

```
const express = require('express');
const path    = require('path');
const session = require('express-session');
const flash   = require('connect-flash');
const { router: postsRouter } = require('./routes/posts');

const app = express();

// View engine
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

// Static files
app.use(express.static(path.join(__dirname, 'public')));

// Body parsing
app.use(express.urlencoded({ extended: false }));
app.use(express.json());

// Session + Flash
app.use(session({ secret: 'blog-dev-secret', resave: false,
saveUninitialized: false }));
app.use(flash());

// Đưa flash messages vào locals (dùng được trong mọi template)
app.use((req, res, next) => {
  res.locals.success = req.flash('success');
  res.locals.error   = req.flash('error');
  res.locals.warning = req.flash('warning');
  next();
});

// Routes
app.get('/', (req, res) => res.redirect('/posts'));
app.use('/posts', postsRouter);
```

```
// 404 handler
app.use((req, res) => {
  res.status(404).render('404', { title: 'Trang không tồn tại' });
});

// Error handler
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).render('404', { title: 'Lỗi server: ' + err.message });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Blog đang chạy tại http://localhost:${PORT}`);
});
```

HTML views/posts/show.ejs – Chi tiết bài viết Copy

```
<%- include('../partials/header', { title: post.title }) %>
<%- include('../partials/flash') %>

<article>
  <header style="margin-bottom:24px;padding-bottom:20px;border-bottom:1px
solid #e2e8f0;">
    <h1 style="font-size:1.8rem;font-weight:800;margin-bottom:8px;"><%=
post.title %></h1>
    <div style="display:flex;gap:16px;color:#64748b;font-size:0.85rem;">
      <span>✍️ <%= post.author %></span>
      <span>🗓️ <%= new Date(post.createdAt).toLocaleString('vi-VN') %>
</span>
      <% if (post.updatedAt !== post.createdAt) { %>
        <span>(đã cập nhật <%= new Date(post.updatedAt).toLocaleString('vi-
VN') %>)</span>
      <% } %>
    </div>
  </header>

  <div style="line-height:1.8;font-size:1rem;">
    <!-- Dùng <%- %> vì content là do admin nhập, và đã whitelist-sanitize -
->
    <%= post.content %>
  </div>

  <div style="margin-top:32px;padding-top:20px;border-top:1px solid #e2e8f0;
              display:flex;gap:12px;align-items:center;">
    <a href="/posts/<%= post.id %>/edit"
       style="padding:8px 16px;background:#0ea5e9;color:#fff;border-
radius:6px;
              text-decoration:none;font-size:0.85rem;">
      ✏️ Sửa bài
    </a>

    <!-- Form xóa bài – dùng POST vì HTML form không support DELETE -->
```

```
    <form action="/posts/<%= post.id %>/delete" method="POST"
          onsubmit="return confirm('Xác nhận xóa bài viết này?')">
      <button type="submit"
              style="padding:8px
16px;background:#ef4444;color:#fff;border:none;
                     border-radius:6px;cursor:pointer;font-size:0.85rem;">
        🗑 Xóa bài
      </button>
    </form>

    <a href="/posts" style="color:#64748b;font-size:0.85rem;margin-
left:auto;">
      ← Tất cả bài viết
    </a>
  </div>
</article>

<%- include('../partials/footer') %>
```

HTML views/posts/new.ejs — Form tạo bài mới Copy

```
<%- include('../partials/header', { title: 'Viết bài mới' }) %>
<%- include('../partials/flash') %>

<div style="max-width:680px;">
  <h1 style="font-size:1.6rem;font-weight:800;margin-bottom:24px;">✍️ Viết
bài mới</h1>

  <!-- Hiển thị lỗi validation -->
  <% if (errors.length > 0) { %>
    <div style="background:#fee2e2;border-left:4px solid #ef4444;
                border-radius:8px;padding:16px;margin-bottom:20px;">
      <p style="font-weight:700;color:#991b1b;margin-bottom:8px;">
        Vui lòng sửa các lỗi sau:
      </p>
      <ul style="margin-left:20px;color:#991b1b;">
        <% errors.forEach(err => { %>
          <li><%= err %></li>
        <% }) %>
      </ul>
    </div>
  <% } %>

  <form action="/posts" method="POST">
    <div style="margin-bottom:20px;">
      <label style="display:block;font-weight:600;margin-bottom:6px;">
        Tiêu đề <span style="color:#ef4444;">*</span>
      </label>
      <input type="text" name="title"
             value="<%= typeof old !== 'undefined' ? old.title || '' : ''
%>"
             placeholder="Nhập tiêu đề bài viết..."
             style="width:100%;padding:10px 14px;border:1px solid #e2e8f0;
                    border-radius:8px;font-size:1rem;font-family:inherit;"
             required>
    </div>
```

```
    <div style="margin-bottom:20px;">
      <label style="display:block;font-weight:600;margin-bottom:6px;">
        Tác giả <span style="color:#ef4444;">*</span>
      </label>
      <input type="text" name="author"
             value="<%= typeof old !== 'undefined' ? old.author || '' : ''
%>"
             placeholder="Tên của bạn ..."
             style="width:100%;padding:10px 14px;border:1px solid #e2e8f0;
                    border-radius:8px;font-size:1rem;font-family:inherit;"
             required>
    </div>

    <div style="margin-bottom:24px;">
      <label style="display:block;font-weight:600;margin-bottom:6px;">
        Nội dung <span style="color:#ef4444;">*</span>
      </label>
      <textarea name="content" rows="12"
                placeholder="Viết nội dung bài viết ..."
                style="width:100%;padding:10px 14px;border:1px solid
#e2e8f0;
                       border-radius:8px;font-size:1rem;font-family:inherit;
                       resize:vertical;line-height:1.7;"
                required><%= typeof old !== 'undefined' ? old.content || ''
: '' %></textarea>
    </div>

    <div style="display:flex;gap:12px;">
      <button type="submit"
              style="padding:10px
24px;background:#0ea5e9;color:#fff;border:none;
                     border-radius:8px;font-size:1rem;font-
weight:600;cursor:pointer;">
        📝 Đăng bài
      </button>
      <a href="/posts"
         style="padding:10px 20px;border:1px solid #e2e8f0;border-
radius:8px;
                color:#64748b;text-decoration:none;font-size:1rem;">
        Hủy
```

```
      </a>
    </div>
  </form>
</div>

<%- include('../partials/footer') %>
```

HTML views/posts/edit.ejs — Form sửa bài viết Copy

```
<%- include('../partials/header', { title: 'Sửa bài: ' + post.title }) %>
<%- include('../partials/flash') %>

<div style="max-width:680px;">
  <h1 style="font-size:1.6rem;font-weight:800;margin-bottom:24px;">✏️ Sửa
bài viết</h1>

  <% if (errors.length > 0) { %>
    <div style="background:#fee2e2;border-left:4px solid #ef4444;
                border-radius:8px;padding:16px;margin-bottom:20px;">
      <ul style="margin-left:20px;color:#991b1b;">
        <% errors.forEach(err => { %><li><%= err %></li><% }) %>
      </ul>
    </div>
  <% } %>

  <form action="/posts/<%= post.id %>/update" method="POST">
    <div style="margin-bottom:20px;">
      <label style="display:block;font-weight:600;margin-bottom:6px;">Tiêu
đề</label>
      <input type="text" name="title" value="<%= post.title %>"
             style="width:100%;padding:10px 14px;border:1px solid #e2e8f0;
                    border-radius:8px;font-size:1rem;font-family:inherit;"
required>
    </div>

    <div style="margin-bottom:20px;">
      <label style="display:block;font-weight:600;margin-bottom:6px;">Tác
giả</label>
      <input type="text" name="author" value="<%= post.author %>"
             style="width:100%;padding:10px 14px;border:1px solid #e2e8f0;
                    border-radius:8px;font-size:1rem;font-family:inherit;"
required>
    </div>
```

```
    <div style="margin-bottom:24px;">
      <label style="display:block;font-weight:600;margin-bottom:6px;">Nội
dung</label>
      <textarea name="content" rows="12"
                style="width:100%;padding:10px 14px;border:1px solid
#e2e8f0;
                       border-radius:8px;font-size:1rem;font-family:inherit;
                       resize:vertical;line-height:1.7;"
                required><%= post.content %></textarea>
    </div>

    <div style="display:flex;gap:12px;">
      <button type="submit"
              style="padding:10px
24px;background:#0ea5e9;color:#fff;border:none;
                     border-radius:8px;font-size:1rem;font-
weight:600;cursor:pointer;">
        💾 Lưu thay đổi
      </button>
      <a href="/posts/<%= post.id %>"
         style="padding:10px 20px;border:1px solid #e2e8f0;border-
radius:8px;
                color:#64748b;text-decoration:none;">Hủy</a>

      <form action="/posts/<%= post.id %>/delete" method="POST"
            style="margin-left:auto;"
            onsubmit="return confirm('Xác nhận xóa?')">
        <button type="submit"
                style="padding:10px 16px;background:none;color:#ef4444;
                       border:1px solid #ef4444;border-
radius:8px;cursor:pointer;">
          🗑 Xóa
        </button>
      </form>
    </div>
  </form>
</div>

<%- include('../partials/footer') %>
```

CHẠY DỰ ÁN BLOG

Sau khi tạo xong tất cả files:

```shell
cd blog-ejs
npm install express ejs express-session connect-flash
node app.js
# Mở http://localhost:3000
```

METHOD-OVERRIDE CHO PUT VÀ DELETE

HTML form chỉ hỗ trợ GET và POST. Để dùng PUT/PATCH/DELETE, cài `npm install method-override` và thêm `?_method=DELETE` vào action URL. Trong chương này chúng ta dùng route `/posts/:id/delete` (POST) để đơn giản hơn. Cách dùng `method-override` sẽ được giới thiệu ở chương REST API.

Tóm tắt chương 11

- **SSR** phù hợp cho nội dung động cần SEO; **CSR** cho app phức tạp; **SSG** cho nội dung tĩnh
- **EJS** nhúng JS vào HTML: `<%= %>` (escaped), `<%- %>` (raw), `<% %>` (code), `<%- include() %>` (partial)
- **Partials** giải quyết code lặp: header/footer dùng chung, include với biến truyền vào
- **POST-Redirect-GET** tránh form resubmission khi refresh trang
- **Flash messages** thông báo kết quả sau redirect, tự xóa sau khi đọc một lần
- Luôn dùng `<%= %>` (escaped) với user input để tránh XSS

Bài tập thực hành

DỄ

11.1 Trang About

Thêm trang `/about` vào blog: tạo `views/about.ejs`, thêm route `GET /about` trong `app.js`, và thêm link "Về chúng tôi" vào navigation bar trong `views/partials/header.ejs`. Trang about hiển thị thông tin về blog và tác giả.

TRUNG BÌNH

11.2 Phân trang (Pagination)

Implement phân trang trên trang chủ: 5 bài mỗi trang. Route `GET /posts?page=2` hiển thị trang 2. Hiển thị nút Prev/Next và số trang hiện tại (`Trang 2 / 4`). Ẩn nút Prev ở trang 1, ẩn nút Next ở trang cuối. Truyền `{ posts, currentPage, totalPages, hasPrev, hasNext }` vào template.

KHÓ

11.3 Tính năng tìm kiếm với keyword highlight

Implement `GET /posts?q=keyword` : lọc bài viết theo title và content. Trong template kết quả, highlight từ khóa trong tiêu đề bằng thẻ `<mark>` (dùng `<%⟶` với string đã sanitize). Nếu không có kết quả, hiện thông báo "Không tìm thấy bài nào cho '*keyword*'". Thêm form search vào navigation bar. Kết hợp với phân trang từ bài 11.2.

PHẦN 4 — DATABASE VÀ LƯU TRỮ

Chương 12: MongoDB với Mongoose — Database NoSQL

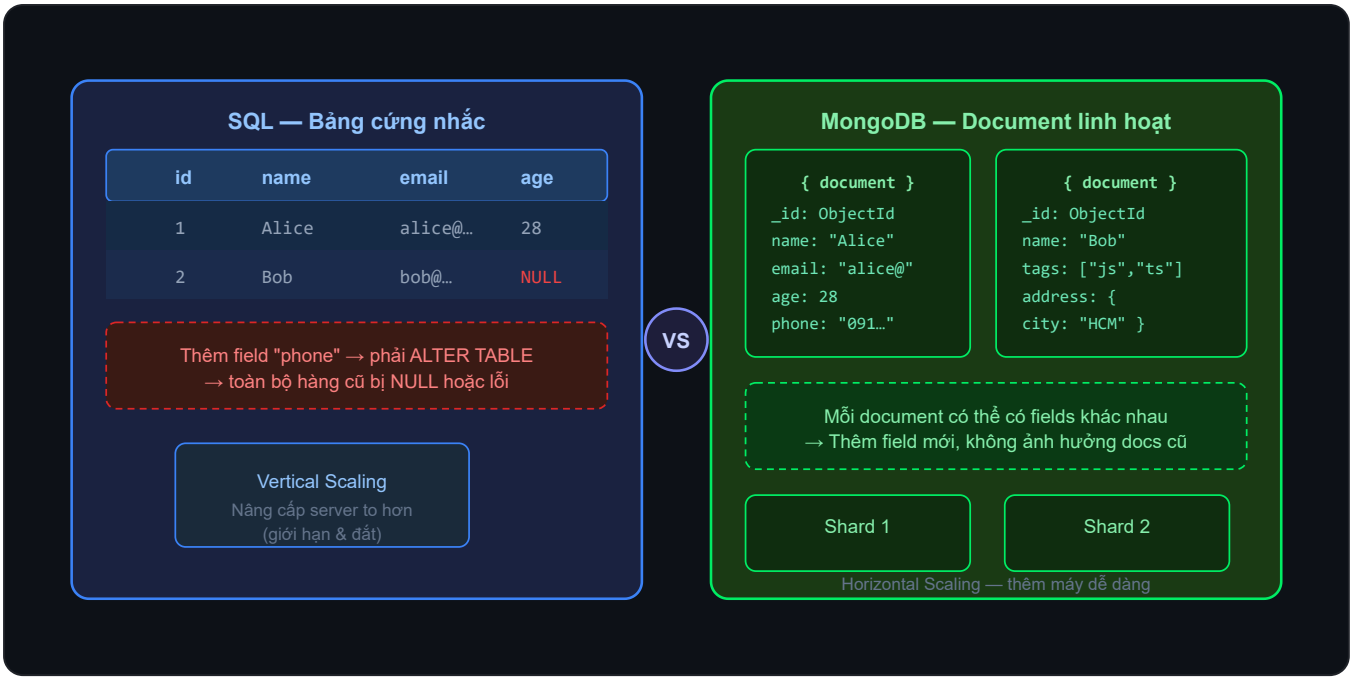
MongoDB lưu data như JSON object — không cần schema cứng nhắc, scale dễ dàng. Mongoose thêm validation, hooks, và relationships trên nền MongoDB, biến việc làm việc với database thành trải nghiệm tự nhiên và an toàn.

🎯 **SAU CHƯƠNG NÀY BẠN SẼ**

- Hiểu sự khác biệt NoSQL vs SQL và khi nào chọn MongoDB
- Kết nối NodeJS với MongoDB Atlas (cloud miễn phí)
- Định nghĩa Schema và Model với Mongoose
- Thực hiện CRUD đầy đủ: Create, Read, Update, Delete
- Dùng Population để liên kết các document
- Viết Middleware hooks (pre/post) để tự động hóa logic
- Aggregation pipeline cho thống kê phức tạp
- Tạo Indexes để tăng tốc truy vấn

12.1 NoSQL vs SQL — Chọn Gì Cho Đúng?

Trước khi học MongoDB, cần hiểu rõ **tại sao** bạn chọn nó thay vì SQL. Đây không phải là "cái nào tốt hơn" — mà là "cái nào phù hợp hơn cho bài toán này".



Tiêu chí	SQL (MySQL, PostgreSQL)	MongoDB (NoSQL)
Cấu trúc dữ liệu	Bảng với hàng và cột cố định	Collection chứa Documents (JSON)
Schema	Cứng nhắc — phải định nghĩa trước	Linh hoạt — có thể thay đổi
Quan hệ	JOIN giữa các bảng	Embed hoặc Reference + populate
ACID	Đầy đủ natively	Hỗ trợ từ v4.0 (multi-doc transactions)
Scale	Vertical (nâng cấp server)	Horizontal (sharding)
Khi nào dùng	Tài chính, quan hệ phức tạp, báo cáo	Social, catalog, content, real-time

MongoDB phù hợp khi nào?

- **Social media** — Mỗi user profile có fields khác nhau (bio, links, preferences)
- **Product catalog** — Sản phẩm có attributes rất khác nhau (điện thoại vs áo quần)
- **Content management** — Blog, CMS, nơi mỗi loại content có cấu trúc riêng
- **Real-time analytics** — Ghi log nhanh, không cần schema chặt
- **IoT data** — Cảm biến gửi dữ liệu với format biến đổi

12.2 MongoDB Atlas — Setup Cloud Miễn Phí

MongoDB Atlas là dịch vụ cloud chính thức của MongoDB. Tier M0 miễn phí hoàn toàn — đủ dùng cho học tập và project nhỏ.

Bước 1: Tạo tài khoản và cluster

1. Vào `cloud.mongodb.com` → Sign up miễn phí
2. Chọn **Create a deployment** → **M0 Free**
3. Chọn provider (AWS/GCP/Azure) và region gần nhất (Singapore)
4. Đặt tên cluster và tạo
5. Tạo **Database User**: username + password mạnh
6. **Network Access** → Add IP Address → `0.0.0.0/0` (cho phép mọi IP trong development)

Bước 2: Lấy connection string

TEXT — Connection String — Copy

```
mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/<dbname>?
retryWrites=true&w=majority

# Ví dụ thực tế:
mongodb+srv://admin:Passw0rd@cluster0.abc12.mongodb.net/myapp?
retryWrites=true&w=majority
```

Bước 3: Cài Mongoose và kết nối

SHELL — Terminal — Copy

```
npm install mongoose dotenv
```

JAVASCRIPT config/database.js Copy

```javascript
// config/database.js
const mongoose = require('mongoose');

const MONGODB_URI = process.env.MONGODB_URI;
const MAX_RETRIES = 5;
let retryCount = 0;

async function connectDB() {
  try {
    await mongoose.connect(MONGODB_URI, {
      // Mongoose 6+ không cần useNewUrlParser, useUnifiedTopology nữa
      serverSelectionTimeoutMS: 5000,  // Timeout sau 5s nếu không kết nối được
      heartbeatFrequencyMS: 10000,     // Kiểm tra kết nối mỗi 10s
    });

    console.log('✅ MongoDB kết nối thành công');
    retryCount = 0; // Reset counter khi thành công

  } catch (error) {
    console.error(`❌ MongoDB lỗi (lần ${retryCount + 1}/${MAX_RETRIES}):`, error.message);

    if (retryCount < MAX_RETRIES) {
      retryCount++;
      const delay = Math.min(1000 * 2 ** retryCount, 30000); // Exponential backoff, max 30s
      console.log(`Thử lại sau ${delay / 1000}s ... `);
      setTimeout(connectDB, delay);
    } else {
      console.error('Đã thử quá nhiều lần. Thoát ... ');
      process.exit(1);
    }
  }
}

// Lắng nghe các events của Mongoose
mongoose.connection.on('connected', () => {
```

```
  console.log('🟢 Mongoose connected to MongoDB');
});

mongoose.connection.on('error', (err) => {
  console.error('🔴 Mongoose connection error:', err);
});

mongoose.connection.on('disconnected', () => {
  console.log('🟠 Mongoose disconnected - Đang thử kết nối lại...');
  connectDB();
});

// Đóng kết nối khi app tắt
process.on('SIGINT', async () => {
  await mongoose.connection.close();
  console.log('MongoDB kết nối đã đóng (app shutdown)');
  process.exit(0);
});

module.exports = { connectDB };
```

JAVASCRIPT app.js Copy

```
require('dotenv').config();
const express = require('express');
const { connectDB } = require('./config/database');

const app = express();
app.use(express.json());

// Kết nối DB trước khi start server
connectDB().then(() => {
  app.listen(3000, () => console.log('Server chạy trên port 3000'));
});
```

12.3 Schema và Model — Blueprint của Document

Mongoose thêm lớp **Schema** lên MongoDB để định nghĩa cấu trúc, kiểu dữ liệu và validation. Dù MongoDB linh hoạt, trong thực tế bạn vẫn cần đảm bảo data nhất quán — đó là việc của Schema.

Các kiểu dữ liệu trong Mongoose

Type	Ví dụ	Ghi chú
`String`	`name: String`	Text, có thể dùng trim, lowercase, uppercase
`Number`	`age: Number`	Integer hoặc float, có min/max
`Boolean`	`isActive: Boolean`	true/false
`Date`	`birthday: Date`	Lưu dạng ISO, so sánh được
`mongoose.Schema.Types.ObjectId`	`author: ObjectId`	Reference đến document khác
`[String]`	`tags: [String]`	Mảng các giá trị
`Mixed`	`metadata: Mixed`	Bất kỳ kiểu gì (dùng cẩn thận)

Schema đầy đủ cho hệ thống user

JAVASCRIPT models/User.js Copy

```javascript
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');

const userSchema = new mongoose.Schema({
  // String với validation
  name: {
    type: String,
    required: [true, 'Tên không được để trống'],
    trim: true,              // Tự cắt khoảng trắng 2 đầu
    minlength: [2, 'Tên phải từ 2 ký tự'],
    maxlength: [50, 'Tên không quá 50 ký tự'],
  },

  email: {
    type: String,
    required: [true, 'Email không được để trống'],
    unique: true,            // Tạo unique index
    lowercase: true,         // Tự chuyển thành chữ thường
    trim: true,
    match: [/^\S+@\S+\.\S+$/, 'Email không hợp lệ'],
  },

  password: {
    type: String,
    required: true,
    minlength: [6, 'Mật khẩu tối thiểu 6 ký tự'],
    select: false,           // Mặc định KHÔNG trả về trong query (bảo mật!)
  },

  role: {
    type: String,
    enum: {
      values: ['user', 'moderator', 'admin'],
      message: 'Role không hợp lệ: {VALUE}',
    },
    default: 'user',
  },
```

```
  avatar: {
    type: String,
    default: null,
  },

  bio: {
    type: String,
    maxlength: 200,
    default: '',
  },

  // Reference đến document khác
  followers: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',  // Tên model để populate
  }],

  isActive: {
    type: Boolean,
    default: true,
  },

  lastLogin: {
    type: Date,
    default: null,
  },

}, {
  timestamps: true,         // Tự thêm createdAt và updatedAt
  toJSON: { virtuals: true },  // Include virtual fields khi convert sang
JSON
  toObject: { virtuals: true },
});

// ── Virtual field (không lưu DB, tính khi cần) ──
userSchema.virtual('followerCount').get(function () {
  return this.followers ? this.followers.length : 0;
});

// ── Instance method ──
userSchema.methods.comparePassword = async function (candidatePassword) {
```

```
  return bcrypt.compare(candidatePassword, this.password);
};

// ── Static method ──
userSchema.statics.findByEmail = function (email) {
  return this.findOne({ email: email.toLowerCase() });
};

const User = mongoose.model('User', userSchema);

module.exports = User;
```

> 💡 **SELECT: FALSE — QUAN TRỌNG CHO BẢO MẬT**
>
> Đặt `select: false` cho field `password` nghĩa là mọi query mặc định sẽ **không trả về password**. Khi cần (đăng nhập), dùng `.select('+password')` để lấy về.

12.4 CRUD Operations — Làm Việc Với Dữ Liệu

Create — Tạo mới

JAVASCRIPT Tạo document Copy

```
const User = require('./models/User');

// Cách 1: Dùng new + save (có thể add logic trước save)
const user = new User({
  name: 'Nguyễn Văn A',
  email: 'nguyenvana@gmail.com',
  password: 'hashedPassword123',
});
const saved = await user.save();
console.log(saved._id); // ObjectId tự generate

// Cách 2: User.create — ngắn gọn hơn
const user2 = await User.create({
  name: 'Trần Thị B',
  email: 'tranthib@gmail.com',
  password: 'hashedPassword456',
});

// Tạo nhiều cùng lúc
const users = await User.insertMany([
  { name: 'A', email: 'a@example.com', password: 'pass1' },
  { name: 'B', email: 'b@example.com', password: 'pass2' },
], { ordered: false }); // ordered: false — các document khác vẫn insert dù
có lỗi
```

Read — Đọc dữ liệu

JAVASCRIPT Query dữ liệu Copy

```
// Tìm tất cả (có filter)
const admins = await User.find({ role: 'admin', isActive: true });

// Tìm 1 document
const user = await User.findOne({ email: 'nguyenvana@gmail.com' });

// Tìm theo _id
const userById = await User.findById('507f1f77bcf86cd799439011');

// ── Query operators ──
const users = await User.find({
  role: { $in: ['admin', 'moderator'] },   // role là admin hoặc moderator
  createdAt: { $gte: new Date('2024-01-01') }, // tạo từ đầu 2024
  name: { $regex: /nguyen/i },              // tên chứa "nguyen" (case-
insensitive)
});

// ── Chaining methods ──
const result = await User
  .find({ isActive: true })
  .select('name email avatar role')   // Chỉ lấy các fields này
  .sort({ createdAt: -1 })           // Mới nhất trước (-1) hoặc cũ nhất (1)
  .limit(10)                         // Tối đa 10 kết quả
  .skip(20)                          // Bỏ qua 20 đầu (cho pagination)
  .lean();                           // Trả về plain JS object (nhanh hơn
Mongoose document)

// ── Pagination helper ──
async function getUsers({ page = 1, limit = 10, search = '' }) {
  const skip = (page - 1) * limit;
  const filter = search
    ? { $or: [
        { name: { $regex: search, $options: 'i' } },
        { email: { $regex: search, $options: 'i' } },
      ]}
    : {};

  const [users, total] = await Promise.all([
```

```
    User.find(filter).select('-password').sort({ createdAt: -1
}).skip(skip).limit(limit).lean(),
    User.countDocuments(filter),
  ]);

  return {
    data: users,
    pagination: {
      page,
      limit,
      total,
      totalPages: Math.ceil(total / limit),
      hasNext: page * limit < total,
      hasPrev: page > 1,
    },
  };
}
```

Update — Cập nhật

JAVASCRIPT Update operations Copy

```
// findByIdAndUpdate – trả về document sau khi update
const updated = await User.findByIdAndUpdate(
  userId,
  { $set: { name: 'Tên Mới', bio: 'Bio mới' } },
  {
    new: true,              // Trả về document SAU khi update (default là
trước)
    runValidators: true,    // Chạy schema validators khi update
  }
);

// Update nhiều documents
await User.updateMany(
  { isActive: false, lastLogin: { $lt: new Date('2023-01-01') } },
  { $set: { role: 'user' } }
);

// ── Atomic operators ──
// $set: set field value
// $unset: xóa field
// $inc: tăng/giảm số
// $push: thêm vào array
// $pull: xóa khỏi array
// $addToSet: thêm vào array nếu chưa có (unique)

// Ví dụ: tăng view count và thêm tag
await Post.findByIdAndUpdate(postId, {
  $inc: { viewCount: 1 },
  $addToSet: { tags: 'nodejs' },  // Thêm 'nodejs' vào tags nếu chưa có
});

// Ví dụ: xóa follower
await User.findByIdAndUpdate(userId, {
  $pull: { followers: followerId },
});
```

Delete — Xóa

JAVASCRIPT Delete operations Copy

```
// Xóa 1 document theo ID
const deleted = await User.findByIdAndDelete(userId);
if (!deleted) throw new Error('User không tồn tại');

// Xóa nhiều
const result = await User.deleteMany({
  isActive: false,
  createdAt: { $lt: new Date('2022-01-01') },
});
console.log(`Đã xóa ${result.deletedCount} users`);

// Soft delete — không xóa thật, chỉ đánh dấu
await User.findByIdAndUpdate(userId, {
  $set: {
    isDeleted: true,
    deletedAt: new Date(),
  },
});
```

12.5 Population — Liên Kết Documents

MongoDB không có JOIN như SQL, nhưng Mongoose có **populate** — tự động thay thế ObjectId bằng document thực tế từ collection khác.

JAVASCRIPT models/Post.js Copy

```javascript
const mongoose = require('mongoose');

const commentSchema = new mongoose.Schema({
  author: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
  },
  content: { type: String, required: true },
  likes: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  createdAt: { type: Date, default: Date.now },
});

const postSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true },
  content: { type: String, required: true },
  author: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',   // Tên model để populate
    required: true,
  },
  tags: [String],
  comments: [commentSchema],  // Embed sub-documents
  viewCount: { type: Number, default: 0 },
  isPublished: { type: Boolean, default: false },
}, { timestamps: true });

module.exports = mongoose.model('Post', postSchema);
```

JAVASCRIPT Sử dụng populate Copy

```
const Post = require('./models/Post');

// Populate đơn giản
const post = await Post.findById(postId)
  .populate('author', 'name email avatar');  // Chỉ lấy name, email, avatar
từ User

// Populate nhiều fields
const posts = await Post.find({ isPublished: true })
  .populate('author', 'name avatar')
  .populate('comments.author', 'name avatar')
  .sort({ createdAt: -1 });

// Populate với filter
const post2 = await Post.findById(postId)
  .populate({
    path: 'author',
    select: 'name email',
    match: { isActive: true },   // Chỉ populate nếu user đang active
  });

// Lấy posts với thông tin author + số lượng comments
const postsWithStats = await Post.find({ isPublished: true })
  .populate('author', 'name avatar')
  .lean()
  .then(posts => posts.map(p => ({
    ...p,
    commentCount: p.comments.length,
  })));
```

📌 **EMBED VS REFERENCE**

Embed (nhúng vào document — như comments trong post): khi data thường được đọc cùng nhau, không bao giờ đọc riêng, array không quá lớn (<100 items). **Reference** (ObjectId + populate — như author của post): khi data được dùng độc lập, cần update riêng, hoặc array có thể rất lớn.

12.6 Middleware (Hooks) — Tự Động Hóa Logic

Mongoose Middleware (còn gọi là hooks) cho phép thực thi code **trước** hoặc **sau** một operation nhất định. Dùng để hash password, ghi log, xóa data liên quan, v.v.

JAVASCRIPT models/User.js — với middleware Copy

```
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');

const userSchema = new mongoose.Schema({ /* ...fields như trên... */ });

// ── PRE SAVE HOOK — Hash password trước khi lưu ──
userSchema.pre('save', async function (next) {
  // 'this' là document đang được save
  // Chỉ hash khi password thực sự thay đổi
  if (!this.isModified('password')) return next();

  try {
    const saltRounds = 12;
    this.password = await bcrypt.hash(this.password, saltRounds);
    next();
  } catch (error) {
    next(error); // Truyền lỗi về Mongoose
  }
});

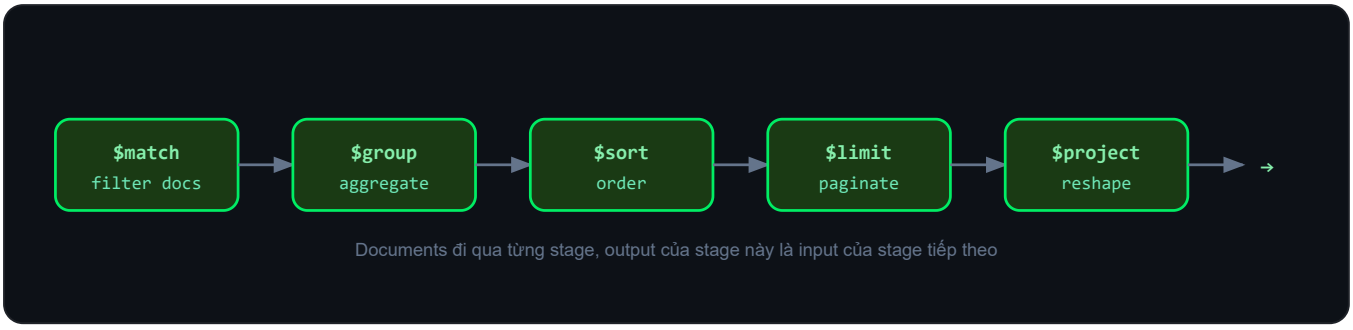
// ── PRE FIND HOOK — Tự filter users bị xóa mềm ──
userSchema.pre(/^find/, function (next) {
  // 'this' là query object
  // Tự động filter isDeleted nếu không có trong query
  if (this._conditions.isDeleted === undefined) {
    this.where({ isDeleted: { $ne: true } });
  }
  next();
});

// ── POST SAVE HOOK — Ghi log sau khi tạo user mới ──
userSchema.post('save', function (doc, next) {
  if (doc.wasNew) {  // Chỉ log khi là document mới
    console.log(`📧 User mới tạo: ${doc.email} lúc ${doc.createdAt}`);
    // Có thể gửi email welcome ở đây
  }
  next();
});
```

```
// ── POST DELETE HOOK – Cleanup data liên quan ──
userSchema.post('findOneAndDelete', async function (doc) {
  if (!doc) return;
  // Xóa tất cả posts của user này
  await mongoose.model('Post').deleteMany({ author: doc._id });
  console.log(`🗑️ Đã xóa posts của user: ${doc.email}`);
});
```

12.7 Aggregation Pipeline — Phân Tích Dữ Liệu

Aggregation pipeline cho phép xử lý và biến đổi data trực tiếp trong MongoDB, rất mạnh cho báo cáo và thống kê. Dữ liệu đi qua các **stages** theo thứ tự.



JAVASCRIPT Aggregation examples Copy

```
const Post = require('./models/Post');

// ── Thống kê bài viết theo tháng ──
const monthlyStats = await Post.aggregate([
  // Stage 1: Chỉ lấy bài đã publish trong năm 2024
  { $match: {
    isPublished: true,
    createdAt: { $gte: new Date('2024-01-01'), $lt: new Date('2025-01-01') }
  }},

  // Stage 2: Group theo năm-tháng
  { $group: {
    _id: {
      year: { $year: '$createdAt' },
      month: { $month: '$createdAt' },
    },
    count: { $sum: 1 },
    totalViews: { $sum: '$viewCount' },
    avgViews: { $avg: '$viewCount' },
  }},

  // Stage 3: Sort từ mới nhất
  { $sort: { '_id.year': -1, '_id.month': -1 } },

  // Stage 4: Format lại output
  { $project: {
    _id: 0,
    period: { $concat: [
      { $toString: '$_id.year' }, '-',
      { $cond: [{ $lt: ['$_id.month', 10] }, '0', ''] },
      { $toString: '$_id.month' },
    ]},
    count: 1,
    totalViews: 1,
    avgViews: { $round: ['$avgViews', 0] },
  }},
]);
```

```
// ── Top 5 tags phổ biến nhất ──
const topTags = await Post.aggregate([
  { $match: { isPublished: true } },
  { $unwind: '$tags' },  // Tách mỗi tag thành 1 document riêng
  { $group: { _id: '$tags', count: { $sum: 1 } }},
  { $sort: { count: -1 } },
  { $limit: 5 },
  { $project: { _id: 0, tag: '$_id', count: 1 } },
]);

// ── $lookup - JOIN giữa 2 collections ──
const postsWithAuthorDetails = await Post.aggregate([
  { $match: { isPublished: true } },
  {
    $lookup: {
      from: 'users',          // Tên collection (lowercase, plural)
      localField: 'author',   // Field trong Post
      foreignField: '_id',    // Field trong User
      as: 'authorInfo',       // Tên field kết quả
    }
  },
  { $unwind: '$authorInfo' },
  { $project: {
    title: 1, viewCount: 1, createdAt: 1,
    'author.name': '$authorInfo.name',
    'author.avatar': '$authorInfo.avatar',
  }},
  { $limit: 10 },
]);
```

12.8 Indexes — Tăng Tốc Truy Vấn

Không có index, MongoDB phải quét toàn bộ collection mỗi query. Index như mục lục sách — giúp tìm nhanh mà không cần đọc từng trang.

JAVASCRIPT Tạo indexes Copy

```
const postSchema = new mongoose.Schema({ /* ... */ });

// Index đơn
postSchema.index({ createdAt: -1 });           // Sort theo thời gian tạo
postSchema.index({ isPublished: 1 });           // Filter theo trạng thái

// Compound index – phù hợp với query có nhiều điều kiện
postSchema.index({ isPublished: 1, createdAt: -1 });  // Lọc published rồi
sort

// Text index – cho full-text search
postSchema.index({ title: 'text', content: 'text' }, {
  weights: { title: 10, content: 1 },  // title quan trọng hơn content (10x)
  name: 'post_search_index',
});

// Unique index
postSchema.index({ slug: 1 }, { unique: true });

// Sparse index – chỉ index documents có field này
postSchema.index({ githubId: 1 }, { sparse: true });

// TTL index – tự xóa document sau một thời gian
const sessionSchema = new mongoose.Schema({
  token: String,
  createdAt: { type: Date, expires: '7d' },  // Tự xóa sau 7 ngày
});
```

JAVASCRIPT Full-text search Copy

```javascript
// Search với text index (phải có text index trước)
const results = await Post.find({
  $text: { $search: 'nodejs async programming' },
  isPublished: true,
}, {
  score: { $meta: 'textScore' },  // Thêm relevance score
})
.sort({ score: { $meta: 'textScore' } })  // Sort theo relevance
.limit(10)
.lean();

// Kiểm tra query plan (explain)
const plan = await Post.find({ isPublished: true })
  .sort({ createdAt: -1 })
  .explain('executionStats');

console.log('Index used:',
plan.queryPlanner.winningPlan.inputStage.indexName);
console.log('Docs scanned:', plan.executionStats.totalDocsExamined);
console.log('Docs returned:', plan.executionStats.nReturned);
```

⚠️ **INDEX KHÔNG PHẢI LÚC NÀO CŨNG TỐT**

Mỗi index chiếm dung lượng RAM và làm chậm Write operations (vì phải cập nhật index). Chỉ tạo index cho các fields thực sự hay được dùng trong query. Dùng `explain()` để kiểm tra query có dùng index không.

📋 Tóm tắt Chương 12

- **MongoDB** lưu data dạng BSON documents, linh hoạt hơn SQL về schema
- **Mongoose** thêm validation, type checking và hooks lên MongoDB
- **Schema + Model**: blueprint định nghĩa cấu trúc, validators, và behavior của documents
- **CRUD**: create/insertMany, find/findById, findByIdAndUpdate, findByIdAndDelete
- **Population**: thay ObjectId bằng document thực tế (tương tự JOIN)

- **Middleware hooks**: pre/post save, find, delete — tự động hóa logic
- **Aggregation pipeline**: $match, $group, $sort, $lookup cho phân tích phức tạp
- **Indexes**: tạo cho fields hay query, text index cho full-text search

🏋️ Bài Tập Thực Hành

DỄ

Bài 1: Mở rộng User Schema

Thêm fields `bio` (String, max 200 ký tự) và `website` (String, phải là URL hợp lệ) vào User schema. Viết route `PUT /users/:id/profile` chỉ cho phép update 2 fields này (không cho phép thay đổi email, role, password qua route này). Trả về user đã update (không có password).

TRUNG BÌNH

Bài 2: Comment System hoàn chỉnh

Xây dựng Comment system: tạo `Comment` schema với các fields `post` (ref Post), `author` (ref User), `content`, `likes` (array of User ref). Viết 5 routes: `GET /posts/:postId/comments` (với pagination + populate author), `POST /posts/:postId/comments`, `PUT /comments/:id`, `DELETE /comments/:id`, và `POST /comments/:id/like` (toggle like dùng $addToSet và $pull).

KHÓ

Bài 3: Full-Text Search có pagination

Implement endpoint `GET /api/search?q=nodejs&page=1&limit=10` dùng MongoDB text index. Kết quả phải: (1) Sort theo relevance score, (2) Có pagination đầy đủ, (3) Highlight matched text trong field `title` và `excerpt` (bọc từ khớp trong `<mark>` tag bằng JavaScript string processing sau khi query), (4) Trả về author name qua populate. Xử lý edge case: query rỗng, không có kết quả.

PHẦN 4 — DATABASE VÀ LƯU TRỮ

Chương 13: SQL với PostgreSQL và Prisma ORM

Khi data có quan hệ chặt chẽ và cần ACID transactions, SQL là lựa chọn tốt nhất. Prisma biến SQL thành code JavaScript/TypeScript type-safe, dễ đọc như viết object — không cần biết SQL phức tạp mà vẫn làm được mọi thứ.

🎯 **SAU CHƯƠNG NÀY BẠN SẼ**

- Hiểu SQL cơ bản: SELECT, INSERT, UPDATE, DELETE, JOIN
- Setup PostgreSQL miễn phí trên Supabase
- Viết Prisma schema, chạy migration, generate client
- Định nghĩa relations: one-to-many và many-to-many
- CRUD với Prisma Client (find, create, update, delete)
- Nested queries và include (tương đương JOIN)
- Transactions để đảm bảo tính toàn vẹn dữ liệu
- Aggregation và groupBy

13.1 SQL Fundamentals — Những Gì Cần Biết

SQL (Structured Query Language) là ngôn ngữ tiêu chuẩn để làm việc với relational database. Bạn không cần thành thạo SQL khi dùng Prisma, nhưng hiểu cơ bản giúp bạn debug và tối ưu hiệu quả hơn.

SELECT — Đọc dữ liệu

SQL — Queries cơ bản — Copy

```sql
-- Lấy tất cả users
SELECT * FROM users;

-- Lấy fields cụ thể
SELECT id, name, email, created_at FROM users;

-- WHERE — filter
SELECT * FROM users
WHERE role = 'admin' AND is_active = true;

-- ORDER BY, LIMIT, OFFSET — sort và pagination
SELECT * FROM users
WHERE role = 'user'
ORDER BY created_at DESC
LIMIT 10 OFFSET 20;  -- Page 3 (skip 20 = 2 pages × 10)

-- LIKE — tìm kiếm text
SELECT * FROM users
WHERE name ILIKE '%nguyen%';  -- ILIKE: case-insensitive (PostgreSQL)

-- COUNT — đếm
SELECT COUNT(*) as total FROM users WHERE is_active = true;

-- Aggregate functions
SELECT
  role,
  COUNT(*) as count,
  MAX(created_at) as latest
FROM users
GROUP BY role
ORDER BY count DESC;
```

JOIN — Kết hợp nhiều bảng

SQL JOIN examples Copy

```sql
-- INNER JOIN — chỉ lấy rows có match cả 2 bảng
SELECT
  posts.id,
  posts.title,
  posts.view_count,
  users.name AS author_name,
  users.email AS author_email
FROM posts
INNER JOIN users ON posts.author_id = users.id
WHERE posts.is_published = true
ORDER BY posts.created_at DESC
LIMIT 20;

-- LEFT JOIN — lấy tất cả posts, kể cả không có author
SELECT
  posts.title,
  COALESCE(users.name, 'Ẩn danh') AS author_name
FROM posts
LEFT JOIN users ON posts.author_id = users.id;

-- JOIN nhiều bảng
SELECT
  orders.id AS order_id,
  users.name AS customer,
  products.name AS product,
  order_items.quantity,
  order_items.price
FROM order_items
INNER JOIN orders ON order_items.order_id = orders.id
INNER JOIN users ON orders.user_id = users.id
INNER JOIN products ON order_items.product_id = products.id
WHERE orders.status = 'DELIVERED';
```

INSERT, UPDATE, DELETE

SQL Mutations Copy

```sql
-- INSERT — dùng $1, $2 để tránh SQL injection (parameterized query)
INSERT INTO posts (title, content, author_id, is_published)
VALUES ($1, $2, $3, $4)
RETURNING *;  -- PostgreSQL: trả về row vừa insert

-- UPDATE
UPDATE users
SET last_login = NOW(), updated_at = NOW()
WHERE id = $1
RETURNING id, name, last_login;

-- DELETE
DELETE FROM comments
WHERE post_id = $1 AND created_at < $2
RETURNING id;
```

> 💡 **DÙNG PG PACKAGE (RAW SQL)**
>
> Trước Prisma, người ta dùng `pg` package để query raw SQL. Prisma được xây trên nền đó và tự động generate SQL tối ưu cho bạn. Hiểu SQL raw giúp bạn debug khi Prisma generate query không như mong muốn.

13.2 Setup PostgreSQL với Supabase

Supabase là platform mã nguồn mở cung cấp PostgreSQL miễn phí với UI đẹp, real-time subscriptions, và Auth. Free tier: 500MB storage, 2 projects.

Bước 1: Tạo project Supabase

1. Vào `supabase.com` → Sign up (free)
2. Chọn **New Project** → đặt tên, tạo password DB (lưu lại!)
3. Chọn region gần nhất (Southeast Asia)
4. Đợi khoảng 2 phút để provision xong
5. Vào **Settings** → **Database** → **Connection string** → chọn **URI**

Bước 2: Cài Prisma

SHELL Terminal Copy

```
# Cài Prisma CLI và client
npm install prisma --save-dev
npm install @prisma/client

# Khởi tạo Prisma (tạo prisma/schema.prisma và .env)
npx prisma init --datasource-provider postgresql
```

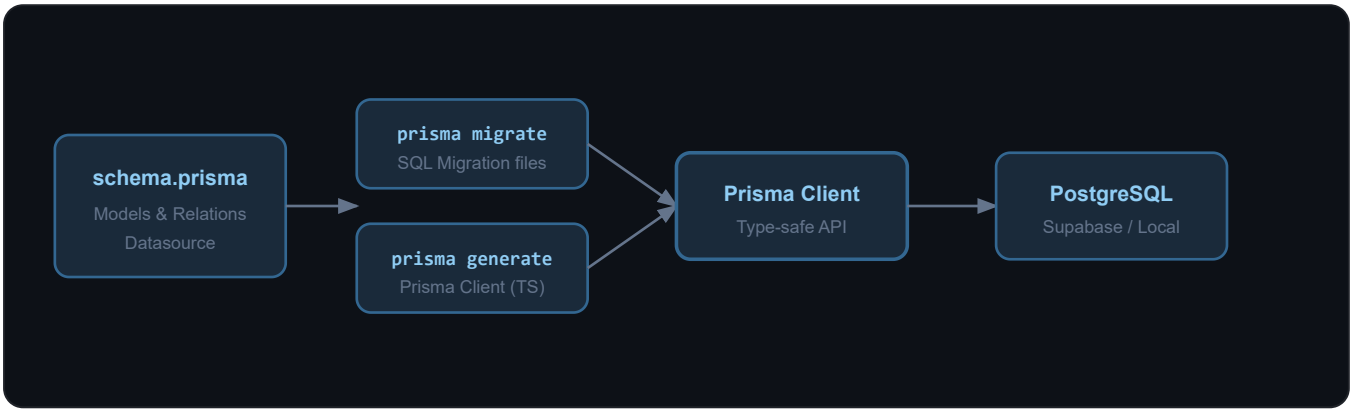
TEXT .env Copy

```
# Thay bằng connection string từ Supabase
DATABASE_URL="postgresql://postgres:[YOUR-
PASSWORD]@db.xxxx.supabase.co:5432/postgres"

# Prisma cần biến này nếu dùng Supabase connection pooler
DIRECT_URL="postgresql://postgres:[YOUR-
PASSWORD]@db.xxxx.supabase.co:5432/postgres"
```

13.3 Prisma Schema — Định Nghĩa Cấu Trúc Database

File `prisma/schema.prisma` là nguồn sự thật duy nhất cho toàn bộ cấu trúc database. Từ đây, Prisma sẽ generate SQL migration và TypeScript client.



Schema đầy đủ cho hệ thống e-commerce

PRISMA prisma/schema.prisma Copy

```prisma
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider  = "postgresql"
  url       = env("DATABASE_URL")
  directUrl = env("DIRECT_URL")  // Cần cho Supabase connection pooler
}

// ─── Models ──────────────────────────────────────────────

model User {
  id        Int      @id @default(autoincrement())
  email     String   @unique
  name      String
  password  String
  role      Role     @default(USER)
  avatar    String?  // ? = nullable (optional)
  bio       String?  @db.VarChar(200)
  orders    Order[]  // Relation: 1 user có nhiều orders
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt  // Tự update khi record thay đổi

  @@index([email])  // Index cho email (ngoài unique)
  @@map("users")    // Tên bảng thực trong DB là "users" (không phải "User")
}

model Category {
  id       Int       @id @default(autoincrement())
  name     String    @unique
  slug     String    @unique
  products Product[] @relation("ProductCategories") // Many-to-many

  @@map("categories")
}
```

```
model Product {
  id          Int         @id @default(autoincrement())
  name        String
  description String?
  price       Decimal     @db.Decimal(10, 2) // Chính xác cho tiền tệ
  stock       Int         @default(0)
  imageUrl    String?
  isActive    Boolean     @default(true)
  categories  Category[]  @relation("ProductCategories") // Many-to-many
  orderItems  OrderItem[]
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt

  @@index([isActive, price])
  @@map("products")
}

model Order {
  id         Int         @id @default(autoincrement())
  user       User        @relation(fields: [userId], references: [id])
  userId     Int
  status     OrderStatus @default(PENDING)
  total      Decimal     @db.Decimal(10, 2)
  address    Json        // Lưu địa chỉ dạng JSON object
  items      OrderItem[]
  note       String?
  createdAt  DateTime    @default(now())
  updatedAt  DateTime    @updatedAt

  @@index([userId, status])
  @@map("orders")
}

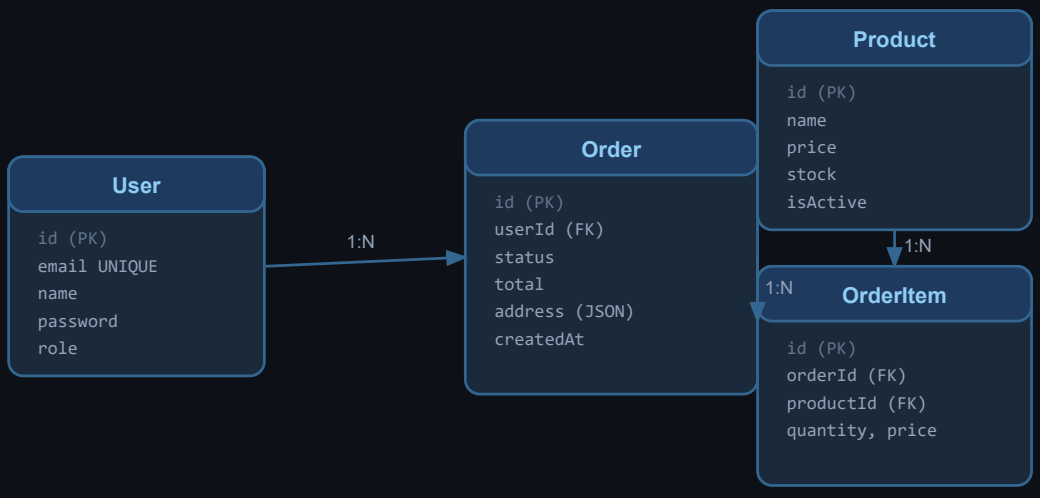
model OrderItem {
  id        Int     @id @default(autoincrement())
  order     Order   @relation(fields: [orderId], references: [id], onDelete:
Cascade)
  orderId   Int
  product   Product @relation(fields: [productId], references: [id])
  productId Int
  quantity  Int
```

```
  price     Decimal @db.Decimal(10, 2) // Lưu giá tại thời điểm mua

  @@unique([orderId, productId]) // Mỗi product chỉ 1 lần trong 1 order
  @@map("order_items")
}

// ─── Enums ──────────────────────────────────────────────
enum Role {
  USER
  MODERATOR
  ADMIN
}

enum OrderStatus {
  PENDING      // Chờ xác nhận
  CONFIRMED    // Đã xác nhận
  SHIPPING     // Đang giao
  DELIVERED    // Đã giao
  CANCELLED    // Đã hủy
}
```



ERD: E-commerce System — User đặt nhiều Order, mỗi Order có nhiều OrderItem liên kết Product

13.4 Migrations — Quản Lý Thay Đổi Schema

SHELL Prisma CLI commands Copy

```
# Tạo migration mới (development) — tự apply vào DB dev
npx prisma migrate dev --name init

# Đặt tên migration theo kiểu: verb_noun
npx prisma migrate dev --name add_category_model
npx prisma migrate dev --name add_index_user_email

# Xem lịch sử migrations
npx prisma migrate status

# Apply migration vào production (không xóa data)
npx prisma migrate deploy

# Mở Prisma Studio — GUI browser để xem và edit data
npx prisma studio

# Regenerate Prisma Client (sau khi sửa schema mà chưa migrate)
npx prisma generate

# Seed database với dữ liệu mẫu
npx prisma db seed

# Reset database (xóa toàn bộ, tạo lại — chỉ dùng dev)
npx prisma migrate reset
```

Seed script

JAVASCRIPT prisma/seed.js Copy

```javascript
// prisma/seed.js
const { PrismaClient } = require('@prisma/client');
const bcrypt = require('bcrypt');

const prisma = new PrismaClient();

async function main() {
  // Tạo categories
  const categories = await Promise.all([
    prisma.category.upsert({
      where: { slug: 'electronics' },
      update: {},
      create: { name: 'Điện tử', slug: 'electronics' },
    }),
    prisma.category.upsert({
      where: { slug: 'clothing' },
      update: {},
      create: { name: 'Thời trang', slug: 'clothing' },
    }),
  ]);

  // Tạo admin user
  const hashedPass = await bcrypt.hash('Admin@123', 12);
  const admin = await prisma.user.upsert({
    where: { email: 'admin@example.com' },
    update: {},
    create: {
      email: 'admin@example.com',
      name: 'Admin',
      password: hashedPass,
      role: 'ADMIN',
    },
  });

  console.log(`✅ Seeded: ${categories.length} categories, admin user`);
}

main()
```

```
  .catch(console.error)
  .finally(() => prisma.$disconnect());
```

JSON package.json – thêm seed script Copy

```
{
  "prisma": {
    "seed": "node prisma/seed.js"
  }
}
```

13.5 Prisma Client — CRUD Đầy Đủ

Setup Prisma Client

JAVASCRIPT lib/prisma.js Copy

```
// lib/prisma.js – Singleton pattern (tránh tạo nhiều instances)
const { PrismaClient } = require('@prisma/client');

// Trong development, tránh nhiều instances do hot-reload
const globalForPrisma = global;

const prisma = globalForPrisma.prisma || new PrismaClient({
  log: process.env.NODE_ENV === 'development'
    ? ['query', 'info', 'warn', 'error']  // Log queries khi dev
    : ['error'],
});

if (process.env.NODE_ENV !== 'production') {
  globalForPrisma.prisma = prisma;
}

module.exports = prisma;
```

Find — Đọc dữ liệu

JAVASCRIPT Prisma queries Copy

```javascript
const prisma = require('./lib/prisma');

// ── findMany – lấy nhiều records ──
const users = await prisma.user.findMany({
  where: {
    role: 'USER',
    isActive: true,
    createdAt: { gte: new Date('2024-01-01') },  // gte = ≥
  },
  select: {
    id: true,
    name: true,
    email: true,
    _count: { select: { orders: true } },  // Đếm số orders
  },
  orderBy: { createdAt: 'desc' },
  take: 10,     // Limit
  skip: 20,     // Offset
});

// ── findUnique – tìm theo unique field ──
const user = await prisma.user.findUnique({
  where: { email: 'alice@example.com' },
});

// ── findFirst – tìm bản ghi đầu tiên thỏa điều kiện ──
const latestOrder = await prisma.order.findFirst({
  where: { userId: 1, status: 'PENDING' },
  orderBy: { createdAt: 'desc' },
});

// ── count – đếm ──
const totalUsers = await prisma.user.count({
  where: { role: 'USER' },
});

// ── Pagination helper ──
async function getProducts({ page = 1, limit = 12, categorySlug }) {
```

```
  const skip = (page - 1) * limit;
  const where = {
    isActive: true,
    ...(categorySlug && {
      categories: { some: { slug: categorySlug } },
    }),
  };

  const [products, total] = await prisma.$transaction([
    prisma.product.findMany({
      where,
      include: { categories: { select: { name: true, slug: true } } },
      orderBy: { createdAt: 'desc' },
      take: limit,
      skip,
    }),
    prisma.product.count({ where }),
  ]);

  return {
    data: products,
    pagination: { page, limit, total, totalPages: Math.ceil(total / limit)
},
  };
}
```

Create — Tạo mới

JAVASCRIPT Create operations Copy

```javascript
// Tạo đơn giản
const user = await prisma.user.create({
  data: {
    name: 'Nguyễn Văn A',
    email: 'nguyenvana@gmail.com',
    password: hashedPassword,
  },
  select: { id: true, name: true, email: true },  // Không trả về password
});

// Nested create — tạo order cùng với items
const order = await prisma.order.create({
  data: {
    userId: 5,
    status: 'PENDING',
    total: 1500000,
    address: { city: 'Hà Nội', street: '123 ABC', phone: '0901234567' },
    items: {
      create: [  // Tạo OrderItems cùng lúc
        { productId: 1, quantity: 2, price: 500000 },
        { productId: 3, quantity: 1, price: 500000 },
      ],
    },
  },
  include: {
    items: {
      include: { product: { select: { name: true } } },
    },
  },
});

// createMany — bulk insert
await prisma.product.createMany({
  data: [
    { name: 'iPhone 15', price: 25000000, stock: 50 },
    { name: 'Samsung S24', price: 22000000, stock: 30 },
  ],
```

```
  skipDuplicates: true,  // Bỏ qua nếu vi phạm unique constraint
});
```

Update — Cập nhật

JAVASCRIPT Update operations Copy

```
// Update đơn giản
const updated = await prisma.user.update({
  where: { id: 1 },
  data: { name: 'Tên Mới', bio: 'Bio mới' },
});

// Atomic operations — increment/decrement
await prisma.product.update({
  where: { id: productId },
  data: {
    stock: { decrement: 2 },  // Giảm stock đi 2
    // increment, multiply, divide cũng available
  },
});

// upsert — update nếu có, create nếu chưa có
const setting = await prisma.setting.upsert({
  where: { key: 'maintenance_mode' },
  update: { value: 'true' },
  create: { key: 'maintenance_mode', value: 'true' },
});

// updateMany
const result = await prisma.order.updateMany({
  where: {
    status: 'PENDING',
    createdAt: { lt: new Date(Date.now() - 7 * 24 * 60 * 60 * 1000) }, //
Quá 7 ngày
  },
  data: { status: 'CANCELLED' },
});
console.log(`Đã hủy ${result.count} orders quá hạn`);
```

Delete — Xóa

JAVASCRIPT Delete operations Copy

```javascript
// Xóa theo unique field
await prisma.user.delete({ where: { id: 1 } });

// deleteMany
await prisma.user.deleteMany({
  where: { isActive: false, createdAt: { lt: new Date('2022-01-01') } },
});
```

13.6 Relations — Queries với Include

JAVASCRIPT Nested queries Copy

```
// Lấy user với tất cả orders và items
const userWithOrders = await prisma.user.findUnique({
  where: { id: 1 },
  include: {
    orders: {
      where: { status: { not: 'CANCELLED' } },
      orderBy: { createdAt: 'desc' },
      take: 5,
      include: {
        items: {
          include: {
            product: { select: { name: true, imageUrl: true } },
          },
        },
      },
    },
  },
});

// Select fields cụ thể (hiệu quả hơn include toàn bộ)
const userProfile = await prisma.user.findUnique({
  where: { id: 1 },
  select: {
    id: true,
    name: true,
    email: true,
    avatar: true,
    _count: {
      select: {
        orders: true,  // Đếm số orders
      },
    },
  },
});
```

```
// Many-to-many query
const electronicsProducts = await prisma.product.findMany({
  where: {
    categories: {
      some: { slug: 'electronics' },  // Có ít nhất 1 category là
electronics
    },
    isActive: true,
  },
  include: {
    categories: { select: { name: true, slug: true } },
  },
});

// Filter on related records
const usersWithRecentOrders = await prisma.user.findMany({
  where: {
    orders: {
      some: {  // Có ít nhất 1 order thỏa điều kiện
        status: 'DELIVERED',
        createdAt: { gte: new Date(Date.now() - 30 * 24 * 60 * 60 * 1000) },
      },
    },
  },
  include: {
    _count: { select: { orders: true } },
  },
});
```

13.7 Transactions — Đảm Bảo Tính Toàn Vẹn

Transaction đảm bảo rằng một nhóm operations hoặc **tất cả thành công** hoặc **tất cả thất bại** — không bao giờ có trạng thái nửa vời. Ví dụ: tạo order và trừ stock phải xảy ra đồng thời.


Không có Transaction (Nguy hiểm)
1. Kiểm tra stock OK ✓
2. Tạo Order → thành công ✓
3. Trừ stock → LỖI MẠNG ✗
Order tạo rồi nhưng stock chưa trừ — dữ liệu không nhất quán!
Có Transaction (An toàn)
1. Kiểm tra stock OK ✓
2. Tạo Order ✓
3. Trừ stock → LỖI ✗ → ROLLBACK
Tất cả rollback — DB sạch, không có trạng thái dở

JAVASCRIPT Tạo order trong transaction Copy

```
const prisma = require('./lib/prisma');

async function checkout(userId, cartItems, shippingAddress) {
  // cartItems = [{ productId: 1, quantity: 2 }, { productId: 3, quantity: 1
}]

  return await prisma.$transaction(async (tx) => {
    // 'tx' là Prisma Client nhưng trong context transaction
    // Mọi operation dùng 'tx' đều trong cùng 1 transaction

    // ── Bước 1: Kiểm tra và lấy thông tin sản phẩm ──
    const productIds = cartItems.map(item => item.productId);
    const products = await tx.product.findMany({
      where: { id: { in: productIds }, isActive: true },
    });

    if (products.length !== productIds.length) {
      throw new Error('Một số sản phẩm không tồn tại hoặc đã bị xóa');
    }

    // ── Bước 2: Kiểm tra tồn kho và tính tổng tiền ──
    const orderItemsData = [];
    let total = 0;

    for (const cartItem of cartItems) {
      const product = products.find(p => p.id === cartItem.productId);

      if (product.stock < cartItem.quantity) {
        throw new Error(`Sản phẩm "${product.name}" chỉ còn ${product.stock}
trong kho`);
      }

      const itemTotal = Number(product.price) * cartItem.quantity;
      total += itemTotal;

      orderItemsData.push({
        productId: product.id,
        quantity: cartItem.quantity,
```

```
        price: product.price,  // Lưu giá tại thời điểm mua
      });
    }

    // ── Bước 3: Tạo order với items ──
    const order = await tx.order.create({
      data: {
        userId,
        total,
        status: 'CONFIRMED',
        address: shippingAddress,
        items: { create: orderItemsData },
      },
      include: {
        items: {
          include: { product: { select: { name: true } } },
        },
      },
    });

    // ── Bước 4: Trừ tồn kho (atomic) ──
    await Promise.all(
      cartItems.map(item =>
        tx.product.update({
          where: { id: item.productId },
          data: { stock: { decrement: item.quantity } },
        })
      )
    );

    // Nếu bất kỳ step nào throw error, toàn bộ sẽ tự động ROLLBACK
    return order;
  }, {
    maxWait: 5000,   // Thời gian chờ lấy connection từ pool (ms)
    timeout: 10000,  // Timeout của toàn bộ transaction (ms)
  });
}

// Sử dụng
app.post('/api/checkout', async (req, res) => {
  try {
```

```
    const order = await checkout(req.user.id, req.body.items,
req.body.address);
    res.json({ success: true, order });
  } catch (error) {
    // Nếu là lỗi business (hết hàng), trả 400
    if (error.message.includes('kho')) {
      return res.status(400).json({ error: error.message });
    }
    res.status(500).json({ error: 'Lỗi hệ thống' });
  }
});
```

Sequential transaction (batch)

JAVASCRIPT — Batch transaction — Copy

```
// prisma.$transaction([ ... ]) - chạy tuần tự, rollback nếu 1 lỗi
const [updateUser, createLog] = await prisma.$transaction([
  prisma.user.update({
    where: { id: 1 },
    data: { lastLogin: new Date() },
  }),
  prisma.auditLog.create({
    data: { userId: 1, action: 'LOGIN', ip: req.ip },
  }),
]);
```

13.8 Aggregation — Thống Kê Nâng Cao

JAVASCRIPT Aggregation và groupBy Copy

```
// ── aggregate – tính toán trên tập dữ liệu ──
const orderStats = await prisma.order.aggregate({
  where: {
    status: 'DELIVERED',
    createdAt: { gte: new Date('2024-01-01') },
  },
  _count: { id: true },
  _sum: { total: true },
  _avg: { total: true },
  _min: { total: true },
  _max: { total: true },
});

console.log(`Tổng orders: ${orderStats._count.id}`);
console.log(`Doanh thu: ${orderStats._sum.total}`);
console.log(`Trung bình: ${orderStats._avg.total}`);

// ── groupBy – nhóm theo field ──
const revenueByMonth = await prisma.order.groupBy({
  by: ['status'],
  where: {
    createdAt: {
      gte: new Date('2024-01-01'),
      lt: new Date('2025-01-01'),
    },
  },
  _count: { id: true },
  _sum: { total: true },
  orderBy: { _sum: { total: 'desc' } },
});

// User stats – dùng raw SQL cho query phức tạp
const topCustomers = await prisma.$queryRaw`
  SELECT
    u.id,
```

```
    u.name,
    u.email,
    COUNT(o.id)::int AS order_count,
    SUM(o.total)::float AS total_spent
  FROM users u
  INNER JOIN orders o ON o.user_id = u.id
  WHERE o.status = 'DELIVERED'
  GROUP BY u.id, u.name, u.email
  ORDER BY total_spent DESC
  LIMIT 10
`;
```

📋 Tóm tắt Chương 13

- **PostgreSQL**: relational DB mạnh nhất open-source, phù hợp khi cần ACID và quan hệ phức tạp
- **Prisma schema**: nguồn sự thật duy nhất — định nghĩa models, relations, enums
- **Migration**: `npx prisma migrate dev` tạo SQL và apply vào DB
- **Prisma Client**: API type-safe với findMany, findUnique, create, update, delete, upsert
- **Relations**: include để join, select để chọn fields, _count để đếm related records
- **Transactions**: `prisma.$transaction(async tx => {})` cho nhóm operations atomic
- **Aggregation**: aggregate, groupBy, $queryRaw cho thống kê phức tạp
- Dùng `$queryRaw` khi cần SQL thuần — không bao giờ interpolate user input trực tiếp

🏋️ Bài Tập Thực Hành

DỄ

Bài 1: Thêm Category và Many-to-Many

Đã có model `Category` và `Product` với quan hệ many-to-many trong schema. Viết: (1) Route `GET /categories` lấy tất cả categories kèm số lượng products; (2) Route `GET /categories/:slug/products` lấy danh sách products theo category với pagination; (3) Route `POST /products/:id/categories` gán categories cho product (nhận array categoryIds, dùng `set` thay vì `connect`).

TRUNG BÌNH

Bài 2: User Dashboard Stats

Viết route `GET /users/:id/stats` trả về object: `totalOrders` (int), `totalSpent` (số tiền), `favoriteCategory` (category mà user mua nhiều nhất theo quantity), `ordersByStatus` (object đếm orders theo từng status). Dùng combination của Prisma aggregate, groupBy và raw SQL nếu cần. Handle case user chưa có order nào.

KHÓ

Bài 3: Cart và Checkout hoàn chỉnh

Thêm model `Cart` và `CartItem` vào schema. Implement: (1) `POST /cart/items` — thêm vào giỏ hàng (nếu đã có thì tăng quantity); (2) `DELETE /cart/items/:productId` — xóa khỏi giỏ; (3) `GET /cart` — xem giỏ hàng với product info và tổng tiền; (4) `POST /cart/checkout` — tạo order từ cart, trừ stock, clear cart, tất cả trong 1 transaction. Xử lý đầy đủ: sản phẩm hết hàng, sản phẩm bị xóa, giỏ hàng rỗng.

PHẦN 4 — DATABASE VÀ LƯU TRỮ

Chương 14: Caching với Redis — Tăng Tốc Ứng Dụng

Redis là in-memory database nhanh nhất thế giới. Dùng làm cache giúp response time giảm từ 200ms xuống 2ms — gấp 100 lần. Chương này học Redis từ cơ bản đến các patterns thực tế: cache-aside, session store, rate limiting, và pub/sub.

🎯 **SAU CHƯƠNG NÀY BẠN SẼ**

- Hiểu Redis là gì và kiến trúc in-memory database
- Làm chủ 5 data types: String, Hash, List, Set, Sorted Set
- Implement Cache-Aside pattern — pattern phổ biến nhất
- Viết Cache middleware tự động cho Express
- Thay thế MemoryStore bằng Redis Session Store
- Xây dựng Rate Limiter bằng Redis từ scratch
- Pub/Sub cơ bản cho real-time notifications
- Distributed lock để tránh race conditions

14.1 Redis là gì?

Redis (Remote Dictionary Server) là in-memory key-value store — nghĩa là nó lưu toàn bộ data trong RAM thay vì ổ cứng. Đây là lý do nó nhanh đến mức khó tin: RAM nhanh hơn SSD tới 1000 lần.

Storage	Tốc độ đọc	Latency điển hình
HDD (ổ cứng cũ)	~200 MB/s	5–10ms
SSD NVMe	~3500 MB/s	0.05–0.1ms
RAM (Redis)	~50,000 MB/s	<0.1ms

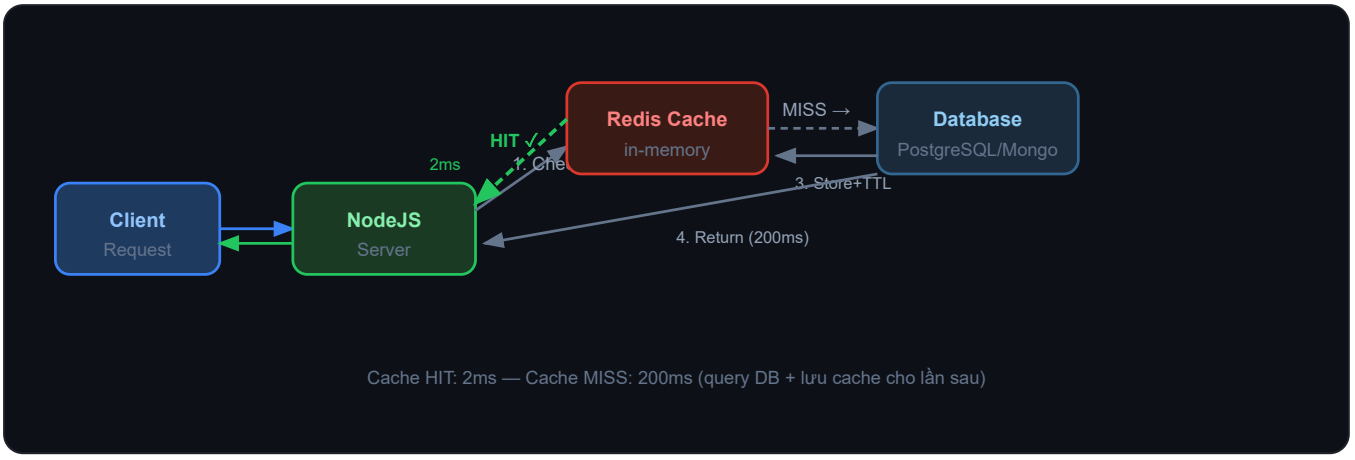
Tại sao Redis không mất data khi tắt máy?

Redis có 2 cơ chế persistence:

- **RDB (Redis Database)**: Snapshot — chụp toàn bộ data ra file `.rdb` định kỳ (mỗi 5 phút). Nếu server crash, mất tối đa vài phút data.
- **AOF (Append-Only File)**: Ghi từng command vào log file. Khi restart, replay lại log. An toàn hơn nhưng tốn dung lượng hơn.

Redis dùng cho gì?

- **Cache**: Lưu kết quả query DB, API responses tốn thời gian
- **Session Store**: Lưu user sessions cho app multi-server
- **Rate Limiting**: Giới hạn số request/phút từ 1 IP
- **Pub/Sub**: Message broker đơn giản giữa các service
- **Leaderboard**: Sorted set cho ranking real-time
- **Job Queue**: FIFO queue cho background jobs (Bull/BullMQ dùng Redis)
- **Distributed Lock**: Đồng bộ hoá giữa nhiều server instances



14.2 Cài Đặt và Kết Nối

Option 1: Redis Cloud (Khuyên dùng)

1. Vào `redis.io/try-free` → Sign up miễn phí (30MB free tier)
2. Tạo subscription → chọn plan Free
3. Lấy **Public endpoint** và **Password**
4. Connection string: `redis://:password@host:port`

Option 2: Docker (Local)

SHELL Docker Copy

```
# Chạy Redis local bằng Docker
docker run -d --name redis-local -p 6379:6379 redis:7-alpine

# Với password
docker run -d --name redis-local -p 6379:6379 redis:7-alpine redis-server --
requirepass mypassword

# Kết nối Redis CLI để test
docker exec -it redis-local redis-cli
# Hoặc
redis-cli -h localhost -p 6379 -a mypassword
```

Cài ioredis và kết nối

SHELL Terminal Copy

```
npm install ioredis
```

JAVASCRIPT lib/redis.js Copy

```javascript
// lib/redis.js — Singleton Redis client
const Redis = require('ioredis');

const redis = new Redis(process.env.REDIS_URL, {
  // Retry khi mất kết nối
  retryStrategy(times) {
    const delay = Math.min(times * 100, 3000); // Tăng dần, tối đa 3s
    console.log(`Redis retry #${times}, waiting ${delay}ms`);
    return delay;
  },
  // Timeout connection
  connectTimeout: 10000,
  // Tự reconnect
  lazyConnect: false,
  // Số lệnh tối đa queue khi đang reconnect
  maxRetriesPerRequest: 3,
});

redis.on('connect', () => console.log('✅ Redis connected'));
redis.on('ready', () => console.log('📗 Redis ready'));
redis.on('error', (err) => console.error('📕 Redis error:', err.message));
redis.on('close', () => console.log('📙 Redis connection closed'));
redis.on('reconnecting', () => console.log('🔄 Redis reconnecting...'));

module.exports = redis;
```

TEXT .env Copy

```
# Redis Cloud
REDIS_URL=redis://:your-password@redis-12345.c1.us-east-1-
2.ec2.cloud.redislabs.com:12345

# Local (không có password)
REDIS_URL=redis://localhost:6379

# Local với password
REDIS_URL=redis://:mypassword@localhost:6379
```

14.3 Data Types — 5 Kiểu Dữ Liệu Cần Biết

String — Kiểu phổ biến nhất

String trong Redis có thể chứa text, numbers, hoặc binary data (JSON, serialized objects).

JAVASCRIPT String operations Copy

```
const redis = require('./lib/redis');

// ── SET với TTL (Time To Live) ──
await redis.set('greeting', 'Hello World');            // Không expire
await redis.set('user:42:name', 'Alice', 'EX', 3600); // Expire sau 1 giờ (seconds)
await redis.set('token:abc123', userId, 'PX', 300000); // Expire sau 5 phút (milliseconds)

// SET NX — chỉ set nếu key chưa tồn tại (NX = Not eXists)
const isSet = await redis.set('lock:resource_1', 'locked', 'EX', 30, 'NX');
// isSet = 'OK' nếu set thành công, null nếu key đã tồn tại

// ── GET ──
const name = await redis.get('user:42:name'); // 'Alice' hoặc null nếu không tồn tại
const json = await redis.get('product:1');    // String, cần JSON.parse

// ── DEL ──
await redis.del('user:42:name');
await redis.del(['key1', 'key2', 'key3']); // Xóa nhiều cùng lúc

// ── EXISTS ──
const exists = await redis.exists('user:42:name'); // 1 nếu có, 0 nếu không

// ── TTL — xem thời gian sống còn lại ──
const ttl = await redis.ttl('user:42:name'); // Số giây còn lại, -1 nếu không expire, -2 nếu không tồn tại
await redis.expire('user:42:name', 7200);    // Đặt TTL mới (2 giờ)

// ── Numeric operations ──
await redis.set('page:views', 0);
await redis.incr('page:views');          // Tăng 1 → atomic!
await redis.incrby('page:views', 5);     // Tăng 5
await redis.decr('page:views');          // Giảm 1
await redis.incrbyfloat('price:btc', 100.5); // Float increment

// ── MSET / MGET — bulk operations ──
```

```
await redis.mset('key1', 'val1', 'key2', 'val2', 'key3', 'val3');
const [v1, v2, v3] = await redis.mget('key1', 'key2', 'key3'); // [val1,
val2, val3]
```

Hash — Lưu Object

Hash lưu field-value pairs, giống như object JavaScript. Tốt hơn JSON string khi cần update từng field riêng lẻ.

JAVASCRIPT Hash operations Copy

```
// ── HSET – set một hoặc nhiều fields ──
await redis.hset('user:42', {
  name: 'Alice',
  email: 'alice@example.com',
  role: 'admin',
  lastLogin: new Date().toISOString(),
});

// ── HGET – lấy một field ──
const name = await redis.hget('user:42', 'name');  // 'Alice'

// ── HGETALL – lấy tất cả fields (trả về object) ──
const user = await redis.hgetall('user:42');
// { name: 'Alice', email: 'alice@example.com', role: 'admin', lastLogin:
'...' }

// ── HMGET – lấy nhiều fields ──
const [email, role] = await redis.hmget('user:42', 'email', 'role');

// ── HDEL – xóa field ──
await redis.hdel('user:42', 'lastLogin');

// ── HEXISTS – kiểm tra field có tồn tại không ──
const hasEmail = await redis.hexists('user:42', 'email'); // 1 hoặc 0

// ── HINCRBY – tăng số trong hash ──
await redis.hset('stats:today', 'visits', 0, 'signups', 0);
await redis.hincrby('stats:today', 'visits', 1);

// ── HKEYS / HVALS / HLEN ──
const keys = await redis.hkeys('user:42');    // ['name', 'email', 'role']
const vals = await redis.hvals('user:42');    // ['Alice', 'alice@...',
'admin']
const count = await redis.hlen('user:42');    // 3
```

List — Queue và Stack

Linked list cho phép push/pop ở 2 đầu. Dùng làm job queue, activity log, hoặc recent items.

JAVASCRIPT List operations Copy

```
// ── RPUSH – thêm vào cuối (Right Push) ──
await redis.rpush('notifications:user:42', 'Bạn có follower mới');
await redis.rpush('job:queue', 'job:email:1', 'job:email:2', 'job:email:3');

// ── LPUSH – thêm vào đầu (Left Push) ──
await redis.lpush('recent:viewed', 'product:50');

// ── LPOP / RPOP – lấy và xóa phần tử ──
const job = await redis.lpop('job:queue');   // FIFO Queue: lấy từ đầu
const last = await redis.rpop('recent:viewed'); // Stack: lấy từ cuối

// ── LRANGE – xem toàn bộ hoặc một phần (không xóa) ──
const allJobs = await redis.lrange('job:queue', 0, -1);     // Tất cả
const first3 = await redis.lrange('job:queue', 0, 2);       // 3 phần tử đầu
const last5 = await redis.lrange('notifications:user:42', -5, -1); // 5 cái
cuối

// ── LLEN – độ dài ──
const queueSize = await redis.llen('job:queue');

// ── LTRIM – giữ lại chỉ N phần tử (rotating buffer) ──
await redis.lpush('recent:searches', searchTerm);
await redis.ltrim('recent:searches', 0, 9); // Chỉ giữ 10 gần nhất

// ── BLPOP – blocking pop (đợi đến khi có data, dùng cho worker) ──
// const [listName, value] = await redis.blpop('job:queue', 30); // Timeout
30s
```

Set — Tập hợp unique

Set chứa các string không trùng lặp. Dùng cho unique visitors, online users, tags.

JAVASCRIPT Set operations Copy

```
// ── SADD – thêm vào set ──
await redis.sadd('online:users', 'user:1', 'user:2', 'user:3');
await redis.sadd('product:1:views', visitorId); // Đếm unique visitors

// ── SISMEMBER – kiểm tra có trong set không ──
const isOnline = await redis.sismember('online:users', 'user:2'); // 1 hoặc
0

// ── SMEMBERS – lấy tất cả members ──
const onlineUsers = await redis.smembers('online:users');

// ── SCARD – số lượng members ──
const onlineCount = await redis.scard('online:users');
const uniqueViews = await redis.scard('product:1:views');

// ── SREM – xóa member ──
await redis.srem('online:users', 'user:2');

// ── Set operations ──
// SUNION – hợp 2 sets
const allFans = await redis.sunion('fans:teamA', 'fans:teamB');
// SINTER – giao 2 sets
const commonFans = await redis.sinter('fans:teamA', 'fans:teamB');
// SDIFF – hiệu (chỉ có trong set đầu, không có ở set sau)
const exclusiveFans = await redis.sdiff('fans:teamA', 'fans:teamB');
```

Sorted Set — Leaderboard

Giống Set nhưng mỗi member có thêm một **score** (số thực). Tự động sort theo score.

JAVASCRIPT Sorted Set – Leaderboard Copy

```
// ── ZADD – thêm với score ──
// score thấp = rank cao nếu sort ascending; dùng desc để sort điểm cao lên đầu
await redis.zadd('leaderboard:global', 1500, 'alice', 2300, 'bob', 980, 'carol');

// ── ZINCRBY – tăng score (atomic) ──
await redis.zincrby('leaderboard:global', 100, 'alice'); // Alice: 1500 → 1600

// ── ZREVRANGE – lấy top N (điểm cao nhất trước) ──
const top10 = await redis.zrevrange('leaderboard:global', 0, 9, 'WITHSCORES');
// ['bob', '2300', 'alice', '1600', 'carol', '980']

// ── ZREVRANK – rank của 1 user (0-indexed, cao nhất là 0) ──
const rank = await redis.zrevrank('leaderboard:global', 'alice'); // 1 (thứ 2)

// ── ZSCORE – điểm của 1 user ──
const score = await redis.zscore('leaderboard:global', 'alice'); // '1600'

// ── ZRANGE + WITHSCORES – toàn bộ bảng xếp hạng ──
const all = await redis.zrange('leaderboard:global', 0, -1, 'WITHSCORES');

// ── ZRANGEBYSCORE – lấy users trong khoảng score ──
const midRange = await redis.zrangebyscore('leaderboard:global', 1000, 2000, 'WITHSCORES');

// Leaderboard helper function
async function getLeaderboard(page = 1, pageSize = 10) {
  const start = (page - 1) * pageSize;
  const end = start + pageSize - 1;

  const raw = await redis.zrevrange('leaderboard:global', start, end, 'WITHSCORES');

  // Parse kết quả: ['alice', '1600', 'bob', '2300', ...]
```

```
  const entries = [];
  for (let i = 0; i < raw.length; i += 2) {
    entries.push({
      rank: start + (i / 2) + 1,
      username: raw[i],
      score: parseInt(raw[i + 1]),
    });
  }

  const total = await redis.zcard('leaderboard:global');
  return { entries, total, totalPages: Math.ceil(total / pageSize) };
}
```

14.4 Cache-Aside Pattern — Pattern Chính

Cache-Aside (còn gọi là Lazy Loading) là pattern cache phổ biến nhất. Application tự quản lý cache: check cache trước, nếu miss thì query DB và lưu vào cache.

JAVASCRIPT Cache-aside implementation Copy

```
const redis = require('./lib/redis');
const User = require('./models/User'); // Mongoose hoặc Prisma

// ── Lấy user với cache ──
async function getUserById(id) {
  const cacheKey = `user:${id}`;

  // 1. Kiểm tra cache
  const cached = await redis.get(cacheKey);
  if (cached) {
    return JSON.parse(cached); // Cache HIT — trả về ngay
  }

  // 2. Cache MISS — query DB
  const user = await User.findById(id).select('-password').lean();
  if (!user) return null;

  // 3. Lưu vào cache với TTL 1 giờ
  await redis.set(cacheKey, JSON.stringify(user), 'EX', 3600);

  return user;
}

// ── Update user — phải invalidate cache ──
async function updateUser(id, data) {
  const user = await User.findByIdAndUpdate(id, data, { new: true
}).select('-password');
  if (!user) return null;

  // Xóa cache cũ để lần sau sẽ lấy fresh data từ DB
  await redis.del(`user:${id}`);

  return user;
}

// ── Xóa user — phải xóa cache ──
async function deleteUser(id) {
  await User.findByIdAndDelete(id);
```

```
  await redis.del(`user:${id}`);
}

// ── Cache danh sách (phức tạp hơn) ──
async function getProductList({ page = 1, limit = 10, categoryId }) {
  // Tạo cache key từ các params
  const cacheKey = `products:page:${page}:limit:${limit}:cat:${categoryId ||
'all'}`;

  const cached = await redis.get(cacheKey);
  if (cached) return JSON.parse(cached);

  // Query DB
  const result = await getProductsFromDB({ page, limit, categoryId });

  // Cache danh sách với TTL ngắn hơn (5 phút) – thay đổi thường xuyên hơn
  await redis.set(cacheKey, JSON.stringify(result), 'EX', 300);

  return result;
}

// Khi có sản phẩm mới/bị xóa – xóa tất cả cache liên quan
async function invalidateProductCache() {
  const keys = await redis.keys('products:page:*');
  if (keys.length > 0) {
    await redis.del(...keys); // Xóa tất cả
  }
}
```

⚠️ CACHE STAMPEDE (THUNDERING HERD)

Khi cache expire, nhiều request đến cùng lúc đều query DB → DB bị quá tải. Giải pháp: **Stale-While-Revalidate** (trả về data cũ trong khi refresh) hoặc **Mutex Lock** (chỉ 1 request query DB, các request khác chờ cache).

14.5 Cache Middleware cho Express

JAVASCRIPT middleware/cache.js Copy

```
// middleware/cache.js
const redis = require('../lib/redis');

/**
 * Cache middleware cho GET requests
 * @param {number} ttl - Thời gian cache tính bằng giây (default: 300 = 5
phút)
 * @param {function} keyFn - Function tạo cache key (optional)
 */
function cacheMiddleware(ttl = 300, keyFn = null) {
  return async (req, res, next) => {
    // Chỉ cache GET requests
    if (req.method !== 'GET') return next();

    // Tạo cache key (có thể include user ID nếu cần per-user cache)
    const cacheKey = keyFn
      ? keyFn(req)
      : `http:${req.path}:${JSON.stringify(req.query)}`;

    try {
      // Kiểm tra cache
      const cached = await redis.get(cacheKey);

      if (cached) {
        // Cache HIT
        const data = JSON.parse(cached);
        res.setHeader('X-Cache', 'HIT');
        res.setHeader('X-Cache-TTL', await redis.ttl(cacheKey));
        return res.json(data);
      }

      // Cache MISS - ghi đè res.json để intercept response
      res.setHeader('X-Cache', 'MISS');

      const originalJson = res.json.bind(res);
```

```
      res.json = async (data) => {
        // Chỉ cache response thành công (status 2xx)
        if (res.statusCode >= 200 && res.statusCode < 300) {
          await redis.set(cacheKey, JSON.stringify(data), 'EX', ttl);
        }
        return originalJson(data);
      };

      next();

    } catch (err) {
      // Nếu Redis lỗi, vẫn cho request đi qua (graceful degradation)
      console.error('Cache middleware error:', err.message);
      next();
    }
  };
}

module.exports = { cacheMiddleware };
```

JAVASCRIPT Sử dụng cache middleware Copy

```
const express = require('express');
const { cacheMiddleware } = require('./middleware/cache');
const redis = require('./lib/redis');
const router = express.Router();

// Cache 5 phút cho danh sách products
router.get('/products', cacheMiddleware(300), async (req, res) => {
  const products = await getProductsFromDB(req.query);
  res.json(products);
});

// Cache 1 giờ cho chi tiết sản phẩm (ít thay đổi)
router.get('/products/:id', cacheMiddleware(3600, req =>
`product:${req.params.id}`), async (req, res) => {
  const product = await getProductById(req.params.id);
  if (!product) return res.status(404).json({ error: 'Không tìm thấy' });
  res.json(product);
});

// Khi update/delete — xóa cache liên quan
router.put('/products/:id', async (req, res) => {
  const product = await updateProduct(req.params.id, req.body);
  // Invalidate cache
  await redis.del(`product:${req.params.id}`);
  await redis.del(`http:/products:${JSON.stringify({})}`);
  res.json(product);
});

module.exports = router;
```

14.6 Session Store với Redis

Mặc định Express lưu sessions trong MemoryStore — bộ nhớ của Node process. Khi restart server, tất cả sessions mất. Khi chạy nhiều server instances, sessions không được chia sẻ. Redis giải quyết cả 2 vấn đề này.

SHELL Terminal Copy

```
npm install express-session connect-redis
```

JAVASCRIPT app.js — Session với Redis Copy

```javascript
const express = require('express');
const session = require('express-session');
const { createClient } = require('redis');
const RedisStore = require('connect-redis').default;

const app = express();

// Tạo Redis client riêng cho session store
const redisClient = createClient({
  url: process.env.REDIS_URL,
});
redisClient.connect().catch(console.error);

// Cấu hình session với Redis store
app.use(session({
  store: new RedisStore({
    client: redisClient,
    prefix: 'sess:',          // Key prefix trong Redis
    ttl: 86400,               // Session expire sau 24 giờ (giây)
  }),
  secret: process.env.SESSION_SECRET,
  resave: false,              // Không lưu lại nếu session không thay đổi
  saveUninitialized: false,   // Không tạo session trống
  cookie: {
    secure: process.env.NODE_ENV === 'production', // Chỉ HTTPS trong prod
    httpOnly: true,
    maxAge: 24 * 60 * 60 * 1000, // 24 giờ (milliseconds)
    sameSite: 'strict',
  },
  name: 'sid',                // Tên cookie (thay mặc định 'connect.sid')
}));

// Sử dụng session
app.post('/login', async (req, res) => {
  const user = await authenticateUser(req.body.email, req.body.password);
  if (!user) return res.status(401).json({ error: 'Sai thông tin đăng nhập'
});
```

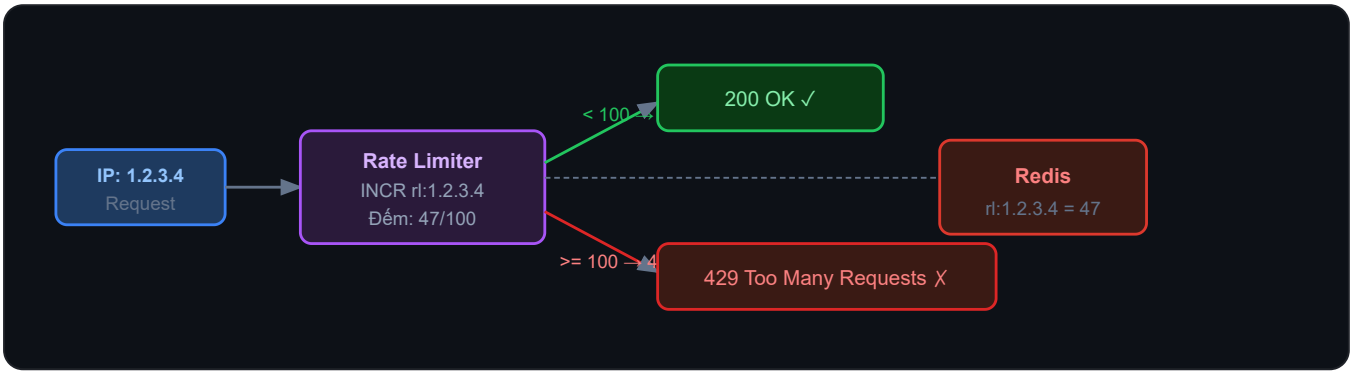
```
  // Lưu thông tin vào session (tự động persist vào Redis)
  req.session.userId = user.id;
  req.session.role = user.role;

  // Regenerate session ID để tránh Session Fixation attack
  req.session.regenerate((err) => {
    if (err) return res.status(500).json({ error: 'Session error' });
    req.session.userId = user.id;
    req.session.role = user.role;
    res.json({ message: 'Đăng nhập thành công', user: { id: user.id, name:
user.name } });
  });
});

app.post('/logout', (req, res) => {
  req.session.destroy((err) => {
    res.clearCookie('sid');
    res.json({ message: 'Đã đăng xuất' });
  });
});
```

14.7 Rate Limiting với Redis

Rate limiting giới hạn số request mỗi user/IP có thể thực hiện trong khoảng thời gian. Bảo vệ API khỏi brute force, DoS, và abuse.



JAVASCRIPT middleware/rateLimiter.js Copy

```
// middleware/rateLimiter.js
const redis = require('../lib/redis');

/**
 * Rate limiter dùng Fixed Window Counter
 * @param {number} maxRequests - Số request tối đa trong window
 * @param {number} windowSecs - Thời gian window (giây)
 * @param {string} prefix - Prefix cho Redis key
 */
function rateLimiter(maxRequests = 100, windowSecs = 60, prefix = 'rl') {
  return async (req, res, next) => {
    // Key theo IP (hoặc user ID nếu authenticated)
    const identifier = req.user?.id || req.ip;
    const key = `${prefix}:${identifier}`;

    try {
      // Pipeline: gộp 2 lệnh vào 1 round-trip
      const pipeline = redis.pipeline();
      pipeline.incr(key);          // Tăng counter
      pipeline.ttl(key);           // Lấy TTL hiện tại
      const results = await pipeline.exec();

      const count = results[0][1];  // Kết quả INCR
      const ttl = results[1][1];    // Kết quả TTL

      // Nếu key mới (TTL = -1), đặt expire
      if (ttl === -1) {
        await redis.expire(key, windowSecs);
      }

      // Set headers để client biết limit
      res.setHeader('X-RateLimit-Limit', maxRequests);
      res.setHeader('X-RateLimit-Remaining', Math.max(0, maxRequests -
count));
      res.setHeader('X-RateLimit-Reset', Math.ceil(Date.now() / 1000) + (ttl
> 0 ? ttl : windowSecs));

      if (count > maxRequests) {
```

```
        return res.status(429).json({
          error: 'Quá nhiều request. Vui lòng thử lại sau.',
          retryAfter: ttl > 0 ? ttl : windowSecs,
        });
      }

      next();

    } catch (err) {
      // Nếu Redis lỗi, cho request đi qua (tránh block toàn bộ service)
      console.error('Rate limiter error:', err.message);
      next();
    }
  };
}

/**
 * Sliding Window Rate Limiter — chính xác hơn Fixed Window
 */
function slidingWindowRateLimiter(maxRequests = 100, windowSecs = 60) {
  return async (req, res, next) => {
    const identifier = req.user?.id || req.ip;
    const key = `sw:rl:${identifier}`;
    const now = Date.now();
    const windowStart = now - windowSecs * 1000;

    try {
      const pipeline = redis.pipeline();
      // Xóa các timestamps cũ hơn window
      pipeline.zremrangebyscore(key, 0, windowStart);
      // Đếm requests trong window
      pipeline.zcard(key);
      // Thêm timestamp hiện tại
      pipeline.zadd(key, now, `${now}-${Math.random()}`);
      // Set TTL
      pipeline.expire(key, windowSecs);

      const results = await pipeline.exec();
      const requestCount = results[1][1]; // ZCARD result

      res.setHeader('X-RateLimit-Limit', maxRequests);
```

```
      res.setHeader('X-RateLimit-Remaining', Math.max(0, maxRequests -
requestCount - 1));

      if (requestCount ≥ maxRequests) {
        return res.status(429).json({
          error: 'Rate limit exceeded',
          retryAfter: windowSecs,
        });
      }

      next();
    } catch (err) {
      console.error('Sliding window rate limiter error:', err.message);
      next();
    }
  };
}

module.exports = { rateLimiter, slidingWindowRateLimiter };
```

JAVASCRIPT — Áp dụng rate limiter — Copy

```
const { rateLimiter, slidingWindowRateLimiter } =
require('./middleware/rateLimiter');

// Global rate limit — toàn bộ API
app.use(rateLimiter(1000, 60)); // 1000 req/phút

// Strict limit cho auth routes — chống brute force
app.use('/api/auth', rateLimiter(5, 900, 'auth')); // 5 lần/15 phút

// Per-user limit cho upload
app.post('/api/upload', rateLimiter(20, 3600, 'upload')); // 20 uploads/giờ
```

14.8 Pub/Sub — Real-Time Events

Redis Pub/Sub cho phép một publisher gửi message đến nhiều subscribers. Dùng để broadcast events khi có thay đổi — ví dụ: thông báo tất cả server instances invalidate cache khi có update.

> **📌 QUAN TRỌNG: CẦN 2 REDIS CONNECTIONS**
>
> Khi một Redis client đang ở chế độ subscribe, nó **chỉ được dùng cho pub/sub**, không thể dùng cho các lệnh khác. Vì vậy cần tạo **2 clients riêng**: 1 cho pub/sub subscriber, 1 cho tất cả operations còn lại.

JAVASCRIPT lib/pubsub.js Copy

```javascript
// lib/pubsub.js
const Redis = require('ioredis');

// Client riêng cho subscriber (không dùng cho operations khác)
const subscriber = new Redis(process.env.REDIS_URL);
const publisher = new Redis(process.env.REDIS_URL);

// Subscribe vào channel
subscriber.subscribe('cache:invalidate', 'user:update', 'order:status',
(err, count) => {
  if (err) console.error('Subscribe error:', err);
  else console.log(`📡 Subscribed to ${count} channels`);
});

// Xử lý messages nhận được
subscriber.on('message', async (channel, message) => {
  console.log(`📨 [${channel}]:`, message);

  if (channel === 'cache:invalidate') {
    const { patterns } = JSON.parse(message);
    // Xóa cache keys match pattern
    for (const pattern of patterns) {
      const keys = await redis.keys(pattern);
      if (keys.length > 0) await redis.del(...keys);
    }
  }

  if (channel === 'user:update') {
    const { userId } = JSON.parse(message);
    await redis.del(`user:${userId}`); // Invalidate user cache
  }
});

// Publisher
async function publishCacheInvalidation(patterns) {
  await publisher.publish('cache:invalidate', JSON.stringify({ patterns }));
}
```

```
async function publishUserUpdate(userId) {
  await publisher.publish('user:update', JSON.stringify({ userId }));
}

async function publishOrderStatus(orderId, status) {
  await publisher.publish('order:status', JSON.stringify({ orderId, status
}));
}

module.exports = { publishCacheInvalidation, publishUserUpdate,
publishOrderStatus };
```

JAVASCRIPT Distributed cache invalidation Copy

```
// Khi update user trên bất kỳ server nào, broadcast cho tất cả servers
const { publishUserUpdate } = require('./lib/pubsub');

app.put('/api/users/:id', async (req, res) => {
  const user = await updateUser(req.params.id, req.body);

  // Broadcast cho tất cả server instances (kể cả server này)
  // để invalidate cache user này
  await publishUserUpdate(req.params.id);

  res.json(user);
});
```

14.9 Distributed Lock — Tránh Race Condition

Khi nhiều server instances cùng xử lý một job, race condition có thể xảy ra. Distributed lock đảm bảo chỉ 1 instance thực sự xử lý tại một thời điểm.

JAVASCRIPT lib/lock.js Copy

```
// lib/lock.js - Distributed Lock với Redis
const redis = require('./redis');
const { randomBytes } = require('crypto');

/**
 * Thử lấy lock
 * @param {string} resource - Tên resource cần lock (vd: 'order:123')
 * @param {number} ttl - Thời gian lock tối đa (ms)
 * @returns {string|null} - lockId nếu thành công, null nếu đang bị lock
 */
async function acquireLock(resource, ttl = 10000) {
  const lockKey = `lock:${resource}`;
  const lockId = randomBytes(16).toString('hex'); // Unique ID để xác nhận
owner

  // SET key value NX PX ttl
  // NX = chỉ set nếu chưa tồn tại
  // PX = expire tính bằng milliseconds
  const result = await redis.set(lockKey, lockId, 'PX', ttl, 'NX');

  if (result === 'OK') {
    return lockId; // Lấy lock thành công
  }
  return null; // Lock đã bị giữ bởi instance khác
}

/**
 * Giải phóng lock (chỉ nếu là owner)
 */
async function releaseLock(resource, lockId) {
  const lockKey = `lock:${resource}`;

  // Dùng Lua script để đảm bảo atomic: kiểm tra owner và xóa trong 1 bước
  const luaScript = `
    if redis.call("get", KEYS[1]) == ARGV[1] then
      return redis.call("del", KEYS[1])
    else
      return 0
```

```
    end
  `;

  const result = await redis.eval(luaScript, 1, lockKey, lockId);
  return result === 1;
}

/**
 * Wrapper tiện lợi — tự acquire và release
 */
async function withLock(resource, ttl, fn) {
  const lockId = await acquireLock(resource, ttl);

  if (!lockId) {
    throw new Error(`Resource "${resource}" đang bị khóa. Thử lại sau.`);
  }

  try {
    return await fn();
  } finally {
    await releaseLock(resource, lockId);
  }
}

module.exports = { acquireLock, releaseLock, withLock };
```

JAVASCRIPT — Sử dụng distributed lock — Copy

```javascript
const { withLock } = require('./lib/lock');

// Xử lý payment — chỉ 1 instance được chạy cùng lúc cho cùng orderId
app.post('/api/orders/:id/pay', async (req, res) => {
  const { id: orderId } = req.params;

  try {
    const result = await withLock(`payment:${orderId}`, 30000, async () => {
      // Kiểm tra order chưa được thanh toán
      const order = await getOrder(orderId);
      if (order.status !== 'PENDING') {
        throw new Error('Order đã được xử lý');
      }

      // Xử lý thanh toán
      const payment = await processPayment(order, req.body.paymentMethod);

      // Update order status
      await updateOrderStatus(orderId, 'PAID');

      return payment;
    });

    res.json({ success: true, payment: result });

  } catch (error) {
    if (error.message.includes('đang bị khóa')) {
      return res.status(409).json({ error: 'Đang xử lý, thử lại sau' });
    }
    res.status(400).json({ error: error.message });
  }
});
```

📋 Tóm tắt Chương 14

- **Redis**: In-memory key-value store, nhanh hơn DB 100-1000x, hỗ trợ persistence

- **5 data types**: String (cache), Hash (object), List (queue), Set (unique), Sorted Set (leaderboard)
- **Cache-Aside**: Check cache → HIT return fast / MISS query DB → Store with TTL → Return
- **TTL**: Luôn đặt expire cho cache; invalidate khi data thay đổi (DEL key)
- **Session Store**: connect-redis thay thế MemoryStore, persist qua restarts và share giữa servers
- **Rate Limiting**: INCR + EXPIRE pattern, sliding window dùng Sorted Set chính xác hơn
- **Pub/Sub**: 2 clients riêng (subscriber + publisher), dùng cho cache invalidation broadcast
- **Distributed Lock**: SET NX PX + Lua script release, tránh race condition multi-server
- **Graceful degradation**: Nếu Redis lỗi, app vẫn hoạt động (chậm hơn nhưng không crash)

🏋️ Bài Tập Thực Hành

DỄ

Bài 1: Cache cho Product API

Implement cache cho `GET /api/products` và `GET /api/products/:id`. Cache danh sách 5 phút, cache chi tiết 1 giờ. Khi có `POST`, `PUT`, hoặc `DELETE` trên `/api/products`, tự động xóa cache danh sách. Khi `PUT /api/products/:id`, xóa cache của sản phẩm đó. Thêm header `X-Cache: HIT/MISS` vào response để debug.

TRUNG BÌNH

Bài 2: Leaderboard Real-Time

Xây dựng leaderboard game với Sorted Set. Endpoints cần có: `POST /scores` (nhận `{ userId, score }`, dùng ZADD nếu score cao hơn hiện tại), `GET /leaderboard?page=1` (top 10 mỗi page, kèm rank và score), `GET /leaderboard/me` (rank và score của user hiện tại), `GET /leaderboard/around-me?range=5` (5 người trên và dưới user hiện tại trong bảng xếp hạng).

KHÓ

Bài 3: Distributed Lock chống Double Payment

Implement hệ thống xử lý payment an toàn: (1) Tạo endpoint `POST /api/orders/:id/pay` với distributed lock để tránh double payment; (2) Lock key là `payment:{orderId}`, TTL 30 giây; (3) Dùng Lua script để atomic release lock; (4) Implement retry logic: nếu không lấy được lock, thử lại tối đa 3 lần với delay 500ms; (5) Thêm idempotency key (`X-Idempotency-Key` header) — nếu cùng key được dùng lần 2, trả về kết quả từ Redis cache thay vì xử lý lại.

PHẦN 5 — NÂNG CAO VÀ CLOUD

Chương 15: REST API Best Practices và API Design

Viết API chạy được chỉ mất vài phút. Viết API mà người khác (và tương lai bạn) thực sự muốn dùng đòi hỏi kiến thức về thiết kế. Chương này dạy các quy ước và patterns được cả thế giới dùng.

> 🎯 **SAU CHƯƠNG NÀY BẠN SẼ HIỂU ĐƯỢC**
>
> - Đặt tên resources đúng REST conventions
> - Dùng HTTP status codes đúng ngữ cảnh
> - Implement pagination, filtering, sorting chuẩn
> - Versioning API đúng cách
> - Document API với Swagger/OpenAPI
> - Upload file với Multer

15.1 REST Conventions — Đặt tên đúng

REST (Representational State Transfer) không phải là một chuẩn kỹ thuật cứng nhắc — đây là một tập hợp các quy ước được cộng đồng lập trình thế giới đồng thuận và áp dụng rộng rãi. Khi bạn đặt tên URL theo đúng REST conventions, mọi developer khác đều có thể đoán ra ý nghĩa của endpoint mà không cần đọc tài liệu.

Resources là danh từ số nhiều

Nguyên tắc cốt lõi nhất của REST: URL biểu diễn **resource** (tài nguyên), không phải **action** (hành động). Hành động được biểu diễn bằng HTTP method (GET, POST, PUT, DELETE). Vì vậy URL chỉ chứa danh từ, không chứa động từ.

Tại sao dùng số nhiều? Vì resource collection (tập hợp) về mặt khái niệm luôn ở dạng số nhiều. `/users` là tập hợp tất cả users, `/users/5` là user số 5 trong tập hợp đó. Nhất quán với số nhiều giúp URL dễ đoán hơn.

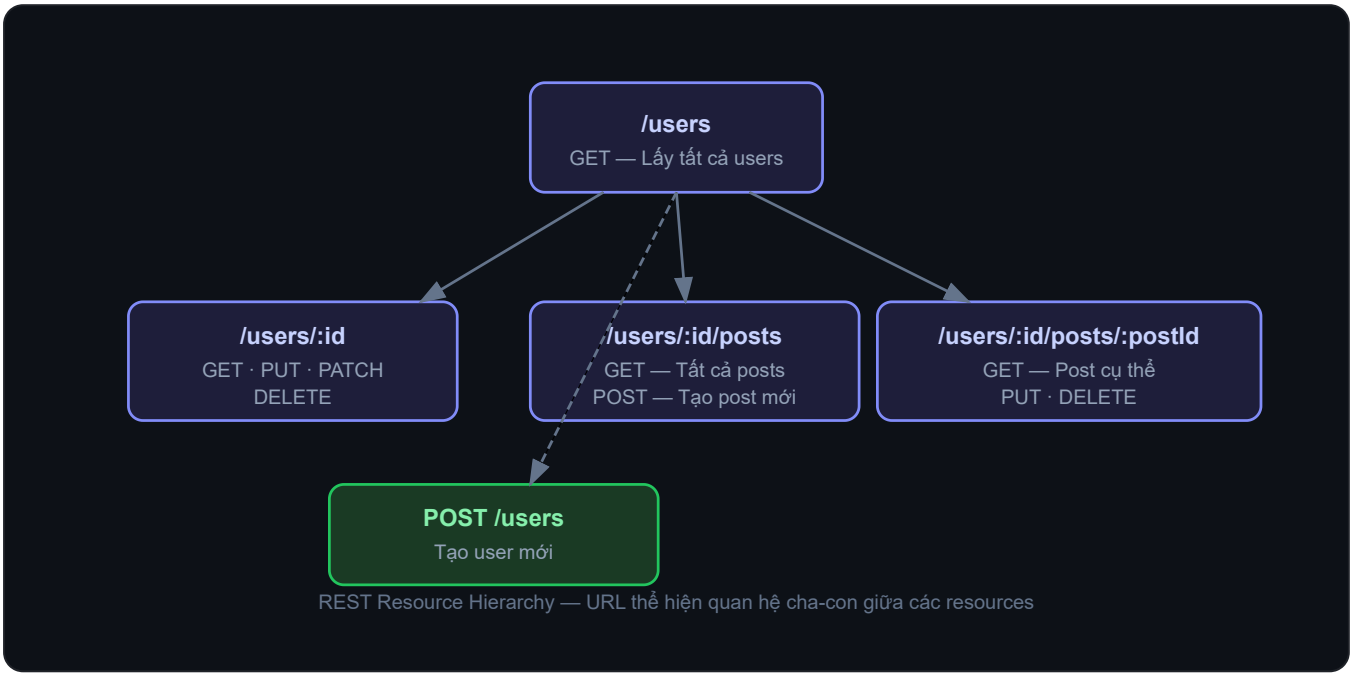
So sánh SAI vs ĐÚNG

SAI ❌ (có động từ)	ĐÚNG ✅ (REST convention)	Giải thích
`GET /getUsers`	`GET /users`	GET đã nói lên "lấy", không cần "get" trong URL
`POST /createUser`	`POST /users`	POST đã nói lên "tạo mới", resource là /users
`PUT /updateUser/5`	`PUT /users/5`	PUT đã nói lên "cập nhật", :id xác định resource cụ thể
`DELETE /deleteUser/5`	`DELETE /users/5`	DELETE đã nói lên "xóa", không cần thêm động từ
`GET /getUserPosts/5`	`GET /users/5/posts`	Nested resource thể hiện quan hệ cha-con rõ ràng

Nested Resources — Biểu diễn quan hệ cha-con

Khi một resource thuộc về một resource khác, dùng URL lồng nhau để thể hiện quan hệ đó. Tuy nhiên không nên lồng quá 2-3 cấp vì URL sẽ trở nên dài và khó đọc.

Ví dụ: `/users/42/posts/7/comments` — đây là comments của bài post số 7, thuộc user số 42. Nếu cần đi sâu hơn, hãy cân nhắc tạo endpoint phẳng hơn thay vì lồng thêm.



REST Resource Hierarchy — URL thể hiện quan hệ cha-con giữa các resources

> 💡 **MẸO**
>
> Khi bạn không chắc một action có phù hợp với REST không (ví dụ: "activate account", "send email"), hãy nghĩ về nó như một resource. `POST /users/:id/activation` hoặc `POST /emails/send` — biến action thành resource phụ là cách phổ biến nhất.

> 📌 **GHI NHỚ**
>
> Dùng **kebab-case** cho URL nhiều từ: `/blog-posts` thay vì `/blogPosts` hay `/blog_posts`. Hầu hết các API lớn (GitHub, Stripe, Twilio) đều dùng kebab-case cho URL và camelCase cho JSON body/response.

15.2 HTTP Status Codes — Dùng đúng ngữ cảnh

HTTP status codes là ngôn ngữ chung giữa server và client. Khi bạn trả về đúng status code, client (trình duyệt, mobile app, hay service khác) biết chính xác điều gì đã xảy ra mà không cần parse response body. Nhiều developer hay dùng 200 OK cho mọi thứ rồi nhét error vào body — đây là anti-pattern nghiêm trọng.

Code	Tên	Dùng khi nào	Ví dụ thực tế
200	OK	Request thành công, có data trả về	GET /users trả danh sách users
201	Created	Tạo resource mới thành công	POST /users tạo user mới; body chứa user vừa tạo
204	No Content	Thành công nhưng không có data trả về	DELETE /users/5 xóa thành công, không cần body
301	Moved Permanently	Resource đã đổi URL vĩnh viễn	API v1 bị deprecated, chuyển hướng sang v2
302	Found	Chuyển hướng tạm thời	Sau login, chuyển về trang dashboard
400	Bad Request	Request sai cú pháp / thiếu field bắt buộc	POST /users thiếu field email trong body
401	Unauthorized	Chưa xác thực (chưa đăng nhập)	Gọi API không kèm token hoặc token sai
403	Forbidden	Đã xác thực nhưng không có quyền	User thường cố xóa tài khoản của admin
404	Not Found	Resource không tồn tại	GET /users/9999 — user ID 9999 không có trong DB
409	Conflict	Conflict với trạng thái hiện tại của resource	POST /users với email đã tồn tại trong hệ thống
422	Unprocessable Entity	Cú pháp đúng nhưng dữ liệu không hợp lệ (validation fail)	Email đúng format nhưng domain không tồn tại
429	Too Many Requests	Client gửi quá nhiều request (rate limiting)	Gọi API quá 100 lần/phút vượt giới hạn cho phép
500	Internal Server Error	Lỗi không xác định phía server	Unhandled exception, DB connection drop đột ngột

Code	Tên	Dùng khi nào	Ví dụ thực tế
503	Service Unavailable	Server tạm thời không phục vụ được	Đang bảo trì, quá tải, hoặc dependency service sập

> ⚠️ **CẢNH BÁO**
>
> Phân biệt rõ **401 vs 403**: 401 có nghĩa "tôi không biết bạn là ai — hãy đăng nhập". 403 có nghĩa "tôi biết bạn là ai nhưng bạn không có quyền làm việc này". Nhầm lẫn hai mã này là lỗi phổ biến nhất khi implement auth middleware.

Một nguyên tắc hữu ích: nhóm theo chữ số đầu tiên. **2xx** — thành công. **3xx** — chuyển hướng. **4xx** — lỗi do client (client phải sửa request). **5xx** — lỗi do server (client không làm gì được, chờ server fix). Khi client nhận 4xx, họ biết cần sửa code của họ. Khi nhận 5xx, họ biết cần báo cáo lỗi.

15.3 Response Format Nhất Quán

Một trong những điều gây khó chịu nhất khi dùng API là mỗi endpoint lại có cấu trúc response khác nhau. Endpoint này trả `{ user: { ... } }`, endpoint kia trả thẳng object, endpoint nọ lại trả `{ result: true, payload: { ... } }`. Client phải handle từng trường hợp riêng — đây là design tệ.

Giải pháp: định nghĩa một response format nhất quán và dùng nó cho **tất cả** endpoints. Dưới đây là ví dụ về 3 loại response phổ biến nhất.

Success — Single Resource

JSON response-success.json Copy

```json
{
  "success": true,
  "message": "Lấy thông tin user thành công",
  "data": {
    "id": "64a1b2c3d4e5f67890123456",
    "name": "Nguyễn Văn An",
    "email": "an@example.com",
    "role": "user",
    "createdAt": "2024-01-15T08:30:00.000Z"
  }
}
```

Error Response

JSON response-error.json Copy

```json
{
  "success": false,
  "error": {
    "code": "USER_NOT_FOUND",
    "message": "Không tìm thấy user với ID đã cung cấp"
  }
}
```

List + Pagination

JSON response-list.json Copy

```json
{
  "success": true,
  "data": [
    { "id": "1", "name": "An", "email": "an@example.com" },
    { "id": "2", "name": "Bình", "email": "binh@example.com" }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 87,
    "totalPages": 9,
    "hasNext": true,
    "hasPrev": false
  }
}
```

Để đảm bảo tính nhất quán, tạo một module helper chứa tất cả các hàm format response. Mọi route handler chỉ gọi hàm từ module này thay vì tự viết `res.json()`.

JAVASCRIPT helpers/apiResponse.js Copy

```
// helpers/apiResponse.js
const success = (res, data, message = 'Thành công', statusCode = 200) => {
  return res.status(statusCode).json({ success: true, data, message });
};

const created = (res, data, message = 'Tạo thành công') => {
  return res.status(201).json({ success: true, data, message });
};

const paginated = (res, data, pagination) => {
  return res.status(200).json({ success: true, data, pagination });
};

const error = (res, message, code, statusCode = 400) => {
  return res.status(statusCode).json({ success: false, error: { code,
message } });
};

module.exports = { success, created, paginated, error };
```

Và cách dùng trong các route handler:

JAVASCRIPT routes/users.js Copy

```
const apiResponse = require('../helpers/apiResponse');

// GET /users/:id
router.get('/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id).lean();
  if (!user) {
    return apiResponse.error(res, 'Không tìm thấy user', 'USER_NOT_FOUND',
404);
  }
  return apiResponse.success(res, user, 'Lấy thông tin user thành công');
}));

// POST /users
router.post('/', asyncHandler(async (req, res) => {
  const existing = await User.findOne({ email: req.body.email });
  if (existing) {
    return apiResponse.error(res, 'Email đã tồn tại', 'EMAIL_CONFLICT',
409);
  }
  const user = await User.create(req.body);
  return apiResponse.created(res, user, 'Tạo user thành công');
}));
```

📌 **GHI NHỚ**

Trường `error.code` là machine-readable string (ví dụ: `USER_NOT_FOUND`, `EMAIL_CONFLICT`). Client có thể switch-case trên đây để hiển thị thông báo lỗi phù hợp với ngôn ngữ của user mà không cần parse message (vốn có thể thay đổi).

15.4 Pagination — Phân trang

Không bao giờ trả về toàn bộ collection trong một request. Nếu database có 1 triệu records, trả về tất cả trong một lần sẽ làm sập server và timeout client. Có hai chiến lược phân trang phổ biến: **offset pagination** và **cursor pagination**.

Offset Pagination (Trang theo số thứ tự)

Đây là cách đơn giản nhất và quen thuộc nhất: user yêu cầu "trang số N, mỗi trang M items". Phù hợp khi người dùng cần nhảy tới một trang cụ thể (ví dụ: "đi tới trang 5"). Nhược điểm: hiệu suất kém khi offset lớn vì DB phải scan và bỏ qua nhiều rows.

JAVASCRIPT middleware/paginate.js Copy

```
// middleware/paginate.js
const paginate = (req) => {
  const page = Math.max(1, parseInt(req.query.page) || 1);
  const limit = Math.min(100, Math.max(1, parseInt(req.query.limit) || 10));
  const skip = (page - 1) * limit;
  return { page, limit, skip };
};

// Usage in route
router.get('/posts', asyncHandler(async (req, res) => {
  const { page, limit, skip } = paginate(req);
  const [data, total] = await Promise.all([
    Post.find().skip(skip).limit(limit).lean(),
    Post.countDocuments()
  ]);
  const totalPages = Math.ceil(total / limit);
  apiResponse.paginated(res, data, {
    page, limit, total, totalPages,
    hasNext: page < totalPages,
    hasPrev: page > 1
  });
}));
```

Chú ý `Math.min(100, ...)` — giới hạn tối đa 100 items/page để tránh client yêu cầu `?limit=999999` gây quá tải server. Luôn có giới hạn trên cho limit.

Cursor Pagination (Phân trang theo con trỏ)

Cursor pagination không dùng số trang mà dùng một "con trỏ" (thường là ID của item cuối cùng) để xác định điểm bắt đầu của trang tiếp theo. Đây là cách tiếp cận của Facebook, Twitter, Instagram — những nơi có feed vô hạn (infinite scroll). Hiệu suất tốt hơn offset rất nhiều vì DB có thể dùng index để tìm điểm bắt đầu.

JAVASCRIPT routes/posts.js Copy

```
router.get('/posts/cursor', asyncHandler(async (req, res) => {
  const { cursor, limit = 10 } = req.query;
  const query = cursor ? { _id: { $lt: cursor } } : {};
  const posts = await Post.find(query)
    .sort({ _id: -1 })
    .limit(parseInt(limit) + 1)
    .lean();
  const hasMore = posts.length > limit;
  if (hasMore) posts.pop();
  res.json({
    success: true,
    data: posts,
    nextCursor: hasMore ? posts[posts.length - 1]._id : null,
    hasMore
  });
}));
```

Trick quan trọng: query `limit + 1` items. Nếu trả về đúng `limit + 1` items, nghĩa là vẫn còn data (hasMore = true) và item cuối cùng bị pop ra làm nextCursor. Nếu trả về ít hơn hoặc bằng `limit` items, là trang cuối (hasMore = false).

> 💡 **MẸO**
>
> Chọn loại pagination nào? Nếu UI cần "Trang 1, 2, 3..." hoặc "Nhảy đến trang 50" → dùng offset. Nếu UI là infinite scroll (TikTok, Instagram feed, chat messages) → dùng cursor. Cursor nhanh hơn và không bị vấn đề "item bị skip hoặc lặp" khi có data mới insert trong lúc user đang phân trang.

15.5 Filtering, Sorting và Search

Sau khi đã implement pagination, bước tiếp theo là cho phép client lọc và sắp xếp data. Tất cả filtering parameters được truyền qua query string. Ví dụ:

- `GET /products?category=electronics&minPrice=100&maxPrice=500` — lọc theo category và giá
- `GET /products?sort=price&order=asc` — sắp xếp theo giá tăng dần
- `GET /products?q=macbook` — full-text search theo từ khóa

- `GET /products?fields=id,name,price` — chỉ lấy các fields cần thiết (field selection)

Thay vì viết logic lặp đi lặp lại ở mỗi route, hãy tách ra thành một helper function:

JAVASCRIPT helpers/buildMongoQuery.js Copy

```
// helpers/buildMongoQuery.js
const buildMongoQuery = (queryParams) => {
  const { q, category, minPrice, maxPrice, inStock, sort, order, fields } =
queryParams;
  const filter = {};

  if (q) filter.$text = { $search: q };
  if (category) filter.category = category;
  if (minPrice || maxPrice) {
    filter.price = {};
    if (minPrice) filter.price.$gte = Number(minPrice);
    if (maxPrice) filter.price.$lte = Number(maxPrice);
  }
  if (inStock !== undefined) filter.inStock = inStock === 'true';

  // Sort: ?sort=price&order=asc hoặc ?sort=-price
  let sortObj = { createdAt: -1 };
  if (sort) {
    const sortField = sort.startsWith('-') ? sort.slice(1) : sort;
    const sortDir = sort.startsWith('-') ? -1 : (order === 'desc' ? -1 : 1);
    sortObj = { [sortField]: sortDir };
  }

  // Field selection: ?fields=id,name,email
  let projection = null;
  if (fields) {
    projection = fields.split(',').reduce((acc, f) => ({ ...acc, [f.trim()]:
1 }), {});
  }

  return { filter, sortObj, projection };
};

module.exports = buildMongoQuery;
```

Và cách dùng helper trong route:

JAVASCRIPT routes/products.js Copy

```
const buildMongoQuery = require('../helpers/buildMongoQuery');
const { paginate } = require('../middleware/paginate');

router.get('/', asyncHandler(async (req, res) => {
  const { filter, sortObj, projection } = buildMongoQuery(req.query);
  const { page, limit, skip } = paginate(req);

  let query = Product.find(filter).sort(sortObj).skip(skip).limit(limit);
  if (projection) query = query.select(projection);

  const [data, total] = await Promise.all([
    query.lean(),
    Product.countDocuments(filter)
  ]);

  const totalPages = Math.ceil(total / limit);
  apiResponse.paginated(res, data, {
    page, limit, total, totalPages,
    hasNext: page < totalPages,
    hasPrev: page > 1
  });
}));
```

⚠️ CẢNH BÁO

Luôn whitelist các fields được phép filter/sort. Nếu client có thể sort theo bất kỳ field nào, họ có thể sort theo field không có index, gây full table scan và làm chậm toàn bộ hệ thống. Hãy kiểm tra `sort` chỉ nằm trong danh sách `['price', 'name', 'createdAt', 'rating']` trước khi áp dụng.

📌 GHI NHỚ

Để dùng full-text search với `$text`, MongoDB cần có text index trên các fields muốn search. Thêm vào Mongoose schema: `ProductSchema.index({ name: 'text', description: 'text' });` Nếu không có index, query sẽ báo lỗi "text index required".

15.6 API Versioning

Khi API của bạn đã có người dùng (internal teams, mobile apps, third-party clients), bạn không thể đột ngột thay đổi response format hay xóa fields — điều đó sẽ break ứng dụng của họ. Đây là lúc cần API versioning.

Ba chiến lược versioning phổ biến

- **URL versioning**: `/api/v1/users`, `/api/v2/users` — rõ ràng nhất, dễ test, được dùng nhiều nhất
- **Header versioning**: `Accept: application/vnd.myapi.v2+json` — "sạch" hơn về mặt REST nhưng khó test với browser
- **Query param versioning**: `/api/users?version=2` — linh hoạt nhưng dễ bị quên, ít phổ biến

Với Express, URL versioning được implement bằng cách mount các router riêng biệt:

JAVASCRIPT app.js Copy

```
// app.js
app.use('/api/v1', require('./routes/v1'));
app.use('/api/v2', require('./routes/v2'));
```

JAVASCRIPT routes/v1/users.js Copy

```
// routes/v1/users.js
const express = require('express');
const router = express.Router();
const User = require('../../models/User');

// V1: chỉ trả về name và email
router.get('/', async (req, res) => {
  const users = await User.find().select('name email').lean();
  res.json({ success: true, data: users });
});

module.exports = router;
```

JAVASCRIPT routes/v2/users.js Copy

```
// routes/v2/users.js
const express = require('express');
const router = express.Router();
const User = require('../../models/User');

// V2: thêm phone, avatar, createdAt và metadata
router.get('/', async (req, res) => {
  const users = await User.find().select('name email phone avatar
createdAt').lean();
  res.json({
    success: true,
    data: users,
    _meta: { version: '2.0', deprecation: null }
  });
});

module.exports = router;
```

Deprecation Policy — Thông báo version cũ sắp bị xóa

Khi bạn ra v2, không nên xóa v1 ngay lập tức. Hãy thêm header `Deprecation` vào tất cả response của v1 để thông báo cho client biết họ cần migrate:

JAVASCRIPT middleware/deprecation.js Copy

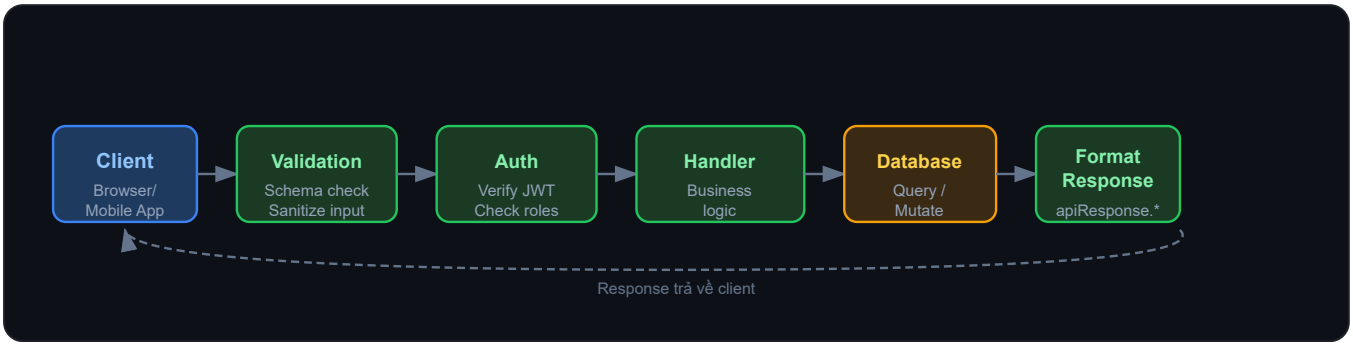
```
// middleware/deprecation.js - thêm vào tất cả v1 routes
const deprecationWarning = (req, res, next) => {
  res.set('Deprecation', 'true');
  res.set('Sunset', 'Sat, 01 Jan 2026 00:00:00 GMT');
  res.set('Link', '<https://api.example.com/v2>; rel="successor-version"');
  next();
};

module.exports = deprecationWarning;

// app.js
app.use('/api/v1', deprecationWarning, require('./routes/v1'));
```

Request Lifecycle Diagram

Mỗi request qua API đều đi qua một vòng đời chuẩn. Hiểu rõ vòng đời này giúp bạn đặt middleware đúng chỗ và debug hiệu quả hơn:



💡 MẸO

Nên dùng URL versioning (`/api/v1/` , `/api/v2/`) cho các public API hoặc API có nhiều client độc lập. Với internal microservices trong cùng team, header versioning hoặc contract testing (xem chương 16) là lựa chọn tốt hơn.

15.7 File Upload với Multer

Multer là middleware phổ biến nhất để handle `multipart/form-data` trong Express — đây là content type được dùng khi upload file. Multer parse request, lưu file vào disk (hoặc memory), và gắn thông tin file vào `req.file` / `req.files`.

BASH Terminal Copy

```
npm install multer
```

JAVASCRIPT middleware/upload.js Copy

```
// middleware/upload.js
const multer = require('multer');
const path = require('path');

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/avatars/');
  },
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname);
    cb(null, `avatar-${req.user.id}-${Date.now()}${ext}`);
  }
});

const fileFilter = (req, file, cb) => {
  const allowed = ['image/jpeg', 'image/png', 'image/webp'];
  if (allowed.includes(file.mimetype)) {
    cb(null, true);
  } else {
    cb(new Error('Chỉ chấp nhận file JPEG, PNG hoặc WebP'), false);
  }
};

const upload = multer({
  storage,
  limits: { fileSize: 5 * 1024 * 1024 }, // 5MB
  fileFilter
});

module.exports = upload;
```

Sau khi có middleware, kết nối vào route. Lưu ý middleware `upload.single('avatar')` phải được đặt trước route handler:

JAVASCRIPT routes/users.js Copy

```
// routes/users.js
const upload = require('../middleware/upload');
const apiResponse = require('../helpers/apiResponse');

router.post('/:id/avatar',
  authenticate,
  upload.single('avatar'),
  asyncHandler(async (req, res) => {
    if (!req.file) {
      return apiResponse.error(res, 'Vui lòng chọn file', 'FILE_REQUIRED',
400);
    }
    const user = await User.findByIdAndUpdate(
      req.params.id,
      { avatar: `/uploads/avatars/${req.file.filename}` },
      { new: true }
    );
    apiResponse.success(res, { avatar: user.avatar }, 'Upload avatar thành
công');
  })
);
```

Upload nhiều file cùng lúc

Multer cũng hỗ trợ upload nhiều file. Dùng `upload.array('images', 5)` để nhận tối đa 5 file từ field tên "images", hoặc `upload.fields([ ... ])` cho nhiều field khác tên:

JAVASCRIPT routes/posts.js Copy

```
// Upload nhiều ảnh cho một bài post
router.post('/:id/images',
  authenticate,
  upload.array('images', 5),
  asyncHandler(async (req, res) => {
    if (!req.files || req.files.length === 0) {
      return apiResponse.error(res, 'Vui lòng chọn ít nhất một ảnh',
'FILES_REQUIRED', 400);
    }
    const imageUrls = req.files.map(f => `/uploads/posts/${f.filename}`);
    const post = await Post.findByIdAndUpdate(
      req.params.id,
      { $push: { images: { $each: imageUrls } } },
      { new: true }
    );
    apiResponse.success(res, { images: post.images }, `Upload
${req.files.length} ảnh thành công`);
  })
);

// Upload nhiều loại file khác nhau
router.post('/submit',
  authenticate,
  upload.fields([
    { name: 'resume', maxCount: 1 },
    { name: 'portfolio', maxCount: 3 }
  ]),
  asyncHandler(async (req, res) => {
    const resume = req.files['resume']?.[0];
    const portfolios = req.files['portfolio'] || [];
    // xử lý tiếp ...
    apiResponse.success(res, { received: true }, 'Nộp hồ sơ thành công');
  })
);
```

⚠ CẢNH BÁO

Không lưu file trực tiếp trên server production vì file sẽ mất khi deploy lại hoặc server restart. Dùng cloud storage: **AWS S3**, **Google Cloud Storage**, hoặc **Cloudinary** (tiện nhất cho ảnh vì tự động resize/optimize). Package `multer-s3` cho phép stream file thẳng lên S3 mà không cần lưu tạm.

Handle lỗi từ Multer

Multer throw lỗi riêng có type là `MulterError`. Cần bắt riêng để trả về thông báo lỗi có nghĩa cho client:

JAVASCRIPT middleware/errorHandler.js Copy

```
const multer = require('multer');

const handleMulterError = (err, req, res, next) => {
  if (err instanceof multer.MulterError) {
    if (err.code === 'LIMIT_FILE_SIZE') {
      return res.status(400).json({
        success: false,
        error: { code: 'FILE_TOO_LARGE', message: 'File không được vượt quá
5MB' }
      });
    }
    if (err.code === 'LIMIT_FILE_COUNT') {
      return res.status(400).json({
        success: false,
        error: { code: 'TOO_MANY_FILES', message: 'Tối đa 5 file mỗi lần
upload' }
      });
    }
  }
  // Lỗi từ fileFilter
  if (err.message === 'Chỉ chấp nhận file JPEG, PNG hoặc WebP') {
    return res.status(400).json({
      success: false,
      error: { code: 'INVALID_FILE_TYPE', message: err.message }
    });
  }
  next(err);
};

module.exports = handleMulterError;
```

15.8 OpenAPI/Swagger Documentation

API documentation không phải là luxury — đây là yêu cầu cơ bản. Không có documentation, không ai biết cách dùng API của bạn (kể cả bạn sau 3 tháng). Swagger/OpenAPI là chuẩn công nghiệp để document REST API: viết một lần, tự động generate interactive UI, client SDK, và test cases.

BASH Terminal Copy

```bash
npm install swagger-jsdoc swagger-ui-express
```

JAVASCRIPT config/swagger.js Copy

```javascript
// config/swagger.js
const swaggerJsdoc = require('swagger-jsdoc');

const options = {
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'My API',
      version: '1.0.0',
      description: 'API documentation cho NodeJS project'
    },
    servers: [{ url: 'http://localhost:3000/api/v1' }],
    components: {
      securitySchemes: {
        bearerAuth: { type: 'http', scheme: 'bearer', bearerFormat: 'JWT' }
      }
    }
  },
  apis: ['./routes/**/*.js']
};

module.exports = swaggerJsdoc(options);
```

JAVASCRIPT app.js Copy

```javascript
// app.js
const swaggerUi = require('swagger-ui-express');
const swaggerSpec = require('./config/swagger');

app.use('/api/docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
```

Sau khi mount, vào `http://localhost:3000/api/docs` để xem Swagger UI — giao diện interactive cho phép test API trực tiếp từ browser mà không cần Postman.

Viết JSDoc Annotations cho Swagger

`swagger-jsdoc` đọc các comment JSDoc đặc biệt trong file routes để tự động generate spec. Đây là cách tốt nhất vì documentation nằm ngay cạnh code:

JAVASCRIPT routes/v1/users.js Copy

```
/**
 * @swagger
 * /users:
 *   get:
 *     summary: Lấy danh sách users
 *     tags: [Users]
 *     security:
 *       - bearerAuth: []
 *     parameters:
 *       - in: query
 *         name: page
 *         schema:
 *           type: integer
 *           default: 1
 *         description: Số trang
 *       - in: query
 *         name: limit
 *         schema:
 *           type: integer
 *           default: 10
 *         description: Số items mỗi trang
 *     responses:
 *       200:
 *         description: Danh sách users thành công
 *         content:
 *           application/json:
 *             schema:
 *               type: object
 *               properties:
 *                 success:
 *                   type: boolean
 *                 data:
 *                   type: array
 *                 pagination:
 *                   type: object
 *       401:
 *         description: Chưa xác thực
 */
```

```
router.get('/', authenticate, asyncHandler(async (req, res) => {
  // implementation ...
}));

/**
 * @swagger
 * /users/{id}:
 *   get:
 *     summary: Lấy thông tin một user
 *     tags: [Users]
 *     security:
 *       - bearerAuth: []
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema:
 *           type: string
 *         description: MongoDB ObjectId của user
 *     responses:
 *       200:
 *         description: Thông tin user
 *       404:
 *         description: Không tìm thấy user
 */
router.get('/:id', authenticate, asyncHandler(async (req, res) => {
  // implementation ...
}));
```

Định nghĩa Schemas tái sử dụng

Thay vì lặp lại schema ở mỗi endpoint, định nghĩa một lần trong `components/schemas` rồi tham chiếu bằng `$ref`:

JAVASCRIPT config/swagger.js Copy

```
// Thêm vào phần components trong swagger options
components: {
  schemas: {
    User: {
      type: 'object',
      properties: {
        id: { type: 'string', example: '64a1b2c3d4e5f678' },
        name: { type: 'string', example: 'Nguyễn Văn An' },
        email: { type: 'string', format: 'email', example: 'an@example.com'
},
        role: { type: 'string', enum: ['user', 'admin'], example: 'user' },
        createdAt: { type: 'string', format: 'date-time' }
      }
    },
    ApiError: {
      type: 'object',
      properties: {
        success: { type: 'boolean', example: false },
        error: {
          type: 'object',
          properties: {
            code: { type: 'string', example: 'USER_NOT_FOUND' },
            message: { type: 'string', example: 'Không tìm thấy user' }
          }
        }
      }
    }
  },
  securitySchemes: {
    bearerAuth: { type: 'http', scheme: 'bearer', bearerFormat: 'JWT' }
  }
}
```

💡 **MẸO**

Ngoài `swagger-jsdoc`, có thể dùng **Postman** để generate OpenAPI spec từ collection đã có, hoặc dùng **Insomnia** với plugin OpenAPI export. Một lựa chọn khác là viết spec YAML trước (design-first), rồi dùng spec

để generate route stubs. Approach này đặc biệt tốt khi làm việc nhóm: backend và frontend đồng ý spec trước, rồi làm song song.

Bảo vệ Swagger UI trong môi trường production

Không nên để Swagger UI public trên production vì nó expose toàn bộ cấu trúc API. Cách đơn giản nhất là chỉ bật Swagger trong development:

JAVASCRIPT app.js Copy

```
// Chỉ mount Swagger UI trong môi trường development
if (process.env.NODE_ENV !== 'production') {
  const swaggerUi = require('swagger-ui-express');
  const swaggerSpec = require('./config/swagger');
  app.use('/api/docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
  console.log('📖 Swagger UI: http://localhost:3000/api/docs');
}
```

15.9 Rate Limiting — Bảo vệ API khỏi bị lạm dụng

Một API không có rate limiting là API không an toàn. Không có giới hạn, client có thể gửi hàng nghìn request mỗi giây, làm sập server hoặc tốn chi phí database/cloud của bạn. Package `express-rate-limit` là giải pháp đơn giản và hiệu quả nhất:

BASH Terminal Copy

```
npm install express-rate-limit
```

JAVASCRIPT middleware/rateLimiter.js Copy

```
const rateLimit = require('express-rate-limit');

// General rate limiter: 100 requests per 15 minutes
const generalLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 phút
  max: 100,
  message: {
    success: false,
    error: {
      code: 'RATE_LIMIT_EXCEEDED',
      message: 'Quá nhiều request. Vui lòng thử lại sau 15 phút.'
    }
  },
  standardHeaders: true,  // Thêm RateLimit-* headers
  legacyHeaders: false
});

// Strict limiter cho auth endpoints
const authLimiter = rateLimit({
  windowMs: 60 * 60 * 1000, // 1 giờ
  max: 10, // Chỉ 10 lần login thất bại mỗi giờ
  message: {
    success: false,
    error: {
      code: 'AUTH_RATE_LIMIT',
      message: 'Quá nhiều lần đăng nhập thất bại. Vui lòng thử lại sau 1
giờ.'
    }
  }
});

module.exports = { generalLimiter, authLimiter };

// app.js
const { generalLimiter, authLimiter } = require('./middleware/rateLimiter');
app.use('/api/', generalLimiter);
app.use('/api/v1/auth', authLimiter);
```

Khi request bị block, server trả về status **429 Too Many Requests** cùng với headers `RateLimit-Limit`, `RateLimit-Remaining`, và `RateLimit-Reset` để client biết khi nào được gửi request tiếp.

> 📌 **GHI NHỚ**
>
> Mặc định `express-rate-limit` lưu trạng thái trong memory của process. Khi deploy với nhiều instances (horizontal scaling), mỗi instance có counter riêng — dẫn đến rate limit không hoạt động đúng. Dùng `rate-limit-redis` store để chia sẻ counter qua Redis (đã học ở chương 14).

15.10 Input Validation với Joi hoặc Zod

Validate input ở mọi endpoint là bắt buộc — không bao giờ tin dữ liệu từ client. Có hai library phổ biến: **Joi** (mature, callback-style) và **Zod** (TypeScript-first, schema tái dùng với inferred types). Dưới đây là ví dụ với Joi:

BASH Terminal Copy

```
npm install joi
```

JAVASCRIPT validation/userSchemas.js Copy

```
const Joi = require('joi');

const createUserSchema = Joi.object({
  name: Joi.string().min(2).max(100).required()
    .messages({ 'string.min': 'Tên phải có ít nhất 2 ký tự' }),
  email: Joi.string().email().required()
    .messages({ 'string.email': 'Email không hợp lệ' }),
  password: Joi.string().min(8).pattern(/^(?=.*[A-Z])(?=.*[0-
9])/).required()
    .messages({
      'string.min': 'Mật khẩu phải có ít nhất 8 ký tự',
      'string.pattern.base': 'Mật khẩu phải có ít nhất 1 chữ hoa và 1 số'
    }),
  role: Joi.string().valid('user', 'admin').default('user')
});

const updateUserSchema = Joi.object({
  name: Joi.string().min(2).max(100),
  email: Joi.string().email(),
  phone: Joi.string().pattern(/^[0-9]{10,11}$/)
}).min(1); // Phải có ít nhất 1 field

module.exports = { createUserSchema, updateUserSchema };
```

JAVASCRIPT middleware/validate.js Copy

```javascript
// Middleware factory: validate(schema) trả về middleware function
const validate = (schema, source = 'body') => {
  return (req, res, next) => {
    const { error, value } = schema.validate(req[source], {
      abortEarly: false,  // Gom tất cả lỗi, không dừng ở lỗi đầu tiên
      stripUnknown: true  // Xóa các fields không có trong schema
    });
    if (error) {
      const details = error.details.map(d => ({
        field: d.path.join('.'),
        message: d.message
      }));
      return res.status(422).json({
        success: false,
        error: {
          code: 'VALIDATION_ERROR',
          message: 'Dữ liệu không hợp lệ',
          details
        }
      });
    }
    req[source] = value; // Thay thế bằng data đã được validate và sanitize
    next();
  };
};

module.exports = validate;

// Dùng trong route:
// router.post('/', validate(createUserSchema), asyncHandler(handler));
```

💡 **MẸO**

Dùng `abortEarly: false` để Joi thu thập tất cả lỗi validation trong một lần, thay vì dừng ở lỗi đầu tiên. Client nhận được toàn bộ danh sách lỗi trong một request, giúp họ sửa form nhanh hơn mà không phải submit nhiều lần.

Bài tập thực hành

Các bài tập dưới đây yêu cầu bạn tự triển khai các tính năng đã học. Khởi tạo một Express project mới với Mongoose và áp dụng từng pattern theo thứ tự.

DỄ

15.1 Field Selection Middleware

Thêm field selection vào `GET /users`: query param `?fields=id,name,email` chỉ trả về các fields được chỉ định thay vì toàn bộ document. Gợi ý: dùng `.select()` của Mongoose với danh sách fields từ query string. Đảm bảo whitelist các fields được phép chọn (không cho phép chọn `password` hay `__v`).

TRUNG BÌNH

15.2 Cursor Pagination

Implement cursor-based pagination thay cho offset pagination cho `GET /posts` (sort by `_id` giảm dần). Đảm bảo response có `nextCursor` và `hasMore`. Test với Postman: lấy trang đầu, copy `nextCursor` vào request tiếp theo, xác nhận không có item bị lặp hoặc bị skip. Xử lý edge case: cursor không hợp lệ hoặc đã là trang cuối.

KHÓ

15.3 API Versioning Demo

Build demo versioning hoàn chỉnh: `v1` trả `{ name, email }`, `v2` thêm `{ phone, avatar, joinedAt }`. Thêm header `X-API-Version` vào mọi response để client biết đang dùng version nào. Thêm `Deprecation: true` và `Sunset: <date>` header cho tất cả endpoints của v1. Document cả hai version với Swagger (hai server trong `servers` array của swagger config). Test với curl và verify headers.

Tổng kết chương 15

Chúng ta đã đi qua toàn bộ các best practices cốt lõi để thiết kế REST API chuyên nghiệp. Đây là bản tóm tắt nhanh những điểm quan trọng nhất:

Chủ đề	Best Practice	Anti-pattern cần tránh
URL Design	Danh từ số nhiều, không có động từ	`/getUsers`, `/createPost`
Status Codes	Dùng đúng code theo ngữ cảnh	Trả 200 cho mọi thứ kể cả lỗi
Response Format	Nhất quán, có `success` flag và `error.code`	Mỗi endpoint một kiểu format khác nhau
Pagination	Luôn phân trang, giới hạn max limit	Trả toàn bộ collection, không giới hạn
Versioning	URL versioning rõ ràng, có deprecation policy	Break changes mà không báo trước
File Upload	Validate mimetype, giới hạn size, lưu cloud	Accept mọi file, lưu thẳng trên local disk production
Documentation	Swagger/OpenAPI, annotations ngay trong code	Không có docs hoặc docs lỗi thời
Rate Limiting	Giới hạn request, trả 429 với thông tin rõ ràng	Không có giới hạn, để API bị abuse
Validation	Validate mọi input, trả về tất cả lỗi cùng lúc	Tin dữ liệu từ client, chỉ validate ở frontend

📌 GHI NHỚ

Các nguyên tắc trong chương này không phải "nice to have" — đây là những gì phân biệt một junior developer với một senior developer trong mắt team lead và technical interviewer. Khi bạn apply vào các công ty công nghệ, họ sẽ review thiết kế API của bạn theo chính những tiêu chuẩn này.

PHẦN 5 — NÂNG CAO VÀ CLOUD

Chương 16: Testing và Code Quality

Code không có test là code bạn không dám sửa. Test không chỉ bắt bugs — chúng là tài liệu sống, là mạng lưới an toàn giúp bạn refactor tự tin. Chương này xây dựng test suite đầy đủ.

> 🎯 **SAU CHƯƠNG NÀY BẠN SẼ HIỂU ĐƯỢC**
>
> - Phân biệt Unit Test, Integration Test, E2E Test
> - Setup Jest và viết tests từ cơ bản đến nâng cao
> - Mock dependencies để test độc lập
> - Integration test HTTP endpoints với Supertest
> - Đo lường test coverage và đặt threshold
> - Cấu hình ESLint và Prettier cho code quality

16.1 Tại sao cần Test?

Câu hỏi này nghe có vẻ đơn giản, nhưng nhiều lập trình viên vẫn bỏ qua testing vì áp lực deadline, hoặc vì nghĩ rằng "code nhỏ thì không cần test". Thực tế cho thấy điều ngược lại: **chi phí sửa một bug tăng theo cấp số nhân theo thời gian bug tồn tại**.

Chi phí của Bug theo thời gian

Một nghiên cứu nổi tiếng của IBM và nhiều tổ chức phần mềm lớn chỉ ra rằng chi phí sửa một bug phụ thuộc rất lớn vào thời điểm bug được phát hiện:

Giai đoạn phát hiện	Chi phí tương đối	Mô tả
Trong lúc dev viết code	1x	Dev tự phát hiện ngay, sửa trong vài phút
Trong Code Review	5x	Reviewer thấy, dev phải sửa lại và review lại
QA / Testing team	15x	QA tạo ticket, dev fix, QA retest — mất nhiều ngày
Production (user phát hiện)	100x+	Hotfix khẩn cấp, downtime, mất dữ liệu, thiệt hại thương hiệu

Điều này có nghĩa là đầu tư thời gian viết tests ngay từ đầu là một quyết định kinh tế rất khôn ngoan. Một test suite tốt giúp bạn phát hiện bugs ngay khi chúng được tạo ra, với chi phí thấp nhất có thể.

Test là Documentation không thể Outdated

Tài liệu thông thường (README, wiki, comments) rất dễ bị lỗi thời. Khi code thay đổi, documentation thường không được cập nhật theo. Nhưng tests thì khác — nếu bạn thay đổi behavior của code mà không cập nhật tests, tests sẽ fail ngay lập tức. Điều này đảm bảo tests luôn phản ánh đúng hành vi thực tế của hệ thống.

Một bộ tests tốt có thể trả lời câu hỏi: "*Hàm này làm gì với input này?*" rõ ràng hơn bất kỳ comment nào. Khi một developer mới join team, đọc tests là cách nhanh nhất để hiểu codebase hoạt động như thế nào.

Refactor với Confidence

Đây là một trong những lợi ích lớn nhất của testing. Khi bạn có test suite đầy đủ, bạn có thể refactor code mạnh dạn hơn. Thay vì lo lắng "tôi đổi cái này liệu có break gì không?", bạn chỉ cần chạy tests và để chúng trả lời câu hỏi đó.

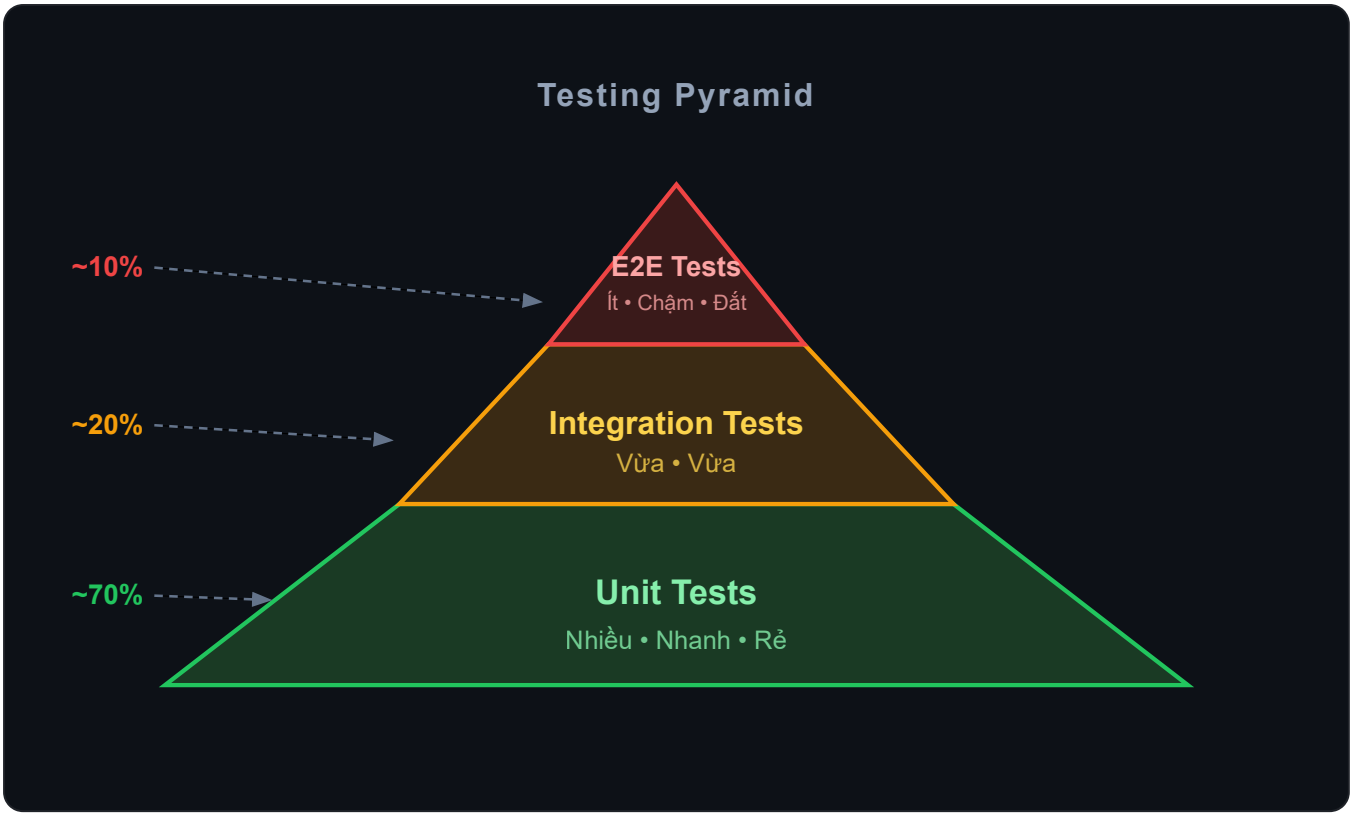
> 💡 **MẸO**
>
> Một quy tắc vàng: **không bao giờ refactor mà không có tests**. Nếu cần refactor một module chưa có tests, hãy viết tests trước (characterization tests — tests mô tả behavior hiện tại), rồi mới refactor.

Testing Pyramid

Martin Fowler đã phổ biến khái niệm **Testing Pyramid** — một mô hình hướng dẫn cách phân bổ các loại tests. Hình tháp có ba tầng, từ dưới lên trên:

- **Unit Tests (tầng đáy)**: Test từng hàm, module riêng lẻ. Nhanh, rẻ, nhiều nhất.
- **Integration Tests (tầng giữa)**: Test sự tương tác giữa các module, database, API. Chậm hơn, ít hơn.

- **E2E Tests (đỉnh)**: Test toàn bộ luồng từ UI đến database. Chậm nhất, đắt nhất, ít nhất.



Lý do tỉ lệ này được khuyến nghị: Unit tests cung cấp phản hồi nhanh nhất (chạy trong mili-giây), dễ debug nhất khi fail, và ít tốn kém nhất để viết và maintain. E2E tests thì ngược lại — chạy chậm (có thể mất nhiều phút), khó debug, và dễ bị flaky (đôi khi pass đôi khi fail không có lý do rõ ràng).

> 📌 **GHI NHỚ**
>
> Testing pyramid chỉ là hướng dẫn, không phải quy tắc cứng nhắc. Tùy thuộc vào dự án, đôi khi tầng integration tests có thể chiếm tỉ lệ lớn hơn (ví dụ như các API-heavy backends). Mục tiêu là có *đủ* tests ở mỗi tầng để đảm bảo độ tin cậy.

16.2 Setup Jest

Jest là testing framework phổ biến nhất trong hệ sinh thái JavaScript/NodeJS, được phát triển bởi Meta (Facebook). Jest có đầy đủ mọi thứ cần thiết trong một package: test runner, assertion library, mocking, và coverage reporting — không cần cài thêm nhiều packages khác nhau như với Mocha/Chai/Sinon.

Cài đặt

BASH terminal Copy

```bash
# Cài Jest và Supertest (dùng cho integration tests)
npm install --save-dev jest supertest

# Nếu dùng TypeScript
npm install --save-dev ts-jest @types/jest

# Kiểm tra version
npx jest --version
```

Cấu hình npm scripts

Thêm các scripts vào `package.json` để có thể chạy tests theo nhiều mode khác nhau:

JSON package.json Copy

```json
{
  "scripts": {
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage",
    "test:ci": "jest --ci --coverage --forceExit"
  }
}
```

Giải thích từng script:

- `jest`: Chạy tất cả tests một lần rồi thoát.
- `jest --watch`: Chế độ watch — tự động re-run tests khi file thay đổi. Rất hữu ích khi đang phát triển.
- `jest --coverage`: Chạy tests và sinh báo cáo coverage.
- `jest --ci --coverage --forceExit`: Mode dùng cho CI/CD. `--ci` tắt interactive mode, `--forceExit` đảm bảo process thoát sau khi tests xong (quan trọng khi có database connections chưa đóng).

jest.config.js

File cấu hình Jest cho phép bạn tùy chỉnh nhiều thứ — từ test environment, file patterns, đến coverage thresholds:

JAVASCRIPT jest.config.js Copy

```javascript
module.exports = {
  // Environment: 'node' cho backend (mặc định), 'jsdom' cho browser
  testEnvironment: 'node',

  // Pattern để tìm test files
  testMatch: [
    '**/tests/**/*.test.js',
    '**/__tests__/**/*.js'
  ],

  // Các file được tính vào coverage report
  collectCoverageFrom: [
    'src/**/*.js',
    '!src/index.js',         // Exclude entry point
    '!**/node_modules/**'
  ],

  // Ngưỡng coverage tối thiểu — build fail nếu dưới ngưỡng
  coverageThresholds: {
    global: {
      branches: 70,
      functions: 75,
      lines: 75,
      statements: 75
    }
  },

  // File chạy trước khi test suite bắt đầu (setup DB, global mocks ...)
  setupFilesAfterFramework: ['./tests/setup.js'],

  // Verbose output để dễ đọc
  verbose: true
};
```

⚠ CẢNH BÁO

> Đặt `coverageThresholds` quá cao (ví dụ 100%) từ đầu có thể gây ra anti-pattern "test chỉ để đạt con số" thay vì test có nghĩa. Bắt đầu với 70-75% và tăng dần là chiến lược hợp lý.

Cấu trúc thư mục Tests

Tổ chức thư mục tests rõ ràng giúp dễ tìm và maintain. Quy ước phổ biến nhất là tách riêng unit tests và integration tests:

TEXT project structure Copy

```
project/
├── src/
│   ├── app.js              # Express app (export, không listen)
│   ├── index.js            # Entry point (gọi app.listen)
│   ├── utils/
│   │   └── format.js
│   ├── services/
│   │   ├── emailService.js
│   │   └── userService.js
│   ├── models/
│   │   └── User.js
│   └── routes/
│       └── auth.js
└── tests/
    ├── setup.js            # Global setup/teardown
    ├── helpers/
    │   └── testDb.js       # DB helper cho integration tests
    ├── unit/
    │   ├── format.test.js
    │   ├── emailService.test.js
    │   └── userService.test.js
    └── integration/
        ├── auth.test.js
        └── posts.test.js
```

Lưu ý quan trọng: file `src/app.js` nên **export** Express app mà không gọi `app.listen()`. File `src/index.js` mới gọi `listen()`. Điều này giúp Supertest có thể import app và tự quản lý port trong quá trình test.

16.3 Unit Testing Cơ Bản

Unit test kiểm tra từng đơn vị code nhỏ nhất (thường là một hàm) một cách độc lập. Mục tiêu là: **với input X, hàm có trả về output Y đúng như mong đợi không?** Unit tests không cần database, không cần network — chỉ cần logic của hàm đó.

Jest Matchers — Các phép kiểm tra phổ biến

Jest cung cấp rất nhiều *matchers* — các hàm kiểm tra (assertion) có cú pháp đọc gần như tiếng Anh tự nhiên:

Matcher	Ý nghĩa	Ví dụ
`toBe(val)`	So sánh strict equality (===)	`expect(1 + 1).toBe(2)`
`toEqual(val)`	So sánh deep equality (cho objects/arrays)	`expect({a:1}).toEqual({a:1})`
`toContain(item)`	Array/string chứa item	`expect([1,2,3]).toContain(2)`
`toHaveLength(n)`	Array/string có độ dài n	`expect('abc').toHaveLength(3)`
`toThrow(msg)`	Hàm throw error	`expect(() ⇒ fn()).toThrow('err')`
`toBeTruthy()`	Giá trị truthy	`expect('hello').toBeTruthy()`
`toBeFalsy()`	Giá trị falsy	`expect(null).toBeFalsy()`
`toBeNull()`	Giá trị null	`expect(val).toBeNull()`
`toBeUndefined()`	Giá trị undefined	`expect(val).toBeUndefined()`
`toBeGreaterThan(n)`	Số lớn hơn n	`expect(5).toBeGreaterThan(3)`
`toMatch(regex)`	String khớp regex/substring	`expect('hello').toMatch(/ell/)`
`toHaveProperty(key)`	Object có property	`expect(obj).toHaveProperty('id')`

Bạn có thể thêm `.not` vào trước bất kỳ matcher nào để đảo ngược: `expect(val).not.toBeNull()`.

Ví dụ thực tế: Unit Test cho Utility Functions

Hãy bắt đầu với một module utility đơn giản gồm ba hàm: format tiền tệ, tạo slug từ text, và validate email:

JAVASCRIPT src/utils/format.js Copy

```
// Hàm format tiền tệ Việt Nam
const formatCurrency = (amount) => {
  if (amount < 0) throw new Error('Số tiền không được âm');
  return new Intl.NumberFormat('vi-VN', {
    style: 'currency',
    currency: 'VND'
  }).format(amount);
};

// Hàm tạo slug từ text (kể cả tiếng Việt có dấu)
const slugify = (text) => {
  return text
    .toLowerCase()
    .normalize('NFD')
    .replace(/[̀-ͯ]/g, '')   // Xóa dấu
    .replace(/[^a-z0-9]+/g, '-')       // Thay ký tự đặc biệt bằng gạch
ngang
    .replace(/^-+|-+$/g, '');             // Xóa gạch ngang đầu/cuối
};

// Hàm validate email
const validateEmail = (email) => {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
};

module.exports = { formatCurrency, slugify, validateEmail };
```

Bây giờ viết tests cho module này. Mỗi hàm sẽ có một `describe` block riêng, bên trong là các `it` (hoặc `test`) blocks cho từng case:

JAVASCRIPT tests/unit/format.test.js Copy

```javascript
const { formatCurrency, slugify, validateEmail } =
require('../../src/utils/format');

// describe() nhóm các tests liên quan lại với nhau
describe('formatCurrency', () => {
  it('formats VND correctly', () => {
    // toMatch() cho phép kiểm tra string chứa pattern
    expect(formatCurrency(1000000)).toMatch(/1\.000\.000/);
  });

  it('handles zero', () => {
    expect(formatCurrency(0)).toMatch(/0/);
  });

  it('throws for negative values', () => {
    // Khi test hàm throw, phải bọc trong arrow function
    expect(() => formatCurrency(-100)).toThrow('Số tiền không được âm');
  });
});

describe('slugify', () => {
  it('converts Vietnamese text to slug', () => {
    expect(slugify('Xin chào thế giới')).toBe('xin-chao-the-gioi');
  });

  it('handles special characters', () => {
    expect(slugify('Hello World!')).toBe('hello-world');
  });

  it('removes leading/trailing dashes', () => {
    expect(slugify('  hello  ')).toBe('hello');
  });

  it('handles numbers in text', () => {
    expect(slugify('Chapter 16 Testing')).toBe('chapter-16-testing');
  });
});
```

```
describe('validateEmail', () => {
  it('validates correct email', () => {
    expect(validateEmail('user@example.com')).toBe(true);
    expect(validateEmail('name.last@domain.org')).toBe(true);
  });

  it('rejects invalid email', () => {
    expect(validateEmail('not-an-email')).toBe(false);
    expect(validateEmail('missing@tld')).toBe(false);
    expect(validateEmail('@domain.com')).toBe(false);
    expect(validateEmail('')).toBe(false);
  });
});
```

Lifecycle Hooks: beforeEach / afterEach / beforeAll / afterAll

Jest cung cấp các hàm *lifecycle hooks* để chạy code trước/sau tests. Hiểu rõ sự khác biệt giữa chúng rất quan trọng:

- `beforeAll(fn)`: Chạy **một lần** trước tất cả tests trong describe block. Dùng cho setup tốn kém như kết nối database.
- `afterAll(fn)`: Chạy **một lần** sau tất cả tests. Dùng cho cleanup như đóng kết nối database.
- `beforeEach(fn)`: Chạy **trước mỗi** test. Dùng cho reset state, tạo test data mới.
- `afterEach(fn)`: Chạy **sau mỗi** test. Dùng cho cleanup state, clear mocks.

JAVASCRIPT tests/unit/lifecycle-example.test.js Copy

```javascript
describe('Lifecycle Hooks Demo', () => {
  let db;
  let user;

  // Chạy 1 lần - kết nối database (ví dụ concept)
  beforeAll(async () => {
    db = await createTestDatabase();
    console.log('DB connected');
  });

  // Chạy trước MỖI test - tạo data sạch
  beforeEach(async () => {
    user = await db.users.create({
      name: 'Test User',
      email: 'test@example.com'
    });
  });

  // Chạy sau MỖI test - dọn dẹp data
  afterEach(async () => {
    await db.users.deleteMany({});
    jest.clearAllMocks(); // Reset tất cả mock function calls
  });

  // Chạy 1 lần sau tất cả tests - đóng kết nối
  afterAll(async () => {
    await db.close();
  });

  it('test 1 - user tồn tại', () => {
    expect(user).toBeDefined();
    expect(user.email).toBe('test@example.com');
  });

  it('test 2 - user được tạo mới (không bị ảnh hưởng từ test 1)', () => {
    // Mỗi test có user riêng nhờ beforeEach
    expect(user.name).toBe('Test User');
```

```
  });
});
```

> 💡 **MẸO**
>
> Dùng `beforeEach` thay vì `beforeAll` khi muốn đảm bảo mỗi test bắt đầu với state hoàn toàn sạch (test isolation). Tests phụ thuộc vào nhau là một anti-pattern nguy hiểm — thứ tự chạy tests có thể thay đổi.

16.4 Mocking với Jest

Một trong những thách thức lớn nhất khi viết unit tests là **dependencies**. Hàm của bạn có thể gọi database, gửi email, gọi API bên ngoài — những thứ này không nên thực sự chạy trong unit tests (chậm, không ổn định, có side effects). Đây là lúc **mocking** phát huy tác dụng.

Mocking là kỹ thuật thay thế một dependency thật bằng một "giả" (mock) có behavior do bạn kiểm soát. Jest có ba cơ chế mocking chính:

- `jest.fn()`: Tạo một mock function từ đầu.
- `jest.spyOn(obj, 'method')`: "Spy" lên một method có sẵn, theo dõi nó được gọi như thế nào mà không cần thay thế behavior (hoặc có thể override).
- `jest.mock('module')`: Mock toàn bộ một module — mọi export của module đó đều trở thành mock function tự động.

Ví dụ 1: Mock External Service (Nodemailer)

Giả sử có hàm `sendWelcomeEmail` dùng Nodemailer để gửi email. Trong unit test, ta không muốn gửi email thật — ta chỉ muốn kiểm tra hàm gọi nodemailer đúng cách không:

JAVASCRIPT src/services/emailService.js Copy

```javascript
const nodemailer = require('nodemailer');

const sendWelcomeEmail = async (user) => {
  const transporter = nodemailer.createTransport({
    host: process.env.SMTP_HOST,
    port: 587,
    auth: {
      user: process.env.SMTP_USER,
      pass: process.env.SMTP_PASS
    }
  });

  await transporter.sendMail({
    from: '"MyApp" <noreply@myapp.com>',
    to: user.email,
    subject: `Chào mừng ${user.name}!`,
    html: `<h1>Xin chào ${user.name}</h1><p>Cảm ơn bạn đã đăng ký.</p>`
  });
};

module.exports = { sendWelcomeEmail };
```

JAVASCRIPT tests/unit/emailService.test.js Copy

```
// jest.mock() PHẢI được gọi trước khi require module cần test
jest.mock('nodemailer');

const nodemailer = require('nodemailer');
const { sendWelcomeEmail } = require('../../src/services/emailService');

describe('sendWelcomeEmail', () => {
  let mockSendMail;

  beforeEach(() => {
    // Tạo mock function cho sendMail
    // mockResolvedValue() - mock async function trả về resolved Promise
    mockSendMail = jest.fn().mockResolvedValue({ messageId: 'test-id' });

    // Khi code gọi nodemailer.createTransport(), nó nhận object giả
    nodemailer.createTransport.mockReturnValue({
      sendMail: mockSendMail
    });
  });

  afterEach(() => {
    // Reset tất cả mock sau mỗi test để tránh ảnh hưởng chéo
    jest.clearAllMocks();
  });

  it('sends email to correct recipient', async () => {
    const user = { name: 'Alice', email: 'alice@test.com' };
    await sendWelcomeEmail(user);

    // Kiểm tra sendMail được gọi đúng 1 lần
    expect(mockSendMail).toHaveBeenCalledTimes(1);

    // expect.objectContaining() - kiểm tra object có các field này (có thể
có thêm field khác)
    expect(mockSendMail).toHaveBeenCalledWith(
      expect.objectContaining({
        to: 'alice@test.com',
        subject: 'Chào mừng Alice!'
```

```
      })
    );
  });

  it('propagates send errors', async () => {
    // mockRejectedValue() - mock async function throw error
    mockSendMail.mockRejectedValue(new Error('SMTP error'));

    // Test async throw: dùng await + expect().rejects.toThrow()
    await expect(
      sendWelcomeEmail({ name: 'Bob', email: 'bob@test.com' })
    ).rejects.toThrow('SMTP error');
  });

  it('creates transporter with correct config', async () => {
    const user = { name: 'Alice', email: 'alice@test.com' };
    await sendWelcomeEmail(user);

    expect(nodemailer.createTransport).toHaveBeenCalledWith(
      expect.objectContaining({
        host: process.env.SMTP_HOST,
        port: 587
      })
    );
  });
});
```

Ví dụ 2: Mock Database Model

Trong unit tests cho service layer, ta không muốn thực sự kết nối MongoDB. Thay vào đó, ta mock Mongoose model để kiểm soát data trả về:

JAVASCRIPT tests/unit/userService.test.js Copy

```
// Mock toàn bộ module User model
jest.mock('../../src/models/User');

const User = require('../../src/models/User');
const userService = require('../../src/services/userService');

describe('userService', () => {
  afterEach(() => {
    jest.clearAllMocks();
  });

  describe('getUserById', () => {
    it('returns user when found', async () => {
      const mockUser = {
        _id: '507f1f77bcf86cd799439011',
        name: 'Alice',
        email: 'alice@test.com'
      };

      // Khi code gọi User.findById(), nó nhận mockUser thay vì query DB
thật
      User.findById.mockResolvedValue(mockUser);

      const result = await
userService.getUserById('507f1f77bcf86cd799439011');

      expect(result).toEqual(mockUser);

expect(User.findById).toHaveBeenCalledWith('507f1f77bcf86cd799439011');
    });

    it('throws when user not found', async () => {
      User.findById.mockResolvedValue(null);

      await expect(userService.getUserById('999')).rejects.toThrow('User not
found');
    });
  });
```

```
  describe('updateUser', () => {
    it('updates user fields', async () => {
      const mockUpdatedUser = {
        _id: '123',
        name: 'Alice Updated',
        email: 'alice@test.com'
      };

      User.findByIdAndUpdate.mockResolvedValue(mockUpdatedUser);

      const result = await userService.updateUser('123', { name: 'Alice
Updated' });

      expect(result).toEqual(mockUpdatedUser);
      expect(User.findByIdAndUpdate).toHaveBeenCalledWith(
        '123',
        { name: 'Alice Updated' },
        { new: true, runValidators: true }
      );
    });
  });
});
```

jest.spyOn — Theo dõi mà không thay thế

Đôi khi bạn chỉ muốn theo dõi xem một method có được gọi không, mà không cần thay đổi behavior của nó. `jest.spyOn()` cho phép điều này:

JAVASCRIPT tests/unit/spy-example.test.js Copy

```
const logger = require('../../src/utils/logger');

describe('Spy Example', () => {
  it('logs error when operation fails', async () => {
    // Spy lên logger.error - không thay đổi behavior
    const logSpy = jest.spyOn(logger, 'error');

    // Thực thi logic gây ra error...
    await someServiceThatLogs();

    // Kiểm tra logger.error đã được gọi
    expect(logSpy).toHaveBeenCalled();
    expect(logSpy).toHaveBeenCalledWith(expect.stringContaining('Error:'));

    // Restore method về implementation thật
    logSpy.mockRestore();
  });
});
```

> 📌 **GHI NHỚ**
>
> Luôn gọi `jest.clearAllMocks()` trong `afterEach` để reset mock call counts và implementations. Nếu không, mock state từ test trước sẽ "rò rỉ" sang test sau, gây ra kết quả không đoán trước được.

16.5 Integration Testing với Supertest

Integration tests kiểm tra sự phối hợp giữa nhiều thành phần: routes, middleware, service, và database. Thay vì mock mọi thứ, integration tests sử dụng một test database thật (hoặc in-memory database) để đảm bảo tất cả các thành phần hoạt động đúng khi kết hợp với nhau.

Supertest là thư viện cho phép test HTTP endpoints mà không cần start một HTTP server thật — nó tự xử lý việc binding port, gửi request, và nhận response.

Global Test Setup

File `tests/setup.js` được chạy một lần cho toàn bộ test suite — đây là nơi kết nối và cleanup database:

JAVASCRIPT tests/setup.js Copy

```
const mongoose = require('mongoose');

// Chạy một lần trước TẤT CẢ test files
beforeAll(async () => {
  const uri = process.env.TEST_MONGODB_URI ||
'mongodb://localhost:27017/testdb';
  await mongoose.connect(uri, {
    useNewUrlParser: true,
    useUnifiedTopology: true
  });
});

// Chạy sau MỖI test – xóa data để tests độc lập nhau
afterEach(async () => {
  const collections = mongoose.connection.collections;
  for (const key in collections) {
    await collections[key].deleteMany({});
  }
});

// Chạy một lần sau TẤT CẢ test files
afterAll(async () => {
  await mongoose.connection.close();
});
```

Integration Test cho Auth Routes

Đây là ví dụ điển hình về integration test: test toàn bộ luồng đăng ký, đăng nhập, và truy cập protected route:

JAVASCRIPT tests/integration/auth.test.js Copy

```javascript
const request = require('supertest');
const app = require('../../src/app');

describe('Auth Routes', () => {

  // ═══ POST /api/v1/auth/register ═══
  describe('POST /api/v1/auth/register', () => {
    it('should create user with valid data', async () => {
      const res = await request(app)
        .post('/api/v1/auth/register')
        .send({
          name: 'Alice Nguyen',
          email: 'alice@test.com',
          password: 'SecurePass123!'
        });

      expect(res.statusCode).toBe(201);
      expect(res.body.success).toBe(true);
      expect(res.body.data).toHaveProperty('token');

      // Password KHÔNG được trả về trong response
      expect(res.body.data).not.toHaveProperty('password');
      expect(res.body.data.user.email).toBe('alice@test.com');
    });

    it('should fail with missing required fields', async () => {
      const res = await request(app)
        .post('/api/v1/auth/register')
        .send({ email: 'test@test.com' }); // Thiếu name và password

      expect(res.statusCode).toBe(400);
      expect(res.body.success).toBe(false);
      expect(res.body.error).toBeDefined();
    });

    it('should fail with invalid email format', async () => {
      const res = await request(app)
        .post('/api/v1/auth/register')
```

```
        .send({
          name: 'Test User',
          email: 'not-an-email',
          password: 'Password123!'
        });

      expect(res.statusCode).toBe(400);
    });

    it('should fail with duplicate email', async () => {
      const userData = {
        name: 'Bob',
        email: 'bob@test.com',
        password: 'Password123!'
      };

      // Đăng ký lần 1 - thành công
      await request(app).post('/api/v1/auth/register').send(userData);

      // Đăng ký lần 2 với cùng email - phải fail
      const res = await
request(app).post('/api/v1/auth/register').send(userData);

      expect(res.statusCode).toBe(409);
      expect(res.body.error.code).toBe('EMAIL_ALREADY_EXISTS');
    });
  });

  // ═══ POST /api/v1/auth/login ═══
  describe('POST /api/v1/auth/login', () => {
    // Tạo user trước mỗi test trong describe block này
    beforeEach(async () => {
      await request(app).post('/api/v1/auth/register').send({
        name: 'Alice',
        email: 'alice@test.com',
        password: 'Password123!'
      });
    });

    it('should login with correct credentials', async () => {
      const res = await request(app)
```

```
        .post('/api/v1/auth/login')
        .send({ email: 'alice@test.com', password: 'Password123!' });

      expect(res.statusCode).toBe(200);
      expect(res.body.data).toHaveProperty('token');
      expect(typeof res.body.data.token).toBe('string');
    });

    it('should fail with wrong password', async () => {
      const res = await request(app)
        .post('/api/v1/auth/login')
        .send({ email: 'alice@test.com', password: 'wrongpassword' });

      expect(res.statusCode).toBe(401);
      expect(res.body.success).toBe(false);
    });

    it('should fail with non-existent email', async () => {
      const res = await request(app)
        .post('/api/v1/auth/login')
        .send({ email: 'nobody@test.com', password: 'Password123!' });

      expect(res.statusCode).toBe(401);
    });
  });

  // ═══ GET /api/v1/users/me (Protected Route) ═══
  describe('GET /api/v1/users/me', () => {
    it('should return current user with valid token', async () => {
      // Setup: đăng ký để lấy token
      const registerRes = await request(app)
        .post('/api/v1/auth/register')
        .send({ name: 'Alice', email: 'alice@test.com', password:
'Password123!' });

      const token = registerRes.body.data.token;

      // Dùng token để gọi protected route
      const res = await request(app)
        .get('/api/v1/users/me')
        .set('Authorization', `Bearer ${token}`);
```

```
      expect(res.statusCode).toBe(200);
      expect(res.body.data.email).toBe('alice@test.com');
      expect(res.body.data).not.toHaveProperty('password');
    });

    it('should fail without token', async () => {
      const res = await request(app).get('/api/v1/users/me');
      expect(res.statusCode).toBe(401);
    });

    it('should fail with invalid/expired token', async () => {
      const res = await request(app)
        .get('/api/v1/users/me')
        .set('Authorization', 'Bearer invalid.token.here');

      expect(res.statusCode).toBe(401);
    });
  });
});
```

> 💡 **MẸO**
>
> Khi viết integration tests, hãy test theo **user stories**: "Khi user đăng ký với email hợp lệ, hệ thống tạo account và trả về token". Điều này giúp tests có ý nghĩa business và dễ hiểu hơn.

16.6 Test Coverage

Test coverage (độ bao phủ test) đo lường phần trăm code đã được tests "chạm tới". Đây là metric quan trọng nhưng thường bị hiểu sai. Coverage cao không đảm bảo code chất lượng — một test chạy qua code mà không assert gì cũng tính vào coverage.

Các loại Coverage Metrics

Jest (dùng Istanbul/nyc bên dưới) theo dõi 4 loại metrics:

Metric	Ý nghĩa	Ví dụ
Statements	% câu lệnh đã được thực thi	`const x = 1;` là 1 statement
Branches	% nhánh rẽ đã được test (if/else, ternary, switch)	`if (x) { ... } else { ... }` có 2 branches
Functions	% hàm đã được gọi ít nhất một lần	Mỗi function/arrow function được đếm
Lines	% dòng code đã được thực thi	Giống statements nhưng tính theo dòng

Trong 4 metrics, **Branches** thường là khó đạt nhất vì mỗi điều kiện if/else tạo ra 2 nhánh. Nếu tests chỉ test happy path mà bỏ qua error cases, branches coverage sẽ thấp.

Sinh và đọc Coverage Report

BASH terminal Copy

```
# Chạy tests với coverage report
npx jest --coverage

# Output trong terminal:
# ----------|---------|----------|---------|---------|-------------------
# File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Lines
# ----------|---------|----------|---------|---------|-------------------
# src/utils/format.js   |   95.23 |    87.50 |  100.00 |   95.23 | 28
# src/services/email.js |   82.14 |    75.00 |   88.88 |   82.14 | 45, 67-70
# src/routes/auth.js    |   91.30 |    83.33 |   94.73 |   91.30 | 120
# ----------|---------|----------|---------|---------|-------------------
# All files             |   89.55 |    81.94 |   94.54 |   89.55 |
# ----------|---------|----------|---------|---------|-------------------

# Report HTML được tạo tại:
# coverage/lcov-report/index.html
# Mở file này trong browser để xem từng dòng code được cover hay không
```

Cột **"Uncovered Lines"** cho biết chính xác dòng nào chưa được test — đây là thông tin quý giá để bạn biết cần viết thêm tests gì.

Đặt Coverage Threshold

Coverage threshold trong `jest.config.js` đảm bảo CI/CD pipeline fail nếu coverage giảm xuống dưới mức cho phép — một cơ chế quan trọng để bảo vệ chất lượng code theo thời gian:

JAVASCRIPT jest.config.js Copy

```
module.exports = {
  coverageThresholds: {
    // Ngưỡng global cho toàn bộ codebase
    global: {
      branches: 70,
      functions: 75,
      lines: 75,
      statements: 75
    },
    // Có thể đặt ngưỡng riêng cho từng file/thư mục quan trọng
    './src/services/': {
      branches: 80,
      functions: 85,
      lines: 85,
      statements: 85
    }
  }
};
```

> ⚠️ **CẢNH BÁO**
>
> **Đừng chase 100% coverage.** Một số code không cần test (configuration files, simple getters/setters). Quan trọng hơn là test *critical paths*: authentication logic, payment processing, data validation. Tập trung vào chất lượng tests, không phải số lượng.

> 📌 **GHI NHỚ**

> 70-80% là ngưỡng coverage hợp lý cho hầu hết dự án. Tăng dần theo thời gian khi team quen với TDD. Đừng đột ngột đặt 90% cho codebase cũ — sẽ tạo ra nhiều tests vô nghĩa chỉ để đạt số.

16.7 ESLint và Prettier

Testing kiểm tra code có chạy đúng không. **ESLint** và **Prettier** kiểm tra code có được viết đúng chuẩn không. Cùng nhau, chúng tạo thành bức tường bảo vệ code quality toàn diện.

- **ESLint**: Phân tích code để tìm lỗi tiềm ẩn, code smells, và vi phạm style rules. ESLint hiểu *ý nghĩa* của code.
- **Prettier**: Formatter tự động — chỉ quan tâm đến formatting (indent, quotes, trailing commas), không quan tâm đến logic. Prettier đảm bảo toàn team viết code trông giống nhau.

Cài đặt

BASH terminal Copy

```
# ESLint và Prettier core
npm install --save-dev eslint prettier

# Plugin tích hợp Prettier vào ESLint (tắt ESLint rules xung đột với
Prettier)
npm install --save-dev eslint-config-prettier

# Khởi tạo ESLint config
npx eslint --init
```

Cấu hình ESLint

JAVASCRIPT .eslintrc.js Copy

```
module.exports = {
  env: {
    node: true,      // Biết các global của NodeJS (process, __dirname,
etc.)
    es2022: true,    // Cho phép syntax ES2022+
    jest: true       // Biết các global của Jest (describe, it, expect,
etc.)
  },

  extends: [
    'eslint:recommended',  // ESLint recommended rules
    'prettier'             // TẮT các ESLint rules xung đột với Prettier
(luôn để cuối)
  ],

  parserOptions: {
    ecmaVersion: 'latest'
  },

  rules: {
    // warn: cảnh báo nhưng không fail build
    // error: fail build nếu vi phạm

    'no-unused-vars': ['warn', {
      argsIgnorePattern: '^_'  // Bỏ qua args bắt đầu bằng _
    }],
    'no-console': ['warn', {
      allow: ['warn', 'error']  // Cho phép console.warn và console.error
    }],
    'prefer-const': 'error',   // Dùng const thay vì let khi có thể
    'no-var': 'error',         // Không dùng var
    'eqeqeq': ['error', 'always'],  // Phải dùng === thay vì ==
    'curly': 'error',               // Luôn có {} cho if/else/for
    'no-duplicate-imports': 'error',
    'no-return-await': 'warn'  // return await trong async function thường
không cần thiết
  },
```

```
  // Override riêng cho test files
  overrides: [
    {
      files: ['tests/**/*.js', '**/*.test.js'],
      rules: {
        'no-unused-expressions': 'off'  // Cho phép expect().toBe() style
      }
    }
  ]
};
```

Cấu hình Prettier

JSON .prettierrc Copy

```
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5",
  "printWidth": 100,
  "arrowParens": "always",
  "endOfLine": "lf"
}
```

Thêm scripts vào `package.json` để dễ sử dụng:

JSON package.json Copy

```json
{
  "scripts": {
    "lint": "eslint src/ tests/",
    "lint:fix": "eslint src/ tests/ --fix",
    "format": "prettier --write \"src/**/*.js\" \"tests/**/*.js\"",
    "format:check": "prettier --check \"src/**/*.js\" \"tests/**/*.js\""
  }
}
```

Husky + lint-staged: Tự động hóa trước khi commit

Cài ESLint và Prettier là một chuyện, nhưng đảm bảo mọi người trong team thực sự chạy chúng trước khi commit là chuyện khác. **Husky** và **lint-staged** giải quyết vấn đề này bằng cách chạy checks tự động như một Git pre-commit hook.

BASH terminal Copy

```bash
# Cài Husky và lint-staged
npm install --save-dev husky lint-staged

# Khởi tạo Husky
npx husky init

# Tạo pre-commit hook
echo "npx lint-staged" > .husky/pre-commit
```

Cấu hình lint-staged trong `package.json` — chỉ chạy trên các file được staged (staged files):

JSON package.json Copy

```
{
  "lint-staged": {
    "*.js": [
      "eslint --fix",
      "prettier --write"
    ]
  }
}
```

Với cấu hình này, mỗi lần bạn chạy `git commit`, lint-staged sẽ:

1. Lấy danh sách các file .js được staged.
2. Chạy `eslint --fix` để tự động sửa các lỗi có thể fix được.
3. Chạy `prettier --write` để format code.
4. Tự động stage lại các file đã được sửa.
5. Nếu còn lỗi ESLint không thể tự fix — commit bị block.

> 💡 **MẸO**
>
> Ngoài lint-staged, bạn có thể thêm pre-push hook để chạy tests trước khi push: `echo "npm test" > .husky/pre-push`. Điều này đảm bảo không ai push code broken lên remote.

16.8 Test Database Strategy với MongoDB Memory Server

Một thách thức lớn khi integration testing với MongoDB là: bạn cần một database thật để test, nhưng không muốn test can thiệp vào data production hay development. Có ba chiến lược phổ biến:

1. **Dùng database riêng cho test**: Simple nhưng cần MongoDB running, dễ bị conflicts khi nhiều người test cùng lúc.
2. **Docker container MongoDB**: Isolated nhưng cần Docker setup và start/stop container.
3. **MongoDB Memory Server**: Chạy MongoDB trong memory, không cần cài MongoDB. Khởi động nhanh, hoàn toàn isolated — đây là lựa chọn tốt nhất cho testing.

Cài đặt MongoDB Memory Server

BASH terminal Copy

```
npm install --save-dev mongodb-memory-server

# Lần đầu chạy, mongodb-memory-server sẽ tự download binary MongoDB
# (~70MB) và cache lại cho lần sau
```

Test Database Helper

Tạo một helper module để tái sử dụng logic connect/disconnect/clear trong nhiều test files:

JAVASCRIPT tests/helpers/testDb.js Copy

```
const { MongoMemoryServer } = require('mongodb-memory-server');
const mongoose = require('mongoose');

let mongod; // MongoMemoryServer instance

/**
 * Khởi động in-memory MongoDB và kết nối Mongoose
 */
const connect = async () => {
  mongod = await MongoMemoryServer.create();
  const uri = mongod.getUri();

  await mongoose.connect(uri, {
    useNewUrlParser: true,
    useUnifiedTopology: true
  });
};

/**
 * Drop database, đóng kết nối, và dừng in-memory server
 */
const disconnect = async () => {
  await mongoose.connection.dropDatabase();
  await mongoose.connection.close();
  await mongod.stop();
};

/**
 * Xóa tất cả documents trong tất cả collections
 * (Dùng giữa các tests để đảm bảo isolation)
 */
const clearDatabase = async () => {
  const collections = mongoose.connection.collections;
  for (const key in collections) {
    await collections[key].deleteMany({});
  }
};
```

```
module.exports = { connect, disconnect, clearDatabase };
```

Sử dụng trong Integration Tests

JAVASCRIPT tests/integration/posts.test.js Copy

```
const db = require('../helpers/testDb');
const request = require('supertest');
const app = require('../../src/app');

// Lifecycle cho test file này
beforeAll(async () => await db.connect());
afterEach(async () => await db.clearDatabase()); // Sạch sau mỗi test
afterAll(async () => await db.disconnect());

describe('Posts API', () => {
  let authToken;

  // Setup: đăng nhập để lấy token cho các tests
  beforeEach(async () => {
    const res = await request(app)
      .post('/api/v1/auth/register')
      .send({ name: 'Test User', email: 'test@test.com', password:
'Pass123!' });
    authToken = res.body.data.token;
  });

  describe('GET /api/v1/posts', () => {
    it('should return empty array when no posts', async () => {
      const res = await request(app)
        .get('/api/v1/posts')
        .set('Authorization', `Bearer ${authToken}`);

      expect(res.statusCode).toBe(200);
      expect(res.body.data.posts).toEqual([]);
      expect(res.body.data.total).toBe(0);
    });

    it('should return list of posts', async () => {
      // Tạo 2 posts trước
      await request(app)
        .post('/api/v1/posts')
        .set('Authorization', `Bearer ${authToken}`)
        .send({ title: 'Post 1', content: 'Content 1', tags: ['nodejs'] });
```

```
      await request(app)
        .post('/api/v1/posts')
        .set('Authorization', `Bearer ${authToken}`)
        .send({ title: 'Post 2', content: 'Content 2', tags: ['javascript']
});

      const res = await request(app)
        .get('/api/v1/posts')
        .set('Authorization', `Bearer ${authToken}`);

      expect(res.statusCode).toBe(200);
      expect(res.body.data.posts).toHaveLength(2);
      expect(res.body.data.total).toBe(2);
    });

    it('should filter posts by tag', async () => {
      await request(app)
        .post('/api/v1/posts')
        .set('Authorization', `Bearer ${authToken}`)
        .send({ title: 'NodeJS Post', content: '...', tags: ['nodejs'] });

      await request(app)
        .post('/api/v1/posts')
        .set('Authorization', `Bearer ${authToken}`)
        .send({ title: 'React Post', content: '...', tags: ['react'] });

      const res = await request(app)
        .get('/api/v1/posts?tag=nodejs')
        .set('Authorization', `Bearer ${authToken}`);

      expect(res.statusCode).toBe(200);
      expect(res.body.data.posts).toHaveLength(1);
      expect(res.body.data.posts[0].title).toBe('NodeJS Post');
    });
  });

  describe('POST /api/v1/posts', () => {
    it('should create post with valid data', async () => {
      const res = await request(app)
        .post('/api/v1/posts')
```

```
        .set('Authorization', `Bearer ${authToken}`)
        .send({
          title: 'Hướng dẫn Jest',
          content: 'Nội dung bài viết...',
          tags: ['nodejs', 'testing']
        });

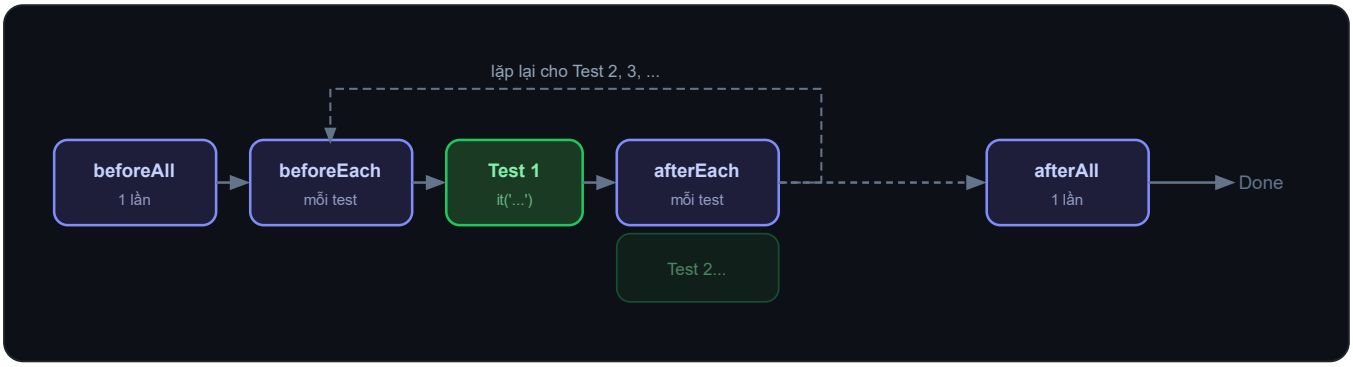
      expect(res.statusCode).toBe(201);
      expect(res.body.data.title).toBe('Hướng dẫn Jest');
      expect(res.body.data.tags).toContain('testing');
      expect(res.body.data).toHaveProperty('_id');
      expect(res.body.data).toHaveProperty('createdAt');
    });

    it('should require authentication', async () => {
      const res = await request(app)
        .post('/api/v1/posts')
        .send({ title: 'Unauthorized', content: '...' });

      expect(res.statusCode).toBe(401);
    });
  });
});
```

Test Lifecycle Flow

Hiểu rõ thứ tự chạy các lifecycle hooks giúp bạn đặt setup và teardown đúng chỗ. Đây là sơ đồ trực quan:



Tối ưu Performance cho Test Suite lớn

Khi test suite phát triển, thời gian chạy có thể tăng đáng kể. Một số kỹ thuật tối ưu:

JAVASCRIPT jest.config.js Copy

```javascript
module.exports = {
  // Chạy tests song song trên nhiều worker (mặc định = số CPU cores - 1)
  maxWorkers: '50%',

  // Chỉ chạy lại tests liên quan đến files đã thay đổi (khi dùng --watch)
  watchPlugins: [
    'jest-watch-typeahead/filename',
    'jest-watch-typeahead/testname'
  ],

  // Cache transform results để lần chạy sau nhanh hơn
  cache: true,
  cacheDirectory: '.jest-cache',

  // Timeout cho mỗi test (mặc định 5000ms)
  testTimeout: 10000
};
```

📌 **GHI NHỚ**

Mỗi Jest worker process có MongoDB Memory Server riêng khi dùng `maxWorkers > 1`. Điều này tốt cho isolation nhưng tốn nhiều memory hơn. Trên CI/CD, có thể dùng `--runInBand` (chạy tuần tự) để tiết kiệm memory và tránh race conditions.

Bài tập thực hành

Sau đây là các bài tập giúp bạn củng cố kiến thức từ chương này. Mỗi bài được đánh giá độ khó để bạn lên kế hoạch học tập phù hợp.

DỄ

16.1 Test CRUD Routes

Viết integration tests cho tất cả routes của Posts API: `GET /api/posts` (lấy danh sách), `GET /api/posts?tag=nodejs` (lọc theo tag), `GET /api/posts?page=2&limit=5` (pagination). Mỗi test case cần kiểm tra: status code trả về đúng, cấu trúc response có đủ fields cần thiết (`success`, `data`, `pagination`), và dữ liệu trả về khớp với data đã tạo. Tạo ít nhất 3 posts với các tags khác nhau làm test data trước khi test.

TRUNG BÌNH

16.2 Mock Email Service

Mock `nodemailer.sendMail()` trong unit tests để test email flow mà không gửi email thật. Cụ thể: (1) Viết unit test xác nhận email được gọi với đúng recipient và subject khi user đăng ký mới. (2) Viết test cho trường hợp SMTP server fail — đảm bảo error được propagate và user không được tạo nếu email fail. (3) Viết test kiểm tra nội dung HTML email có chứa tên của user. Dùng `jest.mock('nodemailer')` và `expect.objectContaining()`.

KHÓ

16.3 CI Pipeline với GitHub Actions

Setup CI test pipeline hoàn chỉnh với GitHub Actions: (1) Tạo `.github/workflows/ci.yml` để chạy tests tự động khi push hoặc tạo PR vào `main` / `develop`. (2) Pipeline phải fail PR nếu coverage dưới 70% ở bất kỳ metric nào. (3) Chạy ESLint check trong CI — fail nếu có errors. (4) Cache `node_modules` để tăng tốc CI. (5) Tạo coverage badge trong README bằng Shields.io hoặc Codecov. Thêm environment variables cần thiết vào GitHub Secrets.

PHẦN 5 — NÂNG CAO VÀ CLOUD

Chương 17: Deploy và DevOps Cơ Bản

Code chạy trên máy local chưa phải sản phẩm. Chương này đưa ứng dụng lên internet — từ cấu hình môi trường, Docker, Nginx, SSL đến CI/CD pipeline tự động deploy mỗi khi bạn push code.

🎯 **SAU CHƯƠNG NÀY BẠN SẼ**

- Quản lý environment variables đúng cách cho production
- Dùng PM2 giữ cho Node process luôn chạy
- Đóng gói ứng dụng bằng Docker và docker-compose
- Deploy lên VPS với Nginx làm reverse proxy
- Cài SSL miễn phí với Let's Encrypt
- Tự động deploy với GitHub Actions CI/CD

17.1 Environment Variables — Cấu Hình Production

Hardcode URL database, API key, hay secret vào source code là lỗi phổ biến nhất của developer mới. Nếu commit lên GitHub, bất kỳ ai cũng đọc được. **Environment variables** giải quyết vấn đề này.

SHELL .env Copy

```
# .env — KHÔNG commit file này!
NODE_ENV=development
PORT=3000
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/mydb
JWT_SECRET=super-secret-key-ít-nhất-32-ký-tự
REDIS_URL=redis://localhost:6379
CORS_ORIGIN=http://localhost:5173
```

SHELL .env.example Copy

```
# .env.example – COMMIT file này, làm template cho team
NODE_ENV=development
PORT=3000
MONGODB_URI=mongodb+srv://USER:PASS@cluster.mongodb.net/DBNAME
JWT_SECRET=your-secret-here-min-32-chars
REDIS_URL=redis://localhost:6379
CORS_ORIGIN=http://localhost:5173
```

JAVASCRIPT src/config/env.js Copy

```
require('dotenv').config();

const requiredVars = [
  'MONGODB_URI',
  'JWT_SECRET',
];

// Fail fast: kiểm tra ngay khi khởi động
for (const key of requiredVars) {
  if (!process.env[key]) {
    console.error(`❌ Missing required env var: ${key}`);
    process.exit(1);
  }
}

module.exports = {
  nodeEnv:     process.env.NODE_ENV || 'development',
  port:        parseInt(process.env.PORT) || 3000,
  mongoUri:    process.env.MONGODB_URI,
  jwtSecret:   process.env.JWT_SECRET,
  redisUrl:    process.env.REDIS_URL || 'redis://localhost:6379',
  corsOrigin:  process.env.CORS_ORIGIN || '*',
  isDev:       process.env.NODE_ENV !== 'production',
};
```

TEXT .gitignore Copy

```
node_modules/
.env
.env.local
.env.*.local
dist/
coverage/
*.log
```

17.2 PM2 — Process Manager

Khi chạy `node app.js` thẳng, nếu server crash thì… chết. PM2 tự động restart khi crash, quản lý logs, và tận dụng nhiều CPU core qua cluster mode.

SHELL Terminal Copy

```
# Cài global
npm install -g pm2

# Chạy app
pm2 start src/index.js --name my-api

# Cluster mode — dùng hết CPU cores
pm2 start src/index.js --name my-api -i max

# Xem status
pm2 status
pm2 monit        # Dashboard real-time

# Logs
pm2 logs my-api
pm2 logs my-api --lines 100

# Restart / reload (zero-downtime)
pm2 restart my-api
pm2 reload my-api    # graceful — không drop connections

# Tự start lại khi server reboot
pm2 startup          # follow hướng dẫn in ra
pm2 save             # lưu danh sách process hiện tại
```

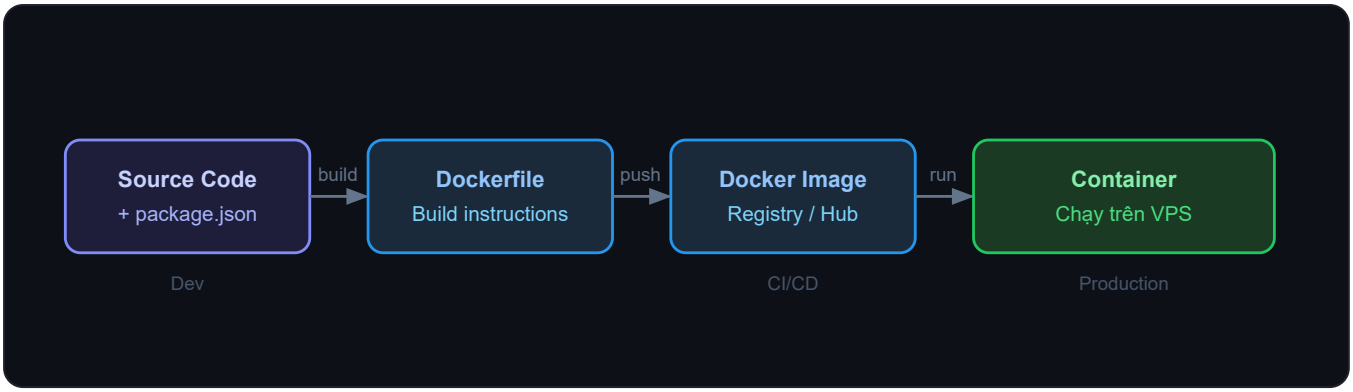
JAVASCRIPT ecosystem.config.js Copy

```
module.exports = {
  apps: [{
    name: 'my-api',
    script: 'src/index.js',
    instances: 'max',        // dùng tất cả CPU cores
    exec_mode: 'cluster',
    watch: false,            // tắt trong production
    max_memory_restart: '500M',
    env: {
      NODE_ENV: 'development',
      PORT: 3000,
    },
    env_production: {
      NODE_ENV: 'production',
      PORT: 3000,
    },
    log_date_format: 'YYYY-MM-DD HH:mm:ss',
    error_file: 'logs/error.log',
    out_file: 'logs/out.log',
    merge_logs: true,
  }],
};

// Chạy với: pm2 start ecosystem.config.js --env production
```

17.3 Docker — Đóng Gói Ứng Dụng

"Chạy được trên máy tôi mà" — câu nói kinh điển của developer. Docker giải quyết bằng cách đóng gói cả code, runtime, và dependencies vào một **container** chạy được ở mọi nơi giống hệt nhau.



DOCKERFILE — Dockerfile — Copy

```
# ——— Build stage ———
FROM node:20-alpine AS builder
WORKDIR /app
# Copy package files trước để cache layer khi code thay đổi
COPY package*.json ./
RUN npm ci --only=production

# ——— Production stage ———
FROM node:20-alpine
WORKDIR /app
# Copy chỉ node_modules và source, không copy devDependencies
COPY --from=builder /app/node_modules ./node_modules
COPY . .
# Không chạy với root user
USER node
EXPOSE 3000
CMD ["node", "src/index.js"]
```

TEXT — .dockerignore — Copy

```
node_modules
.env
.git
*.log
coverage
.nyc_output
README.md
```

SHELL Build và chạy Copy

```shell
# Build image
docker build -t my-api:latest .

# Chạy container
docker run -d \
  --name my-api \
  -p 3000:3000 \
  --env-file .env \
  my-api:latest

# Xem logs
docker logs -f my-api

# Vào trong container để debug
docker exec -it my-api sh
```

docker-compose — Chạy nhiều service cùng nhau

YAML docker-compose.yml Copy

```yaml
version: '3.9'

services:
  api:
    build: .
    ports:
      - "3000:3000"
    env_file: .env
    environment:
      MONGODB_URI: mongodb://mongo:27017/mydb
      REDIS_URL: redis://redis:6379
    depends_on:
      - mongo
      - redis
    restart: unless-stopped

  mongo:
    image: mongo:7
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    restart: unless-stopped

volumes:
  mongo_data:
  redis_data:
```

SHELL Dùng docker-compose Copy

```shell
# Khởi động tất cả services
docker-compose up -d

# Xem logs của từng service
docker-compose logs -f api
docker-compose logs mongo

# Dừng và xóa containers
docker-compose down

# Rebuild image khi code thay đổi
docker-compose up -d --build api
```

17.4 Deploy lên VPS

Hướng dẫn từng bước với DigitalOcean Droplet (Ubuntu 22.04, ~$6/tháng) — tương tự mọi VPS khác.

SHELL Trên server – Lần đầu setup Copy

```
# 1. SSH vào server
ssh root@YOUR_SERVER_IP

# 2. Update hệ thống
apt update && apt upgrade -y

# 3. Cài NodeJS với nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh |
bash
source ~/.bashrc
nvm install --lts
nvm use --lts

# 4. Cài PM2, Git
npm install -g pm2
apt install -y git

# 5. Clone project
git clone https://github.com/your-user/your-repo.git /app
cd /app

# 6. Cài dependencies
npm install --production

# 7. Tạo file .env
cp .env.example .env
nano .env          # điền các giá trị thực

# 8. Start với PM2
pm2 start ecosystem.config.js --env production
pm2 save
pm2 startup        # copy paste lệnh in ra

# 9. Firewall
ufw allow 22        # SSH
ufw allow 80        # HTTP
```

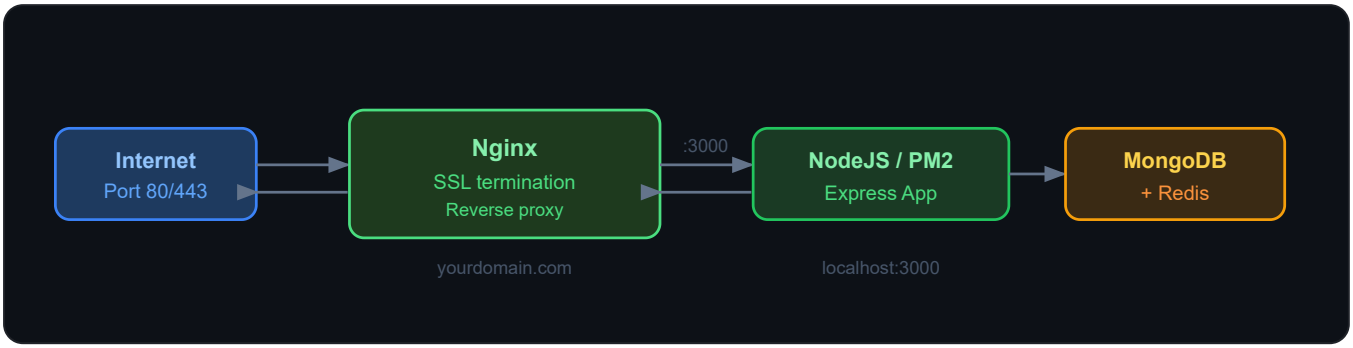
```
ufw allow 443       # HTTPS
ufw enable
```

SHELL — Deploy code mới sau này — Copy

```
cd /app
git pull origin main
npm install --production
pm2 reload my-api    # zero-downtime restart
```

17.5 Nginx — Reverse Proxy

Không nên expose NodeJS thẳng ra internet. Nginx đứng trước làm reverse proxy: xử lý SSL, static files, load balancing, và bảo vệ server.



SHELL — Cài và cấu hình Nginx — Copy

```
# Cài Nginx
apt install -y nginx

# Tạo config site
nano /etc/nginx/sites-available/my-api
```

NGINX /etc/nginx/sites-available/my-api Copy

```
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;

    # Giới hạn body size (upload)
    client_max_body_size 10M;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;

        # WebSocket support
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';

        # Forward thông tin thực của client
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Timeout
        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }

    # Serve static files trực tiếp, không qua NodeJS
    location /uploads {
        alias /app/uploads;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }
}
```

SHELL — Kích hoạt site — Copy

```
# Tạo symlink để enable site
ln -s /etc/nginx/sites-available/my-api /etc/nginx/sites-enabled/

# Test config hợp lệ
nginx -t

# Reload Nginx
systemctl reload nginx
```

17.6 SSL với Let's Encrypt

HTTPS là bắt buộc cho mọi site production. Certbot tự động cài SSL miễn phí từ Let's Encrypt.

SHELL — Cài SSL — Copy

```
# Cài Certbot
snap install --classic certbot

# Cài SSL và tự cấu hình Nginx
certbot --nginx -d yourdomain.com -d www.yourdomain.com

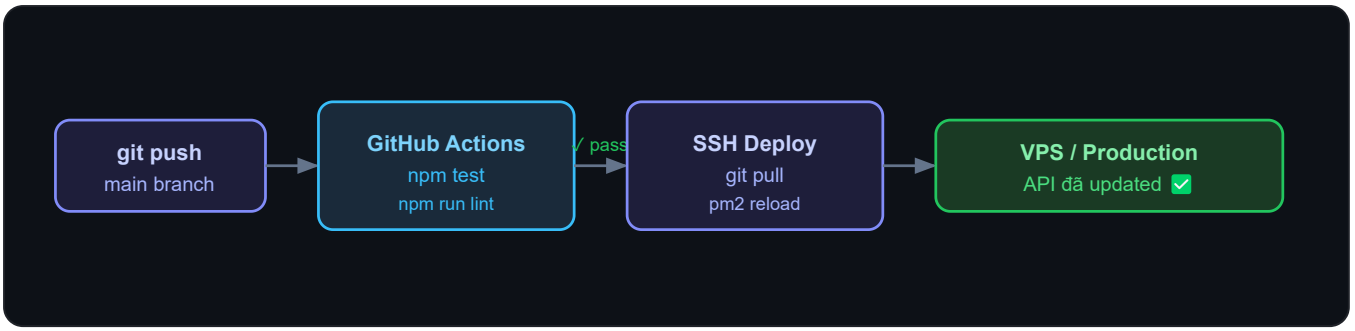
# Certbot sẽ:
# 1. Xác thực domain ownership
# 2. Lấy certificate từ Let's Encrypt
# 3. Cấu hình Nginx tự động (thêm HTTPS, redirect HTTP → HTTPS)
# 4. Cài cron job tự renew (cert có hiệu lực 90 ngày)

# Test tự động renew
certbot renew --dry-run
```

> 💡 **DOMAIN PHẢI TRỎ VỀ SERVER TRƯỚC**
>
> Certbot cần verify bạn sở hữu domain. Trỏ DNS A record của domain về IP của VPS ít nhất 5-10 phút trước khi chạy certbot.

17.7 GitHub Actions — CI/CD Pipeline

Thay vì SSH vào server bằng tay mỗi lần deploy, GitHub Actions tự động chạy test và deploy mỗi khi push code lên branch main.



YAML .github/workflows/deploy.yml Copy

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  # Job 1: Chạy tests
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run linter
        run: npm run lint

      - name: Run tests
        run: npm test
        env:
          NODE_ENV: test
          MONGODB_URI: ${{ secrets.TEST_MONGODB_URI }}
          JWT_SECRET: test-secret

  # Job 2: Deploy (chỉ chạy khi push main, sau khi test pass)
  deploy:
    needs: test          # chờ test pass mới chạy
```

```
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.SERVER_IP }}
          username: ${{ secrets.SERVER_USER }}
          key: ${{ secrets.SSH_PRIVATE_KEY }}
          script: |
            cd /app
            git pull origin main
            npm install --production
            pm2 reload ecosystem.config.js --env production
            echo "✅ Deploy thành công!"
```

Cấu hình GitHub Secrets

Vào GitHub repo → Settings → Secrets and variables → Actions → New repository secret:

- `SERVER_IP` — IP address của VPS
- `SERVER_USER` — username SSH (thường là `root` hoặc `ubuntu`)
- `SSH_PRIVATE_KEY` — nội dung file `~/.ssh/id_rsa` trên máy local
- `TEST_MONGODB_URI` — MongoDB URI cho test environment

> ⚠️ **BẢO MẬT SSH KEY**
>
> Chạy `ssh-keygen -t ed25519 -C "github-actions"` tạo key riêng cho CI/CD. Copy public key vào `~/.ssh/authorized_keys` trên server. Copy private key vào GitHub Secret. KHÔNG dùng key của bản thân.

📚 Tóm tắt chương 17

- **Environment variables**: dùng `.env` + `dotenv`, validate required vars lúc startup, commit `.env.example`
- **PM2**: giữ Node process luôn chạy, cluster mode tận dụng CPU, `pm2 startup` để auto-start

- **Docker**: đóng gói app + dependencies, multi-stage build để image nhỏ, docker-compose cho local dev
- **VPS setup**: NodeJS qua nvm, clone code, PM2, firewall cơ bản
- **Nginx**: reverse proxy trước NodeJS, xử lý SSL, serve static files hiệu quả hơn
- **Let's Encrypt**: SSL miễn phí, tự renew, Certbot cấu hình Nginx tự động
- **GitHub Actions**: CI/CD pipeline — test → deploy khi push main, không cần SSH tay

🏋️ Bài tập thực hành

DỄ

17.1 Dockerize Express App

Tạo `Dockerfile` multi-stage cho project Express API đã làm ở chương 8. Build image: `docker build -t my-api .` Chạy với env file: `docker run --env-file .env -p 3000:3000 my-api` Verify API hoạt động bình thường. Kiểm tra image size trước và sau multi-stage build.

TRUNG BÌNH

17.2 docker-compose Full Stack

Viết `docker-compose.yml` chạy 3 services: API (NodeJS), MongoDB, Redis. API phải kết nối MongoDB và Redis qua service names (không phải localhost). Thêm `healthcheck` cho MongoDB. Thêm volume để data không mất khi restart. Chạy `docker-compose up -d` và test toàn bộ flow login → cache → logout.

KHÓ

17.3 CI/CD Pipeline hoàn chỉnh

Setup GitHub Actions CI/CD thực tế: (1) Job test chạy Jest + coverage report, fail nếu coverage < 70%. (2) Job build Docker image và push lên Docker Hub. (3) Job deploy SSH vào server, pull image mới, restart container. Dùng Oracle Cloud Free Tier hoặc Fly.io làm server miễn phí. Bonus: thêm Slack notification khi deploy thành công/thất bại.

PHẦN 5 — NÂNG CAO VÀ CLOUD

Chương 18: Serverless và Cloud Functions

"Serverless" không có nghĩa là không có server — có nghĩa là *bạn* không cần quản lý server. Viết function, deploy, và chỉ trả tiền khi có request. Chương này giới thiệu paradigm hiện đại nhất của cloud computing và khi nào nên dùng nó.

> 🎯 **SAU CHƯƠNG NÀY BẠN SẼ**
>
> - Hiểu serverless khác traditional server ở điểm nào
> - Biết cold start là gì và cách giảm thiểu
> - Deploy API lên Vercel Functions trong 5 phút
> - Viết AWS Lambda handler chuẩn
> - Hiểu Cloudflare Workers và edge computing
> - Quyết định khi nào dùng serverless, khi nào dùng server truyền thống

18.1 Serverless là gì?

Mô hình traditional: bạn thuê VPS chạy 24/7 kể cả lúc không có request nào. Bạn phải lo cập nhật OS, scale khi traffic tăng, xử lý khi server chết.

Mô hình serverless: bạn chỉ upload code (một function). Cloud provider tự lo chạy, scale từ 0 đến hàng nghìn instances, và bạn chỉ trả tiền cho số lần function được gọi.

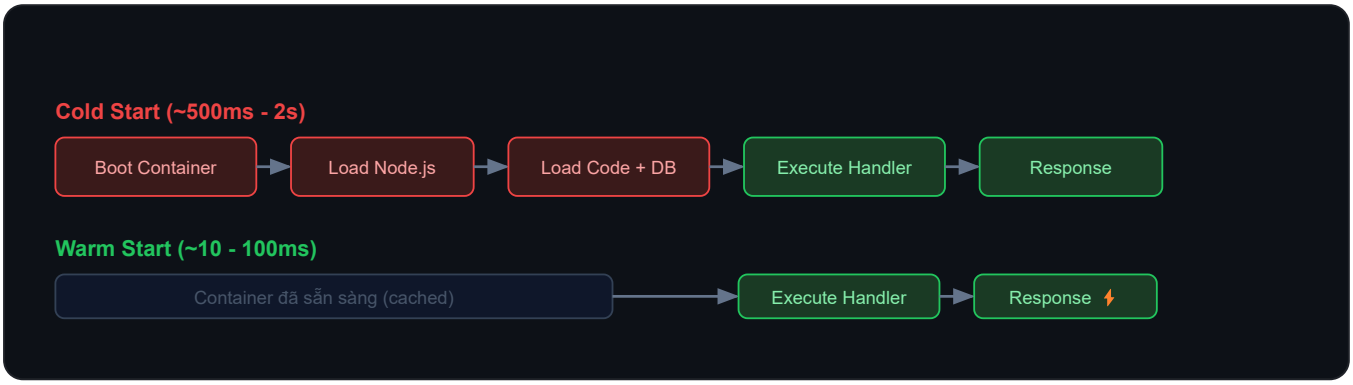


Ưu và nhược điểm

	Ưu điểm ✅	Nhược điểm ❌
Serverless	Không quản lý infra, auto scale, pay-per-use, deploy nhanh	Cold start, stateless, time limit (~15 min), vendor lock-in
Traditional	Full control, persistent connections (WebSocket), dễ debug local	Quản lý server, trả tiền idle, phải tự scale

18.2 Cold Start — Vấn Đề Đặc Trưng

Khi function chưa được gọi gần đây, cloud platform phải **boot container mới**, load code, khởi tạo connections — trước khi xử lý request. Đây gọi là **cold start**.



Cách giảm cold start

- **Giảm dependencies**: chỉ import những gì cần, tránh heavy packages
- **Keep connections ngoài handler**: kết nối DB một lần, tái dùng giữa các invocations

- **Provisioned Concurrency** (AWS Lambda): giữ N instances luôn warm, tốn thêm tiền
- **Cloudflare Workers**: không có cold start vì dùng V8 isolates thay vì containers

JAVASCRIPT Kết nối DB ngoài handler — tái dùng giữa invocations Copy

```
const mongoose = require('mongoose');

// Kết nối một lần, tái dùng trong mọi invocation
let isConnected = false;

async function connectDB() {
  if (isConnected) return;  // Đã kết nối rồi
  await mongoose.connect(process.env.MONGODB_URI);
  isConnected = true;
}

// Handler
exports.handler = async (event) => {
  await connectDB();  // Chỉ thực sự connect lần đầu
  // ... xử lý request
};
```

18.3 Vercel Functions — Dễ nhất để bắt đầu

Vercel là nền tảng deploy phổ biến nhất cho Next.js và static sites, nhưng cũng có Serverless Functions rất dễ dùng — không cần cấu hình gì.

Cách hoạt động

Mọi file trong thư mục `api/` tự động trở thành một serverless endpoint.

JAVASCRIPT api/hello.js → GET /api/hello Copy

```javascript
// Vercel Function đơn giản nhất
export default function handler(req, res) {
  const { name = 'World' } = req.query;
  res.status(200).json({
    message: `Xin chào ${name}!`,
    timestamp: new Date().toISOString(),
    method: req.method,
  });
}
```

JAVASCRIPT api/users/[id].js → GET /api/users/123 (dynamic route) Copy

```
import mongoose from 'mongoose';

let isConnected = false;

async function dbConnect() {
  if (isConnected) return;
  await mongoose.connect(process.env.MONGODB_URI);
  isConnected = true;
}

const UserSchema = new mongoose.Schema({ name: String, email: String });
const User = mongoose.models.User || mongoose.model('User', UserSchema);

export default async function handler(req, res) {
  await dbConnect();

  const { id } = req.query;

  if (req.method === 'GET') {
    const user = await User.findById(id);
    if (!user) return res.status(404).json({ error: 'Không tìm thấy user'
});
    return res.json({ data: user });
  }

  if (req.method === 'PUT') {
    const user = await User.findByIdAndUpdate(id, req.body, { new: true });
    return res.json({ data: user });
  }

  if (req.method === 'DELETE') {
    await User.findByIdAndDelete(id);
    return res.status(204).end();
  }

  res.status(405).json({ error: 'Method not allowed' });
}
```

JSON vercel.json Copy

```json
{
  "version": 2,
  "env": {
    "MONGODB_URI": "@mongodb-uri"
  },
  "functions": {
    "api/**/*.js": {
      "maxDuration": 30
    }
  }
}
```

SHELL Deploy lên Vercel Copy

```shell
# Cài Vercel CLI
npm install -g vercel

# Deploy (lần đầu sẽ hỏi thiết lập project)
vercel

# Deploy production
vercel --prod

# Xem logs
vercel logs

# Set env var
vercel env add MONGODB_URI
```

Migrate Express API sang Vercel

JAVASCRIPT api/index.js – Wrap toàn bộ Express app Copy

```javascript
// Cách này migrate Express app sang Vercel mà không cần rewrite
const express = require('express');
const app = express();

app.use(express.json());

app.get('/api/posts', async (req, res) => {
  // ... logic
  res.json({ posts: [] });
});

app.post('/api/posts', async (req, res) => {
  // ...
});

// Export cho Vercel
module.exports = app;
```

18.4 AWS Lambda

AWS Lambda là serverless platform lớn nhất thế giới — mạnh mẽ hơn nhưng cũng phức tạp hơn Vercel. Phù hợp cho các ứng dụng enterprise hoặc khi cần tích hợp sâu với các AWS services.

JAVASCRIPT handler.js — Lambda function chuẩn Copy

```
// Lambda nhận event từ API Gateway
exports.handler = async (event, context) => {
  // context.callbackWaitsForEmptyEventLoop = false; // Cho phép reuse DB
connection

  const { httpMethod, path, body, queryStringParameters, pathParameters } =
event;

  try {
    // Router đơn giản
    if (httpMethod === 'GET' && path === '/users') {
      const users = [{ id: 1, name: 'Alice' }]; // thay bằng DB query
      return response(200, { data: users });
    }

    if (httpMethod === 'GET' && path.startsWith('/users/')) {
      const { id } = pathParameters;
      return response(200, { data: { id, name: 'Alice' } });
    }

    if (httpMethod === 'POST' && path === '/users') {
      const data = JSON.parse(body || '{}');
      // validate + save to DB
      return response(201, { data: { id: Date.now(), ...data } });
    }

    return response(404, { error: 'Route không tồn tại' });

  } catch (err) {
    console.error('Lambda error:', err);
    return response(500, { error: 'Lỗi server' });
  }
};

function response(statusCode, body) {
  return {
    statusCode,
    headers: {
```

```
      'Content-Type': 'application/json',
      'Access-Control-Allow-Origin': '*',
    },
    body: JSON.stringify(body),
  };
}
```

YAML serverless.yml — Dùng Serverless Framework Copy

```yaml
service: my-api

provider:
  name: aws
  runtime: nodejs20.x
  region: ap-southeast-1  # Singapore — gần Việt Nam nhất
  environment:
    MONGODB_URI: ${env:MONGODB_URI}
    JWT_SECRET: ${env:JWT_SECRET}

functions:
  api:
    handler: handler.handler
    events:
      - httpApi:
          path: /{proxy+}   # catch-all route
          method: ANY
      - httpApi:
          path: /
          method: ANY
```

SHELL Deploy với Serverless Framework Copy

```
# Cài
npm install -g serverless

# Deploy
serverless deploy

# Xem logs
serverless logs -f api --tail

# Remove
serverless remove
```

💡 **CHI PHÍ AWS LAMBDA**

Free tier: 1 triệu requests/tháng + 400,000 GB-seconds compute miễn phí. Sau đó: $0.20/1 triệu requests. Với traffic nhỏ đến vừa, Lambda thường rẻ hơn VPS nhiều.

18.5 Cloudflare Workers — Edge Computing

Cloudflare Workers chạy code ở **edge** — tức là ở hơn 300 datacenter gần người dùng nhất trên toàn cầu. Response time < 50ms vì code chạy ngay gần user, không phải round trip về US hay Singapore.

📌 **WORKERS KHÁC LAMBDA Ở CHỖ NÀO?**

Workers dùng **V8 Isolates** thay vì Docker containers. Mỗi isolate boot trong microseconds thay vì hàng trăm ms → **không có cold start**. Nhưng Workers có giới hạn: không có filesystem, hạn chế Node.js APIs, không có native modules.

JAVASCRIPT src/worker.js Copy

```javascript
// Cloudflare Workers dùng Web Standard APIs
export default {
  async fetch(request, env, ctx) {
    const url = new URL(request.url);

    // Router
    if (url.pathname === '/api/hello') {
      return Response.json({
        message: 'Xin chào từ Edge!',
        location: request.cf?.city || 'Unknown',
        country: request.cf?.country || 'Unknown',
      });
    }

    if (url.pathname === '/api/cache-demo') {
      const cacheKey = new Request(url.toString());
      const cache = caches.default;

      // Check cache
      let response = await cache.match(cacheKey);
      if (response) {
        return new Response(response.body, {
          headers: { ...response.headers, 'X-Cache': 'HIT' }
        });
      }

      // Cache miss: fetch data
      const data = { timestamp: Date.now(), data: 'expensive computation' };
      response = Response.json(data, {
        headers: { 'Cache-Control': 'public, max-age=60' }
      });

      ctx.waitUntil(cache.put(cacheKey, response.clone()));
      return new Response(response.body, {
        headers: { ...response.headers, 'X-Cache': 'MISS' }
      });
    }
```

```
    // KV Storage (Cloudflare's global key-value store)
    if (url.pathname.startsWith('/api/kv/')) {
      const key = url.pathname.slice(8);
      if (request.method === 'GET') {
        const value = await env.MY_KV.get(key);
        return value
          ? Response.json({ key, value })
          : new Response('Not found', { status: 404 });
      }
      if (request.method === 'PUT') {
        const body = await request.text();
        await env.MY_KV.put(key, body, { expirationTtl: 3600 });
        return Response.json({ ok: true });
      }
    }

    return new Response('Not Found', { status: 404 });
  }
};
```

TOML wrangler.toml Copy

```
name = "my-worker"
main = "src/worker.js"
compatibility_date = "2024-01-01"

[[kv_namespaces]]
binding = "MY_KV"
id = "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
```

SHELL Dev và deploy Copy

```shell
# Cài Wrangler CLI
npm install -g wrangler

# Login
wrangler login

# Dev locally
wrangler dev

# Deploy
wrangler deploy
```

18.6 Khi nào dùng Serverless vs Server truyền thống?

Tiêu chí	Serverless ✅	Traditional Server ✅
Traffic pattern	Spiky, không đều	Steady, cao liên tục
Request type	Short-lived (< 15 phút)	Long-running, WebSocket
Team	Startup, MVP, ít DevOps	Có DevOps, cần full control
Cost (low traffic)	Rất rẻ (có thể $0)	Tốn tiền ngay cả lúc idle
Cost (high traffic)	Có thể đắt hơn VPS	Dự đoán được, thường rẻ hơn
Latency	Cold start có thể chậm	Consistent, không có cold start
Ví dụ phù hợp	API, webhooks, cron jobs, image processing	Chat app, game server, admin dashboard

💡 **HYBRID ARCHITECTURE**

Không cần chọn một. Nhiều app dùng **cả hai**: Express trên VPS cho core API, Cloudflare Workers cho edge caching, Lambda cho background jobs và webhooks.

📚 Tóm tắt chương 18

- **Serverless**: code chạy theo yêu cầu, auto scale, pay-per-use — không phải không có server mà là không quản lý server
- **Cold start**: delay lần đầu invoke khi container chưa warm — giảm bằng cách giữ connections ngoài handler và giảm dependencies
- **Vercel Functions**: file trong `api/` tự thành endpoint — dễ nhất để bắt đầu
- **AWS Lambda**: handler function nhận `event` từ API Gateway, trả về object có `statusCode` và `body`
- **Cloudflare Workers**: V8 isolates, không có cold start, chạy ở edge (300+ locations), dùng Web Standard APIs
- Chọn serverless khi traffic spiky, request ngắn, team nhỏ. Chọn VPS khi traffic đều, cần WebSocket, hoặc high traffic

🏋️ Bài tập thực hành

DỄ

18.1 Hello World trên Vercel

Tạo project mới, tạo file `api/hello.js` trả về JSON với thông tin: thời gian hiện tại, method HTTP, query params. Connect GitHub repo và deploy. Verify bằng curl hoặc browser. Thêm env var `MY_NAME` và đọc trong function.

TRUNG BÌNH

18.2 Migrate Express API lên Vercel

Lấy Express API đã làm ở chương 8 (Blog API). Tạo file `api/index.js` export Express app. Tạo `vercel.json` để route tất cả requests về `api/index.js`. Connect MongoDB Atlas. Deploy và test toàn bộ CRUD endpoints. So sánh response time của Vercel Function vs server Express truyền thống.

KHÓ

18.3 Image Resize Service với Lambda

Xây dựng Lambda function nhận POST request với base64-encoded image. Dùng package `sharp` resize thành 3 kích thước: thumbnail (150x150), medium (600x400), large (1200x800). Upload 3 ảnh lên S3. Trả về object với 3 URLs. Cấu hình memory 512MB (sharp cần RAM), timeout 30 giây. Deploy với Serverless Framework.



Chúc mừng bạn đã hoàn thành!

Bạn vừa học xong **18 chương** về NodeJS — từ cách HTTP hoạt động, viết server thuần, đến Express, database, authentication, testing và serverless cloud.

Bước tiếp theo: hãy xây dựng một dự án thực tế của bạn. Áp dụng kiến thức tốt nhất khi bạn thực sự làm sản phẩm.

← Xem lại mục lục đầy đủ